DSA PATTERN RECOGNITION HANDBOOK

20 Problem-Solving Patterns + 7 Essential Algorithms

JAVA EDITION | LEETCODE INTERVIEW PREPARATION

A working handbook for engineers who already know their data structures and want to recognise the right technique quickly. For each pattern it covers the structural signals, the invariant that makes the technique correct, a reusable Java template, and complete interview-ready solutions to 80 assigned LeetCode problems and 13 algorithm practice problems.

Part I
Recognition framework & decision guides

Part II
20 pattern chapters, 80 problems

Part III
7 essential algorithms

Part IV
Revision toolkit & self-assessment

INTRODUCTION

How to use this handbook

This handbook does not reteach data structures. It is about the step before you write code: looking at an unfamiliar problem and working out which structural property makes an efficient solution possible.

What each chapter gives you

Section	Purpose
Pattern identity	What the technique is for, the input shape it expects, and its key insight in one or two sentences.
Recognition signals	The structural reasons a problem fits, the question to ask yourself, when the pattern is wrong, and the patterns most easily confused with it.
Concepts & invariants	The invariant, monotonicity, recurrence or amortised argument that makes the technique correct.
Reusable Java templates	Compilable templates, with the reusable skeleton separated from the problem-specific logic. Variants that genuinely differ get their own template.
Complexity & pitfalls	Typical costs, off-by-one traps, overflow hazards and common interview follow-ups.
Mastery checklist	Takeaways, a recognition rule, an in-interview checklist and the key distinction from a neighbouring pattern.
Assigned problems	Problem → Key insight → Approach → Java solution → Complexity → Pitfall. Where a problem's primary technique differs from the chapter heading, a *classification note* says so.

Suggested study loop

1. **Read the recognition signals and invariants first.** Skip the template for now.
2. **Cover the solutions.** For each assigned problem, read only the problem number and title, then write down the pattern, the invariant and the complexity you expect.
3. **Implement from memory** on a blank editor, ideally against a 20–25 minute limit for Medium problems.
4. **Compare against the handbook**, paying most attention to the *Key insight* and *Pitfall* lines, since those are where interview solutions usually fail.
5. **Re-derive the template** a few days later without looking, and tick it off in the revision checklist.

Conventions

- **Intervals of indices** are written mathematically: `[l, r]` is inclusive, `[l, r)` is half-open. Each template states which one it uses.
- **Prefix sums** use `pre[0] = 0` and `pre[i+1] = pre[i] + a[i]`, so `sum(a[l..r]) = pre[r+1] - pre[l]`.
- **Complexities** are worst case unless marked *amortised* or *expected*. *Space* means auxiliary space and excludes the input and the required output, unless stated otherwise. Recursion stack depth is always counted.
- **Graphs** use `V` vertices and `E` edges. Every graph solution states whether edges are directed and what the weights are assumed to be.

- **Java.** Code targets Java 17 and LeetCode's method signatures. LeetCode supplies `ListNode`, `TreeNode` and (for 759) `Interval`; they are not repeated in the solutions. Templates are complete, non-public classes that compile on their own.
- **Premium** marks LeetCode problems that need a subscription. They are kept because the syllabus assigns them.

> **Verification.** Every problem number, official title, difficulty and premium flag was checked against LeetCode's public problem catalogue when this handbook was built. Every Java block in the document (templates and solutions) was compiled with JDK 17. Every solution was run against LeetCode's published examples and extra edge cases before inclusion. That testing builds confidence but does not replace LeetCode's full hidden test suites.

NAVIGATION

Contents

Part I — Recognition Framework

I.1 How to use this handbook

I.2 The ten-question recognition framework

I.3 Pattern-selection decision guide

I.4 Similar patterns, different decisions

I.5 Quick reference: the 20 patterns

I.6 Index: the 7 essential algorithms

I.7 Problem validation & classification notes

Part II — Twenty Problem-Solving Patterns

1 Two Pointers: Opposite Direction 167 · 15 · 11 · 42

2 Sliding Window 643 · 3 · 1493 · 76

3 Fast & Slow Pointers 876 · 142 · 547 · 202

4 Prefix Sum 724 · 560 · 523 · 325

5 Binary Search: Index 704 · 34 · 162 · 4

6 Binary Search on Answer 1283 · 875 · 1011 · 410

7 Merge Intervals 56 · 57 · 253 · 759

8 Backtracking 78 · 46 · 39 · 51

9 Recursion / DFS 104 · 200 · 547 · 124

10 BFS & Level Order 733 · 994 · 542 · 127

11 Topological Sort 210 · 207 · 802 · 269

12 Dynamic Programming: 1D 70 · 198 · 1416 · 494

13 Dynamic Programming: 2D 62 · 64 · 1143 · 72

14 Greedy: Interval Scheduling 252 · 435 · 56 · 986

15 Heap / Priority Queue 1046 · 215 · 347 · 295

16 Bit Manipulation 136 · 137 · 260 · 201

17 Union-Find / DSU 1971 · 684 · 547 · 721

18 Shortest Path: Dijkstra 1971 · 743 · 1631 · 787

19 Trie / Prefix Tree 208 · 211 · 648 · 212

20 Segment Tree / Fenwick Tree 307 · 315 · 2407 · 699

Part III — Seven Essential Algorithms Beyond the Core Patterns

III Overview

A1 Monotonic Stack 739 · 907

A2 Dijkstra's Algorithm 1514 · 1976

A3 Union-Find / DSU 1319 · 990

A4 Minimum Spanning Tree 1584 · 1135

A5 Kadane's Algorithm 53 · 918

A6 KMP String Matching 28 · 214

A7 Bellman–Ford 787

Part IV — Revision Toolkit

IV Overview

IV.A Quick-reference sheet

IV.B Interview recognition checklist

IV.C Complexity reference

IV.D Revision checklist

IV.E Self-assessment framework

IV.F References

PART I · RECOGNITION FRAMEWORK

The ten-question recognition framework

Pattern recognition is not keyword matching. A phrase like "subarray" or "shortest" suggests a starting hypothesis, but the decision comes from structure: what the input is, what is being asked, what the constraints allow, and which property you can exploit. Work through these ten questions, in order, for every new problem.

#	Question	What you are really looking for
1	**What is the input structure?**	Array, string, set of intervals, tree, explicit graph, grid (an implicit graph), state space (an implicit graph), or a sequence of operations (a data-structure design problem).
2	**What is being asked?**	*Existence* (boolean), *counting* (number of ways: DP or prefix-count), *optimisation* (min/max: greedy, DP, binary search on answer), *ordering* (sorting, topological order), *connectivity* (DFS/BFS/DSU), or *shortest path* (BFS, Dijkstra, Bellman–Ford).
3	**Which constraints rule out brute force?**	Turn the bound on `n` into a complexity budget (table below). The budget usually removes most candidate patterns at once.
4	**Is there ordering, monotonicity, an invariant or a repeated subproblem?**	Sorted input → pointers or binary search. A monotone predicate → binary search on the answer. A window property that is monotone in its length → sliding window. Overlapping subproblems → DP. A local choice that is never regretted → greedy.
5	**What can the problem be reduced to?**	Maintaining a window, finding a boundary, traversing states, combining partial answers (divide and conquer or tree DP), or answering range queries.
6	**Does it need a specific data structure?**	Heap for repeated min/max, monotonic stack for nearest greater/smaller, deque for window max, hash map for complements and prefix counts, DSU for incremental connectivity, trie for prefixes, segment/Fenwick tree for dynamic range aggregates.
7	**What makes the algorithm correct?**	State the invariant or exchange argument in one sentence. If you cannot, the pattern choice is probably wrong.
8	**What are the edge cases?**	Empty or single-element input, all-equal values, negatives, zero, duplicates, overflow, disconnected components, cycles, unreachable targets, and touching interval boundaries.
9	**What are the time and space costs?**	Include sorting, the recursion stack, heap size and hash-map size, then compare against the budget from question 3.
10	**Which plausible alternative is wrong, and why?**	Say it explicitly, for example: "Sliding window fails because values can be negative, so I use prefix sums with a hash map." Interviewers listen for this.

Constraints → complexity budget

The usual rule of thumb is about 10^8 simple operations per second. Java is slightly slower than C++, so treat these as order-of-magnitude bounds, not exact thresholds.

Bound on n	Feasible complexity	Typical techniques
$n \le 10$–12	$O(n!)$, $O(n \cdot n!)$	Permutation backtracking
$n \le 20$–25	$O(2^n \cdot n)$	Subset enumeration, bitmask DP, pruned backtracking
$n \le 40$	$O(2^{n/2})$	Meet-in-the-middle
$n \le 500$	$O(n^3)$	Interval DP, Floyd–Warshall
$n \le 5 \cdot 10^3$	$O(n^2)$	2D DP, all pairs, O(n) work per start point
$n \le 10^5$–10^6	O(n log n) or O(n)	Sorting, heaps, binary search, two pointers, sliding window, prefix sums, DSU, segment/Fenwick trees
value $\le 10^9$–10^{18}	O(log V), $O(\sqrt{V})$	Binary search on the answer, math, bit tricks

PART I · DECISION GUIDE

Pattern-selection decision guide

Each row maps a problem characteristic to an *initial hypothesis*. It is not an automatic rule. Always confirm with the structural test in the third column before committing to a pattern.

Problem characteristic	Hypothesis	Confirm by checking…	Common trap
Pair or triplet with a target, on sorted (or sortable) data	Two Pointers	Moving one pointer provably discards a whole row or column of candidates	Sorting destroys original indices (Two Sum I needs a hash map)
Contiguous subarray or substring, non-negative values or a count-type constraint	Sliding Window	Shrinking a valid or invalid window keeps it monotone	Negative numbers break monotonicity, so use a prefix sum
Contiguous subarray with sum = k, divisible by k, or negatives present	Prefix Sum + hash map	Condition is expressible as f(pre[j]) = g(pre[i])	Forgetting the empty prefix `pre = 0`
Many range-sum queries on static data	Prefix Sum	No updates between queries	Updates present → Fenwick or segment tree
Sorted array, boundary or first/last occurrence	Binary Search	A predicate that is false…false true…true over the indices	Inclusive and half-open bounds mixed together
"Minimise the maximum", "smallest capacity/speed such that…"	Binary Search on Answer	feasible(x) ⇒ feasible(x+1) (or the reverse), and a cheap feasibility check exists	A non-monotone predicate, or a wrong upper bound
Overlapping intervals, rooms, free time	Merge Intervals / Sweep	Sorting by start makes overlap a local check against the last interval	Closed vs half-open touching endpoints
Enumerate all subsets, permutations, placements	Backtracking	Output is exponential anyway; n is small	Counting only, with overlapping states, needs DP
Tree properties, connected components, path aggregates	DFS	Answer at a node combines answers from its children	Recursion depth on degenerate trees or large grids
Fewest steps, unweighted edges, grid spread	BFS	All edges cost the same	Weighted edges need Dijkstra; 0/1 weights need 0-1 BFS
Prerequisites, dependency order, build order	Topological Sort	Directed graph; a cycle means infeasible	Undirected cycle detection is a different algorithm

Problem characteristic	Hypothesis	Confirm by checking...	Common trap
Count ways or optimise, with overlapping subproblems	DP 1D / DP 2D	A state captures everything the future needs	State too small (missing a dimension)
Choose a maximum set of compatible intervals or jobs	Greedy	Exchange argument: an optimal solution can be transformed to include the greedy choice	Weighted versions need DP
Top-K, k-th largest, repeated min/max, streaming	Heap	Only the extreme element matters at each step	Comparator overflow via subtraction
Elements appear k times except some; ranges of integers; masks	Bit Manipulation	Per-bit independence, or an XOR cancellation	Sign bit and `>>` vs `>>>`
Dynamic connectivity, merging groups, cycle on edge insertion	Union-Find	Only unions, never splits	Needing the actual path, or deletions
Weighted shortest path, non-negative weights	Dijkstra	All weights ≥ 0 and the path cost is monotone along the path	An extra constraint (≤ K edges) needs a bigger state
Prefix lookup, many words against one structure	Trie	Shared prefixes are queried repeatedly	A single lookup is cheaper with a hash set
Range queries interleaved with updates	Segment / Fenwick Tree	The aggregate is associative (Fenwick needs an inverse for range queries)	Using Fenwick for range max/min with arbitrary updates
Next greater or smaller element, span, histogram	Monotonic Stack	Each element is resolved by the first larger/smaller element to its right or left	Strict vs non-strict comparisons with duplicates
Connect all nodes at minimum total cost	MST	Undirected, any tree is acceptable, cost is the sum of edges	Shortest-path tree ≠ MST
Maximum-sum contiguous subarray	Kadane	The best subarray ending at i extends the best one ending at i−1 or restarts	All-negative input with zero initialisation
Find a pattern in text, border or period of a string	KMP	Mismatches can reuse the longest proper border	Off-by-one in the failure links
Negative edge weights, or at most K edges	Bellman–Ford	V−1 relaxation rounds; round i ⇒ paths of ≤ i edges	Updating in place when an edge budget is in force

PART I · DISTINCTIONS

Similar patterns, different decisions

Most wrong choices come from confusing two neighbouring techniques. For each pair below, the deciding question is the property that separates them.

Pair	Deciding question	Choose the first when...	Choose the second when...
Sliding Window vs Prefix Sum	If I extend a window, can its validity flip back and forth?	Validity is monotone in the window (non-negative values, counts, "at most k distinct"); O(1) space beyond the window state	Values can be negative, or the condition is exact equality or divisibility: count via `pre[j] - pre[i]` with a hash map

Pair	Deciding question	Choose the first when…	Choose the second when…
Two Pointers vs Binary Search	Am I searching a pair or one boundary?	Pair/triplet over sorted data; pointer moves discard candidates in O(n) total	A single boundary in a monotone predicate; O(log n) per query, or as an inner step (fix one, search the other)
DFS vs Backtracking	Do I undo state after returning?	Visiting each node or state once (visited set persists): components, tree aggregates	Enumerating paths or choices; state is restored so the same node can be revisited along a different path
BFS vs Dijkstra	Are all edge weights equal?	Unit (or equal) weights: FIFO order is distance order, O(V+E)	Non-negative, different weights: needs a priority queue, O((V+E) log V). With 0/1 weights, 0-1 BFS (deque) sits in between
Greedy vs DP	Can I prove the local choice never needs revisiting?	An exchange argument or "stays ahead" proof exists (unweighted interval scheduling)	Choices interact (weighted intervals, knapsack, coin change with arbitrary denominations)
Binary Search (array) vs on Answer	Is the search space the input or the output?	Searching positions in a sorted or partially ordered array	Searching a value range (speed, capacity, largest sum) with a monotone feasibility check that costs O(n)
Segment Tree vs Fenwick Tree	Is the operation invertible, and are range updates or non-sum aggregates needed?	Segment tree: any associative operation (min, max, gcd, custom merges), lazy range updates	Fenwick: invertible operations (sum, xor, counts), less code and memory, prefix queries
Dijkstra vs Bellman–Ford	Are there negative edges, or a cap on the number of edges?	All weights ≥ 0, no edge-count limit: O((V+E) log V)	Negative edges, negative-cycle detection, or "at most K edges": O(V·E) or O(K·E)
Monotonic Stack vs Heap	Do I need the nearest qualifying element or the global extreme?	Nearest greater/smaller by position, with amortised O(1) per element	Repeated global min/max over a changing set, O(log n) per operation
DSU vs Graph Traversal	Is the graph static, and do I need paths?	Edges arrive incrementally; only "same component?" is asked; cycle detection on insertion	The graph is given up front and you need paths, distances, order, or any directed-graph reasoning

> **When the obvious pattern is wrong.** Be suspicious when (a) your sliding-window proof needs "values are non-negative" and the constraints say otherwise; (b) a greedy proof relies on an example rather than an exchange argument; (c) a "shortest path" has weights and you reached for BFS; (d) a DP state cannot answer "what does the future need to know?"; or (e) binary search on the answer has a predicate you cannot prove monotone.

PART I · QUICK REFERENCE

The 20 patterns at a glance

#	Pattern	Structural signal	Core invariant	Problems
1	Two Pointers	Sorted array; pair, triplet or container from both ends	Each move discards only impossible candidates	167 · 15 · 11 · 42
2	Sliding Window	Contiguous range; monotone validity	Window state equals the aggregate of [l, r]	643 · 3 · 1493 · 76
3	Fast & Slow	Linked structure or an iterated function x → f(x)	The gap grows by 1 per step inside a cycle	876 · 142 · 547 · 202
4	Prefix Sum	Range sums; exact sum or modular conditions	sum(i..j) = pre[j+1] − pre[i]	724 · 560 · 523 · 325
5	Binary Search	Sorted or monotone-predicate index space	Answer always lies within [lo, hi]	704 · 34 · 162 · 4
6	Binary Search on Answer	Min-max / max-min with a feasibility check	The feasible set is a suffix (or prefix)	1283 · 875 · 1011 · 410
7	Merge Intervals	Interval overlap, coverage, rooms	Sorted by start ⇒ overlap is checked against the last interval	56 · 57 · 253 · 759
8	Backtracking	Enumerate all valid configurations	State is restored after each branch	78 · 46 · 39 · 51
9	DFS	Trees, components, path aggregates	A node's return value summarises its subtree	104 · 200 · 547 · 124
10	BFS	Fewest steps on unit-weight edges	The queue holds at most two consecutive distance layers	733 · 994 · 542 · 127
11	Topological Sort	Dependencies in a directed graph	A node is emitted only after all its predecessors	210 · 207 · 802 · 269
12	DP 1D	Linear sequence of decisions	dp[i] depends only on a bounded set of earlier states	70 · 198 · 1416 · 494
13	DP 2D	Grid or two-sequence alignment	dp[i][j] is filled after its dependencies	62 · 64 · 1143 · 72
14	Greedy	Local choice provably safe	Exchange argument / greedy stays ahead	252 · 435 · 56 · 986
15	Heap	Top-K, repeated extremes, streams	The heap root is the current extreme	1046 · 215 · 347 · 295
16	Bit Manipulation	Parity, multiplicity, integer ranges	Bits are independent; x ^ x = 0	136 · 137 · 260 · 201
17	Union-Find	Incremental connectivity, grouping	Each component has exactly one root	1971 · 684 · 547 · 721
18	Dijkstra	Weighted shortest path, weights ≥ 0	A popped (non-stale) node's distance is final	1971 · 743 · 1631 · 787
19	Trie	Prefix queries across many words	Path from the root spells the prefix	208 · 211 · 648 · 212

#	Pattern	Structural signal	Core invariant	Problems
20	Segment / Fenwick	Range queries with updates	Node aggregate = merge of its children	307 · 315 · 2407 · 699

PART I · ALGORITHM INDEX

The 7 essential algorithms

#	Algorithm	Use when…	Complexity	Practice
A1	Monotonic Stack	Nearest greater/smaller, spans, contribution counting	O(n) amortised	739 · 907
A2	Dijkstra	Single-source shortest path, weights ≥ 0	O((V+E) log V)	1514 · 1976
A3	Union-Find	Incremental connectivity, equivalence classes	O(α(n)) amortised	1319 · 990
A4	Minimum Spanning Tree	Connect all vertices at minimum total edge weight	Kruskal O(E log E); Prim O(E log V) or $O(V^2)$	1584 · 1135
A5	Kadane	Maximum-sum contiguous subarray and variants	O(n), O(1)	53 · 918
A6	KMP	Exact pattern matching, borders, periods	O(n + m)	28 · 214
A7	Bellman–Ford	Negative weights, negative-cycle detection, ≤ K edges	O(V·E)	787

PART I · VALIDATION

Problem validation & classification notes

All 93 problem entries (80 assigned slots, of which 76 are distinct problems, plus 13 algorithm practice problems) were checked against LeetCode's public catalogue for frontend number, official title, difficulty and premium status. Premium problems: **325, 253, 759, 252, 269, 1135**.

The syllabus deliberately places some problems in chapters where they are not the most natural fit. They are kept where assigned, but the chapter teaches the technique that actually solves them and labels the relationship:

Problem	Assigned chapter	Primary technique	How it is handled
547	3 · Fast & Slow	Graph traversal / DSU	No pointer-speed structure exists. Solved with iterative BFS and an explicit classification note. Recursive DFS appears in Ch. 9 and DSU in Ch. 17, so each occurrence teaches something different.
202	3 · Fast & Slow	Fast & slow on an implicit sequence	Genuine fit: Floyd's cycle detection on n → digitSquareSum(n). A hash-set alternative is mentioned.
162	5 · Binary Search	Binary search on a local predicate	The array is not sorted. The search is driven by the slope predicate `a[m] < a[m+1]`.

Problem	Assigned chapter	Primary technique	How it is handled
4	5 · Binary Search	Binary search on a partition	Searches the cut position in the shorter array, not a value.
733	10 · BFS	Connected-component fill	No shortest-path requirement; BFS is used for traversal only.
494	12 · DP 1D	0/1 knapsack (2D state compressed)	The natural state is (index, sum). The 1D array is the compressed form, which is why the iteration order matters.
56	14 · Greedy	Sort + sweep / merge	Greedy in the "commit to the current group" sense, not interval selection. Ch. 14 uses a separate-endpoint sweep, a different technique from Ch. 7.
986	14 · Greedy	Two pointers over sorted lists	The greedy step is "advance the interval that ends first". Labelled as two pointers.
1971	17 · DSU	DSU or BFS/DFS	A natural DSU fit; solved with DSU.
1971	18 · Dijkstra	Unweighted reachability	Not a Dijkstra problem: edges are unweighted and only existence is asked. Solved with BFS, with an explanation of why Dijkstra is unnecessary.
1631	18 · Dijkstra	Minimax (bottleneck) Dijkstra	Path cost is the maximum edge, not the sum. Dijkstra still works because the cost is monotone along a path. Alternatives: binary search + BFS, or DSU.
787	18 · Dijkstra	Bounded-edge shortest path	Plain Dijkstra on vertices is incorrect. Solved with Dijkstra on an expanded state (node, edges used). A7 solves it again with Bellman–Ford.
Ch. 20 set	20 · Segment / Fenwick	Selected for this handbook	307 (both structures, point update + range sum), 315 (Fenwick + coordinate compression), 2407 (segment tree range max, which Fenwick cannot do with prefix differences), 699 (lazy range assignment + range max).

PART II · PATTERN 01

Two Pointers: Opposite Direction / Left–Right

Two indices start at opposite ends and move toward each other. Each move is justified by an ordering argument showing that every candidate it skips is impossible or dominated.

1.1 Pattern identity

Primary purpose	Search pairs (or, by fixing outer indices, triplets and k-tuples) over a linear order in O(n) per scan instead of $O(n^2)$.
Typical input	A sorted array, or one that may be sorted without losing information the answer needs. Also an array whose answer depends on a bound set by its two ends (container height, water level).
Interview applications	Two Sum on sorted data, 3Sum / 4Sum, closest-sum variants, container with most water, trapping rain water, palindrome checks, reversing in place, merging from the back.
Key insight	The candidate pairs form an n × n grid. Under a monotone order, comparing `f(l, r)` with the target eliminates a whole *row* (all pairs containing `l`) or a whole *column* (all pairs containing `r`) in O(1).

> **Key insight.** Two pointers is correct only when each move discards a provably useless set of candidates. If you cannot say "every pair containing the pointer I just moved is impossible or no better than one already seen", you do not have a valid two-pointer solution.

1.2 Recognition signals

Signal	Structural reason
Sorted input; find a pair or triplet meeting a sum, difference or closeness condition	Sum is monotone in each pointer: moving `l` right can only increase it, moving `r` left can only decrease it.
Answer is determined by the *smaller* of the two ends (container, water)	The limiting side caps every pair it belongs to, so after evaluating it once it can be discarded.
Output asks for values or a count, not original indices	Sorting is then free (O(n log n)) and does not lose required information.
n up to 10^4–10^5 with an $O(n^2)$ brute force over pairs	The budget suggests O(n) per fixed element or O(n log n) overall.
O(1) extra space is requested	Hash-based complement search uses O(n) space; pointers do not.

> **Ask yourself.** "If `f(l, r)` is too small (or too large, or computed), which pointer's every remaining pair is now useless?" If the answer is unambiguous, two pointers applies.

When it is inappropriate.

- **Original indices required on unsorted data** (LeetCode 1, Two Sum): sorting scrambles indices. Use a hash map, or sort index–value pairs if O(n log n) is acceptable.
- **No monotonicity**: if the pair score can rise or fall as either pointer moves, nothing can be eliminated.
- **Contiguous-range constraints**: when the answer is a *subarray*, the pointers move in the same direction. That is a sliding window (Pattern 2).
- **Counting pairs with exact sum and many duplicates** needs careful block counting; a frequency map is often simpler.

Confused with	How to tell them apart
Sliding window	Window pointers both move right and maintain a contiguous range. Opposite pointers move toward each other and score a *pair* of positions.
Binary search	Fixing one element and binary searching its complement costs O(n log n). Two pointers costs O(n) after sorting because both pointers move monotonically.
Hash-map complement	O(n) time, O(n) space, preserves indices and works unsorted. Two pointers needs order but uses O(1) space and extends naturally to k-sum with duplicate skipping.

1.3 Concepts and invariants

The elimination invariant. For a sorted array `a` and target `t`, maintain:

```
Invariant: if a valid pair (i, j) with i < j exists, then l ≤ i and j ≤ r.

If a[l] + a[r] < t:  for every j ≤ r,  a[l] + a[j] ≤ a[l] + a[r] < t  ⇒  no pair uses l
⇒  l++
If a[l] + a[r] > t:  for every i ≥ l,  a[i] + a[r] ≥ a[l] + a[r] > t  ⇒  no pair uses r
⇒  r--
```

Each step shrinks the search rectangle by one row or column, so the scan ends after at most n − 1 moves.

The dominance invariant (optimisation version). In Container With Most Water, area(l, r) = min(h[l], h[r]) · (r − l). Suppose h[l] ≤ h[r]. Then for every r' < r, the width shrinks and min(h[l], h[r']) ≤ h[l], so area(l, r') ≤ h[l] · (r' − l) < area(l, r). Pointer `l` has achieved its best pair, so discard it. The argument does not show the move is safe for the taller side, which is why you must move the shorter one.

Bounded-side invariant (Trapping Rain Water). Water above index i is `min(maxLeft(i), maxRight(i)) − h[i]`. Track `leftMax` over a[0..l] and `rightMax` over a[r..n−1]. If `leftMax < rightMax`, then for index l the true right maximum is at least `rightMax > leftMax`, so its water level is exactly `leftMax`, even though the rest of the right side has not been seen.

Duplicate handling in k-sum: why skipping is correct

After sorting, equal values are adjacent. Two triplets are duplicates if they have the same multiset of values. Enforce a canonical form: choose the first value from the *first* occurrence of each distinct value (`i > 0 && a[i] == a[i−1] → skip`). After recording a hit (l, r), advance both pointers past their runs of equal values. Any other (l', r') with a[l'] = a[l] would need a[r'] = a[r] to keep the same sum, which is the same triplet.

The skip happens *after* a match, not before evaluating. Skipping before evaluating could jump past the only valid pair such as (−1, −1, 2), where the two −1 values are different indices.

When sorting changes the problem. Sorting is safe when the answer depends only on the multiset of values (3Sum returns values). It is unsafe when it depends on positions: Container With Most Water uses indices as widths, so it must not be sorted, and its two-pointer validity comes from dominance, not order. Two Sum I returns indices, so sort index–value pairs or use a hash map.

1.4 Reusable Java templates

Template A — pair search with elimination. Reusable: the loop and the move rules. Problem-specific: `score` and what "found" means.

JAVA TEMPLATE TwoPointerPairSearch.java

```java
import java.util.*;

/** Opposite-direction pair search on a sorted array. Interval: [l, r] inclusive. */
class TwoPointerPairSearch {
    /** Indices {i, j}, i < j, with a[i] + a[j] == target, or null. a must be sorted. */
    static int[] findPair(int[] a, int target) {
        int l = 0, r = a.length - 1;
        while (l < r) {                       // pairs need two distinct indices
            long sum = (long) a[l] + a[r];    // long: no overflow near Integer.MAX_VALUE
            if (sum == target) return new int[]{l, r};
            if (sum < target) l++;            // pairs (l, j<=r) too small: drop row
            else r--;                         // pairs (i>=l, r) too large: drop col
        }
        return null;
    }

    /** Counts pairs i < j with a[i] + a[j] < target (a sorted), a block at a time. */
    static long countPairsBelow(int[] a, long target) {
        long count = 0;
        int l = 0, r = a.length - 1;
        while (l < r) {
            if ((long) a[l] + a[r] < target) { count += r - l; l++; } // l+1..r all ok
            else r--;
        }
        return count;
    }
}
```

Template B — fix one, two-pointer the rest (k-sum with deduplication).

JAVA TEMPLATE KSumTemplate.java

```java
import java.util.*;

/** 3Sum-style skeleton: sort, fix a[i], two-pointer the suffix, skip duplicate values. */
class KSumTemplate {
    static List<List<Integer>> threeSum(int[] a, int target) {
        Arrays.sort(a);
        List<List<Integer>> res = new ArrayList<>();
        int n = a.length;
        for (int i = 0; i < n - 2; i++) {
            if (i > 0 && a[i] == a[i - 1]) continue;          // canonical first value
            int l = i + 1, r = n - 1;
            while (l < r) {
                long s = (long) a[i] + a[l] + a[r];
                if (s == target) {
                    res.add(Arrays.asList(a[i], a[l], a[r]));
                    while (l < r && a[l] == a[l + 1]) l++;    // skip AFTER recording a hit
                    while (l < r && a[r] == a[r - 1]) r--;
                    l++;
```

```
                    r--;
                } else if (s < target) {
                    l++;
                } else {
                    r--;
                }
            }
        }
        return res;
    }
}
```

Template C — move the limiting side (dominance). Used when the score depends on `min(left, right)` of the two ends.

JAVA TEMPLATE LimitingSideTemplate.java

```java
/** Pair optimisation where the smaller end caps the score. No sorting: indices matter. */
class LimitingSideTemplate {
    static long bestPair(int[] h) {
        int l = 0, r = h.length - 1;
        long best = 0;
        while (l < r) {
            long score = (long) Math.min(h[l], h[r]) * (r - l);   // problem-specific score
            best = Math.max(best, score);
            // Shorter side has met its best partner; any narrower pair with it is worse.
            if (h[l] <= h[r]) l++;
            else r--;
        }
        return best;
    }
}
```

1.5 Complexity and pitfalls

Time	O(n) per scan; O(n log n) if you sort first; $O(n^{k-1})$ for k-sum with k ≥ 2 after sorting.
Auxiliary space	O(1) beyond the sort. Java's `Arrays.sort(int[])` (dual-pivot quicksort) uses O(log n) stack. The output list is not counted.
Off-by-one	Use `l < r` for pairs of distinct indices. `l <= r` pairs an element with itself.
Wrong invariant	Moving the taller side in container problems, or moving both pointers on a non-match.
Overflow	Sums of two or three ints can exceed $2^{31}-1$ (4Sum famously does). Accumulate in `long`.
Breaks naive code	All-equal arrays (duplicate explosion), all-negative arrays, arrays shorter than the tuple size, and the need for original indices.
Interview follow-ups	Closest sum (track |s − t|); count pairs instead of listing them; 4Sum / k-sum recursion; unsorted input with an O(1) space requirement; streaming input (two pointers no longer applies, so use a hash map).

1.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Every pointer move must eliminate a row or column of the pair grid; that is the whole proof.
- Sort only when the answer depends on values, not positions.
- For dominance problems, move the side that limits the score.
- Deduplicate by canonical choice: skip repeated outer values, and skip runs only after a match.

- Accumulate sums in `long` whenever three or more ints are added.

> **Recognition rule.** Pair over an ordered axis + a score that is monotone in each end → shrink from the side that can no longer help.

IN-INTERVIEW CHECKLIST

- ☐ Is the input sorted, or can I sort without losing what the output needs?
- ☐ Which pointer moves when the score is low, high, or computed, and why is that safe?
- ☐ How do I avoid duplicates (outer skip, post-match run skip)?
- ☐ Does the sum fit in an `int`?
- ☐ What happens with n < 2 (or n < 3)?

Key distinction. Opposite pointers score *pairs of endpoints*. A sliding window aggregates *everything between* them. If the elements between l and r matter, it is not this pattern.

1.7 Assigned problems

LC 167 Two Sum II - Input Array Is Sorted — MEDIUM

Task. Given a 1-indexed array sorted in non-decreasing order, return the 1-based indices of the unique pair summing to `target`, using O(1) extra space.

KEY INSIGHT

The sum is monotone in both pointers, so a mismatch identifies exactly one pointer that cannot belong to the answer.

WHY THE PATTERN APPLIES

The input is sorted, a pair is required and O(1) space is demanded. This is the textbook elimination invariant (Template A).

APPROACH

1. Start with `l = 0, r = n - 1`.
2. Too small → `l++`; too large → `r--`; equal → return `{l+1, r+1}`.

JAVA SOLUTION LeetCode 167 · Solution.java

```java
class Solution {
    public int[] twoSum(int[] numbers, int target) {
        int l = 0, r = numbers.length - 1;
        while (l < r) {
            int sum = numbers[l] + numbers[r];  // |values| <= 1000, so int is safe here
            if (sum == target) return new int[]{l + 1, r + 1};  // 1-indexed output
            if (sum < target) l++;
            else r--;
        }
        return new int[]{-1, -1};  // unreachable: exactly one solution is guaranteed
    }
}
```

Time O(n) **Space** O(1)

Pitfall. The output is 1-indexed. Do not "optimise" with binary search per element: that costs O(n log n) and adds no benefit.

LC 15 3Sum

MEDIUM

Task. Return all unique triplets of values (distinct indices) that sum to 0.

KEY INSIGHT

Fix the smallest element; the remaining pair is a sorted Two Sum on the suffix. Uniqueness comes from canonical choices on sorted data, not from a set of lists.

WHY THE PATTERN APPLIES

The output is values, so sorting is allowed. After sorting, each fixed `i` turns into an O(n) elimination scan: $O(n^2)$ total, versus $O(n^3)$ brute force.

APPROACH

1. Sort. For each i with a distinct value (skip `a[i] == a[i-1]`), set `l = i+1, r = n-1`.
2. Early exit: if `a[i] > 0`, no triplet starting here can sum to 0.
3. On a hit, record it, then move `l` and `r` past equal runs.

JAVA SOLUTION LeetCode 15 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        int n = nums.length;
        for (int i = 0; i < n - 2; i++) {
            if (nums[i] > 0) break;                          // smallest value > 0: done
            if (i > 0 && nums[i] == nums[i - 1]) continue;   // first value already handled
            int l = i + 1, r = n - 1;
            while (l < r) {
                int s = nums[i] + nums[l] + nums[r];         // |v| <= 1e5: fits in int
                if (s == 0) {
                    res.add(Arrays.asList(nums[i], nums[l], nums[r]));
                    while (l < r && nums[l] == nums[l + 1]) l++;
                    while (l < r && nums[r] == nums[r - 1]) r--;
                    l++;
                    r--;
                } else if (s < 0) {
                    l++;
                } else {
                    r--;
                }
            }
        }
        return res;
    }
}
```

Time $O(n^2)$ **Space** O(log n) sort stack (output excluded)

Pitfall. Deduplicating with a `HashSet<List<Integer>>` works but hides the invariant and costs extra time. The common bug is skipping duplicates of `l` *before* checking the sum, which loses `[-1,-1,2]`.

LC 11 Container With Most Water

MEDIUM

Task. Choose two lines `i < j` maximising `min(h[i], h[j]) · (j − i)`.

KEY INSIGHT

The shorter line has already met its best partner (the farthest one). Every narrower container using it is no larger, so it can be discarded.

WHY THE PATTERN APPLIES

This is a pair optimisation with a dominance argument (Template C). The array is *not* sorted and must not be, because widths come from indices.

APPROACH

1. Evaluate `(l, r)`, update the best.
2. Move the pointer at the shorter line. On a tie, either is safe: both lines are capped at the same height and any narrower pair is worse.

JAVA SOLUTION LeetCode 11 · Solution.java

```java
class Solution {
    public int maxArea(int[] height) {
        int l = 0, r = height.length - 1, best = 0;
        while (l < r) {
            int h = Math.min(height[l], height[r]);
            best = Math.max(best, h * (r - l));  // <= 1e4 * 1e5 = 1e9 < 2^31 - 1
            if (height[l] <= height[r]) l++;     // l is the limiting side
            else r--;
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

> **Pitfall.** Moving the taller line is the classic wrong invariant: it can only lower the cap or keep it, while the width shrinks. An optional speed-up is to skip lines no taller than the current cap, but the plain loop is already O(n).

LC 42 Trapping Rain Water

HARD

Task. Given bar heights, compute the total water trapped after rain.

KEY INSIGHT

`water[i] = min(maxLeft[i], maxRight[i]) − h[i]`. You do not need both maxima exactly, only to know which one is smaller. If `leftMax < rightMax`, the left index's level is settled at `leftMax`, because its true right maximum is at least `rightMax`.

WHY THE PATTERN APPLIES

This is the bounded-side invariant: process from whichever side has the smaller running maximum, because that side's water level is already determined. It replaces the O(n)-space prefix/suffix-maximum arrays.

APPROACH

1. Maintain `leftMax = max(h[0..l])` and `rightMax = max(h[r..n-1])`.

2. If `h[l] < h[r]`: then `leftMax ≤` the right-side maximum, so settle index l, add `leftMax − h[l]`, and `l++`. Otherwise settle r symmetrically.

JAVA SOLUTION LeetCode 42 · Solution.java

```java
class Solution {
    public int trap(int[] height) {
        int l = 0, r = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (l < r) {
            // Invariant: leftMax = max(height[0..l-1]), rightMax = max(height[r+1..]).
            if (height[l] < height[r]) {
                // Inductively leftMax <= max(height[r..]): l's water level is leftMax.
                leftMax = Math.max(leftMax, height[l]);
                water += leftMax - height[l];
                l++;
            } else {
                rightMax = Math.max(rightMax, height[r]);
                water += rightMax - height[r];
                r--;
            }
        }
        return water;
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Compare the *bars* `height[l]` and `height[r]` (or equivalently the running maxima) consistently; mixing the two comparisons breaks the invariant. Alternatives worth naming: prefix/suffix max arrays (O(n) space, easiest to prove) and a decreasing monotonic stack that adds water layer by layer (A1).

Sliding Window: Fixed and Variable Size

Two same-direction pointers bound a contiguous range [l, r]. The right pointer only ever expands and the left pointer only ever contracts, so every element enters and leaves the window at most once. The work is maintaining an O(1)-updatable summary of the window.

2.1 Pattern identity

Primary purpose	Optimise or test a property over all contiguous subarrays or substrings in O(n) instead of $O(n^2)$ by reusing the previous window's state.
Typical input	An array or string, plus either a fixed length k or a constraint on the window's contents (distinct count, character frequencies, sum bound, number of "bad" elements).
Interview applications	Longest substring without repeats, minimum window substring, longest run after k flips or deletions, max/average of every k-window, anagram and permutation search, at-most-k distinct.
Key insight	If a window that violates the constraint keeps violating it when extended to the right (or a valid window stays valid when shrunk), then for each `r` the best `l` never moves left. Both pointers are monotone, giving O(n) total.

> **Key insight.** A variable window is correct only when validity is *monotone under containment*: every sub-window of a valid window is valid (for "longest" problems), or every super-window of a valid window is valid (for "shortest" problems). Without that property, a greedy shrink can throw away the answer.

2.2 Recognition signals

Signal	Structural reason
Contiguous subarray or substring; "longest", "shortest", "maximum over windows of size k"	Contiguity means adjacent candidate ranges differ by one element at each end, so the state can be updated incrementally.
Constraint is a count: at most k distinct, at most k zeros, no repeated characters, contains all of t	Counts change by ±1 when one element enters or leaves, and the constraint is monotone under shrinking or growing.
Sum constraint with *non-negative* values	With non-negative values the sum is monotone in window length, so "sum ≥ S" persists when extending and "sum ≤ S" persists when shrinking.
Fixed k given explicitly	Fixed window: add `a[r]`, remove `a[r-k]`. No validity logic needed.
n up to 10^5, $O(n^2)$ enumeration of ranges is obvious	The budget asks for O(n) or O(n log n).

> **Ask yourself.** "If [l, r] is invalid, is [l, r+1] also invalid? If [l, r] is valid, is [l+1, r] also valid?" A yes to the one you need means each pointer can move monotonically.

When it is inappropriate.

- **Negative values with a sum condition.** In `[5, -10, 6]` with "shortest subarray with sum ≥ 6", the window expands to [5, −10, 6] with sum 1. It never becomes valid, so it never shrinks, and the valid answer [6] is missed. A super-window of a valid window can be invalid, so validity is not monotone and the greedy shrink is unjustified. Use prefix sums plus a hash map (exact sum), or a monotonic deque over prefix sums (LeetCode 862).
- **Exact-count constraints** ("exactly k distinct") are not monotone directly. Use `exactly(k) = atMost(k) − atMost(k−1)`.
- **Non-contiguous selections** (subsequences) belong to DP or greedy, not windows.
- **Window maximum or minimum** needs a monotonic deque. A plain counter cannot evict the maximum in O(1).

Confused with	How to tell them apart
Prefix sum + hash map	Exact target sums, divisibility, or negative values: prefix sums handle any value sign but need O(n) space. Windows need monotone validity but use only O(window-state) space.
Opposite two pointers	Opposite pointers score a pair of endpoints. Window pointers aggregate everything between them and move the same direction.
Monotonic deque	A deque *is* a sliding-window state when the summary is a max or min. Choose it when the aggregate is not invertible.

2.3 Concepts and invariants

Fixed window. The state equals the aggregate of exactly the last k elements:

```
S(r) = S(r−1) + a[r] − a[r−k]          for r ≥ k; window = [r−k+1, r]
```

The aggregate must be *invertible* (sum, count, xor, frequency map) to remove `a[r-k]` in O(1). Max and min are not invertible, which is why they need a deque.

Variable window, "longest valid". Invariant after processing r: `[l, r]` is the longest valid window ending at r. When adding `a[r]` breaks validity, shrink until valid. Correctness: if `[l, r]` is invalid then every `[l', r']` with `l' ≤ l, r' ≥ r` is invalid (it contains an invalid window), so no future answer starts at or before the discarded `l`.

Variable window, "shortest valid". Invariant: after shrinking, `[l-1, r]` was the last valid window. Expand r until valid, then shrink while still valid, recording the answer at each valid state. Correctness: once `[l, r]` is valid, any window `[l, r']` with `r' > r` is longer, so `l` never needs to pair with a larger r.

```
Window length          = r − l + 1      (inclusive [l, r])
Longest with ≤ k bad    : grow r; while bad > k: drop a[l++]; ans = max(ans, r − l + 1)
Shortest with condition : grow r; while valid:  ans = min(ans, r − l + 1); drop a[l++]
Exactly k               = atMost(k) − atMost(k − 1)    (counting problems)
```

Window state design. Keep a frequency array (`int[26]`, `int[128]`) when the alphabet is small, and a `HashMap` otherwise. Pair it with a single scalar that summarises validity: `distinct`, `bad`, or `missing`. Checking the scalar is O(1); re-scanning the frequency table per step is O(σ) and turns O(n) into O(nσ).

Amortised analysis: why the nested while loop is still O(n)

The inner `while` only increments `l`, and `l` never exceeds `r + 1 ≤ n`. Over the entire run `l` moves at most n times and `r` moves exactly n times, so total pointer movement is at most 2n. Each movement does O(1) state update, giving O(n) total regardless of how the inner loop's iterations are distributed.

The "jump" optimisation in Longest Substring Without Repeating Characters (`l = max(l, last[c] + 1)`) preserves the same bound and replaces the inner loop with O(1). It relies on `max`, because `last[c]` may point to a position already left of the window.

2.4 Reusable Java templates

Template A — fixed size k. Reusable: the add/remove rhythm. Problem-specific: what the state is and what is recorded.

JAVA TEMPLATE FixedWindow.java

```java
/** Fixed-size window over a[0..n-1]; window = [r-k+1, r]. Requires 1 <= k <= n. */
class FixedWindow {
    static long maxWindowSum(int[] a, int k) {
        long sum = 0;
        for (int i = 0; i < k; i++) sum += a[i];       // first full window [0, k-1]
        long best = sum;
        for (int r = k; r < a.length; r++) {
            sum += a[r] - a[r - k];                    // enter a[r], leave a[r-k]
            best = Math.max(best, sum);
        }
        return best;
    }
}
```

Template B — variable size, longest valid window (shrink while invalid).

JAVA TEMPLATE LongestValidWindow.java

```java
import java.util.*;

/** Longest subarray with at most k distinct values; validity is shrink-monotone. */
class LongestValidWindow {
    static int longestAtMostKDistinct(int[] a, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        int l = 0, best = 0;
        for (int r = 0; r < a.length; r++) {
            freq.merge(a[r], 1, Integer::sum);                // expand
            // invalid: shrink from the left
            while (freq.size() > k) {
                if (freq.merge(a[l], -1, Integer::sum) == 0) freq.remove(a[l]);
                l++;
            }
            best = Math.max(best, r - l + 1);                 // [l, r] valid and maximal
        }
        return best;
    }

    /** Exactly-k counting via the at-most difference. */
    static long countExactlyKDistinct(int[] a, int k) {
        return countAtMost(a, k) - countAtMost(a, k - 1);
    }

    private static long countAtMost(int[] a, int k) {
        if (k < 0) return 0;
        Map<Integer, Integer> freq = new HashMap<>();
```

```
        long count = 0;
        int l = 0;
        for (int r = 0; r < a.length; r++) {
            freq.merge(a[r], 1, Integer::sum);
            while (freq.size() > k) {
                if (freq.merge(a[l], -1, Integer::sum) == 0) freq.remove(a[l]);
                l++;
            }
            count += r - l + 1;              // every window [l'..r], l <= l' <= r, is valid
        }
        return count;
    }
}
```

Template C — variable size, shortest valid window (shrink while valid). Uses the "need/missing" counter idiom.

JAVA TEMPLATE ShortestValidWindow.java

```
/** Shortest subarray of NON-NEGATIVE values with sum >= target, else 0 (LC 209). */
class ShortestValidWindow {
    static int minLengthAtLeast(int[] a, long target) {
        long sum = 0;
        int l = 0, best = Integer.MAX_VALUE;
        for (int r = 0; r < a.length; r++) {
            sum += a[r];                                  // expand until valid
            // valid: record, then try smaller
            while (sum >= target) {
                best = Math.min(best, r - l + 1);
                sum -= a[l++];
            }
        }
        return best == Integer.MAX_VALUE ? 0 : best;
    }
}
```

> **Assumption.** Template C is correct only because all values are non-negative, so shrinking never makes an invalid window valid again. With negative values, use prefix sums with a monotonic deque instead.

2.5 Complexity and pitfalls

Time	O(n) pointer movement (amortised 2n). Add O(σ) if the state is compared table-to-table instead of through a counter scalar.
Auxiliary space	O(σ) for a frequency array over the alphabet, or O(min(n, distinct)) for a hash map; O(1) for scalar sums.
Off-by-one	Length is `r - l + 1` for inclusive windows. In a fixed window the outgoing index is `r - k`, not `r - k + 1`. Record the answer *after* shrinking (longest) or *inside* the shrink loop (shortest).
Incorrect invariant	Using `if` instead of `while` to shrink is only safe when a single step always restores validity (e.g. a window that grows by at most one per step and never shrinks: the "non-shrinking window" trick).
Overflow	Window sums of up to 10^5 ints of size 10^9 overflow `int`; use `long`.
Breaks naive code	Negative numbers; k > n; empty target; a frequency map that keeps zero-count keys (so `size()` overstates distinct); `char - 'a'` on input that also has uppercase letters.
Interview follow-ups	Exactly k (use the atMost difference); window max (deque); streaming input (the window is already online); returning the window instead of its length (store `bestL`).

2.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Fixed window: add one, remove one, with an invertible aggregate.
- Variable window: prove validity is monotone under containment before writing the shrink loop.
- Longest → shrink while invalid, record after. Shortest → shrink while valid, record inside.
- Summarise the window with a single scalar (missing, distinct, bad) so validity checks are O(1).
- Negative values usually mean prefix sums, not windows.

> **Recognition rule.** Contiguous range + a constraint that only gets worse as the window grows (or only better) → expand right, shrink left, never move back.

IN-INTERVIEW CHECKLIST

- ☐ Fixed or variable? Longest or shortest?
- ☐ What is the window state, and how is it updated on enter and leave?
- ☐ Is validity monotone? Could negative values or "exactly" break it?
- ☐ Where exactly is the answer recorded relative to the shrink loop?
- ☐ Do I need the length or the actual window (track the start index)?

Key distinction. Use a sliding window when validity is monotone in window size. Use prefix sums plus a hash map when the condition is an exact equality or the values can be negative.

2.7 Assigned problems

LC 643 Maximum Average Subarray I — EASY

Task. Return the maximum average of any contiguous subarray of length exactly k.

KEY INSIGHT

Because k is fixed, maximising the average is the same as maximising the sum. Divide once at the end.

WHY THE PATTERN APPLIES

This is a fixed window with an invertible aggregate (sum): Template A directly.

APPROACH

1. Sum the first k elements.
2. Slide: add `nums[r]`, subtract `nums[r-k]`, track the maximum sum.
3. Return `best / (double) k`.

JAVA SOLUTION LeetCode 643 · Solution.java

```java
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        long sum = 0;                                   // |sum| <= 1e5 * 1e4: int also fits
        for (int i = 0; i < k; i++) sum += nums[i];
        long best = sum;
        for (int r = k; r < nums.length; r++) {
            sum += nums[r] - nums[r - k];
            best = Math.max(best, sum);
```

```
        }
        return (double) best / k;                    // divide once, avoid rounding drift
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Initialising `best = 0` fails when every window sum is negative. Initialise from the first window. Also avoid integer division: cast before dividing.

LC 3 Longest Substring Without Repeating Characters

MEDIUM

Task. Return the length of the longest substring with all distinct characters. Characters are English letters, digits, symbols and spaces (ASCII).

KEY INSIGHT

"No repeats" is monotone under shrinking: a sub-window of a repeat-free window is repeat-free. When `s[r]` repeats, the window must start just after that character's previous occurrence, so `l` can jump there directly.

WHY THE PATTERN APPLIES

A variable "longest valid" window (Template B). Storing the last index replaces the shrink loop with an O(1) jump.

APPROACH

1. `last[c]` = most recent index of c, or −1.
2. For each r: `l = max(l, last[s[r]] + 1)`; update `last`; `best = max(best, r − l + 1)`.

JAVA SOLUTION LeetCode 3 · Solution.java

```
import java.util.*;

class Solution {
    public int lengthOfLongestSubstring(String s) {
        int[] last = new int[128];                   // ASCII alphabet per constraints
        Arrays.fill(last, -1);
        int l = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            // max: last[c] may be left of window
            l = Math.max(l, last[c] + 1);
            last[c] = r;
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

Time O(n) **Space** O(σ), σ = 128

Pitfall. Writing `l = last[c] + 1` without `max` moves the left pointer backwards on input like `"abba"`: at the final `a`, last[a] = 0 is already outside the window [2, 2]. Use `int[128]`, not `int[26]`: spaces and digits are allowed.

LC 1493 Longest Subarray of 1's After Deleting One Element

MEDIUM

Task. In a binary array you *must* delete exactly one element. Return the length of the longest non-empty run of 1s that can result (0 if none).

KEY INSIGHT

Reframe as the longest window containing at most one zero. Deleting one element from that window (the zero, or any element if there is none) leaves all 1s, so the answer is the window length − 1.

WHY THE PATTERN APPLIES

"At most k zeros" is a count constraint that is monotone under shrinking (Template B with k = 1).

APPROACH

1. Expand r, counting zeros.
2. While zeros > 1, shrink from l.
3. Answer = max over r of `(r - l + 1) - 1 = r - l`.

JAVA SOLUTION LeetCode 1493 · Solution.java

```java
class Solution {
    public int longestSubarray(int[] nums) {
        int l = 0, zeros = 0, best = 0;
        for (int r = 0; r < nums.length; r++) {
            if (nums[r] == 0) zeros++;
            while (zeros > 1) {                          // window may hold at most one zero
                if (nums[l] == 0) zeros--;
                l++;
            }
            best = Math.max(best, r - l);                // window length - 1 deleted element
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

> **Pitfall.** The deletion is mandatory. For an all-ones array of length n the answer is n − 1, not n. That comes out automatically from `r - l`; returning `r - l + 1` when there is no zero is wrong.

LC 76 Minimum Window Substring

HARD

Task. Return the shortest substring of `s` that contains every character of `t` with multiplicity, or `""`. Both strings use upper- and lowercase English letters.

KEY INSIGHT

"Covers t" is monotone under extension: a super-window of a covering window still covers. That makes it a shortest-valid window. A single `missing` counter (characters of t not yet covered) makes the validity check O(1).

WHY THE PATTERN APPLIES

Template C with a frequency-difference state. `need[c]` starts as the count in t. It goes down when c enters the window and can become negative, which means surplus copies.

APPROACH

1. For each r: if `need[s[r]] > 0` before decrementing, a required copy was satisfied, so `missing--`. Then decrement.
2. While `missing == 0`: record `[l, r]` if shorter; then remove s[l]: increment `need`, and if it becomes positive, `missing++`; `l++`.

JAVA SOLUTION LeetCode 76 · Solution.java

```java
class Solution {
    public String minWindow(String s, String t) {
        if (t.length() > s.length()) return "";
        // need[c] > 0: still required copies
        int[] need = new int[128];
        for (int i = 0; i < t.length(); i++) need[t.charAt(i)]++;
        // required characters not yet covered
        int missing = t.length();
        int l = 0, bestL = 0, bestLen = Integer.MAX_VALUE;
        for (int r = 0; r < s.length(); r++) {
            if (need[s.charAt(r)]-- > 0) missing--;     // this copy was actually needed
            // [l, r] covers t: shrink while valid
            while (missing == 0) {
                if (r - l + 1 < bestLen) {
                    bestLen = r - l + 1;
                    bestL = l;
                }
                if (++need[s.charAt(l)] > 0) missing++; // removed a required copy
                l++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestL, bestL + bestLen);
    }
}
```

Time O(|s| + |t|) **Space** O(σ), σ = 128

Pitfall. Comparing two frequency tables on every step makes it O(σ·|s|) and is easy to get wrong with duplicates in t. Characters not in t start at `need = 0` and go negative; they must never touch `missing`, which the "> 0 before decrement / > 0 after increment" tests guarantee. Store `bestL` and the length, and call `substring` once at the end.

PART II · PATTERN 03

Fast and Slow Pointers: Cycle Detection / Linked Lists

Two pointers traverse the same sequence at different speeds. The speed difference turns positional questions (where is the middle, does it loop, where does the loop start) into arithmetic on distances, using O(1) memory.

3.1 Pattern identity

Primary purpose	Detect cycles, find a cycle's entry, or locate fractional positions (middle, k-th from end) in a sequence that can only be walked forward, without extra memory.
Typical input	A singly linked list, or an *implicit* functional sequence x_0, $f(x_0)$, $f(f(x_0))$, … where every state has exactly one successor (array-as-pointer problems, digit transforms).
Interview applications	Linked list cycle / cycle entry, middle node, palindrome linked list (middle + reverse), reorder list, Happy Number, Find the Duplicate Number (LeetCode 287).
Key insight	Inside a cycle of length L the fast pointer gains one step per iteration on the slow one, so it must land on it within L iterations. The meeting point's distance arithmetic then locates the cycle entry.

> **Key insight.** Fast/slow works on any structure where each state has *exactly one* next state (a functional graph). Its paths look like the Greek letter ρ: a tail leading into one cycle. It does not apply to general graphs, where a node can have several neighbours.

> **Classification note for this chapter.** Of the four assigned problems, 876 and 142 are genuine linked-list fast/slow problems and 202 is fast/slow on an implicit sequence. **547 (Number of Provinces) is not a fast/slow problem**: it is connected components in an undirected graph, where nodes have many neighbours and there is no "next" pointer. It is solved below with iterative BFS, the honest technique, and revisited with DFS in Ch. 9 and DSU in Ch. 17.

3.2 Recognition signals

Signal	Structural reason
Linked list with possible cycle; "return where the cycle begins"	Each node has one `next`, so the structure is a ρ shape and Floyd's arithmetic applies.
Middle, or "split into halves", of a list of unknown length	A 2:1 speed ratio places slow at n/2 when fast reaches the end, in a single pass.
Repeatedly apply a deterministic function; "does it end at X or loop forever?"	The sequence lives in a finite (or eventually bounded) state space, so it must become periodic.
"O(1) extra space", "do not modify the list/array"	Rules out a visited `HashSet` and in-place marking.
Array of n+1 values in [1, n] treated as pointers	`i → a[i]` is a function, and a duplicate value is a cycle entry (LeetCode 287).

> **Ask yourself.** "Does every state have exactly one successor, and am I asked about position, looping, or the loop's start?" If a state can have two successors, use graph traversal instead.

When it is inappropriate.

- **General graphs** (adjacency lists/matrices): use DFS colouring (directed) or a parent check, DSU or BFS (undirected).
- **When O(n) memory is fine and clarity matters**: a `HashSet` of visited nodes is simpler and equally fast. Fast/slow earns its place through O(1) space.
- **Random access is available** (arrays): the middle is just `n / 2`.

Confused with	How to tell them apart
Hash-set visited	Same O(n) time. The hash set uses O(n) space and finds the entry trivially. Floyd uses O(1) space but needs the second phase to find the entry.
Two pointers with a gap	"k-th from the end" uses equal speeds with a k-step head start, which is a fixed offset rather than a speed ratio.
Directed-graph cycle detection	Multiple out-edges per node need DFS three-colour or Kahn's algorithm (Ch. 11).

3.3 Concepts and invariants

Convergence. Once both pointers are in the cycle (length L), let the gap be d = (fast − slow) mod L. Each iteration fast moves 2 and slow moves 1, so the gap changes by exactly one each step. It therefore passes through 0 within L steps and the two meet, with no risk of fast "jumping over" slow. Detection costs O(μ + L), where μ is the tail length.

Entry point derivation. Let μ = distance from head to entry, and suppose they meet k steps into the cycle.

```
slow travelled  s = μ + k  (+ whole loops, absorbed below)
fast travelled 2s = μ + k + m·L            for some integer m ≥ 1
⇒  s = m·L     ⇒   μ + k = m·L     ⇒   μ = m·L − k = (m − 1)·L + (L − k)
```

From the meeting point, walking μ steps lands on `k + μ ≡ 0 (mod L)`, which is the entry. So reset one pointer to the head and move both one step at a time; they meet exactly at the entry after μ steps.

Midpoint conventions. With `slow = fast = head` and the loop guard `fast != null && fast.next != null`, slow ends at index ⌊n/2⌋ (0-based): the *second* middle for even n. With the guard `fast.next != null && fast.next.next != null`, slow ends at ⌊(n−1)/2⌋: the *first* middle, which is what you need to split a list before reversing its second half.

Termination for implicit sequences (why Happy Number cannot run forever)

Floyd needs the sequence to be eventually periodic. For the digit-square map f, a d-digit number $n \geq 10^{d-1}$ maps to at most 81d. For $d \geq 4$, $81d < 10^{d-1}$, so $f(n) < n$ whenever $n \geq 1000$. The sequence therefore enters [1, 999] and stays inside a finite set (f(999) = 243), so by the pigeonhole principle it must repeat. 1 is a fixed point (f(1) = 1), so "happy" means the cycle reached is {1}.

3.4 Reusable Java templates

Template A — middle node (both conventions).

JAVA TEMPLATE ListMiddle.java (uses LeetCode ListNode)

```java
/** Middle of a singly linked list in one pass. ListNode is the LeetCode definition. */
class ListMiddle {
    /** Second middle for even length: index n/2. */
    static ListNode secondMiddle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {   // fast moves 2, slow moves 1
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    /** First middle for even length: index (n-1)/2. Use it to split before reversing. */
    static ListNode firstMiddle(ListNode head) {
        if (head == null) return null;
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

Template B — Floyd on any functional sequence. The same two phases work for list nodes, array indices (`i -> a[i]`) or numbers (`x -> f(x)`). Only `next` changes.

JAVA TEMPLATE FloydCycle.java

```java
import java.util.function.IntUnaryOperator;

/** Floyd's cycle detection on x0, f(x0), f(f(x0)), ... over a finite state space. */
class FloydCycle {
    /** Returns the first state on the cycle (the entry); requires a finite state space. */
    static int cycleEntry(int x0, IntUnaryOperator f) {
        int slow = f.applyAsInt(x0), fast = f.applyAsInt(f.applyAsInt(x0));
        while (slow != fast) {                          // phase 1: meet inside the cycle
            slow = f.applyAsInt(slow);
            fast = f.applyAsInt(f.applyAsInt(fast));
        }
        slow = x0;                                      // phase 2: mu steps from both ends
        while (slow != fast) {
            slow = f.applyAsInt(slow);
            fast = f.applyAsInt(fast);
        }
        return slow;
    }

    /** Cycle length: walk once around from any state known to be on the cycle. */
    static int cycleLength(int onCycle, IntUnaryOperator f) {
        int len = 1;
        for (int x = f.applyAsInt(onCycle); x != onCycle; x = f.applyAsInt(x)) len++;
        return len;
    }
}
```

3.5 Complexity and pitfalls

Time	O(μ + L) for detection and entry, O(n) for middle. Each pointer advances O(n) times.
Auxiliary space	O(1). The hash-set alternative is O(n).
Termination conditions	Check `fast != null && fast.next != null` before `fast.next.next`. The order of the two checks matters.
Incorrect invariant	Starting phase 1 with `slow = head, fast = head.next` shifts the meeting point, so the phase 2 "reset to head" no longer lands on the entry. Start both at head (or both one/two steps from x_0, as in Template B).
Breaks naive code	Empty list, single node, a self-loop (`head.next == head`), a cycle that starts at the head (μ = 0).
Interview follow-ups	Cycle length; Find the Duplicate Number via `i → nums[i]`; palindrome list in O(1) space (first middle + reverse + compare + restore); whether speed 3:1 also works (the gap changes by 2 per step, so with an even L and an odd initial gap the pointers never coincide; 2:1 always meets).

3.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- The pattern needs a functional structure: one successor per state.
- The 2:1 speed makes the gap shrink by exactly one per step, so the pointers must meet.
- The entry is μ steps from both the head and the meeting point, because μ ≡ L − k (mod L).
- The loop guard selects which middle you get.
- Implicit sequences need a termination argument (a bounded state space).

> **Recognition rule.** One-successor sequence + a question about looping or relative position + O(1) memory → tortoise and hare.

IN-INTERVIEW CHECKLIST

- ☐ Is this really a functional sequence, or a general graph?
- ☐ Null-safe guards on `fast` and `fast.next`?
- ☐ Do both pointers start at the same place for phase 2 to be valid?
- ☐ Which middle (first or second) does the follow-up need?
- ☐ Have I handled μ = 0 and a self-loop?

Key distinction. Floyd detects a cycle in a single forward chain. Cycle detection in a general directed graph needs DFS colouring or Kahn's algorithm (Ch. 11), and an undirected graph needs DSU (Ch. 17).

3.7 Assigned problems

LC 876 Middle of the Linked List — EASY

Task. Return the middle node of a non-empty list; for even length, the second of the two middles.

KEY INSIGHT

When fast has taken 2t steps and runs out, slow has taken t steps, so slow is at index ⌊n/2⌋.

WHY THE PATTERN APPLIES

The list length is unknown and a single pass is wanted. This is Template A with the second-middle guard.

APPROACH

1. Advance slow by 1 and fast by 2 while `fast` and `fast.next` are non-null.
2. Return slow.

JAVA SOLUTION LeetCode 876 · Solution.java

```java
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // index n/2: second middle if n even
        return slow;
    }
}
```

Time O(n) **Space** O(1)

> **Pitfall.** The guard `fast.next != null && fast.next.next != null` returns the *first* middle, which is wrong here for even lengths (e.g. [1..6] must return 4).

LC 142 Linked List Cycle II

MEDIUM

Task. Return the node where the cycle begins, or `null` if the list is acyclic. Do not modify the list. The follow-up asks for O(1) memory.

KEY INSIGHT

From the derivation μ = (m − 1)L + (L − k), a pointer restarted at the head and a pointer left at the meeting point reach the entry at the same time.

WHY THE PATTERN APPLIES

This is a pure functional chain where the follow-up demands O(1) memory: Floyd's two phases.

APPROACH

1. Phase 1: 2:1 walk. If fast hits null, return null.
2. Phase 2: `p = head`, then advance `p` and `slow` one step each until equal. Return that node.

JAVA SOLUTION LeetCode 142 · Solution.java

```java
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            // inside the cycle, k steps past entry
            if (slow == fast) {
                ListNode p = head;
                while (p != slow) {                    // both are mu steps from the entry
                    p = p.next;
                    slow = slow.next;
                }
                return p;
```

```
            }
        }
        return null;                                    // fast fell off: no cycle
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Compare node references (`==`), never values: values can repeat. Checking `slow == fast` before the first move would succeed immediately at the head, so check after moving.

LC 547 Number of Provinces

MEDIUM

Task. Given an n × n adjacency matrix `isConnected` (undirected, symmetric, diagonal = 1), return the number of connected components.

Classification note. Fast/slow pointers does not apply. A city can be adjacent to many cities, so there is no single "next" pointer and no ρ-shaped path, and "cycles" are irrelevant to counting components. The primary technique is **graph traversal** (BFS/DFS) or **Union-Find**. This chapter uses iterative BFS; see Ch. 9 (recursive DFS) and Ch. 17 (DSU).

KEY INSIGHT

Each traversal started from an unvisited city marks exactly one whole component, so the number of traversal starts equals the number of provinces.

WHY THIS TECHNIQUE APPLIES

The graph is undirected and given up front, and only component membership matters. BFS with a `visited` array visits each vertex once. With an adjacency matrix, scanning neighbours costs O(n) per vertex.

APPROACH

1. For each unvisited city `s`: `provinces++`, mark it, push it to a queue.
2. Pop `u`; for every `v` with `isConnected[u][v] == 1` and not visited, mark it *when enqueued* and push.

JAVA SOLUTION LeetCode 547 · Solution.java (iterative BFS)

```
import java.util.*;

class Solution {
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, provinces = 0;
        boolean[] visited = new boolean[n];
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int s = 0; s < n; s++) {
            if (visited[s]) continue;
            provinces++;                                // s starts a new component
            visited[s] = true;
            queue.add(s);
            while (!queue.isEmpty()) {
                int u = queue.poll();
                for (int v = 0; v < n; v++) {
                    if (isConnected[u][v] == 1 && !visited[v]) {
                        visited[v] = true;              // mark on enqueue: no duplicates
                        queue.add(v);
                    }
                }
            }
```

```
        }
        return provinces;
    }
}
```

Time O(n^2) (the matrix is read once per row) **Space** O(n) visited + queue

Pitfall. Marking on dequeue instead of on enqueue lets a vertex enter the queue several times (still correct here, but wasteful). The iterative form avoids recursion depth issues that recursive DFS can hit on large inputs, though n ≤ 200 here.

LC 202 Happy Number

EASY

Task. Repeatedly replace n by the sum of the squares of its digits. Return true if the process reaches 1.

KEY INSIGHT

n → f(n) is a functional sequence in a bounded state space (see the termination note in 3.3), so it is eventually periodic. 1 is a fixed point, so n is happy exactly when the cycle Floyd finds is {1}.

WHY THE PATTERN APPLIES

This is fast/slow on an implicit sequence (Template B). No list exists, but every state has exactly one successor.

APPROACH

1. slow = f(n), fast = f(f(n)); advance 1 and 2 steps until they meet.
2. Return `slow == 1`.

JAVA SOLUTION LeetCode 202 · Solution.java

```
class Solution {
    public boolean isHappy(int n) {
        int slow = next(n), fast = next(next(n));
        while (slow != fast) {
            slow = next(slow);
            fast = next(next(fast));
        }
        // the cycle reached is {1} iff happy
        return slow == 1;
    }

    private int next(int x) {
        int sum = 0;
        while (x > 0) {
            int d = x % 10;
            sum += d * d;                                   // <= 81 * 10 digits: no overflow
            x /= 10;
        }
        return sum;
    }
}
```

Time O(log n) per step; O(log n) steps before reaching the bounded range, then O(1) (states < 1000) **Space** O(1)

Pitfall. A `HashSet<Integer>` of seen values is an equally valid and simpler alternative (O(1) extra in practice, since the states are bounded). Mention it, then justify Floyd as the constant-memory version. Without the boundedness argument, "loop until 1" has no termination proof.

PART II · PATTERN 04

Prefix Sum / Cumulative Sum

Precompute cumulative aggregates so that any range aggregate is a difference of two prefixes. Combined with a hash map, the difference becomes an equation: "which earlier prefix makes this range satisfy the condition?"

4.1 Pattern identity

Primary purpose	Answer range-sum queries in O(1) after O(n) preprocessing, and count or optimise subarrays with an exact sum or modular condition in O(n).
Typical input	An integer array (possibly with negatives), a target k or modulus, or many offline range queries on static data. Also 2D grids (submatrix sums) and difference arrays for batch range updates.
Interview applications	Subarray sum equals k, continuous subarray sum (multiple of k), longest subarray with sum k, equal 0/1 counts (map 0 → −1), pivot/equilibrium index, product except self (prefix and suffix products), range-sum query, difference arrays.
Key insight	`sum(i..j) = P[j+1] − P[i]`, so a condition on a subarray becomes a condition on a *pair of prefixes*. Scanning j and looking up the required earlier prefix in a hash map turns $O(n^2)$ pair enumeration into O(n).

> **Key insight.** Prefix sums make no monotonicity assumption, which is exactly why they work with negative values where a sliding window fails. The price is O(n) memory for the map of earlier prefixes.

4.2 Recognition signals

Signal	Structural reason
"Number of subarrays with sum exactly k", values may be negative	Exact equality is not monotone in window length, but `P[j] − P[i] = k ⇔ P[i] = P[j] − k` is a lookup.
"Sum is a multiple of k" / "divisible by k"	`P[j] − P[i] ≡ 0 (mod k) ⇔ P[i] ≡ P[j] (mod k)`: group prefixes by remainder.
Many range queries, no updates	O(n) build, O(1) per query. With updates, use Fenwick or segment trees (Ch. 20).
Balance conditions: equal numbers of 0/1, vowels/consonants, left sum = right sum	Map one class to +1 and the other to −1. "Balanced" becomes "range sum 0", i.e. equal prefixes.
"Longest" or "shortest" subarray with an exact sum	For each j, the best i is the earliest (longest) or latest (shortest) index with the required prefix, so store first or last occurrence.

> **Ask yourself.** "Can the condition on sum(i..j) be rewritten as P[i] = g(P[j]) for a function g I can compute?" If yes, one pass with a map of earlier prefixes solves it.

When it is inappropriate.

- **Inequalities** ("sum ≥ k", "sum ≤ k"): a hash map answers equality only. With non-negative values use a sliding window. With negatives, use a monotonic deque over prefixes or a sorted structure / BIT over prefix values.
- **Range updates interleaved with queries**: a static prefix array goes stale, so use a Fenwick or segment tree.
- **Non-invertible aggregates** (max, min): `max(i..j)` is not a difference of prefixes. Use a sparse table, segment tree or deque.

Confused with	How to tell them apart
Sliding window	A window needs monotone validity (non-negative values, at-most constraints) and uses O(1)–O(σ) space. Prefix + map handles any signs with exact conditions in O(n) space.
Fenwick tree	A Fenwick tree is a prefix-sum array that supports point updates in O(log n). Prefix arrays are the static special case.
Kadane	Maximum subarray sum is `max over j of (P[j] − min over i < j of P[i])`, a running-minimum prefix argument. Kadane is the same thing in O(1) space (A5).

4.3 Concepts and invariants

Indexing convention. Use a length-(n+1) array with an empty prefix:

```
P[0] = 0,   P[i+1] = P[i] + a[i]            P[i] = sum of a[0..i−1]
sum(a[l..r]) = P[r+1] − P[l]                inclusive l..r, 0 ≤ l ≤ r < n
2D:  S[i+1][j+1] = S[i][j+1] + S[i+1][j] − S[i][j] + g[i][j]
     sum(r1..r2, c1..c2) = S[r2+1][c2+1] − S[r1][c2+1] − S[r2+1][c1] + S[r1][c1]
```

The extra slot `P[0] = 0` represents the empty prefix. It is what allows subarrays starting at index 0 to be counted, and it is the reason hash-map solutions seed the map with `{0: 1}` (counting) or `{0: −1}` (index-based).

Hash-map invariant. Process j = 0..n−1 with running prefix `p = P[j+1]`. Invariant: before handling j, the map contains exactly the prefixes P[0..j], which are the starts of subarrays ending at j. Therefore:

1. **Query first**: count += map[p − k] counts subarrays (i..j) with i ≤ j.
2. **Then insert** p, so it can serve as a start for *later* ends.

Inserting before querying makes `P[j+1]` available as its own start, which pairs it with itself and counts the empty subarray whenever k = 0. That is the classic wrong-order bug.

First vs last occurrence. For the longest subarray, store a prefix's *first* index and never overwrite it (use `putIfAbsent`). For the shortest, keep the *latest* index (overwrite). For counting, store frequencies.

Prefix remainders and Java's negative modulo

`(P[j] − P[i]) mod k = 0 ⇔ P[i] mod k = P[j] mod k`, so group prefixes by remainder. Java's `%` follows the sign of the dividend: `−7 % 3 == −1`. With negative values, normalise with `((x % k) + k) % k`, or −1 and 2 land in different buckets despite being congruent. Also keep the running remainder in `long` when `k` can approach `Integer.MAX_VALUE`, because `rem + a[i]` can exceed $2^{31} − 1$.

The number of pairs inside a remainder bucket of size c is c·(c−1)/2. Counting problems such as LeetCode 974 (Subarray Sums Divisible by K) either add the running bucket count during the scan or use this formula at the end.

Difference arrays (the inverse operation). To add v to every element of a[l..r] for many updates, then read the array once: `D[l] += v; D[r+1] -= v`, and the final array is the prefix sum of D. This gives O(1) per update and O(n) to materialise.

4.4 Reusable Java templates

Template A — static range sums (1D and 2D).

JAVA TEMPLATE PrefixSums.java

```java
/** Static prefix sums. Queries are inclusive [l, r]; arrays carry an empty prefix. */
class PrefixSums {
    private final long[] p;                              // p[i] = a[0] + ... + a[i-1]

    PrefixSums(int[] a) {
        p = new long[a.length + 1];
        for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    }

    long sum(int l, int r) { return p[r + 1] - p[l]; }   // 0 <= l <= r < n

    /** 2D: s[i][j] = sum of g[0..i-1][0..j-1]. */
    static long[][] build2D(int[][] g) {
        int m = g.length, n = g[0].length;
        long[][] s = new long[m + 1][n + 1];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                s[i + 1][j + 1] = s[i][j + 1] + s[i + 1][j] - s[i][j] + g[i][j];
        return s;
    }

    static long sum2D(long[][] s, int r1, int c1, int r2, int c2) {   // inclusive corners
        return s[r2 + 1][c2 + 1] - s[r1][c2 + 1] - s[r2 + 1][c1] + s[r1][c1];
    }
}
```

Template B — prefix + hash map (count, longest, modular). Reusable: the scan, the seeding, and query-before-insert. Problem-specific: the key transform and the stored value.

JAVA TEMPLATE PrefixHashMap.java

```java
import java.util.*;

class PrefixHashMap {
    /** Number of subarrays with sum exactly k. Works with negative values. */
    static long countSumK(int[] a, long k) {
        Map<Long, Integer> freq = new HashMap<>();
        freq.put(0L, 1);                                 // empty prefix P[0] = 0
        long p = 0, count = 0;
        for (int x : a) {
            p += x;
            count += freq.getOrDefault(p - k, 0);        // query: starts among P[0..j]
            freq.merge(p, 1, Integer::sum);              // then insert P[j+1]
        }
        return count;
    }

    /** Length of the longest subarray with sum exactly k (0 if none). */
    static int longestSumK(int[] a, long k) {
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);                               // P[0] sits "before" index 0
        long p = 0;
        int best = 0;
```

```
        for (int j = 0; j < a.length; j++) {
            p += a[j];
            Integer i = first.get(p - k);
            if (i != null) best = Math.max(best, j - i);  // subarray (i+1 .. j)
            first.putIfAbsent(p, j);                      // keep the earliest index only
        }
        return best;
    }

    /** Number of subarrays whose sum is divisible by k (k >= 1), values of any sign. */
    static long countDivisible(int[] a, int k) {
        long[] bucket = new long[k];
        bucket[0] = 1;                                    // empty prefix has remainder 0
        long rem = 0, count = 0;
        for (int x : a) {
            rem = ((rem + x) % k + k) % k;                // normalise Java's signed %
            count += bucket[(int) rem];
            bucket[(int) rem]++;
        }
        return count;
    }
}
```

4.5 Complexity and pitfalls

Time	O(n) build, O(1) per static query; O(n) expected for hash-map scans (hashing is expected O(1)).
Auxiliary space	O(n) for the prefix array or the map (O(k) for remainder buckets). Running-prefix variants that only need the total use O(1).
Off-by-one	Mixing `P[i] = sum(a[0..i])` (inclusive) with the formula `P[r+1] - P[l]`. Pick the empty-prefix convention and use it everywhere.
Incorrect invariant	Inserting the current prefix before querying (counts empty subarrays); overwriting the first index in "longest" problems; forgetting the seed `{0:1}` or `{0:-1}`.
Overflow	Prefixes of 10^5 values of size 10^9 reach 10^{14}, so use `long`. With `Map<Long, …>`, look up with `long` keys; `map.get(intValue)` boxes an `Integer` and silently misses.
Breaks naive code	k = 0; all zeros (many equal prefixes); negative values with `%`; length constraints such as "at least 2".
Interview follow-ups	Return the subarray itself; 2D version (fix a row pair, then 1D on column sums: $O(m^2n)$); streaming (the scan is already online); "sum ≥ k" with negatives (monotonic deque, LeetCode 862).

4.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- `P[0] = 0` represents the empty prefix; seeding the map with it covers subarrays starting at 0.
- Rewrite the subarray condition as an equation on two prefixes, then look up the earlier one.
- Query before insert: the map must hold only prefixes strictly before the current end.
- Store frequencies for counting, the first index for longest, the last index for shortest.
- Normalise remainders and use `long` for sums.

> **Recognition rule.** Exact sum, balance or divisibility on a contiguous range, especially with negatives → prefix sums plus a hash map of earlier prefixes.

IN-INTERVIEW CHECKLIST

- ☐ What is the key: P, P − k, P mod k, or a ±1 balance?
- ☐ What is stored: count, first index or last index?
- ☐ Is the seed correct for the empty prefix?
- ☐ Is the update order query → insert?
- ☐ Do I need `long` and modulo normalisation?

Key distinction. Use prefix sums for equality conditions with any signs. Use a sliding window for monotone inequality conditions on non-negative data.

4.7 Assigned problems

LC 724 Find Pivot Index

EASY

Task. Return the leftmost index i where the sum strictly left of i equals the sum strictly right of i, or −1. Values may be negative.

KEY INSIGHT

The right sum is `total − left − nums[i]`, so a single running prefix and the total are enough. No array is needed.

WHY THE PATTERN APPLIES

Each index needs two range sums (left and right). These are complementary prefix and suffix sums, derived from one prefix value and the total.

APPROACH

1. Compute `total`.
2. Scan i with `left` = sum of nums[0..i−1]. If `left == total − left − nums[i]`, return i. Otherwise `left += nums[i]`.

JAVA SOLUTION LeetCode 724 · Solution.java

```java
class Solution {
    public int pivotIndex(int[] nums) {
        // |total| <= 1e4 * 1000: int is safe
        int total = 0;
        for (int x : nums) total += x;
        int left = 0;                                   // sum of nums[0..i-1]
        for (int i = 0; i < nums.length; i++) {
            if (left == total - left - nums[i]) return i;   // first hit is leftmost
            left += nums[i];
        }
        return -1;
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Update `left` *after* the check: the pivot itself belongs to neither side. Index 0 is a valid pivot when the rest sums to 0 (e.g. [2, 1, −1]). Negative values rule out any two-pointer "balance" approach.

LC 560 Subarray Sum Equals K

MEDIUM

Task. Count the contiguous non-empty subarrays whose sum equals k. Values are in [−1000, 1000].

KEY INSIGHT

sum(i..j) = k ⇔ P[i] = P[j+1] − k. For each end j, the number of valid starts is the number of earlier prefixes equal to `P[j+1] - k`.

WHY THE PATTERN APPLIES

Negative values make a sliding window invalid (see 2.2), and the condition is exact equality. This is the canonical prefix-frequency map (Template B, `countSumK`).

APPROACH

1. Seed `{0: 1}`.
2. For each x: `p += x`; `count += freq[p - k]`; `freq[p]++`.

JAVA SOLUTION LeetCode 560 · Solution.java

```java
import java.util.*;

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();   // |prefix| <= 2e4 * 1000: int fits
        freq.put(0, 1);                                 // empty prefix
        int p = 0, count = 0;
        for (int x : nums) {
            p += x;
            count += freq.getOrDefault(p - k, 0);       // query earlier prefixes first
            // then make p available to later ends
            freq.merge(p, 1, Integer::sum);
        }
        return count;
    }
}
```

Time O(n) expected **Space** O(n)

> **Pitfall.** Swapping the two lines inside the loop over-counts by one per element when `k == 0` (the empty subarray). The count fits in `int` here because n ≤ $2 \cdot 10^4$ gives at most about $2 \cdot 10^8$ subarrays.

LC 523 Continuous Subarray Sum

MEDIUM

Task. Return true if some subarray of length at least 2 has a sum that is a multiple of k. Here 0 ≤ nums[i] ≤ 10^9 and 1 ≤ k ≤ $2^{31} - 1$. Zero counts as a multiple.

KEY INSIGHT

The sum of (i..j) is a multiple of k ⇔ P[i] ≡ P[j+1] (mod k). The length constraint j − i + 1 ≥ 2 means the two equal-remainder prefixes must be at least 2 positions apart, so keep the *earliest* index of each remainder.

WHY THE PATTERN APPLIES

A modular equality on two prefixes, with a length condition, maps onto Template B with a first-occurrence map keyed by remainder.

APPROACH

1. Seed `first[0] = -1` (the empty prefix sits before index 0).

2. For each j: `rem = (rem + nums[j]) mod k`. If rem was seen at index i and `j − i ≥ 2`, return true. Otherwise store j only if rem is new.

JAVA SOLUTION LeetCode 523 · Solution.java

```java
import java.util.*;

class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        // remainder -> earliest prefix index
        Map<Long, Integer> first = new HashMap<>();
        first.put(0L, -1);
        // long: rem + nums[j] can exceed 2^31-1
        long rem = 0;
        for (int j = 0; j < nums.length; j++) {
            // nums are non-negative: no sign fix
            rem = (rem + nums[j]) % k;
            Integer i = first.get(rem);
            if (i != null) {
                if (j - i >= 2) return true;            // subarray (i+1 .. j), length >= 2
            } else {
                first.put(rem, j);                      // never overwrite: earliest wins
            }
        }
        return false;
    }
}
```

Time O(n) expected **Space** O(min(n, k))

Pitfall. Overwriting the index on repeat breaks inputs such as `[5, 0, 0]`, k = 3: each step sees its remainder only one position back and never reaches length 2. The earliest occurrence gives the longest candidate, and only length matters. `int rem` overflows when k is near $2^{31} - 1$ and nums[j] = 10^9. Two consecutive zeros always return true, and seeding with −1 handles them.

LC 325 Maximum Size Subarray Sum Equals k

MEDIUM Premium

Task. Return the maximum length of a subarray summing to k, or 0 if none. Values may be negative ($|nums[i]| \le 10^4$, $|k| \le 10^9$, $n \le 2 \cdot 10^5$).

KEY INSIGHT

For end j, the longest valid subarray starts right after the *earliest* i with P[i] = P[j+1] − k. Keeping only first occurrences makes each lookup return the best start.

WHY THE PATTERN APPLIES

Exact sum with negatives rules out a window, and maximal length means storing the first index. This is Template B, `longestSumK`.

APPROACH

1. Seed `{0: -1}`.
2. For each j: update `p`; if `p − k` is present at i, `best = max(best, j − i)`; then `putIfAbsent(p, j)`.

JAVA SOLUTION LeetCode 325 · Solution.java

```java
import java.util.*;

class Solution {
```

```java
    public int maxSubArrayLen(int[] nums, int k) {
        Map<Long, Integer> first = new HashMap<>();     // prefix value -> earliest index
        first.put(0L, -1);
        long p = 0;                                     // long keeps p - k exact
        int best = 0;
        for (int j = 0; j < nums.length; j++) {
            p += nums[j];
            Integer i = first.get(p - k);
            if (i != null) best = Math.max(best, j - i);
            first.putIfAbsent(p, j);
        }
        return best;
    }
}
```

Time O(n) expected **Space** O(n)

Pitfall. `put` instead of `putIfAbsent` keeps the latest index, which yields the *shortest* match. With a `Map<Long, Integer>`, pass a `long` key: `first.get(p - k)` works only because `p` is a `long`.

Binary Search: Index / Sorted Array

Binary search is not "find x in a sorted array". It finds the boundary of a monotone predicate over an index range. Sortedness is just the most common way such a predicate arises.

5.1 Pattern identity

Primary purpose	Locate a boundary (first index where a predicate becomes true) or an element in O(log n) by halving a search space whose invariant guarantees the answer stays inside.
Typical input	A sorted array (possibly with duplicates), a rotated sorted array, an array with a local-order property (peaks, slopes), or two sorted arrays whose combined order is implicit.
Interview applications	Exact search, lower/upper bound, first and last occurrence, insert position, rotated array search and minimum, peak finding, median of two sorted arrays, k-th smallest in a sorted matrix, sqrt(x).
Key insight	Every binary search is: "the predicate is false…false true…true on [lo, hi]; find the first true." Write the predicate down first. Mid-point updates then follow mechanically from the interval convention.

> **Key insight.** Correctness comes from one invariant: "the answer, if it exists, lies within my current interval". Each update must discard only indices that provably are not the answer, and the interval must strictly shrink.

5.2 Recognition signals

Signal	Structural reason
Sorted input; "first", "last", "insert position", "count of x"	`a[i] ≥ x` is monotone in i, so the first true is the lower bound.
"O(log n)" required, or n up to 10^9 implicit indices	Only halving fits the budget.
Not sorted, but some local comparison tells you which half contains an answer	Peaks: if `a[m] < a[m+1]`, a peak exists to the right. That is a predicate, even without global order.
Rotated sorted array	`a[i] ≤ a[n−1]` is false…false true…true, and its first true is the minimum.
Two sorted arrays, a k-th element or median	Search the partition point: "how many elements come from array A" is monotone with respect to the cross-boundary comparison.

> **Ask yourself.** "What exactly is my predicate p(i), and is it false…false true…true over the range?" If you cannot write p(i), you are guessing at the update rules.

When it is inappropriate.

- **No monotone or locally decidable predicate** (for example, an arbitrary unsorted array): a linear scan or a hash set is needed.
- **Linked lists**: there is no O(1) random access, so "binary search" costs O(n) per probe.
- **Many queries with updates**: maintain a balanced BST (`TreeMap`) or Fenwick tree instead of re-sorting.
- **The search space is the answer value, not an index**: that is binary search on the answer (Ch. 6), where the predicate is a feasibility check.

Confused with	How to tell them apart
Two pointers	Pair search over sorted data is O(n) with two pointers, versus O(n log n) for "fix one, binary search the other". Binary search wins when only one boundary is needed or queries are repeated.
Binary search on answer	Same skeleton, but the predicate costs O(n) per evaluation and the range is the output domain.
`TreeMap` floor/ceiling	The dynamic-set version of lower/upper bound: O(log n) with inserts and deletes.

5.3 Concepts and invariants

Two interval conventions. Pick one per problem and never mix them:

Convention	Loop	Shrink rules	Exit state
Inclusive `[lo, hi]`	`while (lo <= hi)`	`lo = mid + 1` / `hi = mid - 1`	Empty interval: lo = hi + 1. Use for exact match with early return.
Half-open `[lo, hi)`	`while (lo < hi)`	`lo = mid + 1` (pred false) / `hi = mid` (pred true)	lo == hi == first true (or n if none). Use for boundaries.

In the half-open form, `hi = mid` keeps a possible answer, and `mid = lo + (hi - lo) / 2 < hi` guarantees progress. If you need `lo = mid` (searching the *last* true), round the midpoint up (`lo + (hi - lo + 1) / 2`) or the loop can stall at a two-element interval.

```
lowerBound(x) = first i with a[i] ≥ x      (n if none)
upperBound(x) = first i with a[i] > x      (n if none)
count(x)      = upperBound(x) - lowerBound(x)
last index of x = upperBound(x) - 1        (valid iff count > 0)
mid = lo + (hi - lo) / 2   or   (lo + hi) >>> 1    never (lo + hi) / 2 when lo + hi may overflow
```

Duplicates. Exact-match search returns *an* occurrence, not a specific one. First/last occurrence must be phrased as a boundary (≥ x, > x), and then duplicates need no special handling.

Binary search without global order: the slope predicate (peak finding)

Let `a[-1] = a[n] = -∞` and adjacent values be distinct. Define `p(i) = a[i] > a[i+1]` (descending at i). Claim: if `a[m] < a[m+1]`, some peak lies in `[m+1, n-1]`. Walk right from m+1: either the values keep rising until index n−1, which is a peak because a[n] = −∞, or they first fall at some j, which is a peak. Symmetrically, `a[m] > a[m+1]` implies a peak in `[0, m]`. The interval always contains a peak, which is exactly the invariant binary search needs. p is not globally monotone, but the *existence* invariant is preserved, and that is sufficient.

Partition search. For the median of A (size m ≤ n) and B (size n), choose i elements from A and j = ⌈(m+n)/2⌉ − i from B for the left half. The cut is valid iff `A[i-1] ≤ B[j]` and `B[j-1] ≤ A[i]`. If `A[i-1] > B[j]`, i is too large; otherwise it is too small. So "i is too large" is a monotone predicate over i ∈ [0, m], and the search costs

O(log min(m, n)).

5.4 Reusable Java templates

Template A — exact match, inclusive interval.

JAVA TEMPLATE ExactSearch.java

```java
/** Exact match on a sorted array. Interval [lo, hi] inclusive; returns -1 if absent. */
class ExactSearch {
    static int indexOf(int[] a, int x) {
        int lo = 0, hi = a.length - 1;
        // invariant: if x exists, it is in [lo, hi]
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;        // overflow-safe midpoint
            if (a[mid] == x) return mid;         // some occurrence (not necessarily first)
            if (a[mid] < x) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
}
```

Template B — boundary search on a predicate, half-open interval. This one shape covers lower bound, upper bound, rotated minimum and "first bad version".

JAVA TEMPLATE BoundarySearch.java

```java
import java.util.function.IntPredicate;

/** First index in [lo, hi) where pred is true; pred must be false...false true...true. */
class BoundarySearch {
    static int firstTrue(int lo, int hi, IntPredicate pred) {
        while (lo < hi) {                        // answer in [lo, hi]; hi means "none"
            int mid = lo + (hi - lo) / 2;        // mid < hi, so hi = mid always shrinks
            if (pred.test(mid)) hi = mid;        // mid may be the answer: keep it
            // mid is false: discard it and everything left
            else lo = mid + 1;
        }
        return lo;                               // == original hi if pred is never true
    }

    static int lowerBound(int[] a, int x) { return firstTrue(0, a.length, i -> a[i] >= x); }

    static int upperBound(int[] a, int x) { return firstTrue(0, a.length, i -> a[i] > x); }

    /** Last index in [lo, hi] with pred true; pred is true...true false...false. */
    static int lastTrue(int lo, int hi, IntPredicate pred) {
        int ans = lo - 1;                        // sentinel: none
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (pred.test(mid)) { ans = mid; lo = mid + 1; }   // record and look right
            else hi = mid - 1;
        }
        return ans;
    }
}
```

> **Answer-recording variant.** `lastTrue` uses the inclusive form with an `ans` variable. It avoids the round-up-midpoint subtlety entirely, at the cost of one extra variable. Many engineers use this form for every boundary search because it never stalls.

5.5 Complexity and pitfalls

Time	O(log n) predicate evaluations; O(log min(m, n)) for the two-array partition.
Auxiliary space	O(1) iterative. Recursive versions use O(log n) stack.
Off-by-one	Using `hi = n − 1` with `while (lo < hi)` means lower bound can never return n ("insert at end"). Mixing `hi = mid − 1` into a half-open loop skips the answer.
Infinite loops	`lo = mid` with a round-down midpoint on a two-element interval never progresses.
Overflow	`(lo + hi) / 2` overflows when both exceed 2^{30} (implicit index spaces, answer spaces). Use `lo + (hi − lo) / 2`. Comparing `a[m] + 1` or `target + 1` near `Integer.MAX_VALUE` also overflows, so phrase bounds with `>` instead of `≥ x + 1`.
Breaks naive code	Empty array; target outside [a[0], a[n−1]]; all equal; n = 1; duplicates (rotated search with duplicates degrades to O(n) worst case).
Interview follow-ups	First/last occurrence (bounds); count occurrences; rotated array (with or without duplicates); k-th smallest of two arrays; search in a 2D sorted matrix (treat as a 1D index space); prove why the loop terminates.

5.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Write the predicate first; the code is just "first true".
- Pick inclusive or half-open per problem and keep the updates consistent with it.
- Duplicates are handled by searching boundaries, not values.
- Global sortedness is not required, only an invariant that some answer stays in the interval.
- Use an overflow-safe midpoint and guarantee the interval strictly shrinks.

> **Recognition rule.** An index space plus a predicate that splits it into "no" then "yes" (or tells you which side holds an answer) → halve it.

IN-INTERVIEW CHECKLIST

- ☐ What is p(i), and is it monotone (or answer-preserving)?
- ☐ Inclusive or half-open? Initial `hi` = n − 1 or n?
- ☐ Does every branch shrink the interval?
- ☐ What is returned when the predicate is never true?
- ☐ Check by hand on sizes 0, 1 and 2.

Key distinction. Index binary search evaluates an O(1) predicate on positions. Binary search on the answer evaluates an O(n) feasibility predicate on candidate output values.

5.7 Assigned problems

LC 704 Binary Search — EASY

Task. Return the index of `target` in an ascending array of distinct integers, or −1, in O(log n).

KEY INSIGHT

With distinct values, exact match has at most one answer, so the inclusive form with an early return is the cleanest fit.

WHY THE PATTERN APPLIES

A sorted array and an O(log n) requirement: Template A.

APPROACH

1. `[lo, hi] = [0, n−1]`; compare `nums[mid]` with the target; discard the half that cannot contain it.

JAVA SOLUTION LeetCode 704 · Solution.java

```java
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
}
```

Time O(log n) **Space** O(1)

Pitfall. `while (lo < hi)` with `hi = mid - 1` never examines the final remaining element. For n = 1 it returns −1 even when `nums[0] == target`.

LC 34 Find First and Last Position of Element in Sorted Array

MEDIUM

Task. In a non-decreasing array, return `[first, last]` indices of `target`, or `[-1, -1]`, in O(log n).

KEY INSIGHT

first = lowerBound(target), the first index with a[i] ≥ target. last = upperBound(target) − 1, using the first index with a[i] > target. Duplicates then need no special handling.

WHY THE PATTERN APPLIES

Two boundary searches on monotone predicates (Template B). An exact search followed by a linear scan outward is O(n) when all elements equal the target.

APPROACH

1. `lo = lowerBound(target)`. If `lo == n` or `nums[lo] != target`, return `[-1, -1]`.
2. Otherwise return `[lo, upperBound(target) - 1]`.

JAVA SOLUTION LeetCode 34 · Solution.java

```java
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int first = firstIndex(nums, target, false);    // first a[i] >= target
        if (first == nums.length || nums[first] != target) return new int[]{-1, -1};
        int last = firstIndex(nums, target, true) - 1;  // (first a[i] > target) - 1
        return new int[]{first, last};
    }

    /** First index in [0, n) with a[i] > target (strict) or a[i] >= target; n if none. */
```

```java
    private int firstIndex(int[] a, int target, boolean strict) {
        int lo = 0, hi = a.length;                    // half-open [lo, hi)
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            boolean pred = strict ? a[mid] > target : a[mid] >= target;
            if (pred) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }
}
```

Time O(log n) **Space** O(1)

Pitfall. Writing the upper bound as `lowerBound(target + 1)` would overflow if `target` could be `Integer.MAX_VALUE`. Here |target| ≤ 10^9, so it happens to be safe, but the habit breaks elsewhere. Use the strict predicate instead. Check `first == n` before reading `nums[first]`, including for an empty array.

LC 162 Find Peak Element

MEDIUM

Task. Return the index of any element strictly greater than its neighbours, with `nums[−1] = nums[n] = −∞` and adjacent elements distinct, in O(log n).

KEY INSIGHT

The slope predicate `nums[m] > nums[m+1]` ("descending at m") tells you a peak exists in [lo, m]; otherwise one exists in [m+1, hi]. The array is unsorted, but the invariant "a peak lies in [lo, hi]" is preserved (proof in 5.3).

WHY THE PATTERN APPLIES

This is a classification note in its own right: binary search here relies on a *locally decidable, answer-preserving* predicate, not on sortedness. The half-open-style shrink (`hi = m` keeps m) is Template B's shape.

APPROACH

1. `lo = 0, hi = n − 1`; while `lo < hi`: `m = mid`; if `nums[m] > nums[m+1]` then `hi = m`, else `lo = m + 1`.
2. Return lo: the interval collapsed onto a peak.

JAVA SOLUTION LeetCode 162 · Solution.java

```java
class Solution {
    public int findPeakElement(int[] nums) {
        // invariant: a peak lies in [lo, hi]
        int lo = 0, hi = nums.length - 1;
        while (lo < hi) {
            int m = lo + (hi - lo) / 2;               // m < hi, so m + 1 is in range
            if (nums[m] > nums[m + 1]) hi = m;         // descending: peak in [lo, m]
            else lo = m + 1;                          // ascending: peak in [m + 1, hi]
        }
        return lo;
    }
}
```

Time O(log n) **Space** O(1)

Pitfall. Do not compare with both neighbours and branch three ways. That requires boundary checks against the −∞ sentinels (and values may equal `Integer.MIN_VALUE`, so a numeric sentinel is unsafe). Comparing `m` with `m+1` only, where m < hi guarantees m+1 exists, avoids both problems.

LC 4 Median of Two Sorted Arrays

HARD

Task. Return the median of the union of two sorted arrays of sizes m and n in O(log(m + n)).

KEY INSIGHT

The median is defined by a *partition*: a left half of size ⌈(m+n)/2⌉ with every left element ≤ every right element. Choosing i elements from the shorter array fixes `j = half − i` from the other, so you are binary searching a single integer i ∈ [0, m].

WHY THE PATTERN APPLIES

"i is too large" (`A[i−1] > B[j]`) is monotone in i. Increasing i adds larger A-elements to the left and removes B-elements, so the search space is the cut position, not a value. Searching the shorter array guarantees `0 ≤ j ≤ n` for every i.

APPROACH

1. Ensure `A` is the shorter array. Let `half = (m + n + 1) / 2`.
2. Binary search i in [0, m]; j = half − i. Use sentinels: `Aleft = i == 0 ? −∞ : A[i−1]`, `Aright = i == m ? +∞ : A[i]`, and similarly for B.
3. If `Aleft > Bright`: hi = i − 1. Else if `Bleft > Aright`: lo = i + 1. Else the cut is valid: the median is `max(Aleft, Bleft)` for odd totals, or the average of `max(lefts)` and `min(rights)` for even totals.

JAVA SOLUTION LeetCode 4 · Solution.java

```java
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);
        int[] a = nums1, b = nums2;                    // a is the shorter array
        int m = a.length, n = b.length;
        int half = (m + n + 1) / 2;                    // size of the left half
        // i = number taken from a, in [0, m]
        int lo = 0, hi = m;
        while (lo <= hi) {
            int i = lo + (hi - lo) / 2;
            int j = half - i;                          // 0 <= j <= n because m <= n
            // Sentinels for empty sides; values are within +-1e6, so long bounds are safe.
            long aLeft = i == 0 ? Long.MIN_VALUE : a[i - 1];
            long aRight = i == m ? Long.MAX_VALUE : a[i];
            long bLeft = j == 0 ? Long.MIN_VALUE : b[j - 1];
            long bRight = j == n ? Long.MAX_VALUE : b[j];
            if (aLeft > bRight) {
                hi = i - 1;                            // took too many from a
            } else if (bLeft > aRight) {
                lo = i + 1;                            // took too few from a
            } else {
                long leftMax = Math.max(aLeft, bLeft);
                if ((m + n) % 2 == 1) return leftMax;
                long rightMin = Math.min(aRight, bRight);
                return (leftMax + rightMin) / 2.0;
            }
        }
        throw new IllegalArgumentException("inputs must be sorted");
    }
}
```

Time O(log min(m, n)) **Space** O(1)

Pitfall. Searching the longer array lets j go negative or exceed n. Using `Integer.MIN_VALUE/MAX_VALUE` as sentinels is fine for this problem's value range, but long sentinels also stay correct if values can be extreme. Compute the even-case average in floating point (or `long`) to avoid int overflow. When one array is empty, i = 0 and the formulas reduce to the other array's median.

Binary Search on Answer: Parametric Search

Instead of searching the input, binary search the space of possible answers. This works when the question "can the target be met with value x?" is cheap to answer and monotone in x.

6.1 Pattern identity

Primary purpose	Turn an optimisation problem ("smallest x such that…", "minimise the maximum…") into O(log range) decision problems.
Typical input	An array of loads, sizes or piles, plus a budget (days, hours, threshold, number of parts). The answer is an integer in a bounded range.
Interview applications	Minimum speed or capacity, smallest divisor, split array largest sum, minimum days to make bouquets, aggressive cows / magnetic force (maximise the minimum), k-th smallest in a sorted matrix, median of large ranges.
Key insight	If `feasible(x)` implies `feasible(x + 1)`, the feasible values form a suffix of the range. The answer is the boundary of that suffix, and binary search finds it with O(log R) calls to a usually greedy O(n) checker.

> **Key insight.** You do not need a formula for the optimum. You need (1) a range guaranteed to contain it, (2) a predicate that is monotone over that range, and (3) a fast, correct way to evaluate the predicate.

6.2 Recognition signals

Signal	Structural reason
"Minimum X such that the task finishes within budget B" / "maximum X such that a constraint still holds"	Relaxing X (more speed, more capacity) can only make the task easier, so the predicate is monotone.
"Minimise the largest part" / "maximise the smallest gap"	Min-max and max-min objectives are the classic parametric forms: fix the threshold, then check greedily.
Answer is a number in $[1, 10^9]$ while $n \le 10^5$	$O(n \log R) \approx 3 \cdot 10^6$ operations. Enumerating every candidate x is impossible.
Checking a candidate is easy; constructing the optimum directly is hard	The decision version admits a greedy that the optimisation version does not obviously admit.

> **Ask yourself.** "If someone handed me the answer x, could I verify in O(n) whether it is achievable? And if x works, does x + 1 (or x − 1) also work?"

When it is inappropriate.

- **Non-monotone predicates**: e.g. "exactly k parts" when feasibility can flip. Reformulate as "at most k parts", which is monotone.
- **No cheap checker**: if verifying x needs exponential search, binary search does not help.

- **A direct formula or a greedy optimum exists**, such as the maximum of an array. Do not add a log factor for nothing.
- **Real-valued answers** need iteration-count termination (e.g. 100 halvings), not `lo < hi` on doubles.

Confused with	How to tell them apart
Binary search on an index (Ch. 5)	Index search looks for a position in a given array. Here the "array" is the virtual sequence `feasible(lo..hi)`, which is never materialised.
Dynamic programming	Split Array Largest Sum has an $O(k \cdot n^2)$ DP. Parametric search wins whenever the checker is greedy and the answer range is narrow in log terms.
Greedy alone	The greedy inside the checker answers "is x enough?". On its own it does not pick the optimal x.

6.3 Concepts and invariants

Defining the answer space. Choose `lo` as a value that is certainly no larger than the answer and `hi` as a value that is certainly feasible. Good bounds come from the problem's extreme cases:

Problem	lo (lower bound)	hi (always feasible)	Why
Koko (875)	1	max(piles)	At speed max, each pile takes one hour and h ≥ n is guaranteed.
Smallest divisor (1283)	1	max(nums)	Every term ceil(v / max) = 1 and threshold ≥ n.
Ship within D days (1011)	max(weights)	sum(weights)	A package cannot be split, and one ship can take everything in one day.
Split array (410)	max(nums)	sum(nums)	Some part contains the largest element, and one part is always allowed.

Monotone predicate and the boundary search. For minimisation, the predicate over [lo, hi] looks like `F F F … F T T … T`. Search the first T with the half-open discipline:

```
Invariant: answer ∈ [lo, hi] and feasible(hi) is true.

mid = lo + (hi − lo) / 2          (rounds down, so mid < hi and the loop always shrinks)
feasible(mid) ⇒ hi = mid          (mid may be the answer: keep it)
otherwise     ⇒ lo = mid + 1      (mid is infeasible, so everything ≤ mid is too)
Loop while lo < hi; on exit lo == hi == answer.
```

For maximisation (`T T … T F F`), search the last T. Round up, `mid = lo + (hi − lo + 1) / 2`, with `lo = mid` on success and `hi = mid − 1` on failure. Rounding down here makes `lo = mid` loop forever when hi = lo + 1.

Proving feasibility monotonicity (and the checker's greedy)

Monotonicity. For capacity C, any shipping schedule valid under C is also valid under C + 1, since no day's load exceeds C < C + 1. Therefore minDays(C) is non-increasing in C, and "minDays(C) ≤ D" is a suffix property. The same one-line argument covers speed (hours are non-increasing in speed) and divisor (each `ceil(v / d)` is non-increasing in d).

Greedy checker. "Given capacity C, fill each day as much as possible before starting a new one" minimises the number of days. Exchange argument: take any valid partition and the greedy one. Where the greedy's first day ends at or after the optimal first day's end, greedy has covered a prefix at least as long. By

induction, after j days greedy covers at least as many items as any valid schedule, so it never uses more days.

Assumption. The greedy relies on all items being non-negative. With negative weights, packing more could *reduce* a day's load, and "fill as much as possible" is no longer optimal.

Integer ceiling without floating point. For positive a and b, `ceil(a / b) = (a + b - 1) / b`, or `(a - 1) / b + 1` for a ≥ 1. The second form avoids overflow when a is near `Integer.MAX_VALUE`. Never use `Math.ceil((double) a / b)` in interviews: it is slow and, for large longs, inexact.

Overflow. The checker's accumulator (total hours, total sum) can exceed $2^{31}-1$. For Koko, 10^4 piles × 10^9 hours each overflows an `int`, so use `long`. Bounds like sum(nums) can also overflow `int` in general; LeetCode 410's constraints keep it within 10^9, but writing `long` costs nothing.

6.4 Reusable Java templates

Template A — minimise x subject to a monotone feasibility predicate. Reusable: the loop. Problem-specific: bounds and `feasible`.

JAVA TEMPLATE MinFeasible.java

```java
import java.util.function.LongPredicate;

/** Smallest x in [lo, hi] with feasible(x) true. Precondition: feasible(hi) is true and
 *  feasible is monotone (false ... false true ... true). Interval [lo, hi] inclusive. */
class MinFeasible {
    static long minFeasible(long lo, long hi, LongPredicate feasible) {
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;   // floor; never overflows, mid < hi
            if (feasible.test(mid)) hi = mid; // mid might be the answer
            else lo = mid + 1;                // mid and everything below fails
        }
        return lo;
    }

    /** Example checker (Koko-style): hours needed at speed k, accumulated in long. */
    static long hoursNeeded(int[] piles, long k) {
        long hours = 0;
        for (int p : piles) hours += (p + k - 1) / k;  // ceil(p / k) for p >= 0, k >= 1
        return hours;
    }
}
```

Template B — maximise x subject to a monotone predicate (max-min problems). Note the upward rounding.

JAVA TEMPLATE MaxFeasible.java

```java
import java.util.*;
import java.util.function.LongPredicate;

/** Largest x in [lo, hi] with ok(x) true. Precondition: ok(lo) is true and the predicate is
 *  true ... true false ... false. */
class MaxFeasible {
    static long maxFeasible(long lo, long hi, LongPredicate ok) {
        while (lo < hi) {
            // ceil: mid > lo, so lo = mid always progresses
            long mid = lo + (hi - lo + 1) / 2;
            if (ok.test(mid)) lo = mid;
```

```
            else hi = mid - 1;
        }
        return lo;
    }

    /** Example: maximise the minimum gap when placing m items on sorted positions. */
    static int maxMinGap(int[] pos, int m) {
        int[] p = pos.clone();
        Arrays.sort(p);
        return (int) maxFeasible(0, p[p.length - 1] - p[0], gap -> {
            int placed = 1;
            long last = p[0];
            for (int i = 1; i < p.length; i++) {
                if (p[i] - last >= gap) { placed++; last = p[i]; }  // greedy: place ASAP
            }
            return placed >= m;
        });
    }
}
```

Template C — greedy partition checker, shared by 1011 and 410: "with segment-sum cap `cap`, how many contiguous segments are needed?"

JAVA TEMPLATE PartitionChecker.java

```
/** Greedy: extend the current segment while it fits under cap. Requires a[i] >= 0 and
 *  cap >= max(a); otherwise some single element cannot be placed. */
class PartitionChecker {
    static int segmentsNeeded(int[] a, long cap) {
        int segments = 1;
        long cur = 0;
        for (int x : a) {
            if (cur + x > cap) { segments++; cur = 0; }   // close the segment before x
            cur += x;
        }
        return segments;
    }
}
```

6.5 Complexity and pitfalls

Time	O(C(n) · log R), where C(n) is the checker cost (usually O(n)) and R = hi − lo + 1.
Auxiliary space	O(1) beyond the input for greedy checkers.
Off-by-one	Pair `lo < hi` + `hi = mid` with floor; pair `lo = mid` with ceil. Mixing `lo <= hi` with `hi = mid` loops forever.
Wrong bounds	lo = 0 for a divisor or speed (division by zero); lo = 1 for capacity (an infeasible region the checker may mis-evaluate, since a single item exceeds the cap).
Overflow	`(lo + hi) / 2` on ints near 2^{31}; checker sums; `(p + k - 1)` when p ≈ 2^{31} in int arithmetic.
Breaks naive code	"Exactly k" instead of "at most k" (non-monotone); a checker that greedily packs negative values; floating-point ceilings.
Interview follow-ups	Prove monotonicity; give the DP alternative (410); handle real-valued answers (iterate ~60–100 times); return the actual partition; reduce the range with tighter bounds such as `ceil(sum / D)`.

6.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- The three ingredients are a valid range, a monotone predicate, and a fast checker. Prove all three aloud.
- Minimisation: floor mid, `hi = mid`. Maximisation: ceil mid, `lo = mid`.
- Choose lo as the "physically necessary" lower bound (max element for unsplittable items).
- Greedy checkers assume non-negative items. State that assumption.
- Use `long` in the checker by default.

> **Recognition rule.** "Smallest / largest value such that a budget is respected" + an O(n) way to test one value → binary search the value.

IN-INTERVIEW CHECKLIST

- ☐ What exactly is x, and is the predicate "x is enough" monotone?
- ☐ Is hi guaranteed feasible? Can lo cause division by zero?
- ☐ Floor or ceil mid, matching the update rule?
- ☐ Can the checker's accumulator overflow?
- ☐ Does the checker's greedy actually compute the optimum for a fixed x?

Key distinction. In Chapter 5 the monotone sequence is the array. Here it is the implicit sequence `feasible(lo), …, feasible(hi)`, and each probe costs O(n) instead of O(1).

6.7 Assigned problems

LC 1283 Find the Smallest Divisor Given a Threshold — MEDIUM

Task. Find the smallest positive integer d such that Σ ceil(nums[i] / d) ≤ threshold, where threshold ≥ nums.length.

KEY INSIGHT

Each term ceil(v / d) is non-increasing in d, so the sum is too. The condition "sum ≤ threshold" therefore holds on a suffix of d values.

WHY THE PATTERN APPLIES

We want a minimum value that meets a budget, the checker is O(n), and the range [1, max(nums)] spans up to 10^6 candidates.

APPROACH

1. `lo = 1`, `hi = max(nums)`. At hi every term is 1 and the sum is n ≤ threshold, so hi is feasible.
2. Minimisation template: feasible → `hi = mid`, otherwise `lo = mid + 1`.

JAVA SOLUTION LeetCode 1283 · Solution.java

```java
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int lo = 1, hi = 1;
        for (int v : nums) hi = Math.max(hi, v);
```

```
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (sumOfCeil(nums, mid) <= threshold) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private long sumOfCeil(int[] nums, int d) {
        long s = 0;
        for (int v : nums) s += (v - 1) / d + 1;  // ceil(v / d) for v >= 1, no overflow
        return s;
    }
}
```

Time O(n log M), M = max(nums) **Space** O(1)

Pitfall. Starting at `lo = 0` divides by zero. `(v + d - 1) / d` is also correct here because v ≤ 10^6, but `(v - 1) / d + 1` is safe for any positive int.

LC 875 Koko Eating Bananas

MEDIUM

Task. Koko eats at most k bananas per hour from a single pile. Find the minimum integer k that finishes all piles within h hours (h ≥ piles.length).

KEY INSIGHT

The hours needed at speed k are Σ ceil(p / k), which is non-increasing in k. The minimum feasible k is the first true in a monotone predicate.

WHY THE PATTERN APPLIES

We want a minimum speed under a time budget, with a clear range [1, max(piles)] and an O(n) check. Speed above max(piles) never helps, because each pile already takes one hour.

APPROACH

1. Search k in [1, max(piles)].
2. Feasible if total hours (in `long`) ≤ h.

JAVA SOLUTION LeetCode 875 · Solution.java

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int lo = 1, hi = 1;
        for (int p : piles) hi = Math.max(hi, p);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (hoursAt(piles, mid) <= h) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private long hoursAt(int[] piles, int k) {
        long hours = 0;                        // up to 1e4 piles * 1e9 hours: needs long
        for (int p : piles) hours += (p - 1) / k + 1;
        return hours;
    }
}
```

Time O(n log M) **Space** O(1)

Pitfall. An `int` hour counter overflows at speed 1 with large piles, wraps negative, and makes a slow speed look feasible. That bug passes the small examples and fails hidden tests.

LC 1011 Capacity To Ship Packages Within D Days

MEDIUM

Task. Ship packages in the given order, with each day's load at most the capacity. Find the minimum capacity that finishes within `days` days.

KEY INSIGHT

For a fixed capacity, greedily packing each day as full as possible minimises the number of days (see the exchange argument in 6.3), and the day count is non-increasing in capacity.

WHY THE PATTERN APPLIES

This is a minimum capacity under a day budget, with an order-preserving contiguous partition and non-negative weights, so the greedy checker is valid.

APPROACH

1. `lo = max(w)` (a package cannot be split), `hi = sum(w)` (one day).
2. Feasible if `segmentsNeeded(w, cap) ≤ days`.

JAVA SOLUTION LeetCode 1011 · Solution.java

```java
class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int lo = 0, hi = 0;
        for (int w : weights) { lo = Math.max(lo, w); hi += w; }  // sum <= 5e4 * 500
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (daysNeeded(weights, mid) <= days) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    /** Greedy: fill the current day until the next package would overflow it. */
    private int daysNeeded(int[] weights, int cap) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d;
    }
}
```

Time O(n log(Σw)) **Space** O(1)

Pitfall. With `lo = 1`, a capacity smaller than one package makes the checker silently "place" an oversized package by starting a new day. It undercounts and can return an impossible capacity. Starting at max(w) removes that region entirely.

LC 410 Split Array Largest Sum

HARD

Task. Split `nums` (non-negative) into k non-empty contiguous subarrays to minimise the largest subarray sum.

KEY INSIGHT

"Can every part be kept ≤ S using at most k parts?" is monotone in S. If the greedy needs at most k parts, exactly k non-empty parts are also achievable, because any part with two or more elements can be split without raising the maximum (n ≥ k is guaranteed).

WHY THE PATTERN APPLIES

This is the canonical min-max objective. It is the same checker as 1011, with "days" renamed to "parts", which shows that the pattern, not the story, determines the solution.

APPROACH

1. Search S in [max(nums), sum(nums)].
2. Feasible if greedy segments ≤ k. This uses "at most k", which is monotone, rather than "exactly k", which is not directly monotone.
3. Alternative: DP `dp[j][i]` = best maximum for the first i elements in j parts, $O(k \cdot n^2)$. Binary search is O(n log Σ).

JAVA SOLUTION LeetCode 410 · Solution.java

```java
class Solution {
    public int splitArray(int[] nums, int k) {
        long lo = 0, hi = 0;
        for (int v : nums) { lo = Math.max(lo, v); hi += v; }
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            if (partsNeeded(nums, mid) <= k) hi = mid;   // "at most k" is monotone in mid
            else lo = mid + 1;
        }
        return (int) lo;
    }

    private int partsNeeded(int[] nums, long cap) {
        int parts = 1;
        long cur = 0;
        for (int v : nums) {
            if (cur + v > cap) { parts++; cur = 0; }
            cur += v;
        }
        return parts;
    }
}
```

Time O(n log Σ) **Space** O(1)

Pitfall. Requiring `partsNeeded == k` breaks monotonicity: the greedy may use fewer parts at a feasible S. The "≤ k" relaxation is justified by the split argument above. Zeros are fine because they never increase a segment sum.

Merge Intervals: Sorting and Sweep

Sort intervals so that overlap becomes a local relationship between neighbours. Then either merge them in one pass or sweep over their endpoints as events.

7.1 Pattern identity

Primary purpose	Compute unions, intersections, gaps or the maximum overlap of a set of intervals in O(n log n).
Typical input	`int[][]` of [start, end] pairs (possibly unsorted), or several already-sorted lists of disjoint intervals.
Interview applications	Merging calendars, inserting a booking, minimum meeting rooms or platforms, free time across schedules, total covered length, CPU or bandwidth peak load.
Key insight	After sorting by start, an interval can overlap the *current merged block* only if its start is ≤ the block's end. It never needs to be compared with earlier blocks, because they all end before the current block starts.

> **Key insight.** Sorting by start converts a pairwise $O(n^2)$ overlap relation into a single running value, the end of the current block. Counting overlaps instead requires splitting each interval into a +1 event and a −1 event and sweeping them in time order.

7.2 Recognition signals

Signal	Structural reason
Input is a set of [start, end] pairs; output is merged ranges, gaps or a count	The interval graph's connected components are contiguous runs once sorted by start.
"Minimum number of rooms / platforms / machines"	The answer is the maximum depth of overlap at any instant: a sweep over events.
Several schedules, each already sorted	A k-way merge (heap) or a two-pointer walk replaces the global sort.
"Remove the fewest to make non-overlapping", "maximum number of non-overlapping"	That is *selection*, not merging: greedy by end time (Pattern 14).

> **Ask yourself.** "Am I combining intervals (merge), counting simultaneous intervals (sweep or heap), or choosing a subset of intervals (greedy)? And do touching endpoints overlap?"

When it is inappropriate. Use a different structure when intervals arrive online and you need fast insert plus overlap queries (a `TreeMap` keyed by start, or a segment tree over compressed coordinates), when intervals carry weights and you select a subset (DP over sorted intervals plus binary search), or when the "intervals" are 2D rectangles (sweep plus a segment tree).

Confused with	How to tell them apart
Greedy interval scheduling (Ch. 14)	Merging keeps *all* intervals and sorts by start. Scheduling discards some and sorts by *end*, because the earliest finish leaves the most room.
Difference array / prefix sum	If coordinates are small integers, `diff[s]++, diff[e]--` plus a prefix sum computes the depth directly. Sweep with sorting is the coordinate-free version.
Two pointers (Ch. 1)	Intersecting two sorted interval lists (LeetCode 986) is two pointers over lists; no merging happens.

7.3 Concepts and invariants

Overlap conditions. Closed intervals [a, b] and [c, d] overlap iff `max(a, c) ≤ min(b, d)`; they are disjoint iff `b < c` or `d < a`. For half-open [a, b) and [c, d), replace ≤ with <, so touching intervals do *not* overlap.

Problem	Convention	[1,4] with [4,5]	Code test
56 Merge Intervals	Closed	Overlap → [1,5]	`cur.start <= last.end`
57 Insert Interval	Closed	Overlap	`iv.end < new.start` → strictly before
253 Meeting Rooms II	Half-open (end frees the room)	Same room reusable	`minEnd <= start` → reuse
759 Employee Free Time	Free time must have positive length	No gap	`start > prevEnd` → gap

Merge invariant. While scanning intervals sorted by start, `last` is the union of everything seen that is connected to the most recent block, and every earlier output block ends strictly before `last.start`. Proof: if `cur.start > last.end`, then every later interval has start ≥ cur.start > last.end, so `last` is final.

Sweep-line invariant. Map each interval to events (start, +1) and (end, −1), and sort by time. At every point the running sum equals the number of intervals covering that instant, and the answer is its maximum. **Tie-breaking encodes the boundary convention**: for half-open intervals, process −1 before +1 at equal times, so a meeting ending at 10 frees its room for one starting at 10. For closed intervals, process +1 first.

```
rooms(t) = #{ i : start_i ≤ t < end_i }        (half-open)
answer   = max over event times t of rooms(t)
Heap view: keep the end times of active meetings; a new meeting at s reuses a room iff
min(end) ≤ s.
```

Sweep, heap, or two sorted arrays: which to use?

- **Event sort**: 2n events, one sort, O(n log n). Generalises to weighted loads (+w/−w) and to "the time of maximum overlap".
- **Min-heap of end times**: sort by start, then pop all ends ≤ start (or one, since only the size matters). The heap size equals the rooms currently in use. Preferable when you must *assign* concrete rooms (store (end, roomId)).
- **Separately sorted starts and ends**: two pointers over the sorted `starts[]` and `ends[]`. This is the leanest constant factor and is valid because only counts matter, not which interval ends.
- **k sorted lists** (759): a k-way heap merge costs O(N log k), versus O(N log N) for flattening and sorting.

7.4 Reusable Java templates

Template A — sort and merge (closed intervals).

JAVA TEMPLATE MergeTemplate.java

```java
import java.util.*;

/** Merge closed intervals [s, e]. Touching intervals ([1,4],[4,5]) are merged. */
class MergeTemplate {
    static int[][] merge(int[][] intervals) {
        // do not reorder the caller's array
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0])); // no subtraction comparator
        List<int[]> out = new ArrayList<>();
        for (int[] iv : a) {
            int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
            if (last != null && iv[0] <= last[1]) {            // overlaps current block
                // a contained interval keeps end
                last[1] = Math.max(last[1], iv[1]);
            } else {
                // copy: never alias the input
                out.add(new int[]{iv[0], iv[1]});
            }
        }
        return out.toArray(new int[0][]);
    }
}
```

Template B — event sweep for maximum overlap. The comparator's tie rule is the boundary convention.

JAVA TEMPLATE SweepTemplate.java

```java
import java.util.*;

/** Maximum number of simultaneously active intervals. halfOpen = true means [s, e). */
class SweepTemplate {
    static int maxOverlap(int[][] intervals, boolean halfOpen) {
        int n = intervals.length;
        int[][] ev = new int[2 * n][];
        for (int i = 0; i < n; i++) {
            ev[2 * i] = new int[]{intervals[i][0], +1};
            ev[2 * i + 1] = new int[]{intervals[i][1], -1};
        }
        // Equal times: half-open -> end (-1) first; closed -> start (+1) first.
        Arrays.sort(ev, (x, y) -> x[0] != y[0] ? Integer.compare(x[0], y[0])
                : halfOpen ? Integer.compare(x[1], y[1]) : Integer.compare(y[1], x[1]));
        int cur = 0, best = 0;
        for (int[] e : ev) {
            cur += e[1];
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Template C — sort by start + min-heap of end times (resource allocation; extend to (end, id) pairs for assignment).

JAVA TEMPLATE HeapAllocationTemplate.java

```java
import java.util.*;

/** Minimum resources for half-open intervals; heap holds end times of busy resources. */
class HeapAllocationTemplate {
    static int minResources(int[][] intervals) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
```

```
        PriorityQueue<Integer> ends = new PriorityQueue<>();
        for (int[] iv : a) {
            // reuse the earliest-free
            if (!ends.isEmpty() && ends.peek() <= iv[0]) ends.poll();
            ends.offer(iv[1]);
        }
        return ends.size();   // the heap never shrinks below the peak overlap
    }
}
```

7.5 Complexity and pitfalls

Time	O(n log n) for the sort; the merge or sweep is O(n); heap variants O(n log n); k-way merge of N intervals O(N log k).
Auxiliary space	O(n) for events or output; O(log n) to O(n) for sorting (object-array sort is TimSort, O(n) worst case); heap O(n) or O(k).
Off-by-one / boundaries	`<` vs `<=` on touching endpoints is the most common bug. Decide the convention from the statement before coding.
Wrong invariant	Setting `last.end = cur.end` instead of `max`, which fails when cur is contained in last ([1,10], [2,3]).
Overflow	Comparator `(x, y) -> x[0] - y[0]` overflows for values near $\pm 2^{31}$. Always use `Integer.compare`.
Breaks naive code	Unsorted input, nested intervals, duplicates, a single interval, an empty list (57: insert into an empty list).
Interview follow-ups	Online insertion (TreeMap floor/ceiling); total covered length; return which rooms are assigned; Meeting Rooms III (rooms by index); the time of maximum overlap.

7.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Sort by start to merge; sort events by time to count; sort by end to select (Ch. 14).
- The merge only ever compares with the last block, and extends it with `max`.
- Boundary conventions turn into the `<` / `<=` choice or the tie-break order of events.
- A min-heap of end times equals the set of busy resources.
- Never use subtraction comparators; never alias input arrays you then mutate.

> **Recognition rule.** Intervals + "combine / gaps / how many at once" → sort by start and sweep, keeping one running end or a heap of ends.

IN-INTERVIEW CHECKLIST

- ☐ Closed or half-open? Do touching intervals overlap?
- ☐ Is the input already sorted (57, 759)? Then avoid the O(n log n) sort.
- ☐ Am I merging, counting depth, or selecting?
- ☐ Nested intervals handled with `max`?
- ☐ Comparator uses `Integer.compare`?

Key distinction. Merge Intervals preserves every interval's coverage. Greedy scheduling (Ch. 14) throws intervals away, which is why the sort key changes from start to end.

7.7 Assigned problems

LC 56 Merge Intervals — MEDIUM

Task. Merge all overlapping closed intervals and return the non-overlapping result covering the same points.

KEY INSIGHT

Sorted by start, each interval either extends the current block (start ≤ block end) or proves the block final.

WHY THE PATTERN APPLIES

This is the defining problem: union of closed intervals, where touching intervals merge ([1,4],[4,5] → [1,5]).

APPROACH

1. Sort by start with `Integer.compare`.
2. Extend `last[1] = max(last[1], cur[1])` on overlap, otherwise start a new block.

JAVA SOLUTION LeetCode 56 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> merged = new ArrayList<>();
        int[] cur = intervals[0].clone();                // 1 <= intervals.length
        for (int i = 1; i < intervals.length; i++) {
            int[] iv = intervals[i];
            if (iv[0] <= cur[1]) {
                cur[1] = Math.max(cur[1], iv[1]);        // contained intervals keep cur[1]
            } else {
                merged.add(cur);
                cur = iv.clone();
            }
        }
        merged.add(cur);                                 // flush the final block
        return merged.toArray(new int[merged.size()][]);
    }
}
```

Time O(n log n) **Space** O(n) output + sort space

Pitfall. Forgetting to add the final block, and `cur[1] = iv[1]` without `max` on nested input such as [[1,10],[2,3]].

LC 57 Insert Interval — MEDIUM

Task. Insert `newInterval` into a sorted list of disjoint closed intervals, merging where necessary.

KEY INSIGHT

The input is already sorted and disjoint, so the result splits into three contiguous phases: intervals strictly before the new one, intervals overlapping it (absorbed into one), and intervals strictly after it. No sort is needed, which gives O(n).

WHY THE PATTERN APPLIES

This is a merge restricted to one block. The overlap test against the growing new interval is the same closed-interval test as in 56.

APPROACH

1. Copy all intervals with `end < new.start`.
2. While `start ≤ new.end`, absorb: `new = [min(starts), max(ends)]`.
3. Append the merged interval, then the remainder.

JAVA SOLUTION LeetCode 57 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> out = new ArrayList<>();
        int i = 0, n = intervals.length;
        int start = newInterval[0], end = newInterval[1];
        // strictly before
        while (i < n && intervals[i][1] < start) out.add(intervals[i++]);
        while (i < n && intervals[i][0] <= end) {                          // overlapping
            start = Math.min(start, intervals[i][0]);
            end = Math.max(end, intervals[i][1]);
            i++;
        }
        out.add(new int[]{start, end});
        // strictly after
        while (i < n) out.add(intervals[i++]);
        return out.toArray(new int[out.size()][]);
    }
}
```

Time O(n) **Space** O(n) output

Pitfall. Use `<` in phase 1 and `<=` in phase 2: [1,2] and new [2,3] touch and must merge. An empty `intervals` array must still return `[newInterval]`, and the loops above handle it naturally.

LC 253 Meeting Rooms II

MEDIUM Premium

Task. Given meeting time intervals, return the minimum number of conference rooms required.

KEY INSIGHT

The answer is the maximum number of meetings in progress at once. Meetings are half-open: a room freed at time 10 can host a meeting starting at 10.

WHY THE PATTERN APPLIES

This counts overlap depth rather than merging. Sorting by start plus a min-heap of end times simulates room allocation exactly: the heap root is the room that frees up first.

APPROACH

1. Sort by start.

2. For each meeting, if the earliest-ending room is free (`peek() <= start`), reuse it (poll). Then push this meeting's end.
3. The final heap size is the number of rooms ever opened, which equals the peak overlap.

JAVA SOLUTION LeetCode 253 · Solution.java

```java
import java.util.*;

class Solution {
    public int minMeetingRooms(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
        // end times of busy rooms
        PriorityQueue<Integer> roomEnds = new PriorityQueue<>();
        for (int[] m : intervals) {
            if (!roomEnds.isEmpty() && roomEnds.peek() <= m[0]) {
                // earliest room is free by m's start: reuse it
                roomEnds.poll();
            }
            roomEnds.offer(m[1]);
        }
        return roomEnds.size();
    }
}
```

Time O(n log n) **Space** O(n) heap

Pitfall. Using `<` instead of `<=` treats back-to-back meetings as conflicting and overcounts. Equivalent alternatives: the event sweep (Template B with end-before-start ties) or two sorted arrays of starts and ends.

LC 759 Employee Free Time

HARD Premium

Task. Each employee has a sorted list of non-overlapping working `Interval`s. Return the finite, positive-length intervals during which *all* employees are free, in sorted order.

KEY INSIGHT

Common free time is exactly the set of gaps in the *union* of all working intervals, restricted to the span between the first start and the last end. So: merge everything, then report the gaps.

WHY THE PATTERN APPLIES

This is a merge over k already-sorted lists. A min-heap keyed on start produces the global start order in O(N log k) without re-sorting, and the merge invariant (track the running maximum end) yields the gaps.

APPROACH

1. Seed a heap with the first interval of every employee as (employee, index), ordered by start.
2. Pop the interval with the smallest start. If `start > prevEnd`, then [prevEnd, start] is free (a strict inequality excludes zero-length gaps). Update `prevEnd = max(prevEnd, end)`.
3. Push that employee's next interval.

JAVA SOLUTION LeetCode 759 · Solution.java (Interval is provided by LeetCode)

```java
import java.util.*;

class Solution {
    public List<Interval> employeeFreeTime(List<List<Interval>> schedule) {
        // Heap entries: {employee, index}; ordered by that interval's start.
```

```
        PriorityQueue<int[]> pq = new PriorityQueue<>((x, y) -> Integer.compare(
                schedule.get(x[0]).get(x[1]).start, schedule.get(y[0]).get(y[1]).start));
        for (int e = 0; e < schedule.size(); e++) {
            if (!schedule.get(e).isEmpty()) pq.offer(new int[]{e, 0});
        }
        List<Interval> free = new ArrayList<>();
        int prevEnd = Integer.MIN_VALUE;            // sentinel: no work seen yet
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            Interval iv = schedule.get(top[0]).get(top[1]);
            if (prevEnd != Integer.MIN_VALUE && iv.start > prevEnd) {
                free.add(new Interval(prevEnd, iv.start));   // positive-length gap only
            }
            prevEnd = Math.max(prevEnd, iv.end);
            if (top[1] + 1 < schedule.get(top[0]).size()) {
                pq.offer(new int[]{top[0], top[1] + 1});
            }
        }
        return free;
    }
}
```

Time O(N log k), N intervals, k employees **Space** O(k) heap + output

Pitfall. Free time before the first start or after the last end is infinite and must not be reported, which is why the sentinel guard exists. Touching intervals [1,2],[2,3] produce no gap. Flattening and sorting all intervals is a simpler O(N log N) alternative that interviewers usually accept.

PART II · PATTERN 08

Backtracking: DFS with Choices

A depth-first walk over an implicit tree of partial solutions: make a choice, explore, undo it, and cut off subtrees that cannot lead to a valid answer.

8.1 Pattern identity

Primary purpose	Enumerate every configuration satisfying constraints (subsets, permutations, partitions, placements), or find one, when the output or search is inherently exponential.
Typical input	A small set of candidates ($n \le 10$–20), a target, a board, or a string to partition.
Interview applications	Subsets, permutations, combinations, combination sum, palindrome partitioning, word search, N-Queens, Sudoku, generating parentheses, letter combinations.
Key insight	Each recursion level makes one decision. The path from the root to a node *is* the partial solution, and restoring state after each child keeps one shared mutable path correct for every branch.

> **Key insight.** Correctness comes from two things: the choice tree generates every valid configuration *exactly once* (canonical ordering), and pruning removes only subtrees that provably contain no solution.

8.2 Recognition signals

Signal	Structural reason
"Return all ..." (subsets, permutations, partitions, boards)	The output itself can be exponential, so no polynomial algorithm can do better than enumerating it.
$n \le 10$ (permutations) or $n \le 20$ (subsets)	$n! \approx 3.6 \cdot 10^6$ at $n = 10$; $2^{20} \approx 10^6$.
Constraint satisfaction: place items so that no two conflict	Partial assignments can be checked incrementally and abandoned early.
Decisions are sequential, and each depends on earlier choices	That is a natural tree with "choose → explore → undo".

> **Ask yourself.** "Do I need the configurations themselves, or only how many or the best one?" If only a count or optimum is needed and subproblems repeat, switch to DP or memoisation.

When it is inappropriate.

- **Counting or optimisation with overlapping states**: the number of combinations summing to a target (Coin Change II) has polynomial DP; plain backtracking is exponential.
- **Large n with a smooth objective**: use greedy, DP or a graph algorithm.
- **Shortest number of steps in a state space**: BFS finds the minimum without exploring every path.

Confused with	How to tell them apart
Graph DFS (Ch. 9)	DFS marks nodes visited *permanently* and visits each once. Backtracking *unmarks* on return, so the same node or element can appear on different paths.
Top-down DP	Memoised DFS caches results by state. Backtracking has no reusable results, because each path's output is distinct.
BFS on states	BFS suits "fewest moves" questions. Backtracking suits "all solutions" or "any solution with deep pruning".

8.3 Concepts and invariants

Choose → explore → undo. Maintain the invariant "*on entry to* `dfs`*, the shared state equals exactly the choices on the current root-to-node path*". Every mutation before the recursive call must be reversed after it. Copy the path into the output only when recording (`new ArrayList<>(path)`), because the path object keeps changing.

Combinations vs permutations.

	Combination / subset	Permutation
Order matters?	No: {1,2} = {2,1}	Yes
Canonical generation	A `start` index: only choose elements at positions ≥ start (increasing indices)	A `used[]` array: any unused element at every level
Reuse allowed (39)	Recurse with `i` instead of `i + 1`	Drop `used[]`
Leaves / outputs	2^n subsets, C(n, k) of size k	n! permutations
Cost	$O(n \cdot 2^n)$ including copying	$O(n \cdot n!)$

Duplicate avoidance with repeated values. Sort first. At one tree level (one loop over i), skip a value equal to its predecessor at that level:

```
Combinations (Subsets II, Combination Sum II):  if (i > start && a[i] == a[i-1])
continue;
Permutations (Permutations II):                 if (i > 0 && a[i] == a[i-1] &&
!used[i-1]) continue;
```

Meaning: among equal values, always pick them left to right. Choosing a later copy first would produce the same multiset or sequence again.

Pruning. A prune is valid when it is *sound*, meaning no solution lies in the cut subtree. Typical sound prunes: remaining target < 0 (with positive candidates), sorted candidates with `a[i] > remain` → `break` (every later candidate is larger), and conflict checks in O(1) using auxiliary arrays (N-Queens columns and diagonals). Pruning does not change the worst case for output-bound problems, but it often decides whether a constraint-satisfaction search finishes at all.

When the problem needs memoisation instead of plain backtracking

Backtracking explores *paths*. If two different paths reach the same *state* and the rest of the computation depends only on that state, the subtree below it is solved repeatedly. Example: "number of combinations of coins summing to T" has state (index, remaining), with only n · T distinct states, but exponentially many paths reach them.

- **Test**: can you write the function's result in terms of a few parameters only (index, remaining, mask), without the path contents? If so, memoise on those parameters.
- **Enumeration remains exponential** even with memoisation, because the output size is exponential. Memoisation pays off for *count / exists / best*.
- **Hybrid**: Word Break II memoises the list of sentences per suffix index; that still has exponential output, but avoids recomputing dead ends.

8.4 Reusable Java templates

Template A — combinations and subsets (start index, optional duplicate skip).

JAVA TEMPLATE CombinationBacktracking.java

```java
import java.util.*;

/** All subsets of a (may contain duplicates; each distinct multiset reported once). */
class CombinationBacktracking {
    static List<List<Integer>> subsetsWithDup(int[] a) {
        int[] s = a.clone();
        Arrays.sort(s);                                  // duplicates adjacent for skipping
        List<List<Integer>> out = new ArrayList<>();
        dfs(s, 0, new ArrayList<>(), out);
        return out;
    }

    private static void dfs(int[] a, int start, List<Integer> path,
                            List<List<Integer>> out) {
        // every node is a subset: record copy
        out.add(new ArrayList<>(path));
        for (int i = start; i < a.length; i++) {
            if (i > start && a[i] == a[i - 1]) continue; // same value at same level: skip
            path.add(a[i]);                              // choose
            dfs(a, i + 1, path, out);                    // explore (i + 1: no reuse)
            path.remove(path.size() - 1);                // undo
        }
    }
}
```

Template B — permutations (used[] array, duplicate-safe).

JAVA TEMPLATE PermutationBacktracking.java

```java
import java.util.*;

/** Distinct permutations of a (handles repeated values). */
class PermutationBacktracking {
    static List<List<Integer>> permuteUnique(int[] a) {
        int[] s = a.clone();
        Arrays.sort(s);
        List<List<Integer>> out = new ArrayList<>();
        dfs(s, new boolean[s.length], new ArrayList<>(), out);
        return out;
    }

    private static void dfs(int[] a, boolean[] used, List<Integer> path,
                            List<List<Integer>> out) {
        if (path.size() == a.length) { out.add(new ArrayList<>(path)); return; }
        for (int i = 0; i < a.length; i++) {
            if (used[i]) continue;
            // Equal values are used left to right: skip copy i while copy i-1 is unused.
```

```
            if (i > 0 && a[i] == a[i - 1] && !used[i - 1]) continue;
            used[i] = true; path.add(a[i]);
            dfs(a, used, path, out);
            path.remove(path.size() - 1); used[i] = false;
        }
    }
}
```

Template C — when only a count is needed: memoise the state, not the path.

JAVA TEMPLATE MemoizedCount.java

```
import java.util.*;

/** Number of multisets from coins (distinct positive values, unlimited reuse) that sum
 *  to target.
 *  Same choice tree as Combination Sum, but the result depends only on (i, remain). */
class MemoizedCount {
    static long countCombinations(int[] coins, int target) {
        long[][] memo = new long[coins.length + 1][target + 1];
        for (long[] row : memo) Arrays.fill(row, -1);
        return count(coins, 0, target, memo);
    }

    private static long count(int[] c, int i, int remain, long[][] memo) {
        if (remain == 0) return 1;
        if (i == c.length || remain < 0) return 0;
        if (memo[i][remain] >= 0) return memo[i][remain];
        long take = remain >= c[i] ? count(c, i, remain - c[i], memo) : 0;  // reuse coin i
        // move past coin i
        long skip = count(c, i + 1, remain, memo);
        return memo[i][remain] = take + skip;
    }
}
```

8.5 Complexity and pitfalls

Time	(number of tree nodes) × (work per node). Subsets $O(n \cdot 2^n)$; permutations $O(n \cdot n!)$; N-Queens $O(n!)$ upper bound with heavy pruning.
Auxiliary space	O(depth) recursion + path. The output is not counted, but it dominates memory in practice.
Off-by-one	Recursing with `start + 1` instead of `i + 1` (generates duplicates or misses some); recording before or after the base check inconsistently.
Wrong invariant	Forgetting to undo (state leaks into sibling branches); adding `path` itself instead of a copy (every output ends up as the final, empty path).
Overflow	Rarely relevant. Memoised counts can exceed `int`, so use `long` or the modulus the problem requires.
Breaks naive code	Duplicate input values (needs sorting + level skip); `break` pruning on unsorted candidates (unsound); a target of 0.
Interview follow-ups	Handle duplicates (Subsets II, Permutations II, Combination Sum II); only count solutions (memoise or DP); iterative generation (bitmask for subsets, next-permutation); estimate the number of nodes visited.

8.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Shared state must equal the current path on every entry: choose, explore, undo.
- A start index gives combinations; `used[]` gives permutations; recursing with `i` allows reuse.
- Sort plus a same-level skip removes duplicates without a set of results.
- Prune only with provable bounds; `break` requires sorted candidates.
- Count or optimise with repeated states → memoise (Template C) or bottom-up DP.

> **Recognition rule.** "Return all valid configurations" with small n → a choice tree with undo. "How many" with repeated states → memoised DFS.

IN-INTERVIEW CHECKLIST

- ☐ What is one decision per level, and what is the base case?
- ☐ Combination (start index) or permutation (`used[]`)? Reuse allowed?
- ☐ Duplicates in the input? Sort + level skip.
- ☐ Which prunes are sound?
- ☐ Am I copying the path when recording?

Key distinction. DFS on a graph visits each node once (visited persists). Backtracking revisits elements on different paths (state is undone). Using a permanent visited set in backtracking silently loses solutions.

8.7 Assigned problems

LC 78 Subsets — MEDIUM

Task. Return all subsets (the power set) of an array of distinct integers.

KEY INSIGHT

Generating subsets in increasing index order makes each one appear exactly once. Every node of the choice tree, not just the leaves, is an answer.

WHY THE PATTERN APPLIES

The output has 2^n elements with n ≤ 10, so enumeration is unavoidable. The start-index template avoids order-duplicates.

APPROACH

1. Record a copy of `path` on entry.
2. For each i ≥ start: choose `nums[i]`, recurse with `i + 1`, undo.

JAVA SOLUTION LeetCode 78 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, 0, new ArrayList<>(), res);
        return res;
```

```
    }

    private void backtrack(int[] nums, int start, List<Integer> path,
                           List<List<Integer>> res) {
        res.add(new ArrayList<>(path));            // each node of the tree is one subset
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            backtrack(nums, i + 1, path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Time $O(n \cdot 2^n)$ **Space** O(n) recursion + path (output excluded)

Pitfall. `res.add(path)` without copying returns 2^n references to the same, finally empty, list. The bitmask alternative (`for mask in 0..2^n-1`) is equally valid and iterative.

LC 46 Permutations

MEDIUM

Task. Return all permutations of an array of distinct integers.

KEY INSIGHT

At depth d, any not-yet-used element can occupy position d. The `used[]` array is the set of choices remaining on the current path.

WHY THE PATTERN APPLIES

n ≤ 6 gives n! outputs. Order matters, so use the permutation template instead of a start index.

APPROACH

1. When the path has length n, record a copy.
2. Otherwise try each unused index: mark, append, recurse, then unmark and remove.

JAVA SOLUTION LeetCode 46 · Solution.java

```
import java.util.*;

class Solution {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
        return res;
    }

    private void backtrack(int[] nums, boolean[] used, List<Integer> path,
                           List<List<Integer>> res) {
        if (path.size() == nums.length) {
            res.add(new ArrayList<>(path));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            used[i] = true;
            path.add(nums[i]);
            backtrack(nums, used, path, res);
            path.remove(path.size() - 1);          // undo in reverse order of doing
            used[i] = false;
        }
```

```
    }
}
```

Time O(n · n!) **Space** O(n) recursion + used[] (output excluded)

Pitfall. With duplicate values (LeetCode 47), this produces repeated permutations. Add sorting and the `!used[i-1]` skip from Template B. A swap-based in-place variant also exists, but its duplicate handling is harder to get right.

LC 39 Combination Sum

MEDIUM

Task. Given distinct positive candidates and a target, return all unique combinations (each candidate reusable without limit) that sum to the target.

KEY INSIGHT

Reuse is allowed, so recurse with the same index `i`. Uniqueness of combinations is still enforced by never going back to smaller indices.

WHY THE PATTERN APPLIES

We need every combination, which is a classic choice tree. With sorted candidates, `candidates[i] > remain` lets us `break`: a sound prune, because all later candidates are larger.

APPROACH

1. Sort. `dfs(start, remain)`: if `remain == 0`, record.
2. For i ≥ start: if `c[i] > remain`, break; otherwise choose, recurse with `(i, remain − c[i])`, undo.

JAVA SOLUTION LeetCode 39 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<Integer>> combinationSum(int[] candidates, int target) {
        Arrays.sort(candidates);                        // enables the break-prune below
        List<List<Integer>> res = new ArrayList<>();
        dfs(candidates, 0, target, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] c, int start, int remain, List<Integer> path,
                     List<List<Integer>> res) {
        if (remain == 0) {
            res.add(new ArrayList<>(path));
            return;
        }
        for (int i = start; i < c.length; i++) {
            // sorted: every later c[j] is larger too
            if (c[i] > remain) break;
            path.add(c[i]);
            dfs(c, i, remain - c[i], path, res);      // i, not i + 1: c[i] may be reused
            path.remove(path.size() - 1);
        }
    }
}
```

Time $O(N^{T/m + 1})$ worst case (T target, m min candidate) **Space** O(T/m) recursion depth

Pitfall. Looping from 0 instead of `start` produces [2,3] and [3,2] as separate answers. If the interviewer asks only for the *number* of combinations, switch to Template C or the coin-change DP. Exponential enumeration is then unnecessary.

LC 51 N-Queens

HARD

Task. Return all distinct placements of n non-attacking queens on an n × n board, as lists of strings.

KEY INSIGHT

Place exactly one queen per row, so rows never conflict. A column, a "\" diagonal (`r - c` constant) or a "/" diagonal (`r + c` constant) is each a single index into a boolean array, which makes every conflict check O(1).

WHY THE PATTERN APPLIES

This is constraint satisfaction with incremental checking. Most partial boards are rejected after a few rows, so the search tree is far smaller than n^n.

APPROACH

1. `dfs(row)`: for each column c, skip if `cols[c]`, `diag[row - c + n - 1]` or `anti[row + c]` is taken.
2. Place the queen (set three flags and `queenCol[row] = c`), recurse to `row + 1`, then clear the flags.
3. At `row == n`, render the board from `queenCol`.

JAVA SOLUTION LeetCode 51 · Solution.java

```java
import java.util.*;

class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        dfs(0, n, new int[n], new boolean[n], new boolean[2 * n - 1],
            new boolean[2 * n - 1], res);
        return res;
    }

    private void dfs(int row, int n, int[] queenCol, boolean[] cols, boolean[] diag,
                     boolean[] anti, List<List<String>> res) {
        if (row == n) {
            res.add(render(queenCol, n));
            return;
        }
        for (int c = 0; c < n; c++) {
            // shift r - c from [-(n-1), n-1] to [0, 2n-2]
            int d = row - c + n - 1;
            int a = row + c;                          // r + c in [0, 2n-2]
            if (cols[c] || diag[d] || anti[a]) continue;
            cols[c] = diag[d] = anti[a] = true;
            queenCol[row] = c;
            dfs(row + 1, n, queenCol, cols, diag, anti, res);
            cols[c] = diag[d] = anti[a] = false;      // undo
        }
    }

    private List<String> render(int[] queenCol, int n) {
        List<String> board = new ArrayList<>(n);
        for (int r = 0; r < n; r++) {
            char[] line = new char[n];
            Arrays.fill(line, '.');
            line[queenCol[r]] = 'Q';
            board.add(new String(line));
        }
        return board;
```

```
    }
}
```

Time O(n!) upper bound (+ $O(n^2)$ per solution to render) **Space** O(n) flags and recursion

Pitfall. The "\" diagonal index `r - c` is negative without the `+ n - 1` offset. Scanning the board to check attacks costs O(n) per placement instead of O(1). For N-Queens II (count only) the same search applies; bitmasks make it faster, but counting still needs the search, because states do not repeat in a way memoisation can exploit.

Recursion / DFS on Trees and Graphs

Solve a structure by solving its substructures: a node's answer is computed from its children's answers. On graphs, a visited set ensures each node is processed once.

9.1 Pattern identity

Primary purpose	Compute tree aggregates (height, sums, best paths), validate tree properties, explore connected components, and detect reachability in grids and graphs.
Typical input	A `TreeNode` root, a 2D grid (an implicit graph with 4- or 8-neighbours), an adjacency matrix or list.
Interview applications	Depth, balance, diameter, LCA, path sums, serialisation, number of islands, flood fill, provinces, cycle detection, bipartiteness, clone graph.
Key insight	Decide what the recursive call *returns* (information flowing up to the parent) separately from what it *records* (a global answer that may combine both children). Most hard tree problems are this distinction.

> **Key insight.** Define the function's contract in one sentence, for example "`gain(node)` = the best downward path sum starting at node". Then trust it for the children. Correctness follows by structural induction: if the contract holds for the children, the combination step makes it hold for the node.

9.2 Recognition signals

Signal	Structural reason
Answer for a tree = f(answers for its subtrees)	Trees are recursively defined, so postorder computes children first.
"Number of islands / regions / provinces / groups"	Count the connected components: one DFS launch per unvisited node.
Reachability, "can we get from A to B", flood a region	DFS visits the full component of the start node. Order and distances are irrelevant.
Path-shaped answers in trees (max path, diameter)	Every path has a unique highest node, so enumerate paths by their apex during the postorder.

> **Ask yourself.** "What must each child tell its parent, and does the final answer need more than that (a global)?" On graphs: "When do I mark visited, and can the recursion get too deep?"

When it is inappropriate. Use BFS for *fewest steps* in unweighted graphs, since DFS finds *a* path, not the shortest. Use DSU for connectivity under incremental edge additions with many queries. Avoid deep recursion on inputs that allow O(n) depth with n around 10^5–10^6 (a skewed tree, a snake-shaped grid); switch to iteration.

Confused with	How to tell them apart
Backtracking (Ch. 8)	DFS marks visited permanently: each node is processed once, O(V + E). Backtracking unmarks and explores paths, which is exponential.

Confused with	How to tell them apart
BFS (Ch. 10)	Both traverse components in O(V + E). Only BFS gives shortest unweighted distances; DFS gives pre/post order, which matters for topological order and tree DP.
Union-Find (Ch. 17)	For a static graph DFS is simplest. DSU is better when edges arrive over time or the question is only "same component?".

9.3 Concepts and invariants

Traversal order is a modelling choice.

Order	Work happens...	Use when
Preorder	Before the children; passes information *down* (depth, path so far, bounds)	Path sum from the root, BST validation with (lo, hi) bounds, serialisation
Inorder	Between the children	BSTs: visits keys in sorted order (k-th smallest, validate BST)
Postorder	After the children; aggregates information *up*	Height, diameter, max path sum, subtree sums, tree DP

Return value vs global state. For Binary Tree Maximum Path Sum the parent can only extend a *downward* path, one that uses at most one child, so that is what `gain()` returns. The best path through a node may use both children, but it cannot be extended upward, so it is recorded in a global maximum. The same split appears in diameter, longest univalue path and house robber III, where the returned value is a pair.

```
gain(v)  = v.val + max(0, gain(v.left), gain(v.right))        (returned: extendable)
best     = max over v of  v.val + max(0, gain(v.left)) + max(0, gain(v.right))    (recorded)
gain(null) = 0
```

Visitation rules on graphs. Mark a node visited *when you first reach it* (before or as you recurse), never after processing it. Otherwise two neighbours may both start on the same node, and cycles recurse forever. In grids you may mark in place (e.g. overwrite '1' with '0'), which saves memory but destroys the input. Say so, or copy the grid.

Connected components. The outer loop over all nodes launches one DFS per unvisited node. The number of launches equals the number of components, and each node and edge is touched O(1) times, so the total is O(V + E): $O(n^2)$ for an adjacency matrix, O(mn) for an m × n grid.

Recursion depth and iterative alternatives

Java's default thread stack (often 512 KB–1 MB) holds roughly 10^4–10^5 frames, depending on frame size. A skewed tree with 10^5 nodes, or a 300 × 300 grid of land in a snake pattern (90,000 cells), can therefore overflow with `StackOverflowError`.

- **Explicit stack** (`ArrayDeque`): push the start, pop, and push unvisited neighbours, marking them when pushed. This handles preorder-style processing directly.
- **Postorder iteratively**: push (node, state) pairs, or use two stacks (reverse of root-right-left preorder), or keep a "children done" flag. This is needed for tree DP without recursion.
- **BFS** is a drop-in replacement whenever order does not matter (counting components, flood fill).
- **Larger stack**: `new Thread(null, task, "dfs", 1 << 26).start()` is a valid engineering fix, but interviewers usually prefer the iterative form.

LeetCode's limits for 104, 200, 547 and 124 are within what recursive Java normally handles, so the solutions below are recursive and Template B shows the iterative grid version.

9.4 Reusable Java templates

Template A — tree DFS: return what the parent needs, record the global answer (illustrated with the tree diameter).

JAVA TEMPLATE TreeDfsTemplate.java (TreeNode provided by LeetCode)

```java
/** Postorder skeleton. height() returns information for the parent; best records answers
 *  that combine both children (here: diameter in edges). */
class TreeDfsTemplate {
    private int best = 0;

    int diameter(TreeNode root) {
        best = 0;
        height(root);
        return best;
    }

    /** Contract: number of nodes on the longest downward path from node (0 for null). */
    private int height(TreeNode node) {
        if (node == null) return 0;
        int l = height(node.left), r = height(node.right);   // children first (postorder)
        // path through node: both sides
        best = Math.max(best, l + r);
        // parent can extend one side only
        return 1 + Math.max(l, r);
    }
}
```

Template B — grid DFS, recursive and iterative. The iterative version is the safe default for very large grids.

JAVA TEMPLATE GridDfsTemplate.java

```java
import java.util.*;

/** Count 4-connected regions of cells equal to target. Visited tracked separately
 *  (input not mutated). */
class GridDfsTemplate {
    private static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    static int countRegions(char[][] g, char target, boolean iterative) {
        int m = g.length, n = g[0].length, regions = 0;
        boolean[][] seen = new boolean[m][n];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                if (g[i][j] == target && !seen[i][j]) {
                    regions++;
                    if (iterative) fillIterative(g, i, j, target, seen);
                    else fillRecursive(g, i, j, target, seen);
                }
        return regions;
    }

    private static void fillRecursive(char[][] g, int r, int c, char t, boolean[][] seen) {
        if (r < 0 || c < 0 || r >= g.length || c >= g[0].length) return;   // bounds first
        if (seen[r][c] || g[r][c] != t) return;
        seen[r][c] = true;                                                 // mark on entry
        for (int[] d : DIRS) fillRecursive(g, r + d[0], c + d[1], t, seen);
    }
```

```
    private static void fillIterative(char[][] g, int r, int c, char t, boolean[][] seen) {
        Deque<int[]> stack = new ArrayDeque<>();
        // mark when pushed
        seen[r][c] = true;
        stack.push(new int[]{r, c});
        while (!stack.isEmpty()) {
            int[] cur = stack.pop();
            for (int[] d : DIRS) {
                int nr = cur[0] + d[0], nc = cur[1] + d[1];
                if (nr < 0 || nc < 0 || nr >= g.length || nc >= g[0].length) continue;
                if (seen[nr][nc] || g[nr][nc] != t) continue;
                seen[nr][nc] = true;
                stack.push(new int[]{nr, nc});
            }
        }
    }
}
```

Template C — components of an undirected graph given as an edge list (adjacency list + recursive DFS).

JAVA TEMPLATE GraphComponents.java

```
import java.util.*;

/** Undirected graph with vertices 0..n-1. Returns comp[v] = component id. O(V + E). */
class GraphComponents {
    static int[] components(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) { adj.get(e[0]).add(e[1]); adj.get(e[1]).add(e[0]); }
        int[] comp = new int[n];
        Arrays.fill(comp, -1);                          // -1 = unvisited
        int id = 0;
        for (int v = 0; v < n; v++) if (comp[v] == -1) dfs(v, id++, adj, comp);
        return comp;
    }

    private static void dfs(int v, int id, List<List<Integer>> adj, int[] comp) {
        comp[v] = id;                                   // mark before visiting neighbours
        for (int w : adj.get(v)) if (comp[w] == -1) dfs(w, id, adj, comp);
    }
}
```

9.5 Complexity and pitfalls

Time	Trees O(n). Graphs O(V + E) with an adjacency list, O(V^2) with an adjacency matrix. Grids O(m · n).
Auxiliary space	Recursion O(h) for trees (h = n when skewed, log n when balanced); O(V) visited plus O(V) worst-case stack for graphs; O(mn) for grids.
Off-by-one	Bounds checks after array access (`g[r][c]` before `r >= 0`); counting depth in edges vs nodes (104 counts nodes).
Wrong invariant	Returning the two-sided path to the parent (124); marking visited after the recursion (infinite loops on cycles).
Overflow	Path sums with values up to 1000 and 3 · 10^4 nodes fit in `int`. Initialise the global max to `Integer.MIN_VALUE`, not 0, when all values may be negative.
Breaks naive code	An empty tree; all-negative values (124); a single-cell grid; self-loops in `isConnected` (diagonal = 1); a skewed tree that overflows the stack.

Interview follow-ups	Make it iterative; don't mutate the input; return the path itself (store parent pointers or the argmax); handle 8-directional grids; count island shapes (canonical path encoding).

9.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Write the function contract first, then trust it for the children (structural induction).
- Separate the returned value (extendable) from the recorded global (complete).
- Mark visited on first contact; one outer-loop launch per component.
- Know the recursion-depth risk and have the explicit-stack version ready.
- Pick pre, in or post order by the direction information flows.

> **Recognition rule.** The answer at a node is composed from its children, or the question is about components or reachability, so use DFS with a clear return contract and a visited set.

IN-INTERVIEW CHECKLIST

- ☐ What does the call return? Is there a separate global?
- ☐ What are the base cases (null, out of bounds, visited)?
- ☐ When is a node marked visited? Is mutating the input acceptable?
- ☐ What is the maximum recursion depth for the given constraints?
- ☐ Adjacency matrix or list, and what does that do to the complexity?

Key distinction. DFS gives connectivity and structural aggregates but not shortest distances. The moment the question says "minimum number of steps", reach for BFS (Ch. 10).

9.7 Assigned problems

LC 104 Maximum Depth of Binary Tree — EASY

Task. Return the number of nodes on the longest root-to-leaf path.

KEY INSIGHT

depth(node) = 1 + max(depth(left), depth(right)), with depth(null) = 0. This is the smallest example of a postorder aggregate.

WHY THE PATTERN APPLIES

The answer for a tree is a function of its subtrees' answers. Nothing flows downward and no global is needed.

APPROACH

1. Base case: null → 0.
2. Return 1 + the larger child depth. (BFS alternative: count levels.)

JAVA SOLUTION LeetCode 104 · Solution.java (TreeNode provided by LeetCode)

```java
class Solution {
    public int maxDepth(TreeNode root) {
        // empty subtree contributes 0 nodes
        if (root == null) return 0;
```

```
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

Time O(n) **Space** O(h) recursion; O(n) when skewed

Pitfall. Depth counts nodes, not edges, so a single node has depth 1. For minimum depth (LeetCode 111) the formula changes: a null child is not a leaf, so `min` must ignore missing children.

LC 200 Number of Islands

MEDIUM

Task. Count the 4-directionally connected groups of '1' cells in a grid of '1'/'0' characters.

KEY INSIGHT

Each island is one connected component of an implicit graph. Every unvisited land cell found by the outer scan starts a new island, and the DFS consumes the whole island so it is never counted twice.

WHY THE PATTERN APPLIES

This counts components without needing distances. DFS visits each cell O(1) times.

APPROACH

1. Scan cells. On a '1', increment the count and sink the island (DFS sets cells to '0').
2. DFS: bounds check, water check, mark, recurse on four neighbours.

JAVA SOLUTION LeetCode 200 · Solution.java

```
class Solution {
    public int numIslands(char[][] grid) {
        int islands = 0;
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == '1') {
                    islands++;
                    sink(grid, i, j);                // remove the whole island
                }
            }
        }
        return islands;
    }

    private void sink(char[][] g, int r, int c) {
        if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
        // mark visited in place (mutates input)
        g[r][c] = '0';
        sink(g, r + 1, c);
        sink(g, r - 1, c);
        sink(g, r, c + 1);
        sink(g, r, c - 1);
    }
}
```

Time O(m · n) **Space** O(m · n) worst-case recursion depth

Pitfall. This mutates the input. If that is not allowed, use a `boolean[][] seen` (Template B). An all-land 300 × 300 grid can reach depth around $9 \cdot 10^4$; use the iterative variant or BFS if the stack is tight.

LC 547 Number of Provinces

MEDIUM

Task. Given an n × n adjacency matrix `isConnected` of an undirected graph, return the number of connected components.

KEY INSIGHT

A province is a connected component. One DFS launch per unvisited city counts them. Because the input is a matrix, scanning a city's neighbours costs O(n), so the total is $O(n^2)$, which is optimal for this input format.

WHY THE PATTERN APPLIES

Static undirected connectivity is the core use case of DFS. The same problem appears in Chapter 3 (solved with BFS, as a classification note) and Chapter 17 (solved with Union-Find).

APPROACH

1. `visited[]` over cities. For each unvisited city: count++, then `dfs(city)`.
2. `dfs(i)`: mark i; for each j with `isConnected[i][j] == 1` and not visited, recurse.

JAVA SOLUTION LeetCode 547 · Solution.java

```java
class Solution {
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length, provinces = 0;
        boolean[] visited = new boolean[n];
        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                provinces++;                          // i starts a new component
                dfs(isConnected, i, visited);
            }
        }
        return provinces;
    }

    private void dfs(int[][] g, int i, boolean[] visited) {
        visited[i] = true;
        for (int j = 0; j < g.length; j++) {          // undirected: matrix is symmetric
            if (g[i][j] == 1 && !visited[j]) dfs(g, j, visited);
        }
    }
}
```

Time $O(n^2)$ **Space** O(n) visited + recursion

> **Pitfall.** The diagonal is always 1 (self-connection). It is harmless because `i` is already visited when scanned. Converting to an adjacency list first does not help: building it already costs $O(n^2)$.

LC 124 Binary Tree Maximum Path Sum

HARD

Task. A path is any sequence of adjacent nodes, each used at most once, with at least one node. Return the maximum path sum; values may be negative.

KEY INSIGHT

Every path has a unique highest node (its apex). At the apex the path is `val + left downward gain + right downward gain`, where a negative gain is replaced by 0 (don't take that branch). The parent, however, can only extend a single downward chain.

WHY THE PATTERN APPLIES

This is postorder tree DP with the return-versus-global split (see 9.3): `gain()` returns the one-sided chain and `best` records the two-sided path.

APPROACH

1. `gain(null) = 0`. Compute `l = max(0, gain(left))` and `r = max(0, gain(right))`.
2. `best = max(best, val + l + r)`.
3. Return `val + max(l, r)`.

JAVA SOLUTION LeetCode 124 · Solution.java (TreeNode provided by LeetCode)

```java
class Solution {
    private int best;

    public int maxPathSum(TreeNode root) {
        best = Integer.MIN_VALUE;                    // all values may be negative
        gain(root);
        return best;
    }

    /** Max sum of a downward path starting at node (may be just node); 0 for null. */
    private int gain(TreeNode node) {
        if (node == null) return 0;
        int l = Math.max(0, gain(node.left));        // drop a branch that only hurts
        int r = Math.max(0, gain(node.right));
        best = Math.max(best, node.val + l + r);     // path with apex at node
        return node.val + Math.max(l, r);            // extendable: one side only
    }
}
```

Time O(n) **Space** O(h) recursion

Pitfall. Initialising `best = 0` returns 0 for an all-negative tree such as [-3], but a path must contain at least one node, so the answer is −3. Returning `val + l + r` to the parent creates a "path" that forks, which is invalid. The instance field must be reset on each call if the object is reused.

BFS and Level Order: Shortest Path in Unweighted Graphs

Breadth-first search explores states in non-decreasing order of edge count. When every edge costs the same, the first time a state is reached is along a shortest path. Grids, word transformations and puzzle states are all graphs, even when no adjacency list is given.

10.1 Pattern identity

Primary purpose	Minimum number of steps (edges) from one or many sources; level-by-level processing; reachability with bounded memory per layer.
Typical input	Grid (4- or 8-directional moves), explicit adjacency list, or an implicit state space with a neighbour generator (words differing by one letter, lock combinations, board states).
Interview applications	Shortest path in a maze, spreading processes (rot, fire, infection), distance to nearest X, word ladder, tree level-order / right-side view, minimum moves in puzzles, bipartiteness check.
Key insight	A FIFO queue processes all states at distance d before any at distance d + 1. With unit weights, a state's first discovery fixes its shortest distance, so marking it visited at enqueue time is both correct and minimal.

> **Key insight.** BFS is Dijkstra with a priority queue that happens to be FIFO, which works only because every edge adds exactly 1. Change one weight and the guarantee disappears.

10.2 Recognition signals

Signal	Structural reason
"Minimum number of moves / steps / transformations"	Each move is one unweighted edge, so the answer is the BFS layer of the target.
"After how many minutes does everything …" / simultaneous spreading	All sources expand in lock-step, so this is a multi-source BFS whose level count is the elapsed time.
"Distance from each cell to the nearest X"	Reverse the direction: start from every X at once. One BFS gives all distances in O(cells).
States are generated by small local edits	An implicit graph: the neighbour generator replaces an adjacency list.
Level-wise output for trees	Snapshot `queue.size()` to delimit levels.

> **Ask yourself.** "Does every move cost the same, and do I need the *fewest* moves (or everything in layer order)?" If yes, BFS. If moves have different costs, it is not BFS.

When it is inappropriate.

- **Weighted edges** with different weights: use Dijkstra (Pattern 18). With weights in {0, 1}, use 0-1 BFS (a deque: push-front for 0, push-back for 1).
- **Pure reachability / component work** (flood fill, islands): BFS works but carries no extra benefit over DFS. Pick whichever avoids recursion-depth risk.
- **Counting all paths or enumerating them**: BFS finds distances, not all paths. You need backtracking or DP over the BFS DAG.
- **Huge branching with a known target**: consider bidirectional BFS, which reduces b^d to about $2 \cdot b^{d/2}$.

Confused with	How to tell them apart
DFS	DFS reaches every node too, but in an order unrelated to distance. Its first visit is not the shortest path. Use DFS for structure (components, cycles, subtree aggregates) and BFS for distance.
Dijkstra	Dijkstra generalises BFS to non-negative weights at a log-factor cost. On unit weights both give identical distances, so BFS is the simpler choice.
DP on a grid	If moves are restricted to right/down (a DAG), DP in row-major order is simpler. If moves go in all four directions (cycles), you need BFS.

10.3 Concepts and invariants

The layer invariant. At any moment the queue contains nodes from at most two consecutive layers, d and d + 1, with every layer-d node ahead of every layer-(d+1) node.

```
Claim: when node v is first enqueued from u, dist(v) = dist(u) + 1.
Proof sketch: nodes are dequeued in non-decreasing dist order (layer invariant).
  If a shorter path to v existed, its predecessor w would have dist(w) < dist(u)
  and was dequeued earlier, so it would already have enqueued v. Contradiction.
```

Visited-state timing. Mark a node visited *when it is enqueued*, not when it is dequeued. Marking at dequeue lets the same node be enqueued many times by different neighbours in the same layer. Distances stay correct, but the queue can grow to O(E), or exponentially in implicit graphs, and lead to TLE or MLE. The exception is weighted search (Dijkstra), where the distance is final only at dequeue time.

Level boundaries. Two equivalent techniques:

- **Size snapshot:** `for (int k = q.size(); k > 0; k--)` processes exactly one layer, and the loop counter is the level.
- **Distance array:** store `dist[v] = dist[u] + 1` at enqueue time; no layer loop is needed. This is more flexible (multi-source, and the visited test becomes `dist[v] == -1`).

Multi-source BFS. Enqueue all sources with distance 0 before starting. This is the same as adding a virtual super-source S with a 0-weight edge to each real source, then running BFS from S. The result is `dist[v] = min over sources s of dist(s, v)` in a single O(V + E) pass, compared with O(k · (V + E)) for k separate BFS runs.

Implicit graph modelling: how to design the state

A state must contain *everything that affects future moves*. In a grid with keys, the state is (row, col, keysMask). With a limit on wall breaks, it is (row, col, breaksLeft). The visited set is over states, not cells. The BFS cost is O(#states × branching), so estimate it before coding.

For word problems, the neighbour generator is either "all words at Hamming distance 1", generated by trying 26 letters at each position (O(L · 26) per word with a hash-set lookup), or pattern buckets like `h*t` that map to the words matching them. For dictionaries of size N with word length L, generation by letter substitution costs O(N · L · 26 · L) including string building and hashing.

10.4 Reusable Java templates

Template A — single/multi-source BFS on a grid with a distance array. Reusable: queue, directions, bounds check, enqueue-time marking. Problem-specific: what counts as passable, and which cells are sources.

JAVA TEMPLATE GridBfs.java

```java
import java.util.*;

/** Multi-source grid BFS: dist[r][c] = fewest 4-directional steps to the nearest source. */
class GridBfs {
    static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    /** passable[r][c] says whether a cell may be entered; sources are {r, c} pairs. */
    static int[][] distances(boolean[][] passable, List<int[]> sources) {
        int m = passable.length, n = passable[0].length;
        int[][] dist = new int[m][n];
        for (int[] row : dist) Arrays.fill(row, -1);     // -1 means "not yet discovered"
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int[] s : sources) {
            if (dist[s[0]][s[1]] == -1) {
                dist[s[0]][s[1]] = 0;                          // all sources are layer 0
                q.add(s);
            }
        }
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || r >= m || c < 0 || c >= n) continue;
                if (!passable[r][c] || dist[r][c] != -1) continue;
                dist[r][c] = dist[cur[0]][cur[1]] + 1;         // mark at ENQUEUE time
                q.add(new int[]{r, c});
            }
        }
        return dist;
    }
}
```

Template B — level-order BFS with a size snapshot (adjacency list or implicit graph).

JAVA TEMPLATE LevelBfs.java

```java
import java.util.*;

/** BFS with explicit levels. Unweighted graph; adj edges are directed as given. */
class LevelBfs {
    /** Returns the fewest edges from src to dst, or -1 if dst is unreachable. */
    static int shortest(List<List<Integer>> adj, int src, int dst) {
        if (src == dst) return 0;
        boolean[] seen = new boolean[adj.size()];
        ArrayDeque<Integer> q = new ArrayDeque<>();
        q.add(src);
        seen[src] = true;
        for (int level = 1; !q.isEmpty(); level++) {
            for (int k = q.size(); k > 0; k--) {               // exactly one layer
```

```
                int u = q.poll();
                for (int v : adj.get(u)) {
                    if (seen[v]) continue;
                    if (v == dst) return level;          // early exit on first discovery
                    seen[v] = true;
                    q.add(v);
                }
            }
        }
        return -1;
    }
}
```

10.5 Complexity and pitfalls

Time	O(V + E); grid O(m · n); implicit graph O(#states × cost of generating neighbours).
Auxiliary space	O(V) for visited plus the queue (the widest layer, up to O(V)).
Off-by-one	Is the answer the number of edges or the number of nodes on the path? Word Ladder counts words, which is edges + 1. Rotting Oranges counts minutes, which is the last layer index, not the number of layers.
Wrong invariant	Marking visited at dequeue (duplicates); mixing levels when multi-source cells are added mid-search.
Overflow	Encoding `r * n + c` fits easily; encoding states with masks can need `long`.
Breaks naive code	No sources (answer 0 or −1?), unreachable targets, source equal to target, a start cell that is itself blocked.
Interview follow-ups	Reconstruct the path (store parents); all shortest paths (BFS DAG + backtracking); bidirectional BFS; 0-1 BFS; obstacles you may remove (state expansion).

10.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- BFS distance is optimal only on unit (equal) weights.
- Mark visited when enqueuing; otherwise the queue can blow up.
- Multi-source BFS = one virtual super-source; seed all sources at distance 0.
- "Nearest X from every cell" → BFS outward from all X, not from every cell.
- Design the state to include everything that changes future moves.

> **Recognition rule.** Fewest equal-cost moves, or simultaneous spreading → BFS (multi-source when there are many starts).

IN-INTERVIEW CHECKLIST

- ☐ What is a node (state), and how are neighbours generated?
- ☐ Are all edges the same cost?
- ☐ Single or multi-source? Should I reverse the direction?
- ☐ Do I need layer counts (size snapshot) or per-node distances (dist array)?
- ☐ Edges vs nodes in the answer; unreachable case.

Key distinction. DFS answers "is it connected / what is the structure?"; BFS answers "how far?". Only BFS's first discovery is a shortest path.

10.7 Assigned problems

LC 733 Flood Fill

EASY

Task. Recolour the 4-connected region of pixels sharing the start pixel's colour with `color`.

Classification note. This is connected-component traversal, not shortest path: no distances are needed, and DFS is equally valid. It is solved with BFS here to practise the queue mechanics and avoid recursion depth on large single-colour images.

KEY INSIGHT

The region is the connected component of the start pixel under the "same original colour" relation. Overwriting the colour can serve as the visited mark, but only when the new colour differs from the old one.

WHY THE PATTERN APPLIES

BFS enumerates a component in O(component size), with the queue replacing the recursion stack.

APPROACH

1. If `image[sr][sc] == color`, return immediately; otherwise recoloured cells would be indistinguishable from unvisited ones and the BFS would loop forever.
2. Recolour the start pixel, enqueue it, and on each pop recolour and enqueue the 4-neighbours that still have the original colour.

JAVA SOLUTION LeetCode 733 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int color) {
        int original = image[sr][sc];
        if (original == color) return image;      // also prevents infinite re-visiting
        int m = image.length, n = image[0].length;
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        ArrayDeque<int[]> q = new ArrayDeque<>();
        image[sr][sc] = color;                    // recolouring doubles as the visited mark
        q.add(new int[]{sr, sc});
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r >= 0 && r < m && c >= 0 && c < n && image[r][c] == original) {
                    image[r][c] = color;          // mark at enqueue time
                    q.add(new int[]{r, c});
                }
            }
        }
        return image;
    }
}
```

Time O(m · n) **Space** O(m · n) queue in the worst case

Pitfall. Leaving out the `original == color` guard causes an infinite loop in the BFS version, or a stack overflow in the DFS version.

LC 994 Rotting Oranges

MEDIUM

Task. Each minute, rotten oranges (2) rot their fresh 4-neighbours (1). Return the minutes until no fresh orange remains, or −1 if that is impossible.

KEY INSIGHT

All rotten oranges spread simultaneously, so the time at which a fresh orange rots is its BFS distance to the *nearest* initially rotten orange. The answer is the maximum such distance.

WHY THE PATTERN APPLIES

This is a multi-source BFS: seed every initial 2 at minute 0. The number of completed layers after the first is the elapsed time.

APPROACH

1. Enqueue all rotten cells and count the fresh ones.
2. Process layer by layer; each newly rotted orange decrements `fresh`. Count a minute only when a layer actually rots something.
3. Return `fresh == 0 ? minutes : -1`.

JAVA SOLUTION LeetCode 994 · Solution.java

```java
import java.util.*;

class Solution {
    public int orangesRotting(int[][] grid) {
        int m = grid.length, n = grid[0].length, fresh = 0;
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 2) q.add(new int[]{r, c});   // every source at minute 0
                else if (grid[r][c] == 1) fresh++;
            }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        int minutes = 0;
        while (!q.isEmpty() && fresh > 0) {        // stop once nothing is left to rot
            minutes++;
            for (int k = q.size(); k > 0; k--) {   // one minute = one BFS layer
                int[] cur = q.poll();
                for (int[] d : dirs) {
                    int r = cur[0] + d[0], c = cur[1] + d[1];
                    if (r < 0 || r >= m || c < 0 || c >= n || grid[r][c] != 1) continue;
                    grid[r][c] = 2;                // mark at enqueue time
                    fresh--;
                    q.add(new int[]{r, c});
                }
            }
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

Time O(m · n) **Space** O(m · n)

Pitfall. Over-counting by one minute: the final layer rots nothing new but still increments the counter. The `fresh > 0` loop guard prevents that. With no fresh oranges at all, the answer is 0 even if there are no rotten ones.

LC 542 01 Matrix

MEDIUM

Task. For each cell of a binary matrix, return the distance (4-directional steps) to the nearest 0. At least one 0 exists.

KEY INSIGHT

Reverse the question: rather than searching outward from every 1 ($O((mn)^2)$), search outward from *all zeros at once*. Each cell is discovered first from its nearest zero.

WHY THE PATTERN APPLIES

This is multi-source BFS with a distance array (Template A). Distances to the nearest member of a set are exactly what the super-source construction computes.

APPROACH

1. Set `dist = 0` for zeros and enqueue them; set `dist = -1` for ones (undiscovered).
2. Run BFS. The first time a 1 is reached, its distance is the predecessor's + 1.

JAVA SOLUTION LeetCode 542 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] updateMatrix(int[][] mat) {
        int m = mat.length, n = mat[0].length;
        int[][] dist = new int[m][n];
        ArrayDeque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                if (mat[r][c] == 0) q.add(new int[]{r, c});   // all zeros are sources
                else dist[r][c] = -1;                          // undiscovered
            }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || r >= m || c < 0 || c >= n || dist[r][c] != -1) continue;
                dist[r][c] = dist[cur[0]][cur[1]] + 1;
                q.add(new int[]{r, c});
            }
        }
        return dist;
    }
}
```

Time $O(m \cdot n)$ **Space** $O(m \cdot n)$ (queue; the result matrix is required output)

Pitfall. A BFS per 1-cell gives the right answer but runs in $O((mn)^2)$. An alternative is a two-pass DP (top-left then bottom-right sweeps, each taking the min over two neighbours). It works because a shortest 4-directional path can be split into monotone pieces handled by the two sweeps.

LC 127 Word Ladder

HARD

Task. Return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, where each step changes one letter and every intermediate word is in `wordList`. Return 0 if none exists.

KEY INSIGHT

Words are nodes of an implicit unweighted graph; edges connect words at Hamming distance 1. Generate neighbours by substituting each of the 26 letters at each position and checking a hash set. This is O(L · 26) per word, instead of comparing against all N words.

WHY THE PATTERN APPLIES

The fewest unit-cost transformations is a BFS distance. The answer counts *words*, so it is the edge count + 1.

APPROACH

1. Put the dictionary in a set; if `endWord` is absent, return 0.
2. BFS level by level from `beginWord`. When a neighbour is found in the set, remove it from the set; removal is the enqueue-time visited mark.
3. Return `level + 1` on generating `endWord`.

JAVA SOLUTION LeetCode 127 · Solution.java

```java
import java.util.*;

class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> unvisited = new HashSet<>(wordList);
        if (!unvisited.contains(endWord)) return 0;
        ArrayDeque<String> q = new ArrayDeque<>();
        q.add(beginWord);
        unvisited.remove(beginWord);
        for (int words = 1; !q.isEmpty(); words++) {     // words = sequence length so far
            for (int k = q.size(); k > 0; k--) {
                char[] w = q.poll().toCharArray();
                for (int i = 0; i < w.length; i++) {
                    char orig = w[i];
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        if (ch == orig) continue;
                        w[i] = ch;
                        String next = new String(w);
                        if (!unvisited.remove(next)) continue;  // not in dict, or seen
                        if (next.equals(endWord)) return words + 1;
                        q.add(next);
                    }
                    w[i] = orig;                                // undo the substitution
                }
            }
        }
        return 0;
    }
}
```

Time O(N · L^2 · 26): N words, each with L positions × 26 letters, and each candidate costs O(L) to build and hash

Space O(N · L)

Pitfall. Off by one: the answer counts words, so a one-step ladder returns 2. `beginWord` need not be in the list. Follow-up: bidirectional BFS, always expanding the smaller frontier, cuts the explored states sharply. Word Ladder II (126) needs a parent map built from the BFS DAG.

Topological Sort: DFS / Kahn's Algorithm

A topological order lists the vertices of a directed graph so that every edge points forward. It exists exactly when the graph is acyclic, so every topological-sort algorithm is also a directed-cycle detector.

11.1 Pattern identity

Primary purpose	Order tasks under "u must come before v" constraints, decide whether such an order exists, and run DP over a DAG in dependency order.
Typical input	n items plus a list of directed pairs (prerequisites, build dependencies, precedence derived from sorted data).
Interview applications	Course schedules, build systems and package managers, task scheduling with dependencies, alien dictionary, longest path in a DAG, detecting deadlock or terminal (safe) nodes.
Key insight	A vertex with no remaining incoming edges can be placed next; removing it may free others (Kahn). Equivalently, in DFS a vertex finishes only after everything reachable from it, so reverse finishing order is topological.

> **Key insight.** Ordering is possible if and only if there is no directed cycle. Kahn's algorithm detects a cycle when it emits fewer than n vertices; DFS detects one when it meets a *gray* (in-progress) vertex.

11.2 Recognition signals

Signal	Structural reason
"Prerequisite", "depends on", "must happen before"	Each constraint is a directed edge; the question is about a linear extension of a partial order.
"Is it possible to finish all ..."	Existence of a topological order is equivalent to the graph being acyclic.
Order must be inferred from sorted examples	Adjacent pairs in sorted data reveal precedence edges (alien dictionary).
Nodes whose every path ends in a terminal / never reaches a cycle	Topological processing on the *reversed* graph peels off terminal nodes.
Longest or shortest path / DP on a DAG	Processing in topological order guarantees predecessors are final before use.

> **Ask yourself.** "Is this a *directed* relation where something must precede something else, and do I need an order or a cycle verdict?"

When it is inappropriate.

- **Cyclic dependency structures**: a topological order does not exist. The algorithms can only report the cycle. Strongly connected components (Tarjan/Kosaraju) are needed to condense cycles into a DAG first.
- **Undirected graphs**: "ordering" is meaningless. Undirected cycle detection uses DSU or DFS with a parent check, not indegrees.

- **Weighted shortest paths on general graphs**: use Dijkstra. Topological-order relaxation works only on DAGs, though there it handles negative weights in O(V + E).

Confused with	How to tell them apart
BFS (Pattern 10)	Kahn's algorithm uses a queue but processes vertices when their *indegree reaches zero*, not when first discovered. It computes no distances.
Plain DFS visited set	Two states (visited / not) cannot tell a back edge (cycle) from a cross edge to a finished vertex. Three colours are required in directed graphs.
Union-Find	DSU ignores direction. It cannot detect directed cycles or produce orders.

11.3 Concepts and invariants

Kahn's invariant. The queue holds exactly the vertices whose indegree has dropped to 0 and which have not yet been emitted. Every emitted vertex has had all its predecessors emitted before it.

```
indeg(v) = number of edges (u → v) with u not yet emitted
Emit u  ⇒  for each u → v: indeg(v)--, and enqueue v when indeg(v) becomes 0

Termination: emitted == n  ⇔  graph is a DAG
If a cycle exists, no vertex on it can ever reach indegree 0 (each has a predecessor on
the cycle).
```

DFS colouring. White = unvisited, gray = on the current recursion stack, black = finished (all descendants explored).

- An edge to a **gray** vertex is a back edge, which proves a directed cycle.
- An edge to a **black** vertex is safe: that vertex's subtree is complete and acyclic.
- When a vertex turns black, every vertex reachable from it is already black. Appending on finish gives *reverse* topological order, so reverse the list (or prepend).

Why reverse postorder is a valid topological order

Take any edge u → v. When DFS explores this edge, v is either white (then v is visited and finished inside u's call, so finish(v) < finish(u)), black (already finished, so finish(v) < finish(u)), or gray (a back edge, meaning a cycle, which a DAG excludes). In a DAG, therefore, finish(v) < finish(u) for every edge, and sorting by decreasing finish time puts u before v.

Choosing between them. Kahn is iterative (no recursion-depth risk), naturally gives a lexicographically smallest order with a priority queue, and supports "process in parallel rounds" questions (the number of BFS layers is the minimum number of semesters). DFS is shorter when you already have a recursive traversal, and its colour states answer "which nodes can reach a cycle?" (Problem 802).

Alien dictionary: special constraints

- Only the **first differing character** of two adjacent words gives information; later characters are unconstrained.
- If word A is a proper extension of word B but appears *before* B (e.g. "abc" before "ab"), no ordering is valid, so return "".

- Comparing only *adjacent* pairs is sufficient: sorted order is transitive, so non-adjacent constraints are implied.
- Every letter that appears must be output, including letters with no constraints.
- Several valid orders may exist; any one is accepted.

11.4 Reusable Java templates

Template A — Kahn's algorithm (BFS on indegrees). Directed graph: `edges[i] = {u, v}` means u → v (u before v). Returns an order, or an empty array if there is a cycle.

JAVA TEMPLATE KahnTopoSort.java

```java
import java.util.*;

/** Kahn's algorithm. Directed edges u -> v mean "u must come before v". */
class KahnTopoSort {
    static int[] order(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[] indeg = new int[n];
        for (int[] e : edges) {
            adj.get(e[0]).add(e[1]);
            indeg[e[1]]++;
        }
        ArrayDeque<Integer> q = new ArrayDeque<>(); // PriorityQueue -> lexicographic
        for (int v = 0; v < n; v++) if (indeg[v] == 0) q.add(v);
        int[] out = new int[n];
        int k = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            out[k++] = u;                          // all predecessors already emitted
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.add(v);
        }
        return k == n ? out : new int[0];        // k < n  <=>  a directed cycle exists
    }
}
```

Template B — DFS with three colours and reverse postorder. Iterative alternatives exist, but recursion is the interview norm. Note that the depth can reach V.

JAVA TEMPLATE DfsTopoSort.java

```java
import java.util.*;

/** DFS topological sort with WHITE/GRAY/BLACK colouring. Directed edges u -> v. */
class DfsTopoSort {
    static final int WHITE = 0, GRAY = 1, BLACK = 2;

    static List<Integer> order(int n, int[][] edges) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) adj.get(e[0]).add(e[1]);
        int[] color = new int[n];
        List<Integer> post = new ArrayList<>();
        for (int v = 0; v < n; v++)                       // cover every component
            if (color[v] == WHITE && !dfs(v, adj, color, post)) return new ArrayList<>();
        Collections.reverse(post);                        // reverse postorder = topological
        return post;
    }
```

```
    /** Returns false if a directed cycle is reachable from u. */
    static boolean dfs(int u, List<List<Integer>> adj, int[] color, List<Integer> post) {
        color[u] = GRAY;                                    // u is on the recursion stack
        for (int v : adj.get(u)) {
            if (color[v] == GRAY) return false;             // back edge: cycle
            if (color[v] == WHITE && !dfs(v, adj, color, post)) return false;
        }
        color[u] = BLACK;                                   // everything reachable is finished
        post.add(u);
        return true;
    }
}
```

11.5 Complexity and pitfalls

Time	O(V + E) for both algorithms; O(V log V + E) for the lexicographic Kahn variant with a heap.
Auxiliary space	O(V + E) for the adjacency list; O(V) for indegrees or colours; DFS recursion O(V) in the worst case.
Edge direction	LeetCode 207/210 give `[a, b]` meaning "to take a you must first take b", so the edge is **b** → **a**. Reversing it silently yields a reversed order (accepted for 207, wrong for 210).
Wrong invariant	A two-state visited array in directed DFS reports false cycles (cross edges) or misses real ones (if visited is reset incorrectly).
Breaks naive code	Disconnected components (start DFS from every vertex / seed every indegree-0 vertex); duplicate edges (each must increment indegree, which is fine if counted consistently); self-loops (immediate cycle).
Interview follow-ups	Lexicographically smallest order (heap); minimum number of semesters (count Kahn layers); is the order unique (queue size always 1); longest path in a DAG; parallel scheduling with durations.

11.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Topological order exists ⇔ the directed graph is acyclic.
- Kahn: emitted count < n ⇒ cycle. DFS: an edge to a gray vertex ⇒ cycle.
- DFS order is *reverse* postorder; forgetting to reverse is the classic bug.
- Get edge direction right from the problem's wording before writing code.
- Reverse the graph to ask "which nodes only lead to terminals?".

> **Recognition rule.** Directed "before/after" constraints + "find an order" or "is it possible" → topological sort (and its built-in cycle check).

IN-INTERVIEW CHECKLIST

- ☐ Which way do the edges point?
- ☐ Kahn (iterative, layers, lexicographic) or DFS (colour states)?
- ☐ Are all vertices covered, including isolated ones?
- ☐ What is returned for a cycle: false, empty array, or ""?
- ☐ Are there duplicate edges or self-loops?

Key distinction. Kahn's queue releases a node when its *last* dependency is satisfied; BFS releases a node when its *first* neighbour is seen.

11.7 Assigned problems

LC 210 Course Schedule II

MEDIUM

Task. Given `numCourses` and pairs `[a, b]` ("take b before a"), return any valid course order, or an empty array if impossible.

KEY INSIGHT

The required output is literally a topological order of the graph with edges b → a, and "impossible" means it contains a directed cycle.

WHY THE PATTERN APPLIES

Kahn's algorithm produces the order and detects the cycle in one pass (Template A).

APPROACH

1. Build adjacency b → a and indegrees.
2. Seed all indegree-0 courses; pop, append, decrement successors.
3. Return the order if all courses were emitted, otherwise `new int[0]`.

JAVA SOLUTION LeetCode 210 · Solution.java

```java
import java.util.*;

class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p : prerequisites) {        // [a, b]: b must precede a  =>  edge b -> a
            adj.get(p[1]).add(p[0]);
            indeg[p[0]]++;
        }
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int c = 0; c < numCourses; c++) if (indeg[c] == 0) q.add(c);
        int[] order = new int[numCourses];
        int k = 0;
        while (!q.isEmpty()) {
            int u = q.poll();
            order[k++] = u;
            for (int v : adj.get(u)) if (--indeg[v] == 0) q.add(v);
        }
        return k == numCourses ? order : new int[0];
    }
}
```

Time O(V + E) **Space** O(V + E)

Pitfall. Building edges as a → b produces a reversed order, which fails. Courses with no prerequisites must still appear, and seeding every indegree-0 vertex handles that.

LC 207 Course Schedule

MEDIUM

Task. Return whether all courses can be finished given the same `[a, b]` prerequisite pairs.

KEY INSIGHT

Only a yes/no verdict is needed: "can finish" ⇔ no directed cycle. DFS colouring answers it without building an order.

WHY THE PATTERN APPLIES

This is the cycle-detection half of topological sort. It is solved here with Template B (the DFS variant) for contrast with 210. Direction does not affect the verdict, because reversing all edges preserves cycles.

APPROACH

1. Build adjacency.
2. DFS from every white vertex; reaching a gray vertex means a cycle, so return false.
3. Black vertices are never re-explored, so the total work is O(V + E).

JAVA SOLUTION LeetCode 207 · Solution.java

```java
import java.util.*;

class Solution {
    private List<List<Integer>> adj;
    private int[] color;          // 0 = white, 1 = gray (on stack), 2 = black (done)

    public boolean canFinish(int numCourses, int[][] prerequisites) {
        adj = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
        for (int[] p : prerequisites) adj.get(p[1]).add(p[0]);   // b -> a
        color = new int[numCourses];
        for (int v = 0; v < numCourses; v++)
            if (color[v] == 0 && hasCycle(v)) return false;
        return true;
    }

    private boolean hasCycle(int u) {
        color[u] = 1;
        for (int v : adj.get(u)) {
            if (color[v] == 1) return true;                      // back edge
            if (color[v] == 0 && hasCycle(v)) return true;
        }
        color[u] = 2;
        return false;
    }
}
```

Time O(V + E) **Space** O(V + E), including O(V) recursion depth

> **Pitfall.** Using a boolean `visited` that is never "un-grayed" flags a cross edge to an already-finished course as a cycle. Example: edges 0→1, 2→1, where DFS from 2 sees 1 already visited. The black state distinguishes the two.

LC 802 Find Eventual Safe States

MEDIUM

Task. In a directed graph (`graph[i]` lists i's successors), a node is *safe* if every path from it ends at a terminal node (out-degree 0). Return all safe nodes in ascending order.

KEY INSIGHT

A node is safe ⇔ it cannot reach a cycle. Terminal nodes are safe; a node becomes safe once *all* its successors are known safe. That is Kahn's algorithm on the **reversed** graph, driven by out-degrees.

WHY THE PATTERN APPLIES

Peeling off nodes whose remaining out-degree is 0 is a topological sort from the sinks backwards. Nodes on or leading into cycles never reach out-degree 0. (The DFS alternative: a node is safe ⇔ its DFS finishes black without meeting gray.)

APPROACH

1. Build reverse edges v → u for each u → v, and `outdeg[u] = graph[u].length`.
2. Queue all terminals. Popping v marks it safe and decrements `outdeg` of each predecessor u; enqueue u when it hits 0.
3. Collect safe nodes by scanning 0..n−1 (ascending without sorting).

JAVA SOLUTION LeetCode 802 · Solution.java

```java
import java.util.*;

class Solution {
    public List<Integer> eventualSafeNodes(int[][] graph) {
        int n = graph.length;
        List<List<Integer>> rev = new ArrayList<>();
        for (int i = 0; i < n; i++) rev.add(new ArrayList<>());
        int[] outdeg = new int[n];
        for (int u = 0; u < n; u++) {
            outdeg[u] = graph[u].length;
            for (int v : graph[u]) rev.get(v).add(u);     // reversed edge v -> u
        }
        ArrayDeque<Integer> q = new ArrayDeque<>();
        for (int u = 0; u < n; u++) if (outdeg[u] == 0) q.add(u);   // terminals are safe
        boolean[] safe = new boolean[n];
        while (!q.isEmpty()) {
            int v = q.poll();
            safe[v] = true;
            for (int u : rev.get(v)) if (--outdeg[u] == 0) q.add(u); // all successors safe
        }
        List<Integer> res = new ArrayList<>();
        for (int u = 0; u < n; u++) if (safe[u]) res.add(u);
        return res;
    }
}
```

Time O(V + E) **Space** O(V + E)

> **Pitfall.** A node pointing to itself (a self-loop) is unsafe; its out-degree never reaches 0, so this is handled automatically. The recursive DFS version can hit about 10^4 depth, while the Kahn version is iterative.

LC 269 Alien Dictionary

HARD Premium

Task. Given words sorted lexicographically in an unknown alphabet, return a string of all letters that appear, in any order consistent with the sorting. Return "" if no valid order exists.

KEY INSIGHT

Each adjacent pair of words gives at most one edge: at the first differing position, `w1[j] → w2[j]`. A word followed by its own proper prefix is a contradiction no alphabet can fix.

WHY THE PATTERN APPLIES

The inferred precedences form a directed graph on letters. A valid alphabet is any topological order, and a cycle means the input is inconsistent.

APPROACH

1. Register every letter that appears, with indegree 0.
2. For each adjacent pair, find the first mismatch and add that edge, deduplicated so indegrees stay consistent. If there is no mismatch and `w1` is longer, return "".
3. Run Kahn over the present letters. If fewer letters are emitted than are present, return "".

JAVA SOLUTION LeetCode 269 · Solution.java

```java
import java.util.*;

class Solution {
    public String alienOrder(String[] words) {
        final int A = 26;
        boolean[] present = new boolean[A];
        boolean[][] edge = new boolean[A][A];          // dedupes repeated constraints
        int[] indeg = new int[A];
        for (String w : words) for (char c : w.toCharArray()) present[c - 'a'] = true;

        for (int i = 0; i + 1 < words.length; i++) {
            String w1 = words[i], w2 = words[i + 1];
            int len = Math.min(w1.length(), w2.length()), j = 0;
            while (j < len && w1.charAt(j) == w2.charAt(j)) j++;
            if (j == len) {
                if (w1.length() > w2.length()) return "";   // "abc" before "ab": invalid
                continue;                                     // prefix in order: no info
            }
            int u = w1.charAt(j) - 'a', v = w2.charAt(j) - 'a';
            if (!edge[u][v]) {                        // only the first mismatch counts
                edge[u][v] = true;
                indeg[v]++;
            }
        }

        ArrayDeque<Integer> q = new ArrayDeque<>();
        int letters = 0;
        for (int c = 0; c < A; c++) {
            if (!present[c]) continue;
            letters++;
            if (indeg[c] == 0) q.add(c);
        }
        StringBuilder sb = new StringBuilder();
        while (!q.isEmpty()) {
            int u = q.poll();
            sb.append((char) ('a' + u));
            for (int v = 0; v < A; v++)
                if (edge[u][v] && --indeg[v] == 0) q.add(v);
        }
        return sb.length() == letters ? sb.toString() : "";  // cycle => inconsistent
    }
}
```

Time $O(C + 26^2)$, where C is the total number of characters **Space** $O(26^2) = O(1)$ alphabet-bounded

Pitfall. Three classic bugs: (1) comparing beyond the first mismatch; (2) missing the invalid-prefix check; (3) counting duplicate edges twice in the indegree without adding them twice to the adjacency, or the reverse. Either inconsistency corrupts Kahn's count.

Dynamic Programming: 1D

One-dimensional DP indexes subproblems by a single position (a prefix, a suffix or an amount). The hard part is not the loop but the *state definition*: a sentence describing exactly what `dp[i]` means, which makes the recurrence almost write itself.

12.1 Pattern identity

Primary purpose	Count ways or optimise over a sequence of decisions where each decision's future depends only on a single index (or one scalar resource).
Typical input	An array or string processed left to right (or right to left); an integer target or amount; a set of items with a single capacity.
Interview applications	Stairs / tiling, house robber, decode ways, word break, coin change, LIS ($O(n^2)$ form), string partitioning with a validity rule, 0/1 and unbounded knapsack (compressed).
Key insight	If the best (or the number of ways) for position i can be built from a *bounded* set of smaller positions, and those answers do not depend on how they were reached, then each subproblem is solved once and the exponential recursion tree collapses to O(n · transitions).

> **Key insight.** A correct state answers "what does the rest of the problem need to know about the past?". If two different histories with the same `i` can lead to different futures, the state is missing a dimension.

12.2 Recognition signals

Signal	Structural reason
"Number of ways" (often mod 10^9+7)	Counting overlapping decompositions: the sum rule over disjoint last choices.
"Maximum / minimum total" over choices with local constraints (no two adjacent)	The optimal substructure: best(i) = best over last choice of (best(earlier) + gain).
A brute-force recursion that recomputes the same arguments	Overlapping subproblems; memoise the arguments, and they become the state.
Splitting a string/array into valid pieces	dp over prefix (or suffix) length; transition = length of the last (or first) piece.
Pick a subset hitting a sum, small total	Pseudo-polynomial knapsack: state = (item, sum), compressed to sum.

> **Ask yourself.** "If I knew the answers for all smaller indices, could I compute this one with a constant or small number of choices, and does the answer for index j depend only on j?"

When it is inappropriate.

- **A greedy choice is provably safe** (for example, interval scheduling by earliest end): DP is correct but slower and wordier.
- **The state needs two indices** (two strings, a grid, a range [i, j]): that is Pattern 13 or interval DP. Forcing 1D gives wrong answers.

- **All solutions must be listed**: DP counts them, but listing requires backtracking (possibly guided by DP).
- **Values are huge** (sum up to 10^9): a sum-indexed DP array is infeasible, so look for another structure.

Confused with	How to tell them apart
Greedy	Greedy commits to one choice per step; DP keeps the best over all choices. A counter-example to greedy (coin denominations {1, 3, 4}, amount 6) means DP.
Backtracking	Same recursion tree; DP adds memoisation keyed by a small state. If the "path so far" cannot be summarised by a small key, memoisation does not help.
Prefix sums	Prefix sums aggregate a fixed function; DP chooses among options at each position.

12.3 Concepts and invariants

The four parts of a DP. Write each down before coding:

1. **State:** a precise sentence, e.g. "`dp[i]` = number of ways to split the suffix `s[i..n−1]`".
2. **Transition:** case analysis on the *last* (or first) decision. The cases must be disjoint for counting and exhaustive for both counting and optimisation.
3. **Base case:** the empty prefix or suffix. In counting DP the empty problem has exactly **1** way, not 0.
4. **Order:** compute every dependency before its use (increasing i for prefix states, decreasing i for suffix states).

```
Counting   :  dp[i] = Σ over disjoint last choices c   dp[prev(i, c)]        (sum rule)
Optimising :  dp[i] = max over last choices c  ( dp[prev(i, c)] + gain(c) )
House Robber: dp[i] = max(dp[i−1], dp[i−2] + a[i−1])    over prefix length i
               (skip house i−1)   (rob house i−1, so house i−2 must be skipped)
```

State dependencies and space optimisation. If `dp[i]` reads only `dp[i−1]` and `dp[i−2]`, keep two variables: O(1) space. If it reads a window of size w (a piece of at most w digits), keep a circular buffer of size w + 1. The rolling update must read the old values before overwriting them.

Counting vs optimisation. Counting needs *disjoint* cases (no double counting) and a modulus. Optimisation tolerates overlapping cases (max is idempotent) but needs a sentinel such as `Integer.MIN_VALUE / 2` or $-\infty$ for impossible states, so that adding to it does not overflow.

When a 1D array is really a compressed 2D state (knapsack)

In 0/1 knapsack the true state is `dp[i][s]` = ways using the first i items to reach sum s:

```
dp[i][s] = dp[i−1][s] + dp[i−1][s − w_i]
```

Row i reads only row i−1, so one array suffices, *if* `dp[s − w_i]` still holds the row-(i−1) value when `dp[s]` is updated. Iterating s in **decreasing** order guarantees this, because smaller indices have not yet been overwritten. Iterating in increasing order lets item i be used again: that is the *unbounded* knapsack recurrence `dp[i][s] = dp[i−1][s] + dp[i][s − w_i]`. Loop direction therefore encodes the problem's semantics.

Top-down vs bottom-up. Memoised recursion is quicker to derive and visits only reachable states, but costs O(depth) stack. Bottom-up allows space compression and avoids stack overflow. Interviewers accept either, as long as you can state the complexity as (#states × transition cost).

12.4 Reusable Java templates

Template A — prefix-indexed optimisation with rolling variables.

JAVA TEMPLATE LinearDp.java

```java
/** dp over prefix length i: dp[i] = best sum of the first i elements, no two adjacent. */
class LinearDp {
    static long bestNonAdjacent(int[] a) {
        long prev2 = 0;          // dp[i-2]  (empty prefix = 0)
        long prev1 = 0;          // dp[i-1]
        for (int x : a) {
            long cur = Math.max(prev1, prev2 + x);   // skip a[i-1]  vs  take it
            prev2 = prev1;                     // shift: read old values first
            prev1 = cur;
        }
        return prev1;            // dp[n]
    }
}
```

Template B — counting partitions of a sequence (suffix DP, modulo). Reusable: the suffix loop and modular sum. Problem-specific: which pieces `s[i..j]` are valid.

JAVA TEMPLATE PartitionCountDp.java

```java
/** dp[i] = ways to split s[i..n-1] into valid pieces; dp[n] = 1 (empty suffix). */
class PartitionCountDp {
    static final int MOD = 1_000_000_007;

    interface Piece { boolean valid(String s, int i, int j); }   // piece s[i..j] inclusive

    static int count(String s, int maxPieceLen, Piece rule) {
        int n = s.length();
        long[] dp = new long[n + 1];
        dp[n] = 1;                                          // one way to split nothing
        for (int i = n - 1; i >= 0; i--) {         // dependencies dp[j+1] lie to the right
            long ways = 0;
            for (int j = i; j < n && j - i < maxPieceLen; j++)
                if (rule.valid(s, i, j)) ways += dp[j + 1];
            dp[i] = ways % MOD;
        }
        return (int) dp[0];
    }
}
```

Template C — 0/1 knapsack count, 2D state compressed to 1D.

JAVA TEMPLATE KnapsackCount.java

```java
/** Number of subsets of w (each item at most once) summing to target; w[i] >= 0. */
class KnapsackCount {
    static long subsets(int[] w, int target) {
        if (target < 0) return 0;
        long[] dp = new long[target + 1];   // after items 0..i: dp[s] = dp[i][s]
        dp[0] = 1;                          // empty subset
        for (int x : w)
            for (int s = target; s >= x; s--)  // DECREASING: dp[s - x] is still row i-1
                dp[s] += dp[s - x];
        return dp[target];                  // increasing s would be unbounded (reuse)
    }
}
```

12.5 Complexity and pitfalls

Time	(#states) × (cost per transition): O(n) for stairs and robber; O(n · L) for partitions with piece length ≤ L; O(n · S) for knapsack with sum S (pseudo-polynomial).
Auxiliary space	O(n) table, O(1) with rolling variables, O(L) with a circular buffer, O(S) for compressed knapsack. Memoised recursion adds O(depth) stack.
Off-by-one	Mixing "dp over prefix *length* i" (size n + 1, element a[i−1]) with "dp over *index* i" (size n). Pick one and say it aloud.
Base cases	The empty sequence counts as 1 way; for climbing stairs, dp[0] = dp[1] = 1. Wrong base values shift every answer.
Overflow	Counts grow exponentially, so take the modulus at each addition and accumulate in `long`. Climbing stairs with n ≤ 45 fits in `int`; n = 92 is the limit for `long`.
Breaks naive code	Leading zeros in digit partitions; zero-weight items in knapsack (they double the count, which is correct); negative targets; n = 0 or 1.
Interview follow-ups	Reconstruct the choices (store argmax or walk back); reduce space; convert to matrix exponentiation for Fibonacci-like recurrences with n ≈ 10^{18} (O(log n)); circular variants (House Robber II: run twice).

12.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Write the state as a sentence before anything else.
- Derive the transition from the last (or first) decision; for counting, cases must be disjoint.
- The empty problem has one way (counting) or value 0 / −∞ (optimisation), whichever is semantically correct.
- Compress space only after the full-table version is correct, and mind the loop direction.
- If histories with the same index diverge, add a dimension (Pattern 13).

> **Recognition rule.** Exponential recursion + small repeated arguments → memoise; the arguments are the state.

IN-INTERVIEW CHECKLIST

- ☐ State definition: prefix length, index or amount?
- ☐ Transition cases: disjoint and exhaustive?
- ☐ Base case and answer location (`dp[n]` or `dp[0]`)?
- ☐ Fill order matches the dependencies?
- ☐ Modulus / sentinel / overflow handled; space reducible?

Key distinction. 1D DP works when one index summarises the past. The moment you need "position in A *and* position in B", or "index *and* remaining capacity" without a safe compression, you are in 2D.

12.7 Assigned problems

LC 70 Climbing Stairs — EASY

Task. Count the distinct ways to climb n steps taking 1 or 2 steps at a time (1 ≤ n ≤ 45).

KEY INSIGHT

Classify by the *last* step: it was 1 (from n−1) or 2 (from n−2). The cases are disjoint, so `ways(n) = ways(n-1) + ways(n-2)`, a Fibonacci recurrence.

WHY THE PATTERN APPLIES

This is the simplest counting DP: one index, two dependencies, and rolling variables give O(1) space.

APPROACH

1. Base cases: `ways(0) = 1` (stand still), `ways(1) = 1`.
2. Roll two variables up to n.

JAVA SOLUTION LeetCode 70 · Solution.java

```java
class Solution {
    public int climbStairs(int n) {
        int prev2 = 1, prev1 = 1;          // ways(0), ways(1)
        for (int i = 2; i <= n; i++) {
            int cur = prev1 + prev2;       // last move was 1 step or 2 steps
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;                      // ways(45) = 1836311903 < 2^31 - 1
    }
}
```

Time O(n) **Space** O(1)

> **Pitfall.** Plain recursion without memoisation is $O(\varphi^n)$. Follow-up: with step sizes from a set S, the transition sums over S. For n ≈ 10^{18}, use 2×2 matrix exponentiation.

LC 198 House Robber — MEDIUM

Task. Maximise the money robbed from houses in a row without robbing two adjacent houses.

KEY INSIGHT

For the prefix of the first i houses, the last house is either skipped (best = dp[i−1]) or robbed (then house i−2 is excluded: dp[i−2] + a[i−1]). The rest of the history does not matter.

WHY THE PATTERN APPLIES

This is optimisation DP over prefix length with two dependencies (Template A). Greedy rules like "take every other house" or "take the largest first" fail on inputs such as [2, 1, 1, 2].

APPROACH

1. `dp[0] = 0`, `dp[1] = a[0]`; `dp[i] = max(dp[i-1], dp[i-2] + a[i-1])`.
2. Keep two rolling variables.

JAVA SOLUTION LeetCode 198 · Solution.java

```java
class Solution {
    public int rob(int[] nums) {
        int prev2 = 0, prev1 = 0;                  // dp[i-2], dp[i-1]; dp[0] = 0
        for (int x : nums) {
            int cur = Math.max(prev1, prev2 + x);  // skip this house, or rob it
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;                              // <= 100 houses * 400 fits easily
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Defining `dp[i]` as "best when robbing house i" forces a max over all earlier states and complicates the answer (max over the last two). The "best for the prefix" definition is cleaner. Follow-ups: circular street (198 → 213: solve [0, n−2] and [1, n−1]) and trees (337: return a pair {rob, skip} per node).

LC 1416 Restore The Array

HARD

Task. A digit string `s` (length ≤ 10^5) was formed by concatenating integers in [1, k] (k ≤ 10^9) with no leading zeros. Count the possible original arrays, mod 10^9+7.

KEY INSIGHT

Let `dp[i]` = number of ways to split the suffix `s[i..]`. The first number starts at i, cannot start with '0', and has at most `digits(k) ≤ 10` digits. So each state has at most 10 transitions, and the scan stops as soon as the number exceeds k.

WHY THE PATTERN APPLIES

This is partition-counting DP (Template B) with a validity rule "value in [1, k], no leading zero". The bounded piece length keeps it 1D and linear: O(n · $\log_{10}$ k).

APPROACH

1. `dp[n] = 1`. For i from n−1 down to 0: if `s[i] == '0'`, `dp[i] = 0`.
2. Otherwise extend `num = num·10 + digit` for j = i, i+1, … while `num ≤ k`, adding `dp[j+1]`.

JAVA SOLUTION LeetCode 1416 · Solution.java

```java
class Solution {
    private static final int MOD = 1_000_000_007;

    public int numberOfArrays(String s, int k) {
        int n = s.length();
        long[] dp = new long[n + 1];            // dp[i] = ways to split s[i..n-1]
        dp[n] = 1;
        for (int i = n - 1; i >= 0; i--) {
            if (s.charAt(i) == '0') continue;   // no number may start with '0' (dp[i] = 0)
            long num = 0, ways = 0;
            for (int j = i; j < n; j++) {
                num = num * 10 + (s.charAt(j) - '0');  // long: up to 11 digits
                if (num > k) break;                    // longer pieces only get bigger
                ways += dp[j + 1];
            }
            dp[i] = ways % MOD;                 // at most 10 terms < MOD: no long overflow
        }
        return (int) dp[0];
```

```
    }
}
```

Time O(n · $\log_{10}$ k) ≤ about 10n **Space** O(n); reducible to O(log k) with a circular buffer of size 11

Pitfall. `num` must be `long`: 10 digits can exceed `Integer.MAX_VALUE` before the `num > k` check fires. Without the early break, the inner loop is O(n) and the whole DP becomes O(n^2).

LC 494 Target Sum

MEDIUM

Task. Assign + or − to each of `nums` (n ≤ 20, 0 ≤ nums[i], Σ ≤ 1000) and count the assignments whose total equals `target`.

Classification note. The natural DP state is two-dimensional, (index, running sum). This is a 0/1 knapsack, solved here in its *compressed* 1D form. The single array is a rolling row of the 2D table, which is why the loop direction is decisive.

KEY INSIGHT

Let P be the elements given +, N the elements given −. Then `P − N = target` and `P + N = total`, so `P = (total + target) / 2`. The problem reduces to counting subsets with sum P.

```
sum(P) − sum(N) = target
sum(P) + sum(N) = total           ⇒   sum(P) = (total + target) / 2
Feasible only if |target| ≤ total and (total + target) is even.
```

WHY THE PATTERN APPLIES

Counting subsets by sum is the 0/1 knapsack count (Template C). The state space is O(n · P) instead of the 2^n sign assignments.

APPROACH

1. Check the parity and range conditions; return 0 if they fail.
2. `dp[0] = 1`; for each x, for s from P down to x: `dp[s] += dp[s − x]`.

JAVA SOLUTION LeetCode 494 · Solution.java

```java
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int total = 0;
        for (int x : nums) total += x;
        if (Math.abs(target) > total || ((total + target) & 1) == 1) return 0;
        int p = (total + target) / 2;              // required sum of the '+' subset, >= 0
        int[] dp = new int[p + 1];           // dp[s]: subsets of items so far with sum s
        dp[0] = 1;
        for (int x : nums)
            for (int s = p; s >= x; s--)       // decreasing: each item used at most once
                dp[s] += dp[s - x];            // x == 0 doubles: +0 and -0 both count
        return dp[p];                              // <= 2^20 assignments: fits in int
    }
}
```

Time O(n · (total + target)/2) **Space** O(total)

Pitfall. Increasing iteration over s turns this into an unbounded knapsack and over-counts. Forgetting the parity check gives answers for an impossible half-integer P. The memoised (index, sum) recursion is also accepted; offset the sum by `total` to index an array.

Dynamic Programming: 2D

Two-dimensional DP appears when one index cannot summarise the past: a cell of a grid (row and column), or a pair of prefixes from two sequences. The table is a DAG of subproblems, and the work is choosing transitions, initialising the borders, and filling in an order that respects every dependency.

13.1 Pattern identity

Primary purpose	Count or optimise paths on a grid with monotone moves; align, compare or transform two sequences.
Typical input	An m × n grid with right/down moves; two strings or arrays of lengths m and n (m, n ≤ a few thousand).
Interview applications	Unique paths (with obstacles), minimum path sum, dungeon game, LCS, edit distance, distinct subsequences, interleaving strings, regex / wildcard matching, longest common substring.
Key insight	For two sequences, define `dp[i][j]` over the *prefixes* `a[0..i)` and `b[0..j)`. The last characters either match, or one of them is dropped or edited. Every alignment decision is local to (i, j) and the three neighbours (i−1, j), (i, j−1), (i−1, j−1).

> **Key insight.** Use prefix *lengths* (an (m+1) × (n+1) table), not indices. Row 0 and column 0 then represent empty prefixes, the base cases become explicit, and the boundary `if`s disappear.

13.2 Recognition signals

Signal	Structural reason
Grid with moves only right/down (or any acyclic move set)	The cell graph is a DAG, so row-major order is a topological order. Each cell combines its predecessors.
Two strings: "common subsequence", "minimum operations to convert", "match pattern"	An alignment of two sequences decomposes on the last pair of characters.
"Subsequence" (not substring) of two inputs	Characters may be skipped independently in each sequence, so there are two indices.
m, n ≤ 10^3–$5 \cdot 10^3$	An O(m · n) table fits in the time budget.
1D attempt fails because the future depends on progress in two things	A missing state dimension.

> **Ask yourself.** "What are the last decisions I could make at (i, j), and which smaller (i', j') does each one lead to?"

When it is inappropriate.

- **Grids with moves in all four directions** contain cycles; there is no fill order. Use BFS (unit cost) or Dijkstra (weighted), Patterns 10 / 18.

- **Substring (contiguous) problems on one string** are often better served by a sliding window or by expanding around centres.
- **Very long sequences** (10^5 each): O(mn) is infeasible. Look for special structure (LCS of permutations reduces to LIS in O(n log n)).

Confused with	How to tell them apart
1D DP	If the state needs two independent positions, it is 2D. A 1D *array* can still implement it via a rolling row, but the state is two-dimensional.
Interval DP	`dp[i][j]` over a single sequence's range [i, j], filled by increasing length (palindromes, burst balloons). Prefix-pair DP is filled row by row.
BFS on grids	BFS handles arbitrary moves with unit cost. Grid DP handles monotone moves with arbitrary cell costs and counting.

13.3 Concepts and invariants

Grid DP. With moves right/down, every path to (i, j) enters from (i−1, j) or (i, j−1):

```
Count:    paths[i][j]  = paths[i-1][j] + paths[i][j-1]          (0 for blocked cells)
Minimise: cost[i][j]   = grid[i][j] + min(cost[i-1][j], cost[i][j-1])
Borders:  first row / column have a single predecessor
```

Two-sequence DP. On prefixes of lengths i and j:

```
LCS:   L[i][j] = L[i-1][j-1] + 1                    if a[i-1] == b[j-1]
               = max(L[i-1][j], L[i][j-1])             otherwise
       L[0][j] = L[i][0] = 0

Edit:  D[i][j] = D[i-1][j-1]                           if a[i-1] == b[j-1]
               = 1 + min( D[i-1][j],    // delete a[i-1]
                          D[i][j-1],    // insert b[j-1]
                          D[i-1][j-1] ) // replace a[i-1] by b[j-1]
       D[i][0] = i,  D[0][j] = j       // convert to / from the empty string
```

Why taking the diagonal on a match is always safe

LCS: if `a[i-1] == b[j-1]`, some optimal common subsequence uses both characters as its last pair. Take any optimal one. If it uses neither, appending the pair makes it longer, a contradiction. If it uses only one of them (say a[i−1] matched to some earlier b[k]), swap that partner to b[j−1]; the length is unchanged. So `L[i-1][j-1] + 1` is optimal and the other two options never beat it. The same exchange argument justifies the free diagonal in edit distance (matching costs 0, which is never worse than editing).

Traversal order. Each cell must be computed after its dependencies. For dependencies on up, left and up-left, row-major order (i ascending, j ascending) works. For dependencies on down/right (such as Dungeon Game), fill from the bottom-right.

Space compression. If row i depends only on rows i and i−1, keep one row:

- Up and left only (grid DP): `dp[j] = f(dp[j] /*up, old*/, dp[j-1] /*left, new*/)`, with no extra variable.

- Also up-left (LCS, edit distance): the old `dp[j-1]` is overwritten before cell (i, j) is computed, so save it in a variable `diag` before each overwrite.
- Compress along the *shorter* dimension to get O(min(m, n)) space. You lose the ability to reconstruct the alignment unless you use Hirschberg's divide-and-conquer.

13.4 Reusable Java templates

Template A — grid DP with a rolling row (monotone moves right/down).

JAVA TEMPLATE GridDp.java

```java
/** Minimum-cost path from (0,0) to (m-1,n-1) moving right/down. O(n) space via one row. */
class GridDp {
    static long minPathCost(int[][] g) {
        int m = g.length, n = g[0].length;
        long[] dp = new long[n];                    // dp[j] = best cost to reach (i, j)
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (i == 0 && j == 0) dp[j] = g[0][0];
                else if (i == 0) dp[j] = dp[j - 1] + g[i][j];        // only from the left
                else if (j == 0) dp[j] = dp[j] + g[i][j];            // only from above
                else dp[j] = Math.min(dp[j], dp[j - 1]) + g[i][j];   // dp[j] = up (old row)
            }
        }
        return dp[n - 1];
    }
}
```

Template B — two-sequence DP on prefix lengths (full table). Reusable: the table shape, the borders and the fill order. Problem-specific: the match and mismatch transitions.

JAVA TEMPLATE TwoSequenceDp.java

```java
/** dp[i][j] over prefixes a[0..i), b[0..j). Shown for LCS; swap transitions as needed. */
class TwoSequenceDp {
    static int lcs(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];          // row 0 / col 0 = empty prefix = 0
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
}
```

Template C — rolling row that also needs the diagonal. This is the compression pattern for LCS, edit distance and similar recurrences.

JAVA TEMPLATE RollingEditDistance.java

```java
/** Edit distance in O(min(m, n)) space; diag saves dp[i-1][j-1] before overwrite. */
class RollingEditDistance {
    static int distance(String a, String b) {
        if (a.length() < b.length()) { String t = a; a = b; b = t; }   // b is the shorter
        int m = a.length(), n = b.length();
        int[] dp = new int[n + 1];
        for (int j = 0; j <= n; j++) dp[j] = j;           // row 0: insert j characters
        for (int i = 1; i <= m; i++) {
```

```
            int diag = dp[0];                                   // dp[i-1][0]
            dp[0] = i;                                          // dp[i][0]: delete i characters
            for (int j = 1; j <= n; j++) {
                int up = dp[j];                             // dp[i-1][j], about to be replaced
                if (a.charAt(i - 1) == b.charAt(j - 1)) dp[j] = diag;
                else dp[j] = 1 + Math.min(diag, Math.min(up, dp[j - 1]));
                diag = up;                                      // becomes dp[i-1][j-1] for j+1
            }
        }
        return dp[n];
    }
}
```

13.5 Complexity and pitfalls

Time	O(m · n) states × O(1) transitions.
Auxiliary space	O(m · n) full table; O(min(m, n)) with a rolling row (alignment reconstruction is then lost).
Off-by-one	In prefix-length tables, character i of the prefix is `a.charAt(i - 1)`. Mixing index and length conventions is the most common bug.
Initialisation	Edit distance borders are i and j, not 0. Grid borders have only one predecessor. Obstacle grids zero out everything after a blocked border cell.
Overflow	Unique paths for 100 × 100 is about 2.3×10^{58}, but LeetCode 62 guarantees the answer ≤ $2 \cdot 10^9$ for its inputs. The binomial approach needs `long` intermediates.
Rolling-row bugs	Reading `dp[j - 1]` expecting the old row (it is already new), or forgetting to save the diagonal.
Interview follow-ups	Reconstruct the LCS or the edit script (walk back from (m, n)); obstacles; only O(n) memory; weighted edit operations; LCS of k strings (k-dimensional state).

13.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Index by prefix lengths; row 0 and column 0 are the empty prefixes.
- Transitions come from the last pair of characters: match, skip in a, skip in b (or edit).
- Fill order must follow the dependency arrows.
- Compress to one row; keep a `diag` variable when up-left is needed.
- Four-directional grids are graphs, not DPs.

> **Recognition rule.** Two sequences compared position by position, or a grid with monotone moves → a (prefix × prefix) or (row × column) table.

IN-INTERVIEW CHECKLIST

- ☐ What do `i` and `j` mean: lengths or indices?
- ☐ What are the base row and column values?
- ☐ Which neighbours does a cell read, and does my loop order compute them first?
- ☐ Can I compress, and do I still need the reconstruction?
- ☐ Are empty strings or 1 × 1 grids handled?

Key distinction. LCS maximises matches (mismatch = skip, cost 0); edit distance minimises edits (mismatch = replace, cost 1, plus insert and delete). The same table shape is used with different transition semantics.

13.7 Assigned problems

LC 62 Unique Paths — MEDIUM

Task. Count the paths from the top-left to the bottom-right of an m × n grid moving only right or down.

KEY INSIGHT

Paths into a cell arrive from above or from the left, and the two sets are disjoint, so the counts add. Equivalently, a path is a sequence of (m−1) downs and (n−1) rights, giving C(m+n−2, m−1).

WHY THE PATTERN APPLIES

This is counting grid DP on a DAG (row-major order). A rolling row gives O(n) space.

APPROACH

1. Row 0 is all ones.
2. For each later row: `dp[j] += dp[j-1]` (old `dp[j]` = from above, new `dp[j-1]` = from the left).

JAVA SOLUTION LeetCode 62 · Solution.java

```java
import java.util.*;

class Solution {
    public int uniquePaths(int m, int n) {
        int[] dp = new int[n];
        Arrays.fill(dp, 1);                    // first row: exactly one path to each cell
        for (int i = 1; i < m; i++)
            for (int j = 1; j < n; j++)
                dp[j] += dp[j - 1];            // from above (old dp[j]) + from left (new)
        return dp[n - 1];                      // problem guarantees answer <= 2 * 10^9
    }
}
```

Time O(m · n) **Space** O(n)

Pitfall. The binomial version computes `res = res * (n - 1 + i) / i` for i = 1..m−1 in `long`. Each partial product is itself a binomial coefficient, so the division is exact, but computing the full factorial first overflows. With obstacles (63), blocked cells reset `dp[j] = 0` and the first row is no longer all ones.

LC 64 Minimum Path Sum — MEDIUM

Task. Find the minimum sum of a right/down path from the top-left to the bottom-right of a non-negative grid.

KEY INSIGHT

The best path to (i, j) extends the cheaper of the best paths to its two possible predecessors (optimal substructure, since costs add).

WHY THE PATTERN APPLIES

This is optimisation grid DP (Template A). Dijkstra would also work, but monotone moves make the DP strictly simpler and O(mn) without a log factor.

APPROACH

1. Initialise the first row and column as running sums.
2. `dp[j] = min(dp[j], dp[j-1]) + grid[i][j]`.

JAVA SOLUTION LeetCode 64 · Solution.java

```java
class Solution {
    public int minPathSum(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        int[] dp = new int[n];
        dp[0] = grid[0][0];
        for (int j = 1; j < n; j++) dp[j] = dp[j - 1] + grid[0][j];   // first row
        for (int i = 1; i < m; i++) {
            dp[0] += grid[i][0];                                       // first column
            for (int j = 1; j < n; j++)
                dp[j] = Math.min(dp[j], dp[j - 1]) + grid[i][j];        // up vs left
        }
        return dp[n - 1];                       // <= (200 + 200) * 200: fits in int
    }
}
```

Time O(m · n) **Space** O(n), or O(1) by writing into `grid` (mutating the input)

Pitfall. Initialising `dp` to 0 and taking `min` for border cells picks up a phantom 0-cost predecessor outside the grid. Handle borders explicitly or initialise outside cells to +∞.

LC 1143 Longest Common Subsequence

MEDIUM

Task. Return the length of the longest common subsequence of two strings (lengths ≤ 1000).

KEY INSIGHT

Look at the last characters of the two prefixes. If they match, they can be paired (the exchange argument above). Otherwise at least one of them is unused, so drop one and take the better result.

WHY THE PATTERN APPLIES

Two independent sequence positions give a 2D state over prefix lengths (Template B). Brute force over subsequences is 2^m.

APPROACH

1. Use a (m+1) × (n+1) table with zero borders.
2. Fill row-major using the LCS recurrence; the answer is `dp[m][n]`.

JAVA SOLUTION LeetCode 1143 · Solution.java

```java
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];  // LCS of text1[0..i) and text2[0..j)
        for (int i = 1; i <= m; i++) {
            char c = text1.charAt(i - 1);
            for (int j = 1; j <= n; j++) {
                if (c == text2.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
                else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
```

```
            }
        }
        return dp[m][n];
    }
}
```

Time O(m · n) **Space** O(m · n); O(min(m, n)) with a rolling row and a diagonal variable

Pitfall. Confusing subsequence with substring. Longest common *substring* resets to 0 on a mismatch and takes a global max, rather than carrying `max(up, left)`. To reconstruct the LCS, walk back from (m, n): go diagonal on a match, otherwise toward the larger neighbour.

LC 72 Edit Distance

MEDIUM

Task. Return the minimum number of single-character insertions, deletions and replacements to turn `word1` into `word2` (lengths ≤ 500).

KEY INSIGHT

In an optimal edit script, the last characters are either matched for free, or the last operation is a delete of `a[i-1]`, an insert of `b[j-1]`, or a replace. Each option leaves a smaller prefix pair. Borders are the cost of building from or reducing to the empty string.

WHY THE PATTERN APPLIES

This is sequence-vs-sequence alignment DP. Each of the three neighbours corresponds to exactly one operation, which makes reconstruction straightforward.

APPROACH

1. `dp[i][0] = i`, `dp[0][j] = j`.
2. On a match, `dp[i][j] = dp[i-1][j-1]`; otherwise 1 + min(delete, insert, replace).

JAVA SOLUTION LeetCode 72 · Solution.java

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) dp[i][0] = i;        // delete all i characters
        for (int j = 0; j <= n; j++) dp[0][j] = j;        // insert all j characters
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];          // match: free
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],          // replace
                                   Math.min(dp[i - 1][j],              // delete word1[i-1]
                                            dp[i][j - 1]));            // insert word2[j-1]
                }
            }
        }
        return dp[m][n];
    }
}
```

Time O(m · n) **Space** O(m · n); O(min(m, n)) with Template C

Pitfall. Zero borders (copied from LCS) are wrong here: converting "abc" to "" costs 3. When compressing, the diagonal must be saved before each overwrite (Template C). Follow-ups: one edit away (161: O(n) two-pointer check) and weighted operations (change the 1s).

Greedy: Interval Scheduling / Local Optimality

Make the locally best irrevocable choice, then prove with an exchange argument that some optimal solution makes the same choice. On intervals the hard part is usually not the loop but picking the right sort key.

14.1 Pattern identity

Primary purpose	Solve optimisation problems in one pass after sorting, by committing to a choice that never needs to be revisited.
Typical input	Intervals `[start, end]`, jobs with deadlines, points on a line, or a sequence where every prefix decision can be fixed immediately.
Interview applications	Maximum set of non-overlapping intervals, minimum arrows or removals, meeting-room feasibility, jump game, gas station, task scheduling, partition labels.
Key insight	For unweighted interval scheduling, the interval that *ends earliest* leaves the most room for everything else. Any optimal schedule can swap its first interval for that one without losing size.

> **Key insight.** A greedy algorithm is only as good as its proof. State the choice ("take the compatible interval with the smallest end") and the exchange ("replacing OPT's first interval with mine keeps OPT feasible and the same size"). If the exchange fails on some example, switch to DP.

14.2 Recognition signals

Signal	Structural reason
"Maximum number of non-overlapping…", "minimum removals so that none overlap", "minimum arrows"	Unweighted selection: every interval is worth 1, so only room for future intervals matters. That is exactly what the earliest end maximises.
"Can one person attend all…?"	After sorting by start, the only possible conflict is between neighbours.
n up to 10^5 intervals	Budget is O(n log n): one sort plus a linear scan. Interval DP at $O(n^2)$ will not fit.
A choice made now restricts the future only through one scalar (last end, farthest reach)	The state collapses to a single number, so no table is needed.

> **Ask yourself.** "After my choice, is the remaining problem the same kind of problem, and is my choice's effect on it never worse than any alternative's?" The second half is the exchange argument.

When it is inappropriate.

- **Weighted intervals** (maximise total profit, LeetCode 1235): one heavy long interval can beat two light short ones. Sort by end, then DP with binary search for the last compatible interval.

- **Choices interact through more than one scalar** (knapsack, coin change with arbitrary coins): local optimality fails on small counterexamples.
- **You need the structure of the union** (merged ranges, free time, room counts): this is sweeping or merging (Pattern 7), not selection.

Confused with	How to tell them apart
Merge Intervals (Pattern 7)	Merging keeps *all* intervals and computes their union; sort by start. Scheduling *discards* intervals to keep a compatible subset; sort by end.
Sweep line / heap (Meeting Rooms II)	Counting the maximum simultaneous overlap is a sweep over events. No interval is chosen or rejected.
Dynamic programming	If you need to compare "take" against "skip" with different payoffs, it is DP. Greedy is the special case where "take the earliest-ending one" is always at least as good.

14.3 Concepts and invariants

Exchange argument for earliest end. Sort intervals by end. Let g_1 be the first interval with the smallest end and let OPT = o_1, o_2, … be an optimal compatible set sorted by end.

```
end(g₁) ≤ end(o₁)                          (g₁ has the globally smallest end)
o₂ starts at or after end(o₁) ≥ end(g₁)   ⇒  {g₁, o₂, o₃, …} is still compatible
|{g₁, o₂, …}| = |OPT|                     ⇒  some optimal solution contains g₁
Induct on the intervals compatible with g₁  ⇒  greedy is optimal.
```

The same argument, phrased as "greedy stays ahead", says that after k picks the greedy's k-th end is ≤ OPT's k-th end, so greedy can never fall behind.

Why sort by end and not by start. Sorting by start and taking the earliest-starting interval fails on `[1,100], [2,3], [4,5]`: you take one long interval instead of two short ones. Sorting by length fails on `[1,5], [4,7], [6,10]`: the shortest, `[4,7]`, blocks both others. The end time is the only key whose exchange argument works.

Boundary convention. LeetCode 435 and 252 treat touching intervals (`[1,2]` and `[2,3]`) as compatible, which means half-open semantics. LeetCode 56 and 986 use closed intervals, where touching intervals overlap or intersect at a point. Write the comparison (`>=` vs `>`) as a deliberate decision.

Counting removals is the complement of scheduling

"Minimum intervals to remove so the rest are non-overlapping" equals `n - (maximum compatible subset)`. Removing a set R leaves a compatible set S = all − R, so minimising |R| is the same as maximising |S|. This reduction is the whole of LeetCode 435. Minimum arrows (452) is the same algorithm with closed intervals: one arrow at the end of each greedy pick bursts every balloon overlapping it.

When the greedy view is not the full story. Merging (56) and intersecting lists (986) are often filed under greedy because each step commits to something permanently. Their correctness comes from sortedness plus a sweep or two-pointer invariant, not from an exchange argument. The assigned problems below label this explicitly.

14.4 Reusable Java templates

Template A — maximum compatible subset (earliest end). Reusable: sort by end, keep `lastEnd`. Problem-specific: the compatibility test (`>=` for half-open, `>` for closed).

JAVA TEMPLATE `IntervalScheduling.java`

```java
import java.util.*;

/** Unweighted interval scheduling on [start, end): touching intervals are compatible. */
class IntervalScheduling {
    static int maxCompatible(int[][] intervals) {
        if (intervals.length == 0) return 0;
        int[][] a = intervals.clone();                        // keep the caller's order
        Arrays.sort(a, (x, y) -> Integer.compare(x[1], y[1])); // by end; never x[1] - y[1]
        int count = 1;
        int lastEnd = a[0][1];
        for (int i = 1; i < a.length; i++) {
            if (a[i][0] >= lastEnd) {                          // closed intervals: use '>'
                count++;
                lastEnd = a[i][1];                             // greedy choice: earliest end
            }
        }
        return count;
    }
}
```

Template B — neighbour conflict check after sorting by start.

JAVA TEMPLATE `NeighbourConflict.java`

```java
import java.util.*;

/** Sorted by start, any overlap implies an overlap between two adjacent intervals. */
class NeighbourConflict {
    static boolean hasOverlap(int[][] intervals, boolean touchingOverlaps) {
        int[][] a = intervals.clone();
        Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
        for (int i = 1; i < a.length; i++) {
            boolean overlap = touchingOverlaps ? a[i][0] <= a[i - 1][1]
                                               : a[i][0] < a[i - 1][1];
            if (overlap) return true;
        }
        return false;
    }
}
```

14.5 Complexity and pitfalls

Time	O(n log n) for the sort plus an O(n) scan.
Auxiliary space	O(log n) to O(n): sorting an `int[][]` uses TimSort (object array), which can allocate up to n/2 references. A copy via `clone()` costs O(n) references.
Off-by-one	`>=` vs `>` for touching endpoints. Read the problem's convention; 435 says `[1,2]` and `[2,3]` do not overlap.
Wrong invariant	Sorting by start (or by length) for selection; updating `lastEnd` on rejected intervals.
Overflow	Comparator `(x, y) -> x[1] - y[1]` overflows when endpoints span more than 2^{31}. 435 allows $-5{\cdot}10^4 \ldots 5{\cdot}10^4$ (safe), but other problems allow $\pm 2^{31}$. Always use `Integer.compare`.

Breaks naive code	Empty input, nested intervals (one inside another), identical intervals, negative coordinates, and `lastEnd` initialised to 0 instead of the first end.
Interview follow-ups	Weighted version (DP + binary search); return the removed intervals; minimum arrows (452); online arrival of intervals (balanced BST or sweep); proof of optimality.

14.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Selection problems sort by end; union or structure problems sort by start.
- Write the exchange argument in two lines before coding; it doubles as your explanation to the interviewer.
- "Minimum removals" = n − "maximum kept".
- Touching endpoints are a per-problem convention; encode it in a single comparison.
- If intervals carry weights or profits, greedy is wrong; use DP.

> **Recognition rule.** Unweighted "keep as many as possible" over intervals → sort by end, take each one that fits.

IN-INTERVIEW CHECKLIST

- ☐ Is every interval worth the same? (If not, DP.)
- ☐ Which endpoint is the sort key, and what is my exchange argument?
- ☐ Do touching endpoints overlap in this problem?
- ☐ Is my comparator overflow-safe?
- ☐ Do I mutate the input order, and is that allowed?

Key distinction. Greedy *selection* rejects intervals to keep a compatible subset (sort by end). *Merging* keeps every interval and builds their union (sort by start, Pattern 7).

14.7 Assigned problems

LC 252 Meeting Rooms

EASY Premium

Task. Given meeting intervals `[start, end]`, decide whether one person can attend all of them. A meeting ending at time t and another starting at t do not conflict.

KEY INSIGHT

After sorting by start, if any two meetings overlap then some *adjacent* pair overlaps. Checking neighbours is enough.

WHY THE PATTERN APPLIES

This is the feasibility special case of scheduling: can we keep all n? The earliest-start order is the order the person would attend them, so each meeting must start no earlier than the previous one ends (Template B).

APPROACH

1. Sort by start with `Integer.compare`.
2. Return false if any `a[i][0] < a[i-1][1]`.

JAVA SOLUTION LeetCode 252 · Solution.java

```java
import java.util.*;

class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (x, y) -> Integer.compare(x[0], y[0]));
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] < intervals[i - 1][1]) return false;  // touching is allowed
        }
        return true;
    }
}
```

Time O(n log n) **Space** O(n) worst case for the object-array sort

Pitfall. Why is the neighbour check sufficient? If meetings i < j overlap (start(j) < end(i)), then start(i+1) ≤ start(j) < end(i), so meeting i+1 already overlaps meeting i. Use `<`, not `<=`: back-to-back meetings are fine.

LC 435 Non-overlapping Intervals

MEDIUM

Task. Return the minimum number of intervals to remove so the rest do not overlap. Touching intervals (`[1,2]`, `[2,3]`) do not overlap.

KEY INSIGHT

Minimum removals = n − maximum number of compatible intervals, and the maximum is achieved by repeatedly keeping the compatible interval with the earliest end.

WHY THE PATTERN APPLIES

This is unweighted interval scheduling exactly (Template A). The exchange argument in 14.3 proves the earliest-end greedy keeps a maximum set.

APPROACH

1. Sort by end.
2. Keep an interval if its start is `>= lastEnd`, otherwise count it as removed. A rejected interval ends no earlier than `lastEnd`, so keeping the current one is never worse.

JAVA SOLUTION LeetCode 435 · Solution.java

```java
import java.util.*;

class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (x, y) -> Integer.compare(x[1], y[1]));
        int kept = 1;
        int lastEnd = intervals[0][1];              // n >= 1 by constraints
        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= lastEnd) {       // compatible (touching allowed)
                kept++;
                lastEnd = intervals[i][1];
            }                                       // else: removed; lastEnd unchanged
        }
        return intervals.length - kept;
    }
}
```

Time O(n log n) **Space** O(n) worst case (sort)

Pitfall. A common alternative sorts by start and, on conflict, keeps the interval with the smaller end (`lastEnd = min(lastEnd, end)`). That is also correct, but forgetting the `min` is a frequent bug. Sorting by end avoids the issue entirely.

LC 56 Merge Intervals

MEDIUM

Task. Merge all overlapping closed intervals (touching ones such as `[1,4]` and `[4,5]` merge) and return the disjoint result.

Classification note. Merge Intervals is a sort-and-sweep problem, not greedy *selection*: nothing is discarded and there is no exchange argument. It fits this chapter only in the weak sense that each closed group is committed permanently. Chapter 7 solves it with the standard "sort by start, extend the last group" sweep (7 · LC 56). This chapter uses a different technique, the **separate-endpoint sweep**, which is worth knowing because it also drives room counting and coverage problems.

KEY INSIGHT

Sort starts and ends *independently*. In sorted order, a merged group ends exactly at position i when `starts[i+1] > ends[i]`: the i+1 smallest ends all come before the next start.

WHY THE TECHNIQUE WORKS

Let x = `ends[i]`, the (i+1)-th smallest end. Every interval ending by x also starts by x, so the set E of intervals with end ≤ x (size ≥ i+1) is contained in the set S of intervals with start ≤ x. If `starts[i+1] > x`, then |S| ≤ i+1, so E = S: exactly i+1 intervals lie entirely in (−∞, x] and every other interval starts after x. That is a genuine gap. Conversely, if a gap exists after the first i+1 intervals by start, all of them end before the next start, so their i+1 ends are the smallest i+1 ends and `ends[i] < starts[i+1]`. The group's end is `ends[i]` (its largest end) and its start is the first start after the previous cut.

APPROACH

1. Copy starts and ends into two arrays and sort both (primitive sort, no comparator).
2. Scan i; when `i == n−1` or `starts[i+1] > ends[i]`, emit `[groupStart, ends[i]]` and set `groupStart = starts[i+1]`.

JAVA SOLUTION LeetCode 56 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] merge(int[][] intervals) {
        int n = intervals.length;
        int[] starts = new int[n], ends = new int[n];
        for (int i = 0; i < n; i++) {
            starts[i] = intervals[i][0];
            ends[i] = intervals[i][1];
        }
        Arrays.sort(starts);                          // primitive sort: no comparator
        Arrays.sort(ends);
        List<int[]> res = new ArrayList<>();
        int groupStart = starts[0];
        for (int i = 0; i < n; i++) {
            // Closed intervals: touching (starts[i+1] == ends[i]) still merges, hence '>'.
            if (i == n - 1 || starts[i + 1] > ends[i]) {
                res.add(new int[]{groupStart, ends[i]});
```

```
                if (i + 1 < n) groupStart = starts[i + 1];
            }
        }
        return res.toArray(new int[0][]);
    }
}
```

Time O(n log n) **Space** O(n) for the two endpoint arrays

Pitfall. The pairing between a start and its own end is deliberately lost. That is fine for the *union*, but this technique cannot report which original intervals formed each group. Use the Chapter 7 sweep when you need that.

LC 986 Interval List Intersections

MEDIUM

Task. Two lists of closed intervals, each pairwise disjoint and sorted, are given. Return their intersection.

Classification note. The primary technique is **two pointers over two sorted lists**, essentially a merge step. The greedy element is the advance rule: discard the interval that *ends first*, because it cannot intersect anything later in the other list.

KEY INSIGHT

Two closed intervals intersect in `[max(starts), min(ends)]` when that range is non-empty. Whichever interval ends first is exhausted: every later interval in the other list starts after the current one, and the current one already started no later than this end.

WHY THE PATTERN APPLIES

Both lists are sorted and disjoint, so each pair of current intervals is examined once and one pointer advances per step, which gives O(m + n). This "advance the one that ends earliest" rule is the same earliest-end reasoning as scheduling.

APPROACH

1. `lo = max(a.start, b.start)`, `hi = min(a.end, b.end)`; if `lo <= hi`, record `[lo, hi]` (a single point counts).
2. Advance the pointer whose interval has the smaller end. On a tie, either one may move; the other is advanced on the next step.

JAVA SOLUTION LeetCode 986 · Solution.java

```java
import java.util.*;

class Solution {
    public int[][] intervalIntersection(int[][] firstList, int[][] secondList) {
        List<int[]> res = new ArrayList<>();
        int i = 0, j = 0;
        while (i < firstList.length && j < secondList.length) {
            int lo = Math.max(firstList[i][0], secondList[j][0]);
            int hi = Math.min(firstList[i][1], secondList[j][1]);
            if (lo <= hi) res.add(new int[]{lo, hi});    // closed: [5,5] is a valid answer
            if (firstList[i][1] < secondList[j][1]) i++;  // earlier end: exhausted
            else j++;
        }
        return res.toArray(new int[0][]);
    }
}
```

Time O(m + n) **Space** O(1) auxiliary (output excluded)

Pitfall. Advancing by the earlier *start* is wrong: a long interval in one list can intersect several intervals in the other. Either list may be empty; the loop condition handles that.

Heap / Priority Queue: Top-K and Streaming

A heap answers one question cheaply and repeatedly: "what is the current extreme?" Use it when the set changes between questions. If the set never changes, sorting or selection is usually simpler.

15.1 Pattern identity

Primary purpose	Maintain the minimum or maximum of a dynamic multiset with O(log n) insert and extract and O(1) peek. Bound the heap at size k to select top-k in O(n log k).
Typical input	A stream or array where the answer depends only on extremes: k largest, k closest, k most frequent, the next event to process, or a running median.
Interview applications	Kth largest, top-k frequent, k closest points, merge k sorted lists, task scheduler, meeting rooms, median of a stream, sliding-window median, Dijkstra.
Key insight	To keep the k *largest* elements, use a *min*-heap of size k: its root is the weakest member of the current top-k, which is exactly the element to evict when something better arrives.

> **Key insight.** The heap type is the opposite of the goal for bounded top-k. The root must be the element you are ready to throw away. For k largest that is the smallest element kept, so use a min-heap.

15.2 Recognition signals

Signal	Structural reason
"k-th largest/smallest", "top k", "k closest"	Only the boundary of the top-k set matters; a size-k heap tracks that boundary in O(log k) per element.
Data arrives online ("stream", `add`/`find` API)	A full sort per query is impossible; the heap is maintained incrementally.
Repeatedly take the best, modify it, and put it back	Simulation with a changing extreme (stones, tasks, events) needs log-time re-insertion.
Median or percentile of a dynamic set	Split the set at the median: max-heap for the lower half, min-heap for the upper half.
k ≪ n, or memory bounded by k	O(n log k) time and O(k) space beat sorting's O(n log n) and O(n).

> **Ask yourself.** "Does each step only need the current minimum or maximum of a changing set?" If yes, use a heap. If you need the nearest larger element by *position*, use a monotonic stack. If you need arbitrary deletions or order statistics, use a `TreeMap` or a balanced BST.

When it is inappropriate.

- **Static data, one query**: quickselect gives expected O(n). Sorting is fine when n log n fits and you want simplicity.
- **Arbitrary removal** (sliding-window maximum): `PriorityQueue.remove(Object)` is O(k). Use a monotonic deque, lazy deletion, or a `TreeMap`.

- **Values in a small range** (frequencies ≤ n): bucket sort gives O(n).

Confused with	How to tell them apart
Sorting	Sort when everything is known up front and you need a full order. Use a heap when elements arrive over time or only k matter.
Monotonic stack/deque	A deque keeps candidates in *position* order with amortised O(1) per element. A heap orders by *value* only.
TreeMap / ordered multiset	Supports removal of any key and floor/ceiling queries in O(log n). Heavier, but needed for sliding-window medians with removals.

15.3 Concepts and invariants

Heap invariant. Every parent compares ≤ its children under the comparator, so the root is the extreme. Java's `PriorityQueue` is a binary min-heap by natural order. Pass a comparator for anything else.

Operation	`PriorityQueue` cost	Note
`offer` / `poll`	O(log n)	Sift up / sift down
`peek`, `size`	O(1)	
`remove(Object)`, `contains`	O(n)	Linear scan; avoid in hot loops
Build from a collection	O(n)	`new PriorityQueue<>(collection)` heapifies bottom-up
Iteration order	Not sorted	Only the root is guaranteed; poll repeatedly for sorted output

Bounded top-k invariant. After processing a prefix, the size-k min-heap holds the k largest elements of that prefix. A new element x enters only if `x > heap.peek()`, and then evicts the root. Cost: O(n log k) time, O(k) space.

Two-heaps median invariant.

```
lo = max-heap (smaller half),  hi = min-heap (larger half)
(1) every x in lo ≤ every y in hi      ⇔  lo.peek() ≤ hi.peek()
(2) |lo| = |hi|  or  |lo| = |hi| + 1
median = lo.peek()                        if sizes differ
       = (lo.peek() + hi.peek()) / 2.0    otherwise
```

Insert into `lo`, move `lo`'s maximum to `hi` (restores (1)), and if `hi` becomes larger move its minimum back (restores (2)). Each add is three heap operations, O(log n).

Comparator correctness: never subtract

`(a, b) -> a - b` is wrong whenever `a - b` can overflow. For `a = Integer.MAX_VALUE` and `b = -1` the difference wraps to a negative number and the heap silently orders them backwards. Use `Integer.compare(a, b)`, `Comparator.comparingInt(...)`, or `Collections.reverseOrder()`. A comparator must also be *consistent*: ties should return 0, and for deterministic output add a tie-breaker (for example, by value when frequencies are equal).

JAVA SNIPPET Comparator pitfalls

```java
PriorityQueue<Integer> bad  = new PriorityQueue<>((a, b) -> b - a);          // overflows
PriorityQueue<Integer> good = new PriorityQueue<>(Collections.reverseOrder()); // max-heap
```

```
PriorityQueue<int[]> byFreqThenValue = new PriorityQueue<>(
    (x, y) -> x[1] != y[1] ? Integer.compare(x[1], y[1]) : Integer.compare(x[0], y[0]));
```

Duplicates. Heaps are multisets: equal values coexist and are popped in an unspecified order among themselves. Count-based problems (top-k frequent) should push one entry per *distinct key*, not per occurrence.

15.4 Reusable Java templates

Template A — bounded top-k (k largest by a key). Reusable: the size-k min-heap and eviction rule. Problem-specific: the key and the comparator.

JAVA TEMPLATE TopK.java

```java
import java.util.*;

/** Keeps the k largest values seen; root = k-th largest. O(n log k) time, O(k) space. */
class TopK {
    static int[] kLargest(int[] nums, int k) {
        PriorityQueue<Integer> heap = new PriorityQueue<>();    // min-heap: root is weakest
        for (int x : nums) {
            if (heap.size() < k) heap.offer(x);
            else if (x > heap.peek()) {                          // x beats the weakest kept
                heap.poll();
                heap.offer(x);
            }
        }
        int[] res = new int[heap.size()];
        for (int i = res.length - 1; i >= 0; i--) res[i] = heap.poll();  // descending order
        return res;
    }
}
```

Template B — two heaps for a running median.

JAVA TEMPLATE RunningMedian.java

```java
import java.util.*;

/** lo: max-heap (smaller half); hi: min-heap (larger half); |lo| - |hi| in {0, 1}. */
class RunningMedian {
    private final PriorityQueue<Integer> lo =
        new PriorityQueue<>(Collections.reverseOrder());
    private final PriorityQueue<Integer> hi = new PriorityQueue<>();

    void add(int x) {
        lo.offer(x);
        hi.offer(lo.poll());                          // max of lower half moves up
        if (hi.size() > lo.size()) lo.offer(hi.poll()); // rebalance sizes
    }

    double median() {                                 // precondition: at least one element
        if (lo.size() > hi.size()) return lo.peek();
        return ((long) lo.peek() + hi.peek()) / 2.0;  // long: the int sum may overflow
    }
}
```

15.5 Complexity and pitfalls

Time	O(log n) per offer/poll; O(n log k) for bounded top-k; O(n) to heapify a collection.
Auxiliary space	O(k) for bounded heaps; O(n) for the full heap or the two median heaps.

Off-by-one	"k-th largest" with a size-k min-heap is `peek()`, not the k-th poll. Check whether k is 1-based.
Wrong invariant	Max-heap for k largest (keeps n elements and loses the O(k) bound); forgetting to rebalance median heaps.
Overflow	Subtraction comparators; summing two ints for a median.
Breaks naive code	Ties in frequency, duplicates, k = n, negative values, an empty heap on `peek()` (returns `null` and then NPE on unboxing).
Interview follow-ups	Quickselect for expected O(n); bucket sort for frequencies; remove arbitrary elements (lazy deletion with a "to delete" map); medians where 99% of values lie in [0, 100] (counting array plus overflow heaps).

15.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- k largest → min-heap of size k; k smallest → max-heap of size k.
- Two heaps split a set at a rank, with sizes balanced within one.
- Never write subtraction comparators; tie-break when output order matters.
- `PriorityQueue` removal of an arbitrary element is O(n). Plan for lazy deletion if you need it.
- For a single static query, compare against quickselect and bucket sort before reaching for a heap.

> **Recognition rule.** Repeatedly need the extreme of a changing set → heap. Need only the top k → bound the heap at k with the opposite order.

IN-INTERVIEW CHECKLIST

- ☐ Min-heap or max-heap, and why?
- ☐ Is the heap bounded? What is evicted?
- ☐ Comparator overflow-safe and consistent on ties?
- ☐ What do I push: values, indices, or (key, value) pairs?
- ☐ Would quickselect or bucket sort be asymptotically better here?

Key distinction. Heap = extreme by *value* in O(log n). Monotonic stack = nearest qualifying element by *position* in amortised O(1).

15.7 Assigned problems

LC 1046 Last Stone Weight — EASY

Task. Repeatedly smash the two heaviest stones. Equal stones are both destroyed; otherwise the difference remains. Return the last stone's weight, or 0 if none is left.

KEY INSIGHT

Each round needs the two current maxima and may insert a new stone. That is a dynamic-maximum simulation, which a max-heap handles directly.

WHY THE PATTERN APPLIES

Sorting once does not work because the inserted difference can belong anywhere in the order. A heap restores order in O(log n) per round.

APPROACH

1. Heapify all stones into a max-heap.
2. While at least two remain: poll y and x (y ≥ x); if `y > x`, offer `y − x`.

JAVA SOLUTION LeetCode 1046 · Solution.java

```java
import java.util.*;

class Solution {
    public int lastStoneWeight(int[] stones) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
        for (int s : stones) pq.offer(s);
        while (pq.size() > 1) {
            int y = pq.poll(), x = pq.poll();  // y >= x
            if (y != x) pq.offer(y - x);
        }
        return pq.isEmpty() ? 0 : pq.peek();
    }
}
```

Time O(n log n) **Space** O(n)

> **Pitfall.** Return 0 when the heap is empty; `peek()` on an empty `PriorityQueue<Integer>` returns `null` and unboxing throws. Each round removes at least one stone, so there are at most n − 1 rounds.

LC 215 Kth Largest Element in an Array

MEDIUM

Task. Return the k-th largest element in sorted order (duplicates count separately), ideally without fully sorting.

KEY INSIGHT

A size-k min-heap's root is the k-th largest of everything seen. Alternatively, quickselect partitions around a pivot and recurses into one side only.

WHY THE PATTERN APPLIES

Only the boundary of the top-k matters, which is exactly bounded top-k (Template A). O(n log k) worst case with no adversarial input risk.

Method	Time	Space	Notes
Sort	O(n log n)	O(log n)	Simplest; often acceptable
Size-k min-heap	O(n log k)	O(k)	Predictable, works on streams
Randomised quickselect	O(n) expected, $O(n^2)$ worst	O(1)	Needs 3-way partition: LeetCode includes large all-equal inputs

APPROACH (HEAP)

1. Offer each value; when the size exceeds k, poll the smallest.
2. Return `peek()`.

JAVA SOLUTION LeetCode 215 · Solution.java (heap)

```java
import java.util.*;
```

```java
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> heap = new PriorityQueue<>();  // min-heap of the k largest
        for (int x : nums) {
            heap.offer(x);
            if (heap.size() > k) heap.poll();                  // evict the weakest
        }
        return heap.peek();                                    // k-th largest overall
    }
}
```

ALTERNATIVE: RANDOMISED QUICKSELECT WITH 3-WAY PARTITION

The k-th largest is at index `n - k` in ascending order. A Dutch-national-flag partition splits the range into `< pivot | = pivot | > pivot`. If the target index falls in the middle block, we are done. This prevents the $O(n^2)$ blow-up on arrays of identical values that a 2-way Lomuto partition suffers.

JAVA SOLUTION LeetCode 215 · Solution.java (quickselect)

```java
import java.util.*;

class Solution {
    private final Random rnd = new Random();

    public int findKthLargest(int[] nums, int k) {
        int target = nums.length - k;              // index in ascending order
        int lo = 0, hi = nums.length - 1;          // search window [lo, hi]
        while (true) {
            int pivot = nums[lo + rnd.nextInt(hi - lo + 1)];
            // Invariant: [lo, lt) < pivot, [lt, i) == pivot, (gt, hi] > pivot
            int lt = lo, i = lo, gt = hi;
            while (i <= gt) {
                if (nums[i] < pivot) swap(nums, lt++, i++);
                else if (nums[i] > pivot) swap(nums, i, gt--);
                else i++;
            }
            if (target < lt) hi = lt - 1;
            else if (target > gt) lo = gt + 1;
            else return pivot;                     // target lies in the == pivot block
        }
    }

    private void swap(int[] a, int i, int j) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
}
```

Time heap O(n log k); quickselect O(n) expected **Space** heap O(k); quickselect O(1)

> **Pitfall.** Using a max-heap of all n elements and polling k times works, but it is O(n + k log n) time and O(n) space. Fine, but say why you chose it. Quickselect mutates the input; mention that if the caller needs the array intact.

LC 347 Top K Frequent Elements

MEDIUM

Task. Return the k most frequent values in any order. The answer is guaranteed unique.

KEY INSIGHT

Count first, then run bounded top-k over the *distinct keys* using frequency as the priority. The heap stores keys, not occurrences.

WHY THE PATTERN APPLIES

After counting there are u ≤ n (key, frequency) pairs and we want the k largest by frequency. A size-k min-heap ordered by frequency gives O(n + u log k).

APPROACH

1. Build a `HashMap<Integer, Integer>` of counts.
2. Push each key into a min-heap compared by count; pop when the size exceeds k.
3. Drain the heap into the result array.

JAVA SOLUTION LeetCode 347 · Solution.java

```java
import java.util.*;

class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.merge(x, 1, Integer::sum);
        // Min-heap by frequency: the root is the least frequent of the current top-k.
        PriorityQueue<Integer> heap =
            new PriorityQueue<>((a, b) -> Integer.compare(freq.get(a), freq.get(b)));
        for (int key : freq.keySet()) {
            heap.offer(key);
            if (heap.size() > k) heap.poll();
        }
        int[] res = new int[k];
        for (int i = k - 1; i >= 0; i--) res[i] = heap.poll();  // most frequent first
        return res;
    }
}
```

Time O(n + u log k), u = distinct values **Space** O(u)

Pitfall. The follow-up asks for better than O(n log n). The heap gives O(n log k); **bucket sort** by frequency (an array of lists indexed 1..n) gives O(n). Do not compare with `freq.get(a) - freq.get(b)`; counts are small here, but the habit is dangerous.

LC 295 Find Median from Data Stream

HARD

Task. Design `MedianFinder` with `addNum(int)` and `findMedian()` returning the median of all numbers so far.

KEY INSIGHT

The median is determined by the boundary between the lower and upper halves. Keep the lower half in a max-heap and the upper half in a min-heap; the median is at one or both roots.

WHY THE PATTERN APPLIES

The data is streaming, and each query only needs the extreme of each half. That is two heaps with the invariant from 15.3.

APPROACH

1. `addNum`: push to `lo`, move `lo`'s top to `hi`, and rebalance if `hi` is larger.

2. `findMedian`: `lo.peek()` for odd counts, otherwise the mean of both roots.

JAVA SOLUTION LeetCode 295 · MedianFinder.java

```java
import java.util.*;

class MedianFinder {
    private final PriorityQueue<Integer> lo;  // max-heap: smaller half
    private final PriorityQueue<Integer> hi;  // min-heap: larger half

    public MedianFinder() {
        lo = new PriorityQueue<>(Collections.reverseOrder());
        hi = new PriorityQueue<>();
    }

    public void addNum(int num) {
        lo.offer(num);
        hi.offer(lo.poll());                        // keeps max(lo) <= min(hi)
        if (hi.size() > lo.size()) lo.offer(hi.poll()); // keeps |lo| in {|hi|, |hi| + 1}
    }

    public double findMedian() {
        if (lo.size() > hi.size()) return lo.peek();
        return ((long) lo.peek() + hi.peek()) / 2.0;
    }
}
```

Time `addNum` O(log n), `findMedian` O(1) **Space** O(n)

Pitfall. Pushing directly into the "right" heap by comparing with a root is also valid, but it is easy to break the ordering invariant. The push-then-transfer sequence enforces both invariants with no case analysis. Follow-ups: if all values lie in [0, 100], use a counting array with O(100) queries; if 99% do, use counts plus two overflow counters or lists.

Bit Manipulation

Treat an integer as 32 independent boolean lanes. Problems become easy when the condition decomposes per bit, or when an algebraic identity such as $x \oplus x = 0$ cancels everything you do not care about.

16.1 Pattern identity

Primary purpose	Achieve O(1) extra space or O(log V) time by working on the binary representation: cancellation (XOR), per-bit counting, masks as sets, and isolating or clearing specific bits.
Typical input	Arrays where elements appear a fixed number of times except a few; integer ranges; subsets of a small universe ($n \le 20$) encoded as masks; power-of-two and parity questions.
Interview applications	Single Number I/II/III, missing number, counting bits, power of two, reverse bits, subset enumeration, bitmask DP, Gray code, bitwise AND/OR of ranges, maximum XOR pair (with a trie).
Key insight	XOR is addition modulo 2 in every bit lane at once, so paired values cancel. When pairs become triples, you need a counter modulo 3 per lane, which is still a constant number of ints.

> **Key insight.** Ask what happens *in a single bit position*. If the answer's bit i depends only on the inputs' bit i (counts, parity, common prefix), solve the one-bit problem and apply it to all 32 lanes.

16.2 Recognition signals

Signal	Structural reason
"Every element appears twice/three times except…" with O(1) extra space	Multiplicities reduce modulo 2 or 3 in each bit lane; a hash map would be O(n) space.
AND/OR/XOR over a numeric range	The result is decided bit by bit; AND of a range keeps only the common binary prefix.
$n \le 20$ and "all subsets" or "visited set" state	A mask is a compact set: $2^{20} \approx 10^6$ states fit in an array.
Power of two, lowest set bit, parity	Two's-complement identities (`x & (x-1)`, `x & -x`) answer these in O(1).

> **Ask yourself.** "Is there an operation under which the unwanted elements cancel, or a per-bit statistic that reveals the answer?"

When it is inappropriate.

- **Multiplicities are arbitrary** (find any duplicate, most frequent element): parity says nothing. Use hashing, sorting, or Floyd's cycle trick (LeetCode 287).
- **Readability over micro-optimisation**: replacing `% 2` with `& 1` earns nothing in an interview unless the problem is about bits.

- **Values exceed 64 bits** or are floating point: use `BitSet` or other techniques.

Confused with	How to tell them apart
Hash map counting	Same answer with O(n) space. Bit tricks win only when the multiplicity structure is fixed and known.
Math (sum formulas)	Missing number via the sum n(n+1)/2 can overflow; XOR of 0..n cannot.
Trie on bits	Maximum XOR pair needs a greedy walk over a binary trie (Pattern 19), not a single XOR fold.

16.3 Concepts and invariants

Identity	Meaning / use
`x ^ x = 0`, `x ^ 0 = x`, XOR is associative and commutative	The order of folding does not matter; pairs cancel.
`x & (x - 1)`	Clears the lowest set bit. Used in Kernighan popcount (loop runs once per set bit) and the power-of-two test `x > 0 && (x & (x-1)) == 0`.
`x & -x`	Isolates the lowest set bit, since `-x = ~x + 1`. Drives Fenwick trees (Pattern 20) and Single Number III.
`(x >> i) & 1`, `x | (1 << i)`, `x & ~(1 << i)`, `x ^ (1 << i)`	Get, set, clear and toggle bit i.
`for (s = m; s > 0; s = (s - 1) & m)`	Enumerates all non-empty submasks of m in decreasing order. Over all m this totals $O(3^n)$.

Java's fixed-width integers. `int` is 32-bit two's complement, `long` is 64-bit; there are no unsigned types.

- `>>` is an *arithmetic* shift (copies the sign bit): `-8 >> 1 == -4`. `>>>` is a *logical* shift (fills with 0): `-8 >>> 1 == 2147483644`. A loop `while (x != 0) x >>= 1` never terminates for negative x; use `>>>`.
- Shift distances are taken modulo the width: `1 << 32 == 1` for an int, and `1 << 31 == Integer.MIN_VALUE`. Use `1L << i` for 64-bit masks.
- `Integer.MIN_VALUE` is its own negation: `-MIN_VALUE == MIN_VALUE`. Therefore `MIN_VALUE & -MIN_VALUE == MIN_VALUE`, which is still the correct lowest set bit (bit 31).
- Built-ins: `Integer.bitCount`, `Integer.numberOfTrailingZeros`, `Integer.highestOneBit`, `Integer.lowestOneBit`, `Integer.toBinaryString` (shows negatives as 32 bits).

Per-bit counting modulo k: the general Single Number method

If every value appears exactly k times except one value that appears once, then for each bit i the number of inputs with bit i set is $k \cdot c + b_i$, where b_i is the unique value's bit. So $b_i = \text{count}_i \bmod k$. Setting bit 31 rebuilds negative numbers correctly, because two's complement is just the bit pattern. This costs O(32·n) time and O(1) space and works for any k. The constant-time state machine for k = 3 (LeetCode 137) is this counter compressed into two ints.

16.4 Reusable Java templates

Template A — bit toolkit. These are reusable as is.

JAVA TEMPLATE BitTricks.java

```
import java.util.*;
```

```java
/** Core bit identities for 32-bit ints (two's complement). */
class BitTricks {
    static boolean getBit(int x, int i)  { return ((x >>> i) & 1) == 1; }
    static int setBit(int x, int i)      { return x | (1 << i); }
    static int clearBit(int x, int i)    { return x & ~(1 << i); }
    static int toggleBit(int x, int i)   { return x ^ (1 << i); }
    static int lowestSetBit(int x)       { return x & -x; }        // 0 if x == 0
    static int clearLowestSetBit(int x)  { return x & (x - 1); }
    static boolean isPowerOfTwo(int x)   { return x > 0 && (x & (x - 1)) == 0; }

    /** Kernighan: one iteration per set bit; correct for negatives (32 iterations max). */
    static int popCount(int x) {
        int c = 0;
        while (x != 0) { x &= x - 1; c++; }
        return c;
    }

    /** All non-empty submasks of mask, largest first. */
    static List<Integer> submasks(int mask) {
        List<Integer> out = new ArrayList<>();
        for (int s = mask; s > 0; s = (s - 1) & mask) out.add(s);
        return out;
    }
}
```

Template B — per-bit counting modulo k. Problem-specific: k, and whether a different multiplicity (other than 1) must be extracted.

JAVA TEMPLATE PerBitCount.java

```java
/** Every value appears k times except one that appears once: rebuild it bit by bit. */
class PerBitCount {
    static int singleAmongK(int[] nums, int k) {
        int result = 0;
        for (int bit = 0; bit < 32; bit++) {
            int count = 0;
            for (int x : nums) count += (x >>> bit) & 1;  // >>> : sign bit is not smeared
            if (count % k != 0) result |= 1 << bit;       // bit 31 set => negative result
        }
        return result;
    }
}
```

16.5 Complexity and pitfalls

Time	O(n) for XOR folds; O(32·n) = O(n) for per-bit counting; O(log V) for range/prefix problems; $O(2^n)$ for mask enumeration.
Auxiliary space	O(1) for the folds and counters; $O(2^n)$ for mask DP tables.
Off-by-one	Bit positions run 0..31; `1 << 31` is negative; looping `bit <= 32` wraps to bit 0.
Wrong invariant	Using `>>` on negatives in a "while non-zero" loop; testing `(x & mask) == 1` instead of `!= 0`.
Overflow	Masks wider than 31 bits need `long` and `1L << i`. `Math.abs(Integer.MIN_VALUE)` is still negative.
Precedence	`==` binds tighter than `&`, `^`, `|`. `x & 1 == 0` does not compile in Java (int & boolean). Always parenthesise.
Interview follow-ups	Generalise to "appears k times"; find two missing numbers; maximum XOR of two numbers (bit trie); count bits for 0..n in O(n) via `bits[i] = bits[i >> 1] + (i & 1)`.

16.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- XOR cancels pairs; per-bit counts modulo k handle k-fold repetition.
- `x & (x-1)` clears the lowest set bit; `x & -x` isolates it.
- Use `>>>` when walking the bits of possibly negative values.
- Range AND = common binary prefix; anything below the first differing bit becomes 0.
- Parenthesise every bitwise expression inside a comparison.

> **Recognition rule.** Fixed multiplicities + O(1) space → XOR or per-bit counting. Range bitwise op → think about the common prefix.

IN-INTERVIEW CHECKLIST

- [ ] Does the condition decompose per bit?
- [ ] Which identity cancels the noise (XOR, AND with x−1, lowbit)?
- [ ] Signed shifts, bit 31, and `Integer.MIN_VALUE` handled?
- [ ] Is `int` wide enough, or do I need `long` masks?
- [ ] Would a hash map be clearer, and is O(1) space actually required?

Key distinction. Bit tricks rely on *exact, known multiplicities*. Hash counting handles arbitrary multiplicities at O(n) space.

16.7 Assigned problems

LC 136 Single Number — EASY

Task. Every element appears twice except one. Find it in O(n) time and O(1) extra space.

KEY INSIGHT

XOR of all elements: each pair contributes `x ^ x = 0`, and order does not matter, so only the single value survives.

WHY THE PATTERN APPLIES

The multiplicity is fixed at 2, and XOR is exactly addition modulo 2 in every bit lane.

APPROACH

1. Fold XOR over the array.

JAVA SOLUTION LeetCode 136 · Solution.java

```java
class Solution {
    public int singleNumber(int[] nums) {
        int x = 0;
        for (int v : nums) x ^= v;  // pairs cancel; order irrelevant
        return x;
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Negative values need no special handling; XOR works on the raw bit patterns. A sum-based trick (`2·sum(set) − sum`) uses O(n) space and can overflow.

LC 137 Single Number II

MEDIUM

Task. Every element appears three times except one, which appears once. Find it in linear time and constant space.

KEY INSIGHT

In each bit lane, the count of ones is 3c + b, where b is the answer's bit. Keep a counter modulo 3 per lane. Two ints `ones` and `twos` store the counter's two bits for all 32 lanes simultaneously.

WHY THE PATTERN APPLIES

The condition decomposes per bit with a fixed multiplicity. Template B solves it in O(32n). The state machine below does the same in O(n) with a few operations per element.

APPROACH

```
per-lane state (twos, ones):  00 → 01 → 10 → 00   on each incoming 1-bit
ones = (ones ^ x) & ~twos    toggle into 'one' unless already at 'two'
twos = (twos ^ x) & ~ones    toggle into 'two' unless we just moved to 'one'
after all input: lanes holding count ≡ 1 (mod 3) are exactly the set bits of 'ones'
```

JAVA SOLUTION LeetCode 137 · Solution.java

```java
class Solution {
    public int singleNumber(int[] nums) {
        int ones = 0, twos = 0;  // per-bit counter mod 3, stored as two bit-planes
        for (int x : nums) {
            ones = (ones ^ x) & ~twos;
            twos = (twos ^ x) & ~ones;
        }
        return ones;             // lanes with count % 3 == 1
    }
}
```

Time O(n) **Space** O(1)

Pitfall. The update order matters: `twos` must use the *new* `ones`. If you cannot derive the state machine under pressure, Template B (count each of the 32 bits modulo 3) is equally linear, easier to defend, and handles negatives because bit 31 is rebuilt directly.

LC 260 Single Number III

MEDIUM

Task. Exactly two elements appear once and all others appear twice. Return the two singles, in any order, in O(n) time and O(1) space.

KEY INSIGHT

The XOR of everything is `a ^ b ≠ 0`. Any set bit of it, for example the lowest via `d & -d`, is a position where a and b differ. Splitting the array by that bit puts a and b in different groups, while each duplicate pair stays together. Each group is then a Single Number I.

WHY THE PATTERN APPLIES

XOR cancellation plus lowest-set-bit isolation turns one problem with two unknowns into two problems with one unknown each.

APPROACH

1. `d = XOR of all`; `mask = d & -d`.
2. XOR separately the elements with `(v & mask) != 0` and those without.

JAVA SOLUTION LeetCode 260 · Solution.java

```java
class Solution {
    public int[] singleNumber(int[] nums) {
        int diff = 0;
        for (int v : nums) diff ^= v;          // = a ^ b, non-zero because a != b
        int mask = diff & -diff;               // lowest bit where a and b differ
        int a = 0, b = 0;                      // diff == MIN_VALUE still works
        for (int v : nums) {
            if ((v & mask) != 0) a ^= v;       // parenthesised: != binds tighter than &
            else b ^= v;
        }
        return new int[]{a, b};
    }
}
```

Time O(n) **Space** O(1)

> **Pitfall.** In languages with overflow traps, `-diff` for `diff = INT_MIN` is undefined. In Java it wraps to itself and the mask is still the correct bit 31. Test `(v & mask) != 0`, not `== 1`: the mask is usually not bit 0.

LC 201 Bitwise AND of Numbers Range

MEDIUM

Task. Given `0 ≤ left ≤ right ≤ 2^31 − 1`, return the AND of every integer in `[left, right]`.

KEY INSIGHT

The result is the common binary prefix of `left` and `right` followed by zeros. Below the highest bit where they differ, the range contains a number ending in `0111…` followed by one ending in `1000…`, so every lower bit is 0 somewhere in the range.

WHY THE PATTERN APPLIES

AND decomposes per bit: bit i survives only if it is 1 for every number in the range. Consecutive integers sweep through all patterns below the first differing bit, so only the shared prefix survives.

APPROACH

1. Shift both right until equal, counting the shifts.
2. Shift the common prefix back left.

JAVA SOLUTION LeetCode 201 · Solution.java

```java
class Solution {
    public int rangeBitwiseAnd(int left, int right) {
        int shift = 0;
        while (left < right) {      // differing bits remain below the common prefix
            left >>= 1;             // inputs are non-negative, so >> and >>> agree
            right >>= 1;
            shift++;
        }
        return left << shift;       // common prefix followed by zeros
    }
}
```

Time O(log right) ≤ 31 iterations **Space** O(1)

Pitfall. Looping from left to right and ANDing is O(range) and times out for ranges of 2^{31}. An equivalent loop is `while (right > left) right &= right - 1;`, which clears right's low bits one at a time until it falls to or below left.

Union-Find / Disjoint Set Union

A forest of parent pointers that answers "are these two in the same group?" and "merge these groups" in near-constant amortised time. It is the tool for connectivity that grows over time and never shrinks.

17.1 Pattern identity

Primary purpose	Maintain a partition of elements under `union(a, b)` and `find(a)` in O(α(n)) amortised time per operation.
Typical input	A list of edges or equivalence relations ("a is connected to b", "a == b", "these two accounts share an email"), processed in any order, often with queries interleaved.
Interview applications	Number of components, cycle detection in undirected graphs, redundant connection, accounts merge, Kruskal's MST, equations satisfiability, islands with incremental land (305), grid percolation.
Key insight	Each group is identified by a single *representative* (its root). Two elements are connected exactly when their roots are equal. Path compression and union by size keep the trees almost flat.

> **Key insight.** DSU does not store *how* elements are connected, only *whether* they are. If you need paths, distances or edge directions, traverse the graph instead. If you only need connectivity under insertions, DSU is the simplest correct tool.

17.2 Recognition signals

Signal	Structural reason
Edges or equivalences arrive one by one with connectivity queries in between	Online incremental connectivity: re-running BFS per query would cost O(V+E) each.
"Which edge creates a cycle?" in an undirected graph	An edge (u, v) closes a cycle iff u and v already share a root before the union.
Grouping by a shared attribute (email, value, row/column)	Transitive equivalence: union every item with the first item that has the same attribute.
Count groups or components	Start with n components and decrement on each successful union.
Kruskal MST, processing edges in sorted order	Accept an edge iff it joins two different components.

> **Ask yourself.** "Do I only ever *merge* groups and ask whether two items share a group?" If yes, use DSU. If groups must split, or the question involves paths or directions, use something else.

When it is inappropriate.

- **Deletions or splits**: DSU cannot undo a union efficiently. Offline tricks exist (process deletions in reverse as insertions; rollback DSU without path compression), but standard DSU cannot split a component.

- **Directed reachability**: union is symmetric, so it loses edge direction. Use DFS/BFS, topological sort, or strongly connected components.
- **Shortest paths or actual paths**: DSU keeps no path information.

Confused with	How to tell them apart
DFS/BFS components	The same O(V+E) for a static graph given up front. DFS is natural when you also need component contents or paths; DSU wins for online edges and for edges given as an unordered list.
Topological sort / directed cycle	DSU detects only *undirected* cycles. A directed cycle needs colouring or Kahn's algorithm.
Hash-map grouping	A map groups by one exact key. DSU merges transitively across different keys (a~b via email 1, b~c via email 2).

17.3 Concepts and invariants

Representation. `parent[x]` is x's parent and roots satisfy `parent[r] == r`. Invariant: following parents from any element reaches its component's unique root.

Two heuristics, used together.

- **Union by size (or rank)**: attach the smaller tree under the larger root. A node's depth increases only when its tree at least doubles in size, so depth ≤ $\log_2 n$ even without compression.
- **Path compression**: during `find`, point visited nodes (all of them, or every other one with path halving) closer to the root, flattening future queries.

```
With both heuristics: m operations on n elements cost O(m · α(n))   (Tarjan)
α = inverse Ackermann; α(n) ≤ 4 for any n that fits in memory  → "effectively constant"
Union by size alone: O(log n) per find.   Compression alone: O(log n) amortised.
```

Cycle detection. Process undirected edges in order. If `find(u) == find(v)` before the union, a u–v path already exists, so adding (u, v) closes a cycle. This is also Kruskal's rejection rule.

Component counting. `components = n − (number of successful unions)`, because each successful union merges exactly two components.

Recursive vs iterative find

Recursive full compression (`parent[x] = find(parent[x])`) is concise, but on a long chain built *without* union by size the first call recurses to depth n and can overflow the stack. With union by size the depth is at most log n, so recursion is safe. Iterative *path halving* (`parent[x] = parent[parent[x]]` inside the loop) has the same amortised bound and no recursion at all, which is why the template uses it.

17.4 Reusable Java templates

Template A — array DSU over 0..n−1. Reusable as is. Problem-specific: what counts as an edge, and what you do when `union` returns false.

JAVA TEMPLATE DSU.java

```java
/** Disjoint Set Union with path halving + union by size. Elements are 0..n-1. */
class DSU {
    private final int[] parent, size;
```

```
    private int components;

    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];   // path halving: skip a level each step
            x = parent[x];
        }
        return x;
    }

    /** Returns false if a and b were already connected (=> the edge a-b closes a cycle). */
    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }  // smaller under larger
        parent[rb] = ra;
        size[ra] += size[rb];
        components--;
        return true;
    }

    boolean connected(int a, int b) { return find(a) == find(b); }
    int componentSize(int x)        { return size[find(x)]; }
    int components()                { return components; }
}
```

Template B — DSU over arbitrary keys. For strings or sparse integers (emails, coordinates), map keys to ids lazily.

JAVA TEMPLATE KeyedDSU.java

```
import java.util.*;

/** DSU over hashable keys; ids are assigned on first sight. */
class KeyedDSU<K> {
    private final Map<K, Integer> id = new HashMap<>();
    private final List<Integer> parent = new ArrayList<>();
    private final List<Integer> size = new ArrayList<>();

    int idOf(K key) {
        Integer i = id.get(key);
        if (i != null) return i;
        int n = parent.size();
        id.put(key, n);
        parent.add(n);
        size.add(1);
        return n;
    }

    int find(int x) {
        while (parent.get(x) != x) {
            parent.set(x, parent.get(parent.get(x)));
            x = parent.get(x);
        }
        return x;
    }

    boolean union(K a, K b) {
```

```
        int ra = find(idOf(a)), rb = find(idOf(b));
        if (ra == rb) return false;
        if (size.get(ra) < size.get(rb)) { int t = ra; ra = rb; rb = t; }
        parent.set(rb, ra);
        size.set(ra, size.get(ra) + size.get(rb));
        return true;
    }

    boolean connected(K a, K b) { return find(idOf(a)) == find(idOf(b)); }
}
```

Note: `parent.get(x) != x` compares an `Integer` with an `int`, which unboxes, so the comparison is numeric and correct. Comparing two `Integer` objects with `!=` would compare references and break above 127.

17.5 Complexity and pitfalls

Time	O(α(n)) amortised per operation with both heuristics; O(n + m α(n)) total. Building from E edges: O(V + E α(V)).
Auxiliary space	O(n) for `parent` and `size`.
Off-by-one	1-indexed vertices (684 uses 1..n): allocate `n + 1` or subtract 1 consistently.
Wrong invariant	`parent[a] = b` instead of `parent[find(a)] = find(b)`, which links non-roots and corrupts components. Comparing `parent[a] == parent[b]` instead of roots.
Boxing	`Integer` reference comparison in map/list-based DSUs (see note above).
Breaks naive code	Self-loops (u == v: union returns false, which is correctly a cycle), duplicate edges, isolated vertices (count them as components), and deep recursion without union by size.
Interview follow-ups	Weighted DSU (ratios, LeetCode 399); DSU with rollback; offline deletions in reverse; component sizes; why DSU cannot detect directed cycles.

17.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Always link roots: `parent[find(a)] = find(b)`.
- Union by size + path compression → effectively O(1) amortised.
- `union` returning false is your cycle detector and Kruskal filter.
- Components = n − successful unions.
- No splits, no directions, no paths: those need other tools.

> **Recognition rule.** Undirected relations merging over time + "same group?" or "how many groups?" → DSU.

IN-INTERVIEW CHECKLIST

- ☐ What are the elements, and how do I map them to 0..n−1?
- ☐ What triggers a union? What does a failed union mean here?
- ☐ 0- or 1-indexed?
- ☐ Are isolated elements counted?
- ☐ Does the problem ever need a split or edge direction? (If so, not DSU.)

Key distinction. DSU answers connectivity *online under insertions*. DFS/BFS answers richer questions (paths, order, distances) on a *static* graph.

17.7 Assigned problems

LC 1971 Find if Path Exists in Graph

EASY

Task. Given an undirected graph on vertices 0..n−1 as an edge list, decide whether `source` and `destination` are connected.

KEY INSIGHT

A path exists iff both endpoints end up in the same component after all edges are unioned. No adjacency list is needed.

WHY THE PATTERN APPLIES

The question is pure undirected connectivity over an edge list, which is DSU's exact interface. BFS/DFS also works (see Chapter 18's version); DSU avoids building the adjacency list.

APPROACH

1. Union every edge.
2. Return `find(source) == find(destination)`.

JAVA SOLUTION LeetCode 1971 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public boolean validPath(int n, int[][] edges, int source, int destination) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        for (int[] e : edges) union(e[0], e[1]);   // undirected, unweighted edges
        return find(source) == find(destination);  // source == destination: trivially true
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
    }
}
```

Time O(n + E·α(n)) **Space** O(n)

Pitfall. An early exit (stop once source and destination connect) is a valid micro-optimisation. With n up to $2 \cdot 10^5$, recursive DFS risks `StackOverflowError` on path-shaped graphs; the iterative DSU has no such risk.

LC 684 Redundant Connection

MEDIUM

Task. A tree on n nodes (labelled 1..n) had one extra edge added. Return an edge whose removal leaves a tree; if several qualify, return the one that occurs last in the input.

KEY INSIGHT

Scan edges in order. The first edge whose endpoints are already connected closes the (unique) cycle. Every other cycle edge appears earlier, so this edge is the last cycle edge in the input, which is exactly the required answer.

WHY THE PATTERN APPLIES

This is undirected cycle detection on edge insertion: `union` returns false exactly at the redundant edge.

APPROACH

1. DSU over 1..n (allocate n + 1).
2. For each edge, if `union` fails, return it.

JAVA SOLUTION LeetCode 684 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int[] findRedundantConnection(int[][] edges) {
        int n = edges.length;                    // tree + 1 edge => edges == nodes
        parent = new int[n + 1];                 // labels are 1..n
        size = new int[n + 1];
        for (int i = 0; i <= n; i++) { parent[i] = i; size[i] = 1; }
        for (int[] e : edges) {
            if (!union(e[0], e[1])) return e;    // already connected: closes the cycle
        }
        return new int[0];                       // unreachable for valid input
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(n·α(n)) **Space** O(n)

Pitfall. Why is the first failing edge the *last* cycle edge in input order? Before it, the edges seen form a forest, so no cycle edge has failed. It is the first moment the cycle is complete, so all other cycle edges came earlier. The directed variant (685) is different: DSU alone is not enough, because nodes with two parents need separate handling.

LC 547 Number of Provinces

MEDIUM

Task. Given an n × n symmetric adjacency matrix `isConnected`, count the connected components.

KEY INSIGHT

Start with n components and decrement on each successful union over the upper triangle of the matrix.

WHY THE PATTERN APPLIES

This is undirected component counting, DSU's "components = n − successful unions" invariant. The same problem appears with BFS in Chapter 3 and recursive DFS in Chapter 9. All three are $O(n^2)$ because the input matrix itself has n^2 cells.

APPROACH

1. For each pair `i < j` with `isConnected[i][j] == 1`, union them.
2. Return the remaining component count.

JAVA SOLUTION LeetCode 547 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        int components = n;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {        // symmetric: upper triangle only
                if (isConnected[i][j] == 1 && union(i, j)) components--;
            }
        }
        return components;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time $O(n^2 \cdot \alpha(n))$ **Space** $O(n)$

Pitfall. Counting distinct values of `parent[i]` at the end is wrong without a final `find` on each element, because non-roots may point to stale intermediate parents. Maintaining the counter during unions avoids this.

LC 721 Accounts Merge

MEDIUM

Task. Each account is `[name, email1, email2, …]`. Two accounts belong to the same person if they share any email. Merge them and return each person's name followed by their emails in sorted order; accounts may be returned in any order.

KEY INSIGHT

Sharing an email is a transitive equivalence between *accounts*. Map each email to the first account index that contained it, and union every later account containing that email with that index.

WHY THE PATTERN APPLIES

Groups are formed transitively across different keys: A~B via one email, B~C via another. DSU merges these incrementally without building an explicit email graph. Two people can share a name, so names must never be used as keys.

APPROACH

1. DSU over account indices. `owner: email → first account index`; union on repeats.
2. Group emails by `find(owner[email])` into a `TreeSet` (sorted, deduplicated).
3. Emit `name(root)` followed by the sorted emails.

JAVA SOLUTION LeetCode 721 · Solution.java

```java
import java.util.*;

class Solution {
    private int[] parent, size;

    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        int n = accounts.size();
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }

        Map<String, Integer> owner = new HashMap<>();        // email -> first account index
        for (int i = 0; i < n; i++) {
            List<String> acc = accounts.get(i);
            for (int j = 1; j < acc.size(); j++) {           // index 0 is the name
                Integer prev = owner.putIfAbsent(acc.get(j), i);
                if (prev != null) union(prev, i);            // shared email => same person
            }
        }

        Map<Integer, TreeSet<String>> groups = new HashMap<>();
        for (Map.Entry<String, Integer> e : owner.entrySet()) {
            int root = find(e.getValue());
            groups.computeIfAbsent(root, k -> new TreeSet<>()).add(e.getKey());
        }

        List<List<String>> res = new ArrayList<>();
        for (Map.Entry<Integer, TreeSet<String>> g : groups.entrySet()) {
            List<String> merged = new ArrayList<>();
```

```
            merged.add(accounts.get(g.getKey()).get(0));     // the group's shared name
            merged.addAll(g.getValue());                     // sorted, no duplicates
            res.add(merged);
        }
        return res;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
    }
}
```

Time O(L log L), L = total emails (sorting dominates; DSU adds O(L·α(n))) **Space** O(L)

Pitfall. Do not merge by name: two different "John"s with disjoint emails must stay separate. Sort with natural `String` order (the problem uses ASCII order, which is what `TreeSet<String>` gives). DFS over an email graph is an equally valid alternative with the same complexity.

Shortest Path: Dijkstra's Algorithm

A priority queue settles vertices in non-decreasing order of distance. With non-negative weights, the first time a vertex leaves the queue its distance is final.

18.1 Pattern identity

Primary purpose	Single-source shortest paths when edges have different *non-negative* costs, or more generally when path cost never decreases as a path is extended.
Typical input	An edge list `[u, v, w]` (directed or undirected), a weighted grid where moving between cells has a cost, or an implicit state graph (node + extra resource).
Interview applications	Network delay, cheapest route, minimum-effort grid path, maximum-probability path, counting shortest paths, shortest path with a resource limit (expanded state).
Key insight	Among the unsettled vertices, the one with the smallest tentative distance cannot be improved later. Every other route to it passes through an unsettled vertex whose distance is already at least as large, and non-negative edges can only add.

> **Key insight.** Dijkstra is BFS with a priority queue instead of a FIFO queue. It needs exactly one property: *extending a path never makes it cheaper*. Sums of non-negative weights have it; so do the maximum edge on a path and the product of probabilities ≤ 1. Negative edges do not.

18.2 Recognition signals

Signal	Structural reason
"Minimum cost / time / effort to reach…" with edge-specific costs	Edge weights differ, so the FIFO order of BFS no longer matches distance order.
All weights ≥ 0 (check the constraints line explicitly)	Required for the settle-once invariant.
One source (or a virtual super-source), many targets	Dijkstra computes the full shortest-path tree from one source in a single run.
Path cost is a max, min or product rather than a sum	Still works if the cost is monotone along the path (bottleneck, probability). Check this, do not assume it.
An extra limit (≤ K edges, fuel, stops)	Distance alone is no longer a sufficient state. Expand the state to (vertex, resource) or switch to bounded Bellman–Ford.
V up to ~10^5, E up to ~$2 \cdot 10^5$	The O((V+E) log V) budget fits; Bellman–Ford's O(VE) does not.

> **Ask yourself.** "If I extend a path by one edge, can its cost decrease, and does the cheapest path to a vertex remain the best prefix for every continuation?" Two yeses (no decrease, cheapest prefix still best) mean Dijkstra on the plain vertex graph. If the cheapest prefix might be the wrong one because of a constraint, the state is too small.

When it is inappropriate.

- **Unweighted or unit-weight graphs**: BFS gives the same answer in O(V+E) without the log factor (Pattern 10).
- **0/1 weights**: 0-1 BFS with a deque is O(V+E).
- **Negative edge weights**: the settled-is-final invariant fails. Use Bellman–Ford (A7), or SPFA with care. Negative cycles make shortest paths undefined.
- **Only reachability is asked**: BFS, DFS or DSU (see 1971).
- **All-pairs on small graphs (V ≤ ~400)**: Floyd–Warshall's $O(V^3)$ is simpler.
- **DAGs**: relaxing in topological order is O(V+E) and allows negative weights.

Confused with	How to tell them apart
BFS	BFS is Dijkstra in the special case where all weights are equal. If weights differ, BFS's first visit is not the shortest.
Bellman–Ford	Handles negative weights and edge-count limits through repeated rounds of relaxing every edge. It is slower: O(VE).
Prim's MST	Same code shape, but Prim keys a vertex by *one edge weight*, while Dijkstra keys by the *whole path cost*. They produce different trees.
A*	Dijkstra plus an admissible heuristic. Rare in interviews; Dijkstra is A* with h = 0.

18.3 Concepts and invariants

Relaxation. For edge (u, v, w): if `dist[u] + w < dist[v]`, set `dist[v] = dist[u] + w` and push `(dist[v], v)`. Every algorithm in this family is a schedule of relaxations. Dijkstra's schedule relaxes the outgoing edges of each vertex exactly once, in order of final distance.

```
Invariant: when u is popped with d == dist[u], dist[u] = δ(s, u)   (true shortest
distance)

Proof sketch: let P be a shortest s→u path, and let (x, y) be its first edge with x
settled, y not.
  dist[y] ≤ δ(s, x) + w(x, y) = δ(s, y) ≤ δ(s, u)        (prefix of a shortest path; w ≥
0)
  u was chosen as the minimum  ⇒  dist[u] ≤ dist[y] ≤ δ(s, u) ≤ dist[u]   ⇒  equality.
The step δ(s, y) ≤ δ(s, u) is exactly where non-negative weights are used.
```

Stale entries (lazy deletion). `java.util.PriorityQueue` has no efficient decrease-key, so push a new entry on every improvement. Old entries stay in the heap. When popped, an entry whose `d > dist[u]` is stale and is skipped. The heap holds at most O(E) entries, so each operation costs O(log E) = O(log V), because $E \le V^2$.

```
Total: O((V + E) log V) time,  O(V + E) space (adjacency list + up to E heap entries)
Dense graphs (E ≈ V²): the array version (scan for the minimum) is O(V²) and beats the
heap.
```

Why negative weights break it. Take the edges s→a (2), s→b (5) and b→a (−4). Dijkstra settles a at distance 2, but the true distance is 1 via b. Once a is settled, its outgoing edges were relaxed with the wrong value, so the error propagates. Adding a constant to every edge does *not* fix it, because paths with more edges are penalised more.

Generalised path costs. Dijkstra works for any cost that satisfies (1) *monotonicity*: cost(P + e) ≥ cost(P), and (2) *isotonicity*: if cost(P) ≤ cost(Q) for two paths to the same vertex, then cost(P + e) ≤ cost(Q + e). Bottleneck `max(cost, w)` (LC 1631) and the probability product with factors in [0, 1] under a max-heap (A2) both qualify.

State expansion. When a constraint makes "cheapest so far" insufficient (for example, a cheap path that has used too many edges), encode the constraint in the state: vertex v becomes (v, edgesUsed). The expanded graph has V·(K+2) states, and Dijkstra applies unchanged. The first time the target appears at the top of the heap, in *any* layer, its cost is optimal.

Grid graphs and representation choices

- **Grid**: vertex = cell `(r, c)`, encoded as `r * cols + c`. There are 4 implicit neighbours and the weight is computed on the fly (for example $|h_a - h_b|$), so no adjacency list is needed. V = R·C and E ≈ 4V.
- **Adjacency list**: `List<int[]>[]` with `{to, w}`. For undirected edges add both directions. Always check whether the problem is 0- or 1-indexed.
- **Distance type**: use `long` when V·maxW can exceed $2^{31}-1$. When the sum is used as a comparator key, compare with `Long.compare`, never by subtraction.
- **Early exit**: for a single target, return when the target is popped (not when it is first pushed).

18.4 Reusable Java templates

Template A — adjacency-list Dijkstra with lazy deletion. Reusable: everything. Problem-specific: graph construction (directed or undirected, index base) and what you read from `dist`.

JAVA TEMPLATE Dijkstra.java

```java
import java.util.*;

/** Single-source shortest paths, non-negative weights. Vertices 0..n-1, edges {u, v, w}. */
class Dijkstra {
    static final long INF = Long.MAX_VALUE / 4;   // headroom: INF + w never overflows

    static long[] shortest(int n, int[][] edges, int src, boolean directed) {
        List<int[]>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] e : edges) {
            adj[e[0]].add(new int[]{e[1], e[2]});
            if (!directed) adj[e[1]].add(new int[]{e[0], e[2]});
        }
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        // Entry {distance, vertex}; Long.compare avoids subtraction overflow.
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
        pq.add(new long[]{0, src});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
            long d = top[0];
            int u = (int) top[1];
            if (d > dist[u]) continue;  // stale entry: u was settled with a smaller d
            for (int[] e : adj[u]) {
                int v = e[0];
                long nd = d + e[1];
                if (nd < dist[v]) {               // relaxation
                    dist[v] = nd;
                    pq.add(new long[]{nd, v});
```

```
                }
            }
        }
        return dist;                                // INF marks unreachable vertices
    }
}
```

Template B — grid Dijkstra with a pluggable path-cost rule. The `combine` line is the only problem-specific part: `d + w` for sums, `Math.max(d, w)` for bottleneck costs.

JAVA TEMPLATE GridDijkstra.java

```
import java.util.*;

/** Dijkstra on an R x C grid, 4-directional; edge weight computed from the two cells. */
class GridDijkstra {
    static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};

    static int minCost(int[][] g, boolean bottleneck) {
        int R = g.length, C = g[0].length;
        int[] dist = new int[R * C];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{0, 0});                            // {cost, cellId}
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int d = top[0], id = top[1];
            if (d > dist[id]) continue;
            if (id == R * C - 1) return d;                  // target popped: final
            int r = id / C, c = id % C;
            for (int k = 0; k < 4; k++) {
                int nr = r + DR[k], nc = c + DC[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                int w = Math.abs(g[nr][nc] - g[r][c]);      // problem-specific weight
                int nd = bottleneck ? Math.max(d, w) : d + w; // problem-specific combine
                int nid = nr * C + nc;
                if (nd < dist[nid]) {
                    dist[nid] = nd;
                    pq.add(new int[]{nd, nid});
                }
            }
        }
        return dist[R * C - 1];
    }
}
```

Template C — Dijkstra on an expanded state (vertex, resource used). Use it when a limit such as "at most K edges" makes the cheapest prefix insufficient.

JAVA TEMPLATE ResourceDijkstra.java

```
import java.util.*;

/** Cheapest src->dst path using at most maxEdges edges. Directed {u, v, w}, w >= 0. */
class ResourceDijkstra {
    static long cheapest(int n, int[][] edges, int src, int dst, int maxEdges) {
        List<int[]>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] e : edges) adj[e[0]].add(new int[]{e[1], e[2]});
        long INF = Long.MAX_VALUE / 4;
        // best[v][e]: cost reaching v with e edges
        long[][] best = new long[n][maxEdges + 1];
```

```
        for (long[] row : best) Arrays.fill(row, INF);
        best[src][0] = 0;
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
        pq.add(new long[]{0, src, 0});                // {cost, vertex, edgesUsed}
        while (!pq.isEmpty()) {
            long[] t = pq.poll();
            long d = t[0];
            int u = (int) t[1], used = (int) t[2];
            if (d > best[u][used]) continue;          // stale state
            if (u == dst) return d;                   // cheapest valid state overall
            if (used == maxEdges) continue;           // resource exhausted
            for (int[] e : adj[u]) {
                long nd = d + e[1];
                if (nd < best[e[0]][used + 1]) {
                    best[e[0]][used + 1] = nd;
                    pq.add(new long[]{nd, e[0], used + 1});
                }
            }
        }
        return -1;
    }
}
```

18.5 Complexity and pitfalls

Time	O((V + E) log V) with a binary heap and lazy deletion; $O(V^2)$ with the array version on dense graphs; expanded state: O(K·E·log(K·E)).
Auxiliary space	O(V + E): adjacency list, `dist`, and up to O(E) heap entries.
Off-by-one	1-indexed vertices (LC 743) sized as `n` instead of `n + 1`; "K stops" means K + 1 edges.
Wrong invariant	Marking a vertex visited when it is *pushed* (that is the BFS habit). In Dijkstra a vertex is final only when *popped* with `d == dist[u]`.
Overflow	`Integer.MAX_VALUE + w` wraps negative and wins every comparison. Use `long` with `INF = Long.MAX_VALUE / 4`, or check for INF before adding. Never write the comparator as `a[0] - b[0]` on large values.
Breaks naive code	Negative weights; zero-weight cycles (fine for Dijkstra, but they defeat naive "visit count" hacks); multiple edges between the same pair (keep all of them, since relaxation handles it); disconnected targets (return −1 / INF).
Interview follow-ups	Reconstruct the path (store `parent[v]` on relaxation); count shortest paths (A2 · 1976); multi-source (push every source at distance 0); a K-edges limit (state expansion or Bellman–Ford); negative weights (Bellman–Ford, A7).

18.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Correctness rests on one property: extending a path never lowers its cost.
- Lazy deletion with the `d > dist[u]` check is the standard Java implementation; never skip that check.
- A vertex is final when popped, not when pushed.
- If a constraint makes "cheapest so far" insufficient, expand the state.
- Use BFS for unit weights, Bellman–Ford for negative weights, and the plain array version for dense graphs.

> **Recognition rule.** Weighted edges + non-negative costs + "minimum total" from a source → Dijkstra. Before coding, ask whether the vertex alone is a sufficient state.

IN-INTERVIEW CHECKLIST

- ☐ Directed or undirected? 0- or 1-indexed?
- ☐ Are all weights ≥ 0? Is the cost a sum, a max or a product?
- ☐ Does the vertex alone determine the future, or do I need (vertex, resource)?
- ☐ Distance type: does V·maxW fit in an `int`?
- ☐ What is returned for unreachable targets?

Key distinction. BFS settles by number of edges and Dijkstra by total weight. If all weights are equal, use BFS; if any weight is negative, use neither.

18.7 Assigned problems

> **Classification overview.** Only **743** is a classic sum-of-weights Dijkstra. **1971** is unweighted reachability, so BFS is the right tool. **1631** is a bottleneck (minimax) variant, valid because the max-edge cost is monotone. **787** has an edge-count limit that breaks plain vertex Dijkstra; it is solved here with an expanded state and in A7 with Bellman–Ford.

LC 1971 Find if Path Exists in Graph — EASY

Task. Given an undirected graph on `n` vertices (0-indexed edge list), decide whether `source` can reach `destination`.

> **Classification note.** This is *not* a Dijkstra problem. Edges carry no weights and only existence is asked, so any traversal answers it in O(V + E). Dijkstra would also be correct, since it degenerates to BFS, but it adds a log factor and signals that you have not looked at the structure. The DSU solution appears in Chapter 17.

KEY INSIGHT

Reachability depends only on connectivity. The first time BFS reaches `destination` is enough; distances are irrelevant.

WHY THE PATTERN APPLIES

It belongs here as a contrast case. Recognising that weights are absent is the shortest-path decision: no weights → BFS/DFS/DSU; weights ≥ 0 → Dijkstra.

APPROACH

1. Build an undirected adjacency list.
2. BFS from `source`, marking vertices when they are *enqueued*; return true as soon as `destination` is reached.

JAVA SOLUTION LeetCode 1971 · Solution.java

```java
import java.util.*;

class Solution {
```

```java
    public boolean validPath(int n, int[][] edges, int source, int destination) {
        if (source == destination) return true;
        List<Integer>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] e : edges) {                  // undirected
            adj[e[0]].add(e[1]);
            adj[e[1]].add(e[0]);
        }
        boolean[] seen = new boolean[n];
        Deque<Integer> q = new ArrayDeque<>();
        q.add(source);
        seen[source] = true;                     // mark on enqueue: each vertex enters once
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : adj[u]) {
                if (seen[v]) continue;
                if (v == destination) return true;
                seen[v] = true;
                q.add(v);
            }
        }
        return false;
    }
}
```

Time O(V + E) **Space** O(V + E)

Pitfall. With n up to $2 \cdot 10^5$, recursive DFS can overflow the Java stack on a path-shaped graph. BFS, iterative DFS or DSU avoid this. Handle `source == destination` (including graphs with no edges).

LC 743 Network Delay Time

MEDIUM

Task. Directed edges `times[i] = [u, v, w]` on nodes 1..n. A signal starts at `k`. Return the time until all nodes have received it, or −1 if some node is unreachable.

KEY INSIGHT

A node receives the signal at its shortest-path distance from `k`, and all nodes are done at the *maximum* of those distances.

WHY THE PATTERN APPLIES

This is a classic single-source shortest path: directed graph, non-negative weights ($0 \le w \le 100$), and the full distance array is needed. Template A applies directly.

APPROACH

1. Adjacency list sized `n + 1` (1-indexed).
2. Run Dijkstra from k; answer = max over dist[1..n], or −1 if any is INF.

JAVA SOLUTION LeetCode 743 · Solution.java

```java
import java.util.*;

class Solution {
    public int networkDelayTime(int[][] times, int n, int k) {
        List<int[]>[] adj = new List[n + 1];          // nodes are 1-indexed
        for (int i = 1; i <= n; i++) adj[i] = new ArrayList<>();
        for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});   // directed
        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;
```

```
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{0, k});
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int d = top[0], u = top[1];
            if (d > dist[u]) continue;                  // stale
            for (int[] e : adj[u]) {
                int nd = d + e[1];                      // d is finite here: no overflow
                if (nd < dist[e[0]]) {
                    dist[e[0]] = nd;
                    pq.add(new int[]{nd, e[0]});
                }
            }
        }
        int ans = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            ans = Math.max(ans, dist[i]);
        }
        return ans;
    }
}
```

Time O((V + E) log V) **Space** O(V + E)

Pitfall. The answer is the maximum distance, not the sum. Distances are at most (n−1)·100 here, so `int` is safe. Only finite `d` values reach the relaxation line, so `d + w` cannot overflow. With n ≤ 100, Bellman–Ford or Floyd–Warshall are also acceptable; say so.

LC 1631 Path With Minimum Effort

MEDIUM

Task. In a height grid, a route's effort is the maximum absolute height difference between consecutive cells. Minimise the effort of a route from the top-left to the bottom-right cell (4-directional moves).

Classification note. This is a *bottleneck (minimax)* shortest path, not a sum. Dijkstra remains correct because `max(cost, w) ≥ cost` (monotone) and max preserves order (isotone). Two equally valid alternatives: **binary search on the answer** (Pattern 6), where feasible(E) asks whether BFS can reach the target using only edges ≤ E, for O(RC log H); and **DSU / Kruskal**, which adds edges in increasing weight until the corners connect (Pattern 17, A4).

KEY INSIGHT

Replace `dist[v] = dist[u] + w` with `dist[v] = max(dist[u], w)`. The first time the target is popped, no other route can have a smaller bottleneck.

WHY THE PATTERN APPLIES

It is an implicit grid graph with non-negative weights and a monotone path cost. Template B with `bottleneck = true` is exactly this problem.

APPROACH

1. The state is a cell id; edge weight is `|h[a] − h[b]|`.
2. Pop the minimum effort; relax neighbours with `max`; return when the target is popped.

JAVA SOLUTION LeetCode 1631 · Solution.java

```java
import java.util.*;

class Solution {
    private static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};

    public int minimumEffortPath(int[][] heights) {
        int R = heights.length, C = heights[0].length, target = R * C - 1;
        int[] effort = new int[R * C];
        Arrays.fill(effort, Integer.MAX_VALUE);
        effort[0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{0, 0});                                // {effort, cellId}
        while (!pq.isEmpty()) {
            int[] top = pq.poll();
            int e = top[0], id = top[1];
            if (e > effort[id]) continue;                       // stale
            if (id == target) return e;  // settled: optimal bottleneck
            int r = id / C, c = id % C;
            for (int k = 0; k < 4; k++) {
                int nr = r + DR[k], nc = c + DC[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
                int w = Math.abs(heights[nr][nc] - heights[r][c]);
                int ne = Math.max(e, w);                        // bottleneck combine
                int nid = nr * C + nc;
                if (ne < effort[nid]) {
                    effort[nid] = ne;
                    pq.add(new int[]{ne, nid});
                }
            }
        }
        return effort[target];                                  // reached only if 1x1 grid
    }
}
```

Time O(RC · log(RC)) **Space** O(RC)

Pitfall. A 1×1 grid has effort 0; the early return handles it because cell 0 is the target. Plain DFS with backtracking is exponential, and BFS with a visited set is wrong because the first visit is not the minimum-effort visit.

LC 787 Cheapest Flights Within K Stops

MEDIUM

Task. Directed flights `[from, to, price]` (price ≥ 1). Return the cheapest price from `src` to `dst` using at most `k` stops (that is, at most k + 1 flights), or −1.

Classification note. Plain vertex Dijkstra is incorrect here. It keeps only the cheapest way to reach each city, but a cheap route that has used too many flights can block a slightly more expensive route with stops to spare. Example: with k = 1, edges 0→1 (1), 1→2 (1), 2→3 (1), 0→2 (5), the best route is 0→2→3 = 6. Vertex Dijkstra settles city 2 at cost 2 via two flights, and from there it cannot take 2→3 within budget. The fix is to make the stop count part of the state. The bounded Bellman–Ford solution is in A7.

KEY INSIGHT

Run Dijkstra on states `(city, flightsUsed)`. In the expanded graph the cheapest prefix is once again sufficient, because two states with the same city and flight count have identical futures.

WHY THE PATTERN APPLIES

Prices are positive, so the cost is monotone. The constraint is captured by enlarging the state space to n · (k + 2) states (Template C).

APPROACH

1. `best[city][f]` = cheapest cost reaching `city` with exactly `f` flights.
2. Pop the cheapest state; if its city is `dst`, return its cost. This is optimal across all flight counts, because the heap is ordered by cost.
3. Expand only if `f < k + 1`.

JAVA SOLUTION LeetCode 787 · Solution.java

```java
import java.util.*;

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        List<int[]>[] adj = new List[n];
        for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
        for (int[] f : flights) adj[f[0]].add(new int[]{f[1], f[2]});   // directed
        int maxFlights = k + 1;                     // k stops = k + 1 edges
        int[][] best = new int[n][maxFlights + 1];
        for (int[] row : best) Arrays.fill(row, Integer.MAX_VALUE);
        best[src][0] = 0;
        PriorityQueue<int[]> pq =
                new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
        pq.add(new int[]{0, src, 0});               // {cost, city, flightsUsed}
        while (!pq.isEmpty()) {
            int[] t = pq.poll();
            int cost = t[0], u = t[1], used = t[2];
            if (cost > best[u][used]) continue;     // stale state
            if (u == dst) return cost;              // cheapest valid state reaching dst
            if (used == maxFlights) continue;       // no flights left
            for (int[] e : adj[u]) {
                int nc = cost + e[1];               // <= (k+1) * 1e4: fits in int
                if (nc < best[e[0]][used + 1]) {
                    best[e[0]][used + 1] = nc;
                    pq.add(new int[]{nc, e[0], used + 1});
                }
            }
        }
        return -1;
    }
}
```

Time O(K·E · log(K·E)) **Space** O(n·K + K·E) states and heap entries

Pitfall. The usual mistakes are treating "stops" as edges (k stops = k + 1 edges) and keeping only `dist[city]`. A valid alternative prune keeps, per city, the fewest flights seen at any popped cost, and skips a state unless it improves on that. The 2D `best` table is easier to justify. Bellman–Ford with k + 1 rounds and a snapshot array (A7) is O(K·E) and is the most common interview answer.

Trie / Prefix Tree

A rooted tree whose edges are characters, so every node is a prefix. Words that share a prefix share the same path, which lets one traversal answer questions about many words at once.

19.1 Pattern identity

Primary purpose	Answer prefix questions (does any word start with p, which dictionary word is a prefix of s, which words match a partial pattern) in time proportional to the query length, independent of the dictionary size.
Typical input	A dictionary of strings over a small alphabet (usually lowercase letters), followed by many queries, or a search space (a grid or a sentence) that must be matched against all words at once.
Interview applications	Autocomplete, spell check, prefix counting, wildcard search, shortest-root replacement, multi-word grid search, maximum XOR (binary trie), streaming suffix matching.
Key insight	A hash set checks one whole word at a time. A trie checks a prefix and keeps going, so when a prefix is absent you stop immediately, and a single walk discovers every word on the path.

> **Key insight.** Use a trie when you need to ask *"can this partial string still become a word?"* while building the string one character at a time. That early "no" is what prunes backtracking in Word Search II.

19.2 Recognition signals

Signal	Structural reason
"starts with", "prefix", "autocomplete", "shortest root"	Prefix relationships are paths from the root; the answer is found by walking down.
Many words searched in one structure (a grid, a text)	Separate searches repeat shared prefixes. One trie walk serves all words with that prefix.
Wildcards that match a single character	A wildcard at depth d branches over the children of the current node only. Without a trie you would scan every word.
Design problem with `insert` and `search` / `startsWith`	The operation set itself describes a trie.
Maximise XOR of pairs (numbers as bit strings)	A binary trie greedily chooses the opposite bit from the most significant bit down.

> **Ask yourself.** "Do I need to stop early on a dead prefix, or ask a question about every word sharing a prefix?" If you only ever test complete words, a `HashSet` is simpler and usually faster.

When it is inappropriate.

- **Exact membership only**: a `HashSet<String>` is O(L) per lookup with less memory and code.

- **Substring (not prefix) queries over one long text**: KMP (A6), Z-function, rolling hashes or suffix structures fit better. A trie of all suffixes is $O(n^2)$ memory.
- **Huge alphabets with sparse use**: 26-slot arrays waste memory; use a `HashMap` for children, or reconsider the approach.
- **Lexicographic ordering of a static list**: sorting is enough.

Confused with	How to tell them apart
Hash set of words	Whole-word lookups only. A set of all prefixes works for prefix tests but costs $O(\Sigma L^2)$ to build, and it cannot report which word ends where.
Sorting + binary search	Prefix existence can be found by binary-searching the insertion point in a sorted list. That works for static dictionaries, but not for incremental character-by-character walks or wildcards.
Backtracking alone	Word Search I (one word) needs no trie. With many words, the trie turns per-word searches into one shared, pruned search.

19.3 Concepts and invariants

Structural invariant. The path from the root to node x spells a unique string prefix(x). Node x exists if and only if some inserted word has prefix(x) as a prefix. `x.end` (or a stored word) is true if and only if prefix(x) itself was inserted.

```
insert(w), search(w), startsWith(p):  O(L) time, L = query length
Total nodes ≤ 1 + Σ |w_i|          (one node per distinct prefix)
Memory (array children): nodes · alphabet · reference size  ≈ ΣL · 26 · 4–8 bytes
```

End-of-word markers. Prefix and word are different questions. After inserting "apple", `search("app")` is false while `startsWith("app")` is true. Storing the full word at the terminal node (`String word` instead of `boolean end`) avoids rebuilding strings during DFS, which is standard in Word Search II.

Node design choices.

Children store	Pros	Cons
`Node[26]`	O(1) child access, fastest, simplest	26 references per node even when sparse
`HashMap<Character, Node>`	Any alphabet, proportional memory	Hashing and boxing overhead; slower constants
Flat arrays `int[nodes][26]`	Cache-friendly, no object headers	Must pre-size the node count (≤ ΣL + 1)

Counters on nodes. A `pass` count (how many words go through a node) answers "how many words have prefix p" in O(|p|) and supports deletion. Decrement along the path, and a node with `pass == 0` can be detached.

Wildcard search cost. A `'.'` at depth d explores every existing child, so the worst case for a pattern of length L is $O(26^k \cdot L)$ for k wildcards, bounded by the number of trie nodes at those depths. The trie does not make wildcards cheap; it confines branching to prefixes that actually exist.

Trie-guided backtracking: pruning that makes Word Search II feasible

Searching each of W words separately on an R×C board costs $O(W \cdot RC \cdot 4 \cdot 3^{L-1})$. With a trie, a single DFS from each cell advances one trie node per step, and three prunes stack:

1. **Dead prefix**: if `node.next[c] == null`, stop. No word continues this path.

2. **Found-word removal**: after reporting `node.word`, set it to `null` so it is not reported twice (no result set needed).
3. **Leaf pruning**: after returning from a child that now has no children and no word, detach it. Later searches stop earlier, which matters on boards full of repeated letters.

Mark the board in place (`board[r][c] = '#'`) and restore on return. This is the choose → explore → undo loop from Pattern 8.

19.4 Reusable Java templates

Template A — array-children trie with prefix counts. Reusable: the node layout and walk. Problem-specific: what is stored at nodes (`end`, `word`, counts).

JAVA TEMPLATE ArrayTrie.java

```java
/** Lowercase-alphabet trie. pass = words through node; endCount = words ending here. */
class ArrayTrie {
    private static final int ALPHA = 26;

    private static final class Node {
        final Node[] next = new Node[ALPHA];
        int pass, endCount;
    }

    private final Node root = new Node();

    void insert(String w) {
        Node cur = root;
        cur.pass++;
        for (int i = 0; i < w.length(); i++) {
            int c = w.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
            cur.pass++;
        }
        cur.endCount++;
    }

    /** Last node of the path spelling s, or null if s is not a prefix of any word. */
    private Node walk(String s) {
        Node cur = root;
        for (int i = 0; i < s.length() && cur != null; i++) {
            cur = cur.next[s.charAt(i) - 'a'];
        }
        return cur;
    }

    boolean search(String w)      { Node n = walk(w); return n != null && n.endCount > 0; }
    // Uses pass, not node existence: the root exists even when the trie is empty.
    boolean startsWith(String p)  { return countPrefix(p) > 0; }
    int countWords(String w)      { Node n = walk(w); return n == null ? 0 : n.endCount; }
    int countPrefix(String p)     { Node n = walk(p); return n == null ? 0 : n.pass; }

    /** Removes one occurrence of w if present; detaches nodes no word passes through. */
    void erase(String w) {
        if (countWords(w) == 0) return;
        Node cur = root;
        cur.pass--;
        for (int i = 0; i < w.length(); i++) {
            int c = w.charAt(i) - 'a';
            // subtree now unused
```

```
            if (--cur.next[c].pass == 0) { cur.next[c] = null; return; }
            cur = cur.next[c];
        }
        cur.endCount--;
    }
}
```

Template B — map-children trie for arbitrary alphabets, with shortest-prefix lookup.

JAVA TEMPLATE MapTrie.java

```
import java.util.*;

/** Trie over arbitrary chars. Children in a HashMap: memory proportional to real edges. */
class MapTrie {
    private static final class Node {
        final Map<Character, Node> next = new HashMap<>();
        boolean end;
    }

    private final Node root = new Node();

    void insert(String w) {
        Node cur = root;
        for (char ch : w.toCharArray()) cur = cur.next.computeIfAbsent(ch, k -> new Node());
        cur.end = true;
    }

    /** Length of the shortest inserted word that is a prefix of s, or -1. */
    int shortestPrefixLength(String s) {
        Node cur = root;
        for (int i = 0; i < s.length(); i++) {
            cur = cur.next.get(s.charAt(i));
            if (cur == null) return -1;      // dead prefix: no word can match further
            if (cur.end) return i + 1;       // first terminal on the path is the shortest
        }
        return -1;
    }
}
```

19.5 Complexity and pitfalls

Time	Insert / search / prefix: O(L). Build: O(ΣL). Wildcard search: up to O(nodes) in the worst case. Word Search II: $O(RC \cdot 4 \cdot 3^{L-1})$ worst case, far less with pruning.
Auxiliary space	O(ΣL · σ) for array children (σ = alphabet size), O(ΣL) for map children; plus O(L) recursion depth for DFS operations.
Off-by-one	Returning the prefix length vs the index; checking `end` before vs after stepping into the child (the root represents the empty string).
Wrong invariant	Treating "node exists" as "word exists": `search` must check the end marker.
Overflow / memory	$3 \cdot 10^4$ words × length 10 × 26 references is about 7.8M slots, which is fine. Million-word dictionaries with array children can exhaust memory; use maps or flat arrays.
Breaks naive code	Duplicate words (count them, or report a found word once); words that are prefixes of other words ("app" and "apple"); the empty string; uppercase or non-letter input that breaks `c - 'a'`.
Interview follow-ups	Delete a word; top-k autocomplete (store a small sorted list or a heap per node); streaming suffix queries (insert reversed words, LC 1032); maximum XOR pair (binary trie, LC 421).

19.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- A node is a prefix; an end marker makes it a word. Keep the two questions separate.
- The trie's value is the early stop on a dead prefix, which multiplies inside backtracking.
- Store the full word at terminal nodes when the search builds paths (grids, DFS).
- Pick array children for small fixed alphabets and maps otherwise; state the memory trade-off.
- For a single exact lookup, use a hash set.

> **Recognition rule.** Many words + prefix questions or character-by-character construction → trie. One word or only whole-word membership → hash set.

IN-INTERVIEW CHECKLIST

- [ ] What is the alphabet and its size? Array or map children?
- [ ] What does each node need: an end flag, the stored word, a pass count?
- [ ] Where does the walk stop early, and what is returned then?
- [ ] How are duplicates and word-is-prefix-of-word cases handled?
- [ ] What is the recursion depth (the word length) for DFS operations?

Key distinction. A trie answers *prefix* questions across many words. KMP answers *substring* questions for one pattern in one text.

19.7 Assigned problems

LC 208 Implement Trie (Prefix Tree) — MEDIUM

Task. Implement `insert`, `search` (whole word) and `startsWith` (prefix) for lowercase words.

KEY INSIGHT

Both queries share one walk. They differ only in whether the final node must carry an end marker.

WHY THE PATTERN APPLIES

The operation set is the definition of a trie (Template A without counts).

APPROACH

1. Create child nodes on demand during insert; mark the last one as an end.
2. Use a shared `find(prefix)` that returns the last node or `null`.

JAVA SOLUTION LeetCode 208 · Trie.java

```java
class Trie {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
    }

    private final Node root = new Node();

    public Trie() {}
```

```
    public void insert(String word) {
        Node cur = root;
        for (int i = 0; i < word.length(); i++) {
            int c = word.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
        }
        cur.end = true;
    }

    public boolean search(String word) {
        Node n = find(word);
        return n != null && n.end;          // node existing only proves a prefix
    }

    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }

    private Node find(String s) {
        Node cur = root;
        for (int i = 0; i < s.length(); i++) {
            cur = cur.next[s.charAt(i) - 'a'];
            if (cur == null) return null;
        }
        return cur;
    }
}
```

Time O(L) per operation **Space** O(ΣL · 26) total nodes

Pitfall. `search` returning true for a prefix is the most common bug. Do not allocate all 26 children eagerly; create them only on insert.

LC 211 Design Add and Search Words Data Structure

MEDIUM

Task. Support `addWord` and `search`, where `'.'` in the query matches any single letter (LeetCode allows at most 2 dots per query).

KEY INSIGHT

A literal character follows one edge. A dot branches over the existing children only, so the trie limits the search to prefixes that are actually present.

WHY THE PATTERN APPLIES

This is prefix-structured wildcard matching. A list of words would be scanned in full for every query; the trie shares all common prefixes.

APPROACH

1. Insert as in 208.
2. `dfs(node, i)`: at the end of the query, return `node.end`; for `'.'`, try every non-null child; otherwise follow one child.

JAVA SOLUTION LeetCode 211 · WordDictionary.java

```
class WordDictionary {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
```

```
    }

    private final Node root = new Node();

    public WordDictionary() {}

    public void addWord(String word) {
        Node cur = root;
        for (int i = 0; i < word.length(); i++) {
            int c = word.charAt(i) - 'a';
            if (cur.next[c] == null) cur.next[c] = new Node();
            cur = cur.next[c];
        }
        cur.end = true;
    }

    public boolean search(String word) {
        return dfs(root, word, 0);
    }

    private boolean dfs(Node node, String w, int i) {
        if (i == w.length()) return node.end;
        char ch = w.charAt(i);
        if (ch == '.') {
            for (Node child : node.next) {          // branch only over existing prefixes
                if (child != null && dfs(child, w, i + 1)) return true;
            }
            return false;
        }
        Node child = node.next[ch - 'a'];
        return child != null && dfs(child, w, i + 1);
    }
}
```

Time add O(L); search O(L) without dots, $O(26^d \cdot L)$ with d dots **Space** $O(\Sigma L \cdot 26) + O(L)$ recursion

Pitfall. Return as soon as a branch succeeds, so the remaining children are not explored. The base case must check `end`; returning true at length L would accept prefixes such as "ba" when only "bad" was inserted.

LC 648 Replace Words

MEDIUM

Task. Replace each word of a sentence with the *shortest* dictionary root that is a prefix of it; leave words with no root unchanged.

KEY INSIGHT

Walking a word down the trie meets candidate roots in increasing length, so the first end marker on the path is the shortest root. The walk stops at that marker or at a dead prefix.

WHY THE PATTERN APPLIES

This is prefix-based replacement: each word is a query that asks which inserted word is its prefix. A hash set would test all L prefixes of each word, costing $O(L^2)$ per word because of substring creation. The trie costs O(L).

APPROACH

1. Insert all roots.
2. For each sentence word, walk until an end marker (replace with that prefix), a missing child, or the word's end (keep the word).

JAVA SOLUTION LeetCode 648 · Solution.java

```java
import java.util.*;

class Solution {
    private static final class Node {
        final Node[] next = new Node[26];
        boolean end;
    }

    public String replaceWords(List<String> dictionary, String sentence) {
        Node root = new Node();
        for (String r : dictionary) {
            Node cur = root;
            for (int i = 0; i < r.length(); i++) {
                int c = r.charAt(i) - 'a';
                if (cur.next[c] == null) cur.next[c] = new Node();
                cur = cur.next[c];
            }
            cur.end = true;
        }
        StringBuilder sb = new StringBuilder();
        for (String w : sentence.split(" ")) {
            if (sb.length() > 0) sb.append(' ');
            sb.append(w, 0, rootLength(root, w));
        }
        return sb.toString();
    }

    /** Length of the shortest root prefixing w, or w.length() if none. */
    private int rootLength(Node root, String w) {
        Node cur = root;
        for (int i = 0; i < w.length(); i++) {
            cur = cur.next[w.charAt(i) - 'a'];
            if (cur == null) return w.length();   // no root continues this prefix
            if (cur.end) return i + 1;            // first terminal = shortest root
        }
        return w.length();
    }
}
```

Time $O(\Sigma L_{dict} + |sentence|)$ **Space** $O(\Sigma L_{dict} \cdot 26) + output$

Pitfall. Check `end` immediately after stepping into each child, so the shortest root wins. LeetCode guarantees single spaces between words; with arbitrary whitespace, split on `"\\s+"` and decide how to preserve spacing.

LC 212 Word Search II

HARD

Task. Return all words from a list that can be traced on an m×n letter board through 4-directionally adjacent cells, without reusing a cell within one word.

KEY INSIGHT

Search the board once for all words together. The DFS carries a trie node, and a missing child ends the branch immediately. Removing words once found, and pruning emptied leaves, keeps later searches short.

WHY THE PATTERN APPLIES

This is trie-guided backtracking. Backtracking explores the paths; the trie supplies the "can this path still become a word?" test and the multi-word sharing. Searching each word separately (up to $3 \cdot 10^4$ words) times out.

APPROACH

1. Build a trie and store each word at its terminal node.
2. From every cell, run `dfs(r, c, parent)`. Step into `parent.next[ch]`; if it is null, return. If `node.word != null`, record the word and clear it.
3. Mark the cell `'#'`, recurse into the 4 neighbours, then restore it. Detach the child if it is now empty (leaf pruning).

JAVA SOLUTION LeetCode 212 · Solution.java

```java
import java.util.*;

class Solution {
    private static final class Node {
        final Node[] next = new Node[26];
        String word;                        // non-null iff a not-yet-found word ends here
        int children;                       // live child count, for leaf pruning
    }

    private static final int[] DR = {1, -1, 0, 0}, DC = {0, 0, 1, -1};
    private char[][] board;
    private List<String> found;

    public List<String> findWords(char[][] board, String[] words) {
        Node root = new Node();
        for (String w : words) {
            Node cur = root;
            for (int i = 0; i < w.length(); i++) {
                int c = w.charAt(i) - 'a';
                if (cur.next[c] == null) { cur.next[c] = new Node(); cur.children++; }
                cur = cur.next[c];
            }
            cur.word = w;                   // duplicates in words collapse to one entry
        }
        this.board = board;
        this.found = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) {
                if (root.next[board[r][c] - 'a'] != null) dfs(r, c, root);
            }
        }
        return found;
    }

    private void dfs(int r, int c, Node parent) {
        char ch = board[r][c];
        int idx = ch - 'a';
        Node node = parent.next[idx];
        if (node == null) return;                       // dead prefix
        if (node.word != null) {
            found.add(node.word);
            node.word = null;                           // report each word once
        }
        board[r][c] = '#';                              // choose: cell used on this path
        for (int k = 0; k < 4; k++) {
            int nr = r + DR[k], nc = c + DC[k];
            if (nr < 0 || nr >= board.length || nc < 0 || nc >= board[0].length) continue;
            if (board[nr][nc] == '#') continue;
            dfs(nr, nc, node);
```

```
        }
        board[r][c] = ch;                                   // undo
        if (node.children == 0 && node.word == null) {      // exhausted branch: prune it
            parent.next[idx] = null;
            parent.children--;
        }
    }
}
```

Time $O(RC \cdot 4 \cdot 3^{L-1})$ worst case, $L \le 10$; build $O(\Sigma L)$ **Space** $O(\Sigma L \cdot 26)$ trie + $O(L)$ recursion

Pitfall. Without clearing `node.word`, a word reachable along several paths is reported several times. Without leaf pruning, adversarial boards (all 'a', with words "aaaa…b") stay exponential. The `'#'` check must happen before indexing `next['#' - 'a']`; here it is done at the neighbour check.

Segment Tree / Fenwick Tree (Binary Indexed Tree)

These structures keep precomputed aggregates over O(n) intervals, so any range can be answered by combining O(log n) of them, and a point change touches only O(log n) of them.

20.1 Pattern identity

Primary purpose	Answer range-aggregate queries (sum, count, min, max, gcd and similar) *interleaved with updates*, each in O(log n). Prefix sums handle queries only; updates would cost O(n) each.
Typical input	An array plus a stream of operations ("update index i", "query [l, r]"), or an offline problem that sweeps an axis and asks "how many / what is the best among values seen so far in this value range?"
Interview applications	Mutable range sums, counting inversions or smaller-after-self, LIS variants with value windows, interval stacking (falling squares), calendar booking, k-th order statistics over counts.
Key insight	Any range [l, r] splits into O(log n) canonical blocks whose aggregates are stored. If the operation is associative, combining the blocks gives the answer. A segment tree stores the blocks explicitly as a tree. A Fenwick tree stores prefix blocks implicitly by low-bit arithmetic and needs an *invertible* operation to turn prefixes into ranges.

> **Key insight.** Two questions decide the structure. (1) Is the merge operation invertible (sum, xor, counts)? If yes and updates are point updates, a Fenwick tree is enough. (2) Do you need non-invertible aggregates (max, min) over arbitrary ranges, range assignments, or custom merges? Then use a segment tree, with lazy propagation if updates cover ranges.

20.2 Recognition signals

Signal	Structural reason
Queries and updates interleave, with both up to 10^5 operations	Static prefix sums cost O(n) per update. Recomputing per query costs O(n) per query. Both are $O(n \cdot q) \approx 10^{10}$.
"Count elements smaller/greater than x to the left/right"	Sweep positions and keep a frequency array indexed by *value*. Each query is a prefix count, which suits a Fenwick tree over (compressed) values.
DP of the form dp[i] = 1 + best over j with value in a window	Index the structure by value and query the best dp in a value range: a range-max segment tree.
Intervals painted, stacked or booked; values up to 10^8–10^9	Range updates plus range queries over a huge axis: coordinate compression plus a lazy segment tree.
The aggregate is associative but not invertible (max, min, gcd)	Prefix differences are impossible, so a range query must be decomposed into tree nodes.

> **Ask yourself.** "What are my update type (point or range) and query type (point or range), and is the operation invertible?" The answer table in 20.3 then chooses the structure for you.

When it is inappropriate.

- **No updates**: prefix sums (Pattern 4) for sums; a sparse table for static range min/max in O(1).
- **Only global min/max with inserts and deletes**: a heap or `TreeMap` is simpler.
- **A sliding-window maximum**: a monotonic deque does it in O(n) total.
- **Few operations ($n \cdot q \le 10^7$)**: brute force is acceptable and much less error-prone.
- **Operations that are not associative** (median of a range, mode of a range): neither structure supports them without fundamentally different techniques (merge-sort trees, Mo's algorithm, persistent trees).

Confused with	How to tell them apart
Prefix sum	Prefix sums are a Fenwick tree with no updates: O(1) query, O(n) update.
`TreeMap` / ordered set	Handles predecessor and successor queries and order statistics poorly (Java's `TreeMap` has no rank query). Use a Fenwick tree over compressed values for "how many ≤ x".
Merge sort counting	Inversion-style counts (LC 315, 493) can be done offline by merge sort in O(n log n). A Fenwick tree is usually shorter and also works online.

20.3 Concepts and invariants

Segment tree — array representation. Node 1 is the root and covers the whole input range `[0, n-1]`. Node `x` covering `[l, r]` has children `2x` covering `[l, mid]` and `2x+1` covering `[mid+1, r]`, with `mid = (l + r) >>> 1`.

```
Invariant:  tree[x] = merge(tree[2x], tree[2x+1]),  leaves hold a[i]
Height h = ⌈log₂ n⌉, so indices are < 2^(h+1) ≤ 4n   ⇒  allocate 4n
Query [ql, qr]: at most 2 nodes per level are partially covered ⇒ O(log n) nodes visited
Point update: one root-to-leaf path, then recompute ancestors ⇒ O(log n)
Build: each of the < 2n real nodes is computed once ⇒ O(n)
```

The segment tree needs only an **associative** merge with an **identity** element for ranges that fall outside the query (0 for sum, `Integer.MIN_VALUE` for max, +∞ for min). Commutativity is not required as long as left and right are merged in order. That is what allows merges such as "max subarray sum within a range" that store a tuple per node.

Bottom-up (iterative) segment tree. Leaves live at `[n, 2n)` and node `x` has children `2x` and `2x+1`. A query over the half-open range `[l, r)` walks both bounds upward. It is short and fast, and its natural convention is *half-open*. The 2407 solution uses it; the recursive Template A uses *inclusive* bounds. Never mix the two conventions inside one implementation.

Lazy propagation (range updates). A range update would touch O(n) leaves, so instead store a pending *tag* on each fully covered node and push it to the children only when a later operation needs to descend. This requires (1) a tag that can be applied to a node aggregate in O(1) (add: `sum += tag · length`; assign: `max = value`), and (2) tags that *compose* (add then add equals one add; assign overrides an earlier assign or add). If you cannot define both, lazy propagation does not apply as written.

Fenwick tree — the low-bit structure. Internally 1-based. `bit[i]` stores the sum of the half-open range `(i - lowbit(i), i]`, where `lowbit(i) = i & -i` is the value of i's lowest set bit.

```
Two's complement:  −i = ~i + 1.  Below the lowest set bit, ~i has 1s and adding 1
carries
through them, flipping back exactly that bit. Above it, i and ~i differ, so i & −i keeps
only the lowest set bit.     e.g. i = 12 = 1100₂,  −i = …0100₂,  i & −i = 4

prefix(i) = bit[i] + prefix(i − lowbit(i))      → strip low bits: ≤ log₂ n steps
update(i, δ): bit[i] += δ; i += lowbit(i)         → climb to every block containing i
rangeSum(l, r) = prefix(r) − prefix(l − 1)        → requires an inverse ("−")
Index 0 is unusable: lowbit(0) = 0, so update(0) would loop forever. Hence 1-based.
```

Zero-based input vs one-based internals. Keep the public API 0-based, matching LeetCode, and shift once at the boundary (`i + 1`) inside `add` and `prefix`. Shifting by hand at call sites is where off-by-one bugs come from.

Why Fenwick fails for range max. With max you cannot recover max(l..r) from max(1..r) and max(1..l−1); the larger prefix hides the window. A Fenwick "prefix max" works only for *prefix* queries with *increase-only* updates. LC 2407 needs a window `[x−k, x−1]`, so it needs a segment tree.

Choosing by update/query type

Updates	Queries	Structure
none	range sum	Prefix sums, O(1) query
point (add / set)	range sum / count / xor	Fenwick (set = add the difference; keep a copy of the array), or a segment tree
range add	point value	Fenwick over the *difference array*: add at l, subtract at r+1; the point value is a prefix sum (Template E)
range add	range sum	Lazy segment tree (Template C), or two Fenwick trees
point set	range max / min / gcd	Segment tree (recursive or bottom-up)
range assign	range max	Lazy segment tree with an assign tag (LC 699)

Coordinate compression: shrinking a 10^9 axis to n slots

Value-indexed structures need an array as large as the value range. When only relative order matters, map each distinct value to its rank among the sorted distinct values. This preserves comparisons (x < y ⇔ rank(x) < rank(y)) and the structure needs only m ≤ n slots.

- **Point values** (LC 315): sort a copy, deduplicate, and look up the rank with `Arrays.binarySearch`. Equal values share a rank, which matters for "strictly smaller".
- **Intervals** (LC 699): compress *endpoints*, and let slot i stand for the half-open cell `[xs[i], xs[i+1])`. An interval `[a, b)` covers slots `rank(a) … rank(b) − 1`. Compressing only starts would merge cells that must stay distinct.
- **Offline requirement**: all coordinates must be known before building. For online problems with unknown coordinates, use a dynamic (pointer-based) segment tree.

20.4 Reusable Java templates

Template A — recursive segment tree, point set, inclusive range query. Shown for sum. For max or min, change `merge` and `IDENTITY` only.

JAVA TEMPLATE SegmentTree.java

```java
/** Array-based segment tree. Public indices are 0-based; ranges [l, r] are inclusive. */
class SegmentTree {
    private static final long IDENTITY = 0;               // max: Long.MIN_VALUE; min: MAX
    private static long merge(long a, long b) { return a + b; }   // must be associative

    private final int n;
    private final long[] tree;                            // node 1 = root over [0, n-1]

    SegmentTree(int[] a) {
        n = a.length;
        tree = new long[4 * Math.max(1, n)];  // height ceil(log2 n) => < 4n nodes
        if (n > 0) build(1, 0, n - 1, a);
    }

    private void build(int x, int l, int r, int[] a) {
        if (l == r) { tree[x] = a[l]; return; }
        int mid = (l + r) >>> 1;
        build(2 * x, l, mid, a);
        build(2 * x + 1, mid + 1, r, a);
        tree[x] = merge(tree[2 * x], tree[2 * x + 1]);
    }

    /** a[i] = value. */
    void set(int i, long value) { set(1, 0, n - 1, i, value); }

    private void set(int x, int l, int r, int i, long value) {
        if (l == r) { tree[x] = value; return; }
        int mid = (l + r) >>> 1;
        if (i <= mid) set(2 * x, l, mid, i, value);
        else set(2 * x + 1, mid + 1, r, i, value);
        tree[x] = merge(tree[2 * x], tree[2 * x + 1]);    // recompute along the path
    }

    /** Aggregate of a[ql..qr], inclusive. */
    long query(int ql, int qr) { return query(1, 0, n - 1, ql, qr); }

    private long query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return IDENTITY;            // disjoint
        if (ql <= l && r <= qr) return tree[x];           // fully covered: canonical block
        int mid = (l + r) >>> 1;
        return merge(query(2 * x, l, mid, ql, qr), query(2 * x + 1, mid + 1, r, ql, qr));
    }
}
```

Template B — Fenwick tree (BIT): point add, prefix and range sums.

JAVA TEMPLATE Fenwick.java

```java
/** Fenwick tree for sums. Public API is 0-based; internal array is 1-based. */
class Fenwick {
    private final long[] bit;                     // bit[i] = sum of (i - lowbit(i), i]

    Fenwick(int n) { bit = new long[n + 1]; }

    /** O(n) build: push each node's value to its immediate parent block. */
    Fenwick(int[] a) {
        this(a.length);
        for (int i = 1; i <= a.length; i++) {
            bit[i] += a[i - 1];
            int parent = i + (i & -i);
            if (parent <= a.length) bit[parent] += bit[i];
        }
    }
```

```java
    /** a[i0] += delta. */
    void add(int i0, long delta) {
        for (int i = i0 + 1; i < bit.length; i += i & -i) bit[i] += delta;
    }

    /** Sum of a[0..i0] inclusive; prefix(-1) == 0. */
    long prefix(int i0) {
        long s = 0;
        for (int i = i0 + 1; i > 0; i -= i & -i) s += bit[i];
        return s;
    }

    /** Sum of a[l..r] inclusive, via prefix difference (needs subtraction). */
    long rangeSum(int l, int r) { return prefix(r) - prefix(l - 1); }
}
```

Template C — lazy segment tree: range add, range sum.

JAVA TEMPLATE LazySegmentTree.java

```java
/** Range add + range sum over [0, n-1]; all ranges inclusive. */
class LazySegmentTree {
    private final int n;
    private final long[] sum, tag;                // tag[x]: pending add for x's children

    LazySegmentTree(int n) {
        this.n = n;
        sum = new long[4 * Math.max(1, n)];
        tag = new long[4 * Math.max(1, n)];
    }

    private void apply(int x, int l, int r, long v) {   // O(1) effect of "add v" on node x
        sum[x] += v * (r - l + 1);
        tag[x] += v;                                     // add tags compose by addition
    }

    private void push(int x, int l, int r) {
        if (tag[x] == 0 || l == r) return;
        int mid = (l + r) >>> 1;
        apply(2 * x, l, mid, tag[x]);
        apply(2 * x + 1, mid + 1, r, tag[x]);
        tag[x] = 0;
    }

    void rangeAdd(int ql, int qr, long v) { rangeAdd(1, 0, n - 1, ql, qr, v); }

    private void rangeAdd(int x, int l, int r, int ql, int qr, long v) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(x, l, r, v); return; }   // stop: defer to children
        push(x, l, r);
        int mid = (l + r) >>> 1;
        rangeAdd(2 * x, l, mid, ql, qr, v);
        rangeAdd(2 * x + 1, mid + 1, r, ql, qr, v);
        sum[x] = sum[2 * x] + sum[2 * x + 1];
    }

    long rangeSum(int ql, int qr) { return rangeSum(1, 0, n - 1, ql, qr); }

    private long rangeSum(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return sum[x];
        push(x, l, r);                                   // children must be up to date
        int mid = (l + r) >>> 1;
        return rangeSum(2 * x, l, mid, ql, qr) + rangeSum(2 * x + 1, mid + 1, r, ql, qr);
```

```
    }
}
```

Template D — coordinate compression.

JAVA TEMPLATE Compressor.java

```java
import java.util.*;

/** Maps values to ranks 0..m-1 preserving order; equal values share a rank. */
class Compressor {
    private final int[] sorted;                    // distinct values, ascending

    Compressor(int[] values) {
        int[] c = values.clone();
        Arrays.sort(c);
        int m = 0;
        for (int i = 0; i < c.length; i++) {
            if (i == 0 || c[i] != c[i - 1]) c[m++] = c[i];
        }
        sorted = Arrays.copyOf(c, m);
    }

    int size() { return sorted.length; }

    /** Rank of a value that was present in the input. */
    int rank(int v) { return Arrays.binarySearch(sorted, v); }

    /** Number of distinct values strictly less than v (works for any v). */
    int countLess(int v) {
        int lo = 0, hi = sorted.length;            // half-open [lo, hi): lower bound
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (sorted[mid] < v) lo = mid + 1; else hi = mid;
        }
        return lo;
    }
}
```

Template E — Fenwick over a difference array: range add, point query.

JAVA TEMPLATE RangeAddPointQuery.java

```java
/** a[l..r] += v in O(log n); read a[i] in O(log n). Indices 0-based, ranges inclusive. */
class RangeAddPointQuery {
    private final long[] bit;  // Fenwick over d, where a[i] = d[0] + ... + d[i]

    RangeAddPointQuery(int n) { bit = new long[n + 2]; }   // r + 1 may equal n

    private void add(int i0, long v) {
        for (int i = i0 + 1; i < bit.length; i += i & -i) bit[i] += v;
    }

    void rangeAdd(int l, int r, long v) {
        add(l, v);
        add(r + 1, -v);                            // cancels the add after r
    }

    long pointQuery(int i0) {
        long s = 0;
        for (int i = i0 + 1; i > 0; i -= i & -i) s += bit[i];
        return s;
    }
}
```

20.5 Complexity and pitfalls

Time	Build O(n) for both. Point update and range query O(log n). Lazy range update and query O(log n). Coordinate compression O(n log n).
Auxiliary space	Segment tree 4n (recursive) or 2n (bottom-up), plus a parallel tag array for lazy. Fenwick n + 1. Recursion depth O(log n).
Off-by-one	Fenwick index 0 (an infinite loop on update); `prefix(l - 1)` when l = 0 (it must return 0); inclusive `[l, r]` in recursive trees vs half-open `[l, r)` in bottom-up trees; array sized n instead of n + 2 for the difference trick.
Wrong invariant	Forgetting to recompute the parent after a child update; forgetting `push` before descending in lazy trees; an identity element that is not neutral (0 for max over negatives).
Overflow	Sums of $3 \cdot 10^4$ values $\le 10^5$ fit in an `int`, but general range sums with updates should be `long`. Lazy `tag · length` overflows an `int` quickly.
Breaks naive code	The Fenwick *set* operation (needs the old value: add `val - a[i]` and store `a[i] = val`); duplicates in compressed ranks for strict comparisons; n = 0 or 1.
Interview follow-ups	k-th smallest via Fenwick binary lifting; a 2D Fenwick for matrix sums (LC 308); a dynamic segment tree for online coordinates; why Fenwick cannot do range max; persistence and offline queries.

20.6 Pattern mastery checklist

ESSENTIAL TAKEAWAYS

- Segment tree: associative merge + identity; range [l, r] splits into O(log n) canonical nodes.
- Fenwick: `bit[i]` covers `(i - (i & -i), i]`; updates climb, queries descend; ranges need an inverse.
- Range updates need lazy tags that apply in O(1) and compose. If they don't, find another approach.
- Index structures by *value* (after compression) to answer "how many / best among values in a range so far".
- Fix one bound convention per implementation and convert at the API boundary.

> **Recognition rule.** Interleaved updates and range queries over the same array (or value axis) → Fenwick if the operation is invertible and updates are point updates; otherwise a segment tree, lazy if updates cover ranges.

IN-INTERVIEW CHECKLIST

- ☐ Update type: point add, point set, range add, range assign?
- ☐ Query type: point or range; sum, count, max, or a custom merge?
- ☐ Is the merge invertible? Associative? What is its identity?
- ☐ Index domain: positions or values? Does it need compression?
- ☐ Bounds: inclusive or half-open; 0-based API vs 1-based internals?

Key distinction. A Fenwick tree is smaller and simpler but limited to invertible prefix-style aggregates with point updates (or the difference-array dual). A segment tree is the general tool for associative range aggregates and range updates.

20.7 Assigned problems

> **Problem selection.** The syllabus leaves this chapter's problems open. The four chosen problems together cover: building both structures and point update + range sum (**307**); a Fenwick tree over compressed values for counting (**315**); a range-max segment tree where Fenwick is unsuitable (**2407**); and coordinate compression with lazy range assignment and range max (**699**).

LC 307 Range Sum Query - Mutable

MEDIUM

Task. Implement `NumArray(nums)`, `update(index, val)` (sets a value) and `sumRange(left, right)` (inclusive), with up to $3 \cdot 10^4$ calls.

KEY INSIGHT

Static prefix sums make each update O(n). Storing O(log n)-sized block sums makes both operations logarithmic. Sum is invertible, so either structure works; this is the problem to practise both.

WHY THE PATTERN APPLIES

Point updates are interleaved with inclusive range-sum queries, which is the defining case. Version 1 is a segment tree (Template A, specialised). Version 2 is a Fenwick tree (Template B), which needs the old value to turn a *set* into an *add*.

APPROACH

1. Segment tree: build O(n); `update` resets a leaf and recomputes its ancestors; `sumRange` merges the canonical nodes.
2. Fenwick: keep `nums`; `update` adds `val - nums[i]`; `sumRange = prefix(r) - prefix(l - 1)`.

JAVA SOLUTION LeetCode 307 · NumArray.java (segment tree)

```java
class NumArray {
    private final int n;
    private final int[] tree;                    // node 1 covers [0, n-1]; sums fit in int

    public NumArray(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(1, 0, n - 1, nums);
    }

    private void build(int x, int l, int r, int[] a) {
        if (l == r) { tree[x] = a[l]; return; }
        int mid = (l + r) >>> 1;
        build(2 * x, l, mid, a);
        build(2 * x + 1, mid + 1, r, a);
        tree[x] = tree[2 * x] + tree[2 * x + 1];
    }

    public void update(int index, int val) { update(1, 0, n - 1, index, val); }

    private void update(int x, int l, int r, int i, int val) {
        if (l == r) { tree[x] = val; return; }
        int mid = (l + r) >>> 1;
        if (i <= mid) update(2 * x, l, mid, i, val);
        else update(2 * x + 1, mid + 1, r, i, val);
        tree[x] = tree[2 * x] + tree[2 * x + 1];
    }

    public int sumRange(int left, int right) { return query(1, 0, n - 1, left, right); }
```

```
    private int query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[x];
        int mid = (l + r) >>> 1;
        return query(2 * x, l, mid, ql, qr) + query(2 * x + 1, mid + 1, r, ql, qr);
    }
}
```

JAVA SOLUTION LeetCode 307 · NumArray.java (Fenwick tree)

```
class NumArray {
    private final int[] nums;  // current values: needed to turn set into add
    private final int[] bit;                    // 1-based Fenwick array

    public NumArray(int[] nums) {
        this.nums = nums.clone();
        bit = new int[nums.length + 1];
        for (int i = 1; i <= nums.length; i++) {     // O(n) build
            bit[i] += nums[i - 1];
            int parent = i + (i & -i);
            if (parent <= nums.length) bit[parent] += bit[i];
        }
    }

    public void update(int index, int val) {
        int delta = val - nums[index];
        nums[index] = val;
        for (int i = index + 1; i < bit.length; i += i & -i) bit[i] += delta;
    }

    public int sumRange(int left, int right) {
        return prefix(right) - prefix(left - 1);
    }

    private int prefix(int i0) {                // sum of nums[0..i0]; prefix(-1) = 0
        int s = 0;
        for (int i = i0 + 1; i > 0; i -= i & -i) s += bit[i];
        return s;
    }
}
```

Time build O(n); update and query O(log n) **Space** O(n) (4n segment tree, n + 1 Fenwick)

Pitfall. In the Fenwick version, `update` is a *set*. Adding `val` directly double-counts the old value. Keeping `nums` in sync is mandatory. Sums are at most $3 \cdot 10^4 \times 100$ in magnitude, so `int` suffices. Use `long` in general.

LC 315 Count of Smaller Numbers After Self

HARD

Task. For each i, count the indices j > i with `nums[j] < nums[i]`; n ≤ 10^5, values in $[-10^4, 10^4]$.

KEY INSIGHT

Sweep from right to left while maintaining frequencies of the values already seen (exactly those with j > i). The answer for i is the count of seen values whose rank is smaller than `rank(nums[i])`: a prefix sum over the value axis.

WHY THE PATTERN APPLIES

Each step does one point update (+1 at a value) and one prefix query. Counts are invertible, so a Fenwick tree is the natural fit. Values are compressed to ranks so the tree has m ≤ n slots and negative values need no offset.

APPROACH

1. Compress: sorted distinct values; `rank(v)` via binary search.
2. For i = n−1 down to 0: `ans[i] = prefix(rank − 1)` (strictly smaller), then `add(rank, 1)`.

JAVA SOLUTION LeetCode 315 · Solution.java

```java
import java.util.*;

class Solution {
    public List<Integer> countSmaller(int[] nums) {
        int n = nums.length;
        int[] sorted = nums.clone();
        Arrays.sort(sorted);
        int m = 0;                                          // compress to distinct ascending
        for (int i = 0; i < n; i++) {
            if (i == 0 || sorted[i] != sorted[i - 1]) sorted[m++] = sorted[i];
        }
        int[] bit = new int[m + 1];  // Fenwick over ranks, 1-based inside

        Integer[] ans = new Integer[n];
        for (int i = n - 1; i >= 0; i--) {
            int rank = Arrays.binarySearch(sorted, 0, m, nums[i]);  // 0-based rank
            int count = 0;
            for (int k = rank; k > 0; k -= k & -k) count += bit[k];  // ranks 0..rank-1
            ans[i] = count;
            for (int k = rank + 1; k <= m; k += k & -k) bit[k]++;    // add value at rank
        }
        return Arrays.asList(ans);
    }
}
```

Time O(n log n) **Space** O(n)

Pitfall. "Strictly smaller" means querying ranks `0..rank-1`, which is `prefix(rank − 1)` in 0-based terms. The inner loop starts at `k = rank` because internal index = 0-based index + 1. Querying before inserting `nums[i]` excludes i itself. An equally valid alternative is merge sort that counts the right-half elements moved before each left element.

LC 2407 Longest Increasing Subsequence II

HARD

Task. Find the longest strictly increasing subsequence in which adjacent elements differ by at most `k`; n, values, k ≤ 10^5.

KEY INSIGHT

Let `best[v]` be the longest valid subsequence seen so far that ends with value v. Processing left to right, `len(x) = 1 + max(best[v] for v in [x − k, x − 1])`. That is a range-max query on the value axis followed by a point update `best[x] = max(best[x], len)`.

WHY THE PATTERN APPLIES

The DP transition is an aggregate over a *window of values*, and the window moves with x. Max is not invertible, so a Fenwick prefix-max cannot answer `[x−k, x−1]`. A segment tree over values 1..maxV does both operations in O(log V). This solution uses the bottom-up variant with *half-open* queries.

APPROACH

1. Size = next power of two ≥ maxV + 1; leaves at `[size, 2·size)`; leaf v stores `best[v]`.
2. For each x: `q = max over [max(1, x−k), x)`; `len = q + 1`; update leaf x with max; track the overall answer.

JAVA SOLUTION LeetCode 2407 · Solution.java

```java
class Solution {
    public int lengthOfLIS(int[] nums, int k) {
        int maxV = 0;
        for (int x : nums) maxV = Math.max(maxV, x);
        int size = 1;
        while (size < maxV + 1) size <<= 1;           // leaves for values 0..maxV
        int[] tree = new int[2 * size];               // bottom-up max tree, identity 0
        int ans = 0;
        for (int x : nums) {
            int len = query(tree, size, Math.max(1, x - k), x) + 1;   // values [x-k, x-1]
            ans = Math.max(ans, len);
            int i = x + size;  // point update: best[x] = max(., len)
            tree[i] = Math.max(tree[i], len);
            for (i >>= 1; i >= 1; i >>= 1) tree[i] = Math.max(tree[2 * i], tree[2 * i + 1]);
        }
        return ans;
    }

    /** Max over leaf positions [l, r) (half-open), 0 if empty. */
    private int query(int[] tree, int size, int l, int r) {
        int res = 0;
        for (l += size, r += size; l < r; l >>= 1, r >>= 1) {
            if ((l & 1) == 1) res = Math.max(res, tree[l++]);   // l is a right child
            if ((r & 1) == 1) res = Math.max(res, tree[--r]);   // r-1 is a left child
        }
        return res;
    }
}
```

Time O(n log V) **Space** O(V)

Pitfall. Strictly increasing means the window excludes x itself, so the query is `[x−k, x)`. A previous occurrence of x must not extend itself. The classic patience-sorting LIS (binary search over tails) does not handle the "difference ≤ k" constraint. The update keeps a maximum (`max(best[x], len)`) because a later, shorter subsequence ending at x must not overwrite a longer one.

LC 699 Falling Squares

HARD

Task. Squares `[left, side]` fall one by one and land on the highest square they overlap (touching edges do not count). After each drop, report the tallest stack height. Coordinates go up to 10^8; up to 1000 squares.

KEY INSIGHT

A square occupies the half-open interval `[left, left + side)`. Its landing height is `max(height over that interval) + side`, and afterwards the whole interval has exactly that height. That is a range-max query followed by a *range assignment*, on a compressed axis.

WHY THE PATTERN APPLIES

Range updates are combined with non-invertible range queries, which calls for a lazy segment tree with an *assign* tag. Assign tags compose (the later assign wins), and applying one to a node sets its max directly. The 10^8 axis is compressed to at most 2000 endpoints.

APPROACH

1. Collect all `left` and `left + side`, sort and deduplicate into `xs`. Slot i is the cell `[xs[i], xs[i+1])`.
2. A square covers slots `rank(left) … rank(left + side) − 1`, so touching squares share no slot.
3. `h = queryMax(range) + side`; `assign(range, h)`; running answer = max(answer, h).

JAVA SOLUTION LeetCode 699 · Solution.java

```java
import java.util.*;

class Solution {
    private int[] mx, tag;                       // tag[x] = pending assignment, -1 if none

    public List<Integer> fallingSquares(int[][] positions) {
        int[] xs = new int[2 * positions.length];
        int p = 0;
        for (int[] s : positions) { xs[p++] = s[0]; xs[p++] = s[0] + s[1]; }
        Arrays.sort(xs);
        int m = 0;
        for (int i = 0; i < xs.length; i++) {
            if (i == 0 || xs[i] != xs[i - 1]) xs[m++] = xs[i];
        }
        // slot i = cell [xs[i], xs[i+1]); m - 1 cells are meaningful, m slots allocated
        mx = new int[4 * m];
        tag = new int[4 * m];
        Arrays.fill(tag, -1);

        List<Integer> res = new ArrayList<>();
        int best = 0;
        for (int[] s : positions) {
            int l = Arrays.binarySearch(xs, 0, m, s[0]);
            // half-open -> inclusive
            int r = Arrays.binarySearch(xs, 0, m, s[0] + s[1]) - 1;
            int h = query(1, 0, m - 1, l, r) + s[1];
            assign(1, 0, m - 1, l, r, h);
            best = Math.max(best, h);
            res.add(best);
        }
        return res;
    }

    private void applyAssign(int x, int v) { mx[x] = v; tag[x] = v; }   // later assign wins

    private void push(int x) {
        if (tag[x] < 0) return;
        applyAssign(2 * x, tag[x]);
        applyAssign(2 * x + 1, tag[x]);
        tag[x] = -1;
    }

    private void assign(int x, int l, int r, int ql, int qr, int v) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { applyAssign(x, v); return; }
        push(x);
```

```
        int mid = (l + r) >>> 1;
        assign(2 * x, l, mid, ql, qr, v);
        assign(2 * x + 1, mid + 1, r, ql, qr, v);
        mx[x] = Math.max(mx[2 * x], mx[2 * x + 1]);
    }

    private int query(int x, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;              // heights are >= 0: 0 is the identity
        if (ql <= l && r <= qr) return mx[x];
        push(x);
        int mid = (l + r) >>> 1;
        return Math.max(query(2 * x, l, mid, ql, qr), query(2 * x + 1, mid + 1, r, ql, qr));
    }
}
```

Time O(n log n) **Space** O(n)

Pitfall. Treating squares as closed intervals `[left, left + side]` makes edge-touching squares stack, which is wrong. The half-open cell mapping prevents it. Heights reach about $1000 \times 10^6 = 10^9$, which still fits in an `int`; use `long` if constraints grow. With $n \le 1000$, an $O(n^2)$ interval scan is also accepted. Mention it, then present the segment tree as the scalable answer.

PART III

Seven Essential Algorithms Beyond the Core Patterns

The pattern chapters taught you to recognise problem families. The seven algorithms here are specific, named procedures that recur inside those families. Each has a precise invariant, a standard implementation you should be able to write from memory, and a known failure boundary. Where an algorithm already appeared in Part II (Dijkstra in Ch. 18, DSU in Ch. 17), this part goes deeper: the correctness proof, variants, and the practice problems are all new.

#	Algorithm	What it adds	Reinforces
A1	Monotonic Stack	Nearest greater/smaller in amortised O(1); contribution counting with duplicates	Ch. 1 (Trapping Rain Water), Ch. 15 (stack vs heap)
A2	Dijkstra	Correctness proof, generalised path costs (products, counts of shortest paths)	Ch. 18, Ch. 10 (BFS as the unit-weight case)
A3	Union-Find / DSU	Amortised analysis, redundancy counting, equivalence constraints	Ch. 17, Ch. 9 (DFS components)
A4	Minimum Spanning Tree	Cut property; Kruskal (DSU) vs Prim (heap or $O(V^2)$ array)	Ch. 17, Ch. 15, Ch. 14 (greedy proofs)
A5	Kadane	Local-vs-global best recurrence; circular and empty-subarray variants	Ch. 12 (1D DP), Ch. 4 (prefix sums)
A6	KMP	Prefix function; linear-time matching without re-examining text	Ch. 2 (string windows), Ch. 19 (prefix structure)
A7	Bellman–Ford	Negative weights, negative-cycle detection, edge-count-bounded paths	Ch. 18 (LC 787), Ch. 13 (DP over rounds)

Monotonic Stack

A stack whose contents stay sorted. Each element is pushed once and popped once, and the moment it is popped is exactly when you learn its nearest greater (or smaller) neighbour.

A1.1 Purpose and practical applications

Purpose	For every index, find the nearest index to the left or right whose value is greater (or smaller), all in O(n) total.
Applications	Next greater element, daily temperatures, stock span, largest rectangle in a histogram, maximal rectangle in a binary matrix, trapping rain water (layer by layer), sum of subarray minimums/maximums, remove-k-digits and other "smallest lexicographic subsequence" problems.
Key insight	An element that is dominated (a smaller value to the left of a larger one, when searching for "next greater") can never be the answer for anything further right, so it can be discarded permanently.

A1.2 Recognition signals

- The answer for position i is the **first** element to its left or right satisfying a comparison with a[i]: "next warmer day", "previous smaller bar", "how far can this extend".
- You need, for each element, the **maximal range in which it is the minimum (or maximum)**. This is contribution counting: Σ a[i] · (#subarrays where a[i] is the minimum).
- A greedy that "removes earlier elements while the new one is better" (e.g. build the smallest number by deleting digits).
- n up to 10^5 with an obvious $O(n^2)$ "scan outward from each i".

> **Ask yourself.** "When a new element arrives, which earlier elements can never again be an answer?" If those elements always form a suffix of what you are holding, the structure is a monotonic stack.

Not appropriate when you need the global extreme of a changing set (use a heap), the extreme of a sliding window (use a monotonic *deque*, which also evicts from the front), or the k-th nearest rather than the nearest.

A1.3 Core concept and invariant

Goal (scanning left → right)	Stack order bottom → top	Pop while…
Next greater element	Decreasing	`a[top] < a[i]`: i is top's next strictly greater
Next greater or equal	Strictly decreasing	`a[top] <= a[i]`
Next smaller element	Increasing	`a[top] > a[i]`
Previous smaller (after pops)	Increasing	Pop `a[top] >= a[i]`; the new top is the previous strictly smaller element

Invariant (next-greater form). After processing index i, the stack holds exactly the indices j ≤ i that do not yet have a next greater element, and their values are non-increasing from bottom to top. A popped index has found its answer, and that answer is final: i is the first index after it with a larger value, because every index between them was either smaller (popped earlier, or never above it) or is still on the stack above it, which contradicts the order.

Index vs value storage. Store *indices*. They give you the distance (`i - top`), the value (`a[top]`) and a write position in the answer array. Store values only when positions never matter (e.g. remove-k-digits builds the output directly).

Amortised O(n). Each index is pushed exactly once and popped at most once, so the inner while loop runs at most n times *in total* across all iterations, even though a single iteration may pop many.

A1.4 Important formulas and derivations

For contribution counting, let `L[i]` be the index of the previous element *strictly smaller* than a[i] (or −1), and `R[i]` be the index of the next element *smaller or equal* (or n). Then a[i] is the minimum of every subarray [l, r] with L[i] < l ≤ i ≤ r < R[i]:

> `count(i) = (i − L[i]) · (R[i] − i)`
> `Σ over subarrays of min = Σᵢ a[i] · (i − L[i]) · (R[i] − i)`
>
> Why one side strict and the other non-strict: with duplicates, a subarray such as [2, 2] has two positions holding its minimum. Making exactly one side non-strict assigns it to exactly one of them (here the rightmost occurrence), so nothing is double-counted or missed.

A1.5 Reusable Java implementation

Template A — next greater (right) and previous smaller (left). `ArrayDeque` used as a stack: `push` / `peek` / `pop` act on the head.

JAVA TEMPLATE MonotonicStack.java

```java
import java.util.*;

class MonotonicStack {
    /** nge[i] = first j > i with a[j] > a[i], or -1. Stack values non-increasing. */
    static int[] nextGreater(int[] a) {
        int n = a.length;
        int[] nge = new int[n];
        Arrays.fill(nge, -1);
        Deque<Integer> st = new ArrayDeque<>();           // holds indices still waiting
        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && a[st.peek()] < a[i]) {  // strict: equal values wait
                nge[st.pop()] = i;
            }
            st.push(i);
        }
        return nge;
    }

    /** pse[i] = last j < i with a[j] < a[i], or -1. Stack values strictly increasing. */
    static int[] previousSmaller(int[] a) {
        int n = a.length;
        int[] pse = new int[n];
        Deque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
```

```
            while (!st.isEmpty() && a[st.peek()] >= a[i]) st.pop();  // drop not-smaller
            pse[i] = st.isEmpty() ? -1 : st.peek();
            st.push(i);
        }
        return pse;
    }
}
```

Template B — contribution boundaries in one pass. An index is popped by the first element to its right that is ≤ it (right boundary, non-strict), and the element below it on the stack is its previous strictly smaller element (left boundary). A sentinel of −∞ at position n flushes the stack.

JAVA TEMPLATE MinContribution.java

```java
import java.util.*;

class MinContribution {
    /** Sum of min over all subarrays, exact (no modulus), using long. */
    static long sumOfSubarrayMins(int[] a) {
        int n = a.length;
        long total = 0;
        Deque<Integer> st = new ArrayDeque<>();   // indices; values strictly increasing
        for (int i = 0; i <= n; i++) {
            long cur = (i == n) ? Long.MIN_VALUE : a[i];    // sentinel pops everything
            while (!st.isEmpty() && a[st.peek()] >= cur) {
                int mid = st.pop();
                int left = st.isEmpty() ? -1 : st.peek();     // previous strictly smaller
                total += (long) a[mid] * (mid - left) * (i - mid);  // i: next <= a[mid]
            }
            st.push(i);
        }
        return total;
    }
}
```

A1.6 Time and space complexity

Time	O(n) amortised: n pushes, at most n pops.
Auxiliary space	O(n) for the stack in the worst case (monotone input never pops), plus the answer array.

A1.7 Common pitfalls

- **Strict vs non-strict.** `<` vs `<=` decides whether equal values resolve each other. For "next strictly warmer", equal temperatures must *not* pop. For contribution counting, exactly one side must be non-strict.
- **Forgetting the leftovers.** Indices still on the stack at the end have no answer. Pre-fill with −1/0/n or use a sentinel.
- **Overflow.** In contribution counting, `a[i] · left · right` can reach $3{\cdot}10^4 \cdot (3{\cdot}10^4)^2 \approx 2.7{\cdot}10^{13}$: compute in `long` and take the modulus only at the end of each product.
- **Using `Stack<Integer>`.** It is synchronised and slower. Prefer `ArrayDeque`, or a plain `int[]` with a top pointer for speed.
- **Circular arrays** (Next Greater Element II): iterate i from 0 to 2n−1 using `i % n`, and push only during the first pass.

A1.8 Comparison with related algorithms

Technique	Answers	Choose it when
Monotonic stack	Nearest greater/smaller per index	Answers depend on position (nearest), offline over the array
Monotonic deque	Max/min of each sliding window	Elements also expire from the old end (window of size k)
Heap	Global extreme of a dynamic set	Insertions/deletions in arbitrary order; no positional "nearest" meaning
Segment tree	Range min/max and "first index ≥ x in a range"	Queries are online or ranges are arbitrary; O(log n) each
Two pointers (LC 42)	Water using running maxima	Only the two global running maxima matter, so O(1) space

A1.9 Practice problems

LC 739 Daily Temperatures

MEDIUM

Task. For each day, return how many days you must wait for a strictly warmer temperature (0 if never).

KEY INSIGHT

"Next strictly greater element to the right", with the answer expressed as an index distance. A day is popped exactly when its first warmer day arrives.

WHY THE ALGORITHM APPLIES

The answer is the nearest qualifying element by position, and a colder day sitting to the left of a warmer one can never be the answer for later days.

APPROACH

1. Keep a stack of indices with non-increasing temperatures.
2. For each i, pop every index with a strictly smaller temperature and set `ans[j] = i - j`.
3. Push i. Remaining indices keep the default 0.

JAVA SOLUTION LeetCode 739 · Solution.java

```java
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] ans = new int[n];          // default 0 = no warmer day
        int[] stack = new int[n];        // array-backed stack of indices
        int top = -1;
        for (int i = 0; i < n; i++) {
            while (top >= 0 && temperatures[stack[top]] < temperatures[i]) {
                int j = stack[top--];
                ans[j] = i - j;
            }
            stack[++top] = i;
        }
        return ans;
    }
}
```

Time O(n) amortised **Space** O(n) stack

Pitfall. Using `<=` in the pop condition would treat an equal temperature as "warmer". An alternative O(1)-extra-space solution scans right-to-left and jumps along already computed answers (`j += ans[j]`), but the stack form is the one to know.

LC 907 Sum of Subarray Minimums

MEDIUM

Task. Return the sum of `min(sub)` over every contiguous subarray, modulo 10^9+7.

KEY INSIGHT

Instead of enumerating O(n^2) subarrays, count how many subarrays each element is the minimum of: `(i − L[i]) · (R[i] − i)`, with one strict and one non-strict boundary so that duplicates are counted once.

WHY THE ALGORITHM APPLIES

L[i] (previous strictly smaller) and R[i] (next smaller-or-equal) are both nearest-smaller queries, which a single increasing stack answers at pop time (Template B).

APPROACH

1. Scan i = 0..n with a sentinel at n; pop while `arr[top] >= cur`.
2. For the popped `mid`: left boundary = new top (or −1), right boundary = i.
3. Add `arr[mid] · (mid − left) · (i − mid)` in `long`, reducing modulo 10^9+7.

JAVA SOLUTION LeetCode 907 · Solution.java

```java
class Solution {
    private static final int MOD = 1_000_000_007;

    public int sumSubarrayMins(int[] arr) {
        int n = arr.length;
        int[] stack = new int[n + 1];   // indices; values strictly increasing bottom -> top
        int top = -1;
        long total = 0;
        for (int i = 0; i <= n; i++) {
            int cur = (i == n) ? 0 : arr[i];   // arr[i] >= 1, so 0 is a -infinity sentinel
            while (top >= 0 && arr[stack[top]] >= cur) {
                int mid = stack[top--];
                int left = (top >= 0) ? stack[top] : -1;   // previous strictly smaller
                long count = (long) (mid - left) * (i - mid);   // i = next smaller-or-equal
                total = (total + count % MOD * arr[mid]) % MOD;
            }
            stack[++top] = i;
        }
        return (int) total;
    }
}
```

Time O(n) **Space** O(n)

Pitfall. Making both boundaries strict double-counts subarrays with repeated minima (e.g. [1,1]); making both non-strict misses them. Also, `(mid − left) · (i − mid)` can reach $9 \cdot 10^8$ and its product with `arr[mid]` exceeds the int range, so cast to `long` *before* multiplying. The sentinel value 0 is valid only because the constraints guarantee `arr[i] ≥ 1`.

Dijkstra's Algorithm

Greedy single-source shortest paths for non-negative weights: always finalise the closest unfinished vertex. Chapter 18 covers the pattern and its problem shapes. This chapter covers the proof, the exact conditions under which the greedy step stays valid, and two generalisations: multiplicative path values and counting shortest paths.

A2.1 Purpose and practical applications

Purpose	Compute `dist[v]`, the minimum total weight of a path from a source s to every vertex v, when every edge weight is ≥ 0.
Applications	Network delay, routing, grid paths with costs, minimum-effort and minimax paths, maximum-reliability paths (products of probabilities ≤ 1), counting shortest paths, shortest paths on expanded state graphs (node, fuel), (node, stops), (cell, keys).
Key insight	With non-negative weights a path never gets cheaper as it grows, so the cheapest frontier vertex cannot be improved by any route through vertices that are farther away.

A2.2 Recognition signals

- A shortest, cheapest, fastest or most reliable route on a graph or grid whose edges have **different, non-negative** costs.
- The path cost is built by an operation that is **monotone along the path**: sum of non-negative weights, maximum of edge weights, product of factors in [0, 1] (maximised).
- Single source (or a super-source for multi-source), V and E up to about 10^5.
- A state richer than a vertex is fine, provided the state space stays small enough: Dijkstra runs on any graph whose vertices are states.

> **Ask yourself.** "Can extending a path ever make it *better*?" If no, a greedy settle-the-best-first order is valid. If yes (negative weights, or a bonus for longer paths), Dijkstra is invalid.

A2.3 Core concept and invariant

Relaxation. For an edge u → v with weight w: `if dist[u] + w < dist[v] then dist[v] = dist[u] + w`. Every shortest-path algorithm is a strategy for ordering relaxations. Dijkstra orders them by settling vertices in increasing distance, so each edge is relaxed once from its final source distance.

Invariant. Let S be the set of settled vertices. (1) For every v ∈ S, `dist[v]` equals the true distance δ(s, v). (2) For every v ∉ S, `dist[v]` is the shortest path to v that uses only vertices of S before its last edge.

Proof that the popped vertex is final

Let u be the unsettled vertex with the smallest tentative `dist[u] = d`. Take any s → u path P. P starts inside S and ends outside it, so it has a first vertex x outside S, reached by an edge y → x with y ∈ S. By invariant (2), the prefix of P up to x costs at least `dist[x] ≥ d` (u was the minimum). The rest of P, from x

to u, has weight ≥ 0. So cost(P) ≥ d, and no path beats `dist[u]`. Settling u preserves (1), and relaxing u's edges restores (2).

The step "the rest of P has weight ≥ 0" is exactly where negative weights break the proof.

Why negative weights break it. Edges s→a (2), s→b (3), b→a (−2). Dijkstra settles a at distance 2 before it settles b. The true distance is 3 + (−2) = 1, found only after a has been finalised. With lazy re-insertion some implementations happen to recover in this case, but then the "settled once" guarantee and the complexity bound both disappear, and negative cycles make the answer undefined. Use Bellman–Ford (A7).

Stale entries. Java's `PriorityQueue` has no decrease-key. Push a new `(dist, v)` entry on every improvement and, when popping, skip any entry whose distance is larger than the current `dist[v]`. The heap holds at most E + 1 entries.

A2.4 Important formulas and derivations

```
Generalised condition: path value f(P · e) is computed from f(P) and w(e) by an
operation ⊕ that is
  (a) monotone:            a ≤ b ⇒ a ⊕ w ≤ b ⊕ w
  (b) non-improving:       a ⊕ w ≥ a          (for minimisation)
Sum with w ≥ 0           : a ⊕ w = a + w              ✓
Bottleneck (LC 1631)     : a ⊕ w = max(a, w)          ✓
Reliability (LC 1514)    : a ⊕ p = a · p, p ∈ [0, 1]  ✓ (maximise; value never
increases)
Sum with w < 0           : violates (b)               ✗  → Bellman–Ford

Counting shortest paths (weights > 0):
  ways[s] = 1
  ways[v] = Σ ways[u]  over edges u → v with dist[u] + w(u, v) = dist[v]
```

The counting recurrence is evaluated correctly in pop order because every predecessor u on a shortest path to v has `dist[u] < dist[v]` when weights are strictly positive. So all of v's contributors are settled, and their counts are final, before v itself is popped. With zero-weight edges this ordering argument fails and the counting needs extra care: zero-weight ties may be popped in either order.

A2.5 Reusable Java implementation

Template A — lazy Dijkstra with path reconstruction. Directed edges `{u, v, w}` with w ≥ 0; add the reverse edge for undirected graphs. Distances are `long` because sums of int weights overflow.

JAVA TEMPLATE Dijkstra.java

```java
import java.util.*;

class Dijkstra {
    static final long INF = Long.MAX_VALUE / 4;   // headroom: INF + w never overflows

    long[] dist;
    int[] parent;

    /** n vertices 0..n-1, directed edges {u, v, w}, w >= 0. */
    Dijkstra(int n, int[][] edges, int src) {
        List<List<int[]>> adj = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) adj.get(e[0]).add(new int[]{e[1], e[2]});

        dist = new long[n];
        parent = new int[n];
        Arrays.fill(dist, INF);
        Arrays.fill(parent, -1);
        dist[src] = 0;
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
        pq.add(new long[]{0, src});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
            long d = top[0];
            int u = (int) top[1];
            if (d > dist[u]) continue;           // stale entry: u already settled cheaper
            for (int[] e : adj.get(u)) {
                int v = e[0];
                long nd = d + e[1];
                if (nd < dist[v]) {                    // relaxation
                    dist[v] = nd;
                    parent[v] = u;
                    pq.add(new long[]{nd, v});
                }
            }
        }
    }

    /** Vertices on a shortest src -> t path, or an empty list if t is unreachable. */
    List<Integer> path(int t) {
        LinkedList<Integer> p = new LinkedList<>();
        if (dist[t] >= INF) return p;
        for (int v = t; v != -1; v = parent[v]) p.addFirst(v);
        return p;
    }
}
```

Template B — dense-graph Dijkstra, O(V²), no heap. When E ≈ V² (a complete graph or adjacency matrix), a linear scan for the minimum beats a heap's O(E log V).

JAVA TEMPLATE DenseDijkstra.java

```
import java.util.*;

class DenseDijkstra {
    /** w[u][v] >= 0 is the edge weight, or -1 if there is no edge (directed). */
    static long[] run(int[][] w, int src) {
        int n = w.length;
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        boolean[] done = new boolean[n];
        dist[src] = 0;
        for (int iter = 0; iter < n; iter++) {
            int u = -1;
            for (int v = 0; v < n; v++) {
                if (done[v] || dist[v] == Long.MAX_VALUE) continue;
                if (u == -1 || dist[v] < dist[u]) u = v;
            }
            if (u == -1) break;                        // remaining vertices unreachable
            done[u] = true;
            for (int v = 0; v < n; v++) {
                if (w[u][v] >= 0 && !done[v] && dist[u] + w[u][v] < dist[v]) {
                    dist[v] = dist[u] + w[u][v];
                }
            }
        }
```

```
        return dist;                                    // Long.MAX_VALUE = unreachable
    }
}
```

A2.6 Time and space complexity

Binary heap, lazy deletion	O((V + E) log E) = O((V + E) log V), since E ≤ V^2. Space O(V + E): adjacency lists plus up to E heap entries.
Array scan (dense)	O(V^2 + E) time, O(V) extra space. Better when E = Θ(V^2).
Fibonacci heap (theory)	O(E + V log V). Not used in interviews.

A2.7 Common pitfalls

- **No stale check.** Without `if (d > dist[u]) continue`, the code stays correct but may re-expand vertices many times. For counting problems it double-counts ways.
- **Marking visited on push** (BFS habit). The first push is not necessarily the shortest, so a vertex is final only when popped.
- **int distances.** 10^5 edges × 10^9 weight overflows int. Use `long`, an INF with headroom, and `Long.compare` in the comparator.
- **Comparator subtraction** `(a, b) -> (int)(a[0] - b[0])` truncates and overflows. Never use it.
- **Early exit** on popping the target is safe; exiting when the target is first *pushed* is not.
- **Directed vs undirected.** Check the statement; LC 743 is directed, LC 1514 and 1976 are undirected.

A2.8 Comparison with related algorithms

Algorithm	Weights	Complexity	Use when
BFS (Ch. 10)	All equal	O(V + E)	Unweighted fewest-steps
0-1 BFS	0 or 1	O(V + E)	Deque: push 0-edges to the front, 1-edges to the back
Dijkstra	≥ 0	O((V + E) log V)	General non-negative single source
Bellman–Ford (A7)	Any; detects negative cycles	O(V · E)	Negative edges, or at most K edges
Floyd–Warshall	Any, no negative cycles	O(V^3)	All pairs, V ≤ ~400
DAG relaxation	Any, acyclic	O(V + E)	Relax in topological order (Ch. 11)

A2.9 Practice problems

LC 1514 Path with Maximum Probability — MEDIUM

Task. In an undirected graph whose edges carry success probabilities in [0, 1], return the maximum probability of a path from `start_node` to `end_node`, or 0 if no path exists.

KEY INSIGHT

Path value is a product of factors in [0, 1], so extending a path never increases its value. That is the mirror image of "never decreases the cost", which makes greedy settle-the-most-probable-first valid with a *max*-heap.

WHY THE ALGORITHM APPLIES

The monotone, non-improving condition from A2.4 holds with $\oplus$ = multiplication and the order reversed. Equivalently, minimise $\Sigma -\log p$, which has all weights ≥ 0. Multiplying directly avoids floating-point log error.

APPROACH

1. Build an undirected adjacency list with probabilities.
2. `best[start] = 1`; max-heap on probability; skip stale entries where `p < best[u]`.
3. Relax `best[v] = max(best[v], best[u] · p)`; return when `end` is popped.

JAVA SOLUTION LeetCode 1514 · Solution.java

```java
import java.util.*;

class Solution {
    public double maxProbability(int n, int[][] edges, double[] succProb,
                                 int start_node, int end_node) {
        List<List<double[]>> adj = new ArrayList<>();        // {neighbour, probability}
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int i = 0; i < edges.length; i++) {             // undirected
            adj.get(edges[i][0]).add(new double[]{edges[i][1], succProb[i]});
            adj.get(edges[i][1]).add(new double[]{edges[i][0], succProb[i]});
        }
        double[] best = new double[n];                       // 0 = unreachable so far
        best[start_node] = 1.0;
        // max-heap on probability
        PriorityQueue<double[]> pq =
            new PriorityQueue<>((a, b) -> Double.compare(b[0], a[0]));
        pq.add(new double[]{1.0, start_node});
        while (!pq.isEmpty()) {
            double[] top = pq.poll();
            double p = top[0];
            int u = (int) top[1];
            if (p < best[u]) continue;                       // stale
            if (u == end_node) return p;                     // popped = final
            for (double[] e : adj.get(u)) {
                int v = (int) e[0];
                double np = p * e[1];
                if (np > best[v]) {
                    best[v] = np;
                    pq.add(new double[]{np, v});
                }
            }
        }
        return 0.0;
    }
}
```

Time O((V + E) log E) **Space** O(V + E)

Pitfall. A min-heap (copied from shortest-path code) silently settles the *least* probable path first. Edges with probability 0 never improve `best[v]` from 0 and are correctly ignored. The node id is stored in a `double` slot, which is exact for integers far beyond $n \leq 10^4$.

LC 1976 Number of Ways to Arrive at Destination

MEDIUM

Task. In an undirected graph with positive travel times, count the shortest-time paths from intersection 0 to n − 1, modulo 10^9+7.

KEY INSIGHT

Shortest-path counting is DP over the shortest-path DAG, and Dijkstra's pop order is a topological order of that DAG when weights are strictly positive. Count while relaxing: a strict improvement *resets* the count, and an equal distance *adds* to it.

WHY THE ALGORITHM APPLIES

Weights are positive (time ≥ 1), so the recurrence from A2.4 is complete for v when v is popped. Distances reach about 200×10^9, so they need `long`.

APPROACH

1. `dist[0] = 0, ways[0] = 1`.
2. Pop (d, u); skip stale entries. For each edge (u, v, t): if `d + t < dist[v]`, set `dist[v]`, `ways[v] = ways[u]`, push; else if equal, `ways[v] += ways[u]`.
3. Return `ways[n-1]`.

JAVA SOLUTION LeetCode 1976 · Solution.java

```java
import java.util.*;

class Solution {
    private static final int MOD = 1_000_000_007;

    public int countPaths(int n, int[][] roads) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] r : roads) {                                  // undirected, time >= 1
            adj.get(r[0]).add(new int[]{r[1], r[2]});
            adj.get(r[1]).add(new int[]{r[0], r[2]});
        }
        long[] dist = new long[n];
        long[] ways = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        dist[0] = 0;
        ways[0] = 1;
        PriorityQueue<long[]> pq = new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
        pq.add(new long[]{0, 0});
        while (!pq.isEmpty()) {
            long[] top = pq.poll();
            long d = top[0];
            int u = (int) top[1];
            if (d > dist[u]) continue;            // stale: would double-count ways
            for (int[] e : adj.get(u)) {
                int v = e[0];
                long nd = d + e[1];
                if (nd < dist[v]) {               // strictly shorter: old ways are obsolete
                    dist[v] = nd;
                    ways[v] = ways[u];
                    pq.add(new long[]{nd, v});
                } else if (nd == dist[v]) {       // another shortest route into v
                    ways[v] = (ways[v] + ways[u]) % MOD;
                }
            }
        }
        return (int) ways[n - 1];
    }
}
```

Time O((V + E) log E) **Space** O(V + E)

Pitfall. Without the stale check, a vertex popped twice at the same distance would push its ways into neighbours twice. Using `int` distances overflows at about $2.1 \cdot 10^9$. The edge case n = 1 returns `ways[0] = 1`.

Union-Find / Disjoint Set Union

A forest of parent pointers that answers "are x and y in the same group?" while groups only ever merge. Chapter 17 applies it to connectivity problems. This chapter covers why the two optimisations matter, what the amortised bound really means, a weighted extension, and the counting arguments that recur in DSU problems.

A3.1 Purpose and practical applications

Purpose	Maintain a partition of n elements under `union(x, y)` and `find(x)` (the canonical representative), in near-constant amortised time per operation.
Applications	Connected components with edges arriving online, cycle detection in undirected graphs, Kruskal's MST (A4), equivalence constraints (`a == b`), account/email merging, percolation, "number of islands II", offline connectivity queries sorted by threshold, and ratio or parity constraints (weighted DSU).
Key insight	Only the *identity* of each group matters, not its shape. That freedom lets you flatten trees aggressively (path compression) and attach small trees under large ones (union by size), keeping every tree almost flat.

A3.2 Recognition signals

- Relations that are reflexive, symmetric and transitive (equivalence): "connected", "same account", "equal", "friends of friends".
- Edges or relations arrive over time, interleaved with "same group?" or "how many groups?" queries.
- Counting components, or counting the edges that are redundant within a component.
- Detecting the first edge that closes a cycle in an undirected graph.

> **Ask yourself.** "Do groups only merge, and do I only need membership, not paths?" If you need splits (edge deletion), paths, distances or directed reachability, DSU is the wrong tool.

A3.3 Core concept and invariant

Representation. `parent[x]` points toward the root of x's tree, and a root has `parent[r] = r`. Two elements are in the same set exactly when their roots coincide. The invariant to preserve: *every element's parent chain ends at its own set's root*. A union only re-points one root, so it never breaks the chains of other elements.

- **Path compression**: during `find`, point visited nodes directly (or every other node, for path halving) at the root. Membership is unchanged; only the shape changes.
- **Union by size / rank**: attach the root of the smaller tree under the root of the larger. A node's depth grows only when its tree at least doubles in size, so depth ≤ $\log_2 n$ even without compression.

A3.4 Important formulas and derivations

```
Union by size alone        : height ≤ ⌊log₂ n⌋            ⇒ find is O(log n) worst case
Path compression alone     : O(log n) amortised per operation (m ≥ n operations)
Both together (Tarjan)     : O(m · α(n)) for m operations; α(n) ≤ 4 for any n < 2^65536

Components after processing edges: components = n − (number of successful unions)
Redundant edges              : redundant = E − (number of successful unions)
Connecting k components needs exactly k − 1 additional edges.
```

Why size beats rank in interviews. Both give the same bound. Size has a direct meaning: it answers "how large is x's group?" with `size[find(x)]` at no extra cost.

A3.5 Reusable Java implementation

Template A — DSU with path halving and union by size. The iterative find avoids recursion depth issues for n up to 10^6.

JAVA TEMPLATE DSU.java

```java
class DSU {
    private final int[] parent, size;
    private int components;

    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];   // path halving: shortcut every other node
            x = parent[x];
        }
        return x;
    }

    /** Returns false if x and y were already connected (edge (x, y) closes a cycle). */
    boolean union(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        if (size[rx] < size[ry]) { int t = rx; rx = ry; ry = t; }  // rx is the larger root
        parent[ry] = rx;
        size[rx] += size[ry];
        components--;
        return true;
    }

    boolean connected(int x, int y) { return find(x) == find(y); }
    int componentSize(int x) { return size[find(x)]; }
    int components() { return components; }
}
```

Template B — weighted DSU (potential to the root). This stores `w[x] = value(x) − value(parent[x])`, so constraints such as `value(b) − value(a) = d` can be merged and checked. It is the basis of LC 399 (Evaluate Division, in multiplicative form) and of parity problems.

JAVA TEMPLATE `WeightedDSU.java`

```java
class WeightedDSU {
    private final int[] parent;
    private final long[] diff;   // diff[x] = value(x) − value(parent[x])

    WeightedDSU(int n) {
        parent = new int[n];
        diff = new long[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    /** Returns the root; afterwards diff[x] = value(x) − value(root). */
    int find(int x) {
        if (parent[x] == x) return x;
        int root = find(parent[x]);
        diff[x] += diff[parent[x]];      // parent[x] is now a root child: full offset
        parent[x] = root;
        return root;
    }

    /** Adds value(b) − value(a) = d. Returns false if it contradicts earlier facts. */
    boolean relate(int a, int b, long d) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return diff[b] − diff[a] == d;
        // value(rb) − value(ra) = d + diff[a] − diff[b]
        parent[rb] = ra;
        diff[rb] = d + diff[a] − diff[b];
        return true;
    }

    /** value(b) − value(a) if known, else null. */
    Long difference(int a, int b) {
        if (find(a) != find(b)) return null;
        return diff[b] − diff[a];
    }
}
```

Template B uses recursion and no union by size, to keep the offset algebra readable. Its depth is O(log n) amortised but can be O(n) on a single first call, so add union by size (the offset formula is symmetric) when n is large.

A3.6 Time and space complexity

Time	O(α(n)) amortised per `find` / `union` with both optimisations; O(n) to initialise.
Space	O(n) for `parent` and `size`.
Without optimisations	O(n) per operation worst case (a chain), so O(n · m) total.

A3.7 Common pitfalls

- **Comparing parents instead of roots.** `parent[x] == parent[y]` is wrong; always compare `find(x) == find(y)`.
- **Updating size on the wrong root**, or reading `size[x]` for a non-root x.
- **Recursive find** on 10^5+ elements with no union by size can overflow the stack on the first call.

- **Labels that are not 0..n−1** (strings, coordinates, 1-indexed cities): map them to dense ids first, or size the array n + 1.
- **Expecting deletions.** DSU cannot split a set. For offline problems, process in reverse (deletions become unions) or use DSU with rollback.
- **Directed graphs.** DSU ignores direction; cycle detection in a directed graph needs DFS colouring (Ch. 11).

A3.8 Comparison with related algorithms

Approach	Strength	Prefer it when
DSU	Online merges, O(α(n)) queries, tiny code	Edges stream in; only membership or component counts are needed
DFS/BFS components	Paths, distances, levels, colouring	The graph is static and you need more than membership (e.g. bipartiteness, actual path)
Weighted DSU	Relative values within a group	Constraints of the form x − y = d, x / y = r, or parity
Tarjan / bridges	Edge-removal effects	You need critical edges or articulation points (DSU cannot split)

A3.9 Practice problems

LC 1319 Number of Operations to Make Network Connected — MEDIUM

Task. n computers, undirected cables `connections`. You may move any cable to any other pair of computers. Return the minimum number of moves to connect every computer, or −1 if impossible.

KEY INSIGHT

A connected graph on n vertices needs at least n − 1 edges. If there are at least n − 1 cables, the redundant cables (those whose endpoints were already connected) are always enough to link the k components with k − 1 moves.

WHY THE ALGORITHM APPLIES

We need the component count and the redundant-edge count. Both are byproducts of union successes and failures (A3.4).

APPROACH

1. If `connections.length < n - 1`, return −1.
2. Union every cable; the answer is `components - 1`.

JAVA SOLUTION LeetCode 1319 · Solution.java

```java
class Solution {
    private int[] parent, size;

    public int makeConnected(int n, int[][] connections) {
        if (connections.length < n - 1) return -1;       // not enough cables in total
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
        int components = n;
        for (int[] c : connections) {
            if (union(c[0], c[1])) components--;         // undirected edge
        }
```

```
        return components - 1;  // E >= n-1 guarantees >= components-1 redundant cables
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(n + E · α(n)) **Space** O(n)

Pitfall. The redundant count is never needed explicitly. Proof: with k components and E ≥ n − 1 edges, redundant = E − (n − k) ≥ (n − 1) − (n − k) = k − 1. Many solutions compute it anyway and compare, which is harmless. Forgetting the up-front −1 check is not.

LC 990 Satisfiability of Equality Equations

MEDIUM

Task. Given equations like `"a==b"` and `"b!=c"` over single lowercase-letter variables, decide whether integers can be assigned so that all hold.

KEY INSIGHT

Equality is an equivalence relation. Its classes are exactly the DSU components after merging every `==`. A `!=` is violated exactly when both variables fall in the same class. Any assignment of distinct values per class satisfies everything else.

WHY THE ALGORITHM APPLIES

All merges must be applied before any check. A `!=` that looks fine early could be contradicted by later equalities, so this is two-phase offline DSU.

APPROACH

1. Pass 1: union the endpoints of every `==` (26 elements).
2. Pass 2: for every `!=`, return false if the endpoints share a root.

JAVA SOLUTION LeetCode 990 · Solution.java

```
class Solution {
    private final int[] parent = new int[26];

    public boolean equationsPossible(String[] equations) {
        for (int i = 0; i < 26; i++) parent[i] = i;
        for (String e : equations) {                        // phase 1: all equalities
            if (e.charAt(1) == '=') union(e.charAt(0) - 'a', e.charAt(3) - 'a');
        }
        for (String e : equations) {                        // phase 2: check inequalities
            if (e.charAt(1) == '!' && find(e.charAt(0) - 'a') == find(e.charAt(3) - 'a')) {
                return false;                               // includes "a!=a"
```

```
            }
        }
        return true;
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private void union(int a, int b) {
        parent[find(a)] = find(b);  // 26 elements: union by size is unnecessary
    }
}
```

Time O(m · α(26)) ≈ O(m) for m equations **Space** O(1) (26-entry array)

Pitfall. Processing the equations in input order (checking `!=` as you go) gives wrong answers for `["a!=b","b==a"]`. The trivial contradiction `"a!=a"` is caught automatically because a shares a root with itself.

Minimum Spanning Tree: Kruskal and Prim

Connect every vertex of an undirected, weighted graph with the minimum total edge weight. Both classic algorithms are greedy, and both are justified by the same *cut property*.

A4.1 Purpose and practical applications

Purpose	Find a set of V − 1 edges that connects all V vertices of an undirected graph with minimum total weight (a minimum spanning forest if the graph is disconnected).
Applications	Network/cable/pipe layout, connecting points at minimum cost, clustering (remove the k − 1 heaviest MST edges), approximation for metric TSP, minimax paths (the MST contains a bottleneck-optimal path between every pair), "critical and pseudo-critical edges".
Key insight	For any partition of the vertices into two sides, the cheapest edge crossing the partition is safe to add. Kruskal and Prim are two different ways of choosing which partition to look at.

A4.2 Recognition signals

- "Connect all", "minimum total cost to link every", "any network is fine as long as everything is reachable".
- Undirected graph; the cost is the **sum of chosen edges**, not the distance between particular pairs.
- Points in a plane with a cost per pair (an implicit complete graph).
- The answer should be −1 or "impossible" when the graph cannot be connected: check that V − 1 edges were taken.

> **Ask yourself.** "Is the objective the total weight of the network, or the distance from a source?" Total weight → MST. Source distances → shortest-path tree (Dijkstra). The two trees are generally different.

Not applicable to directed graphs (that is the minimum arborescence problem, solved by Chu–Liu/Edmonds), or when only a subset of terminals must be connected (the Steiner tree problem, which is NP-hard).

A4.3 Core concept and invariant

Invariant (both algorithms). The chosen edge set F is always a subset of *some* minimum spanning tree.

Cut property and its exchange proof

Let (S, V\S) be a cut that no edge of F crosses, and let e = (u, v) be a minimum-weight edge crossing it. Take an MST T ⊇ F. If e ∈ T, we are done. Otherwise T + e contains a cycle, and that cycle must cross the cut a second time through some edge e' ∈ T. Since w(e) ≤ w(e'), the tree T' = T + e − e' is spanning, has weight ≤ w(T), and contains F + e. So F + e is still inside an MST.

Kruskal scans edges by weight. When edge (u, v) joins two different components, take the cut S = component of u: no lighter edge crosses it (any lighter crossing edge would have been taken already), so the edge is safe. **Prim** grows a single tree S and repeatedly takes the cheapest edge leaving S, which is a direct application of the cut property.

Cycle property (the dual). The heaviest edge on any cycle (if strictly heaviest) is in no MST. That is why Kruskal may safely reject an edge whose endpoints are already connected.

A4.4 Important formulas and derivations

```
A spanning tree of V vertices has exactly V − 1 edges.
Connected ⇔ Kruskal accepts exactly V − 1 edges.
Spanning forest of a graph with c components has V − c edges.

Kruskal : O(E log E) sort + O(E · α(V)) DSU            = O(E log V)
Prim    : binary heap, lazy   O(E log E)                 = O(E log V)
          adjacency matrix / complete graph, array scan   O(V²)
Dense complete graph (E = V(V−1)/2): Prim O(V²) beats Kruskal O(V² log V)
```

If all edge weights are distinct the MST is unique. With ties there may be several MSTs, all with the same total weight.

A4.5 Reusable Java implementation

Template A — Kruskal with DSU. Undirected edges `{u, v, w}`, vertices 0..n−1. Returns the forest weight and reports connectivity.

JAVA TEMPLATE Kruskal.java

```java
import java.util.*;

class Kruskal {
    /** Returns MST weight, or -1 if the graph is disconnected. Edges are undirected. */
    static long mstWeight(int n, int[][] edges) {
        int[][] es = edges.clone();                // do not reorder the caller's array
        Arrays.sort(es, (a, b) -> Integer.compare(a[2], b[2]));
        int[] parent = new int[n], size = new int[n];
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        long total = 0;
        int taken = 0;
        for (int[] e : es) {
            int ru = find(parent, e[0]), rv = find(parent, e[1]);
            if (ru == rv) continue;                // would close a cycle: cycle property
            if (size[ru] < size[rv]) { int t = ru; ru = rv; rv = t; }
            parent[rv] = ru;
            size[ru] += size[rv];
            total += e[2];
            if (++taken == n - 1) break;                    // tree complete
        }
        return taken == n - 1 ? total : -1;        // n - taken = number of components
    }

    private static int find(int[] p, int x) {
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
```

```
    }
}
```

Template B — Prim with a lazy binary heap. Adjacency list; a vertex is added to the tree when it is first popped.

JAVA TEMPLATE PrimHeap.java

```java
import java.util.*;

class PrimHeap {
    /** Undirected edges {u, v, w}. Returns MST weight, or -1 if disconnected. */
    static long mstWeight(int n, int[][] edges) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) {
            adj.get(e[0]).add(new int[]{e[1], e[2]});
            adj.get(e[1]).add(new int[]{e[0], e[2]});
        }
        boolean[] inTree = new boolean[n];
        PriorityQueue<int[]> pq =
            new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
        pq.add(new int[]{0, 0});                    // {vertex, weight of edge into tree}
        long total = 0;
        int count = 0;
        while (!pq.isEmpty() && count < n) {
            int[] top = pq.poll();
            int u = top[0];
            if (inTree[u]) continue;                // stale: u joined via a cheaper edge
            inTree[u] = true;
            total += top[1];
            count++;
            for (int[] e : adj.get(u)) {
                if (!inTree[e[0]]) pq.add(new int[]{e[0], e[1]});  // crossing edges
            }
        }
        return count == n ? total : -1;
    }
}
```

Prim's heap holds *edge weights* (the cost to attach a vertex), not path distances. Writing `dist[u] + w`, the Dijkstra habit, computes a shortest-path tree instead.

A4.6 Time and space complexity

Kruskal	O(E log E) time; O(V) DSU space + O(E) for the sorted copy.
Prim (heap)	O(E log E) time; O(V + E) space for adjacency lists and heap.
Prim (array, dense)	$O(V^2)$ time, O(V) space; no edge list needs to be materialised.

A4.7 Common pitfalls

- **Confusing MST with shortest paths.** Prim's key is the single attaching edge weight, not the path distance from the root.
- **Disconnected input.** Prim started at vertex 0 silently returns a tree of 0's component only. Check that all V vertices were added (or V − 1 edges, for Kruskal).
- **Materialising V^2 edges** for 1000+ points in Kruskal: about 500k edges and an O(E log E) sort works but is slower and heavier than $O(V^2)$ Prim.

- **1-indexed vertices** (LC 1135): size the arrays n + 1, or subtract 1.
- **Overflow.** The total of V − 1 weights can exceed the int range: accumulate in `long`.

A4.8 Comparison with related algorithms

Algorithm	Mechanism	Best for
Kruskal	Global edge order + DSU	Sparse graphs given as edge lists; spanning forests; "process edges by threshold" problems
Prim (heap)	Grow one tree via the cheapest crossing edge	Graphs given as adjacency lists
Prim (array)	Linear scan for the closest outside vertex	Dense or complete graphs (point sets), $O(V^2)$
Dijkstra (A2)	Grow a tree by *path* distance	Shortest paths from one source, a different objective
Borůvka	All components add their cheapest edge in parallel rounds	Parallel/distributed settings; rarely needed in interviews

A4.9 Practice problems

LC 1584 Min Cost to Connect All Points

MEDIUM

Task. Given up to 1000 points, connect them all with minimum total Manhattan distance `|xi − xj| + |yi − yj|`.

KEY INSIGHT

The graph is complete and implicit: every pair is an edge. With V = 1000, E ≈ $5 \cdot 10^5$. Array-based Prim runs in $O(V^2) = 10^6$ steps and never builds an edge list.

WHY THE ALGORITHM APPLIES

"Connect all at minimum total cost" on an undirected graph is MST by definition. Density decides the variant: Prim with an array.

APPROACH

1. `minDist[v]` = cheapest edge from the current tree to v (initially ∞; 0 for point 0).
2. Repeat V times: pick the non-tree vertex with the smallest `minDist`, add it, and update every non-tree vertex with its distance to the new vertex.

JAVA SOLUTION LeetCode 1584 · Solution.java

```java
import java.util.*;

class Solution {
    public int minCostConnectPoints(int[][] points) {
        int n = points.length;
        int[] minDist = new int[n];            // cheapest edge from the tree to each vertex
        boolean[] inTree = new boolean[n];
        Arrays.fill(minDist, Integer.MAX_VALUE);
        minDist[0] = 0;
        long total = 0;
        for (int iter = 0; iter < n; iter++) {
            int u = -1;
            for (int v = 0; v < n; v++) {      // O(V) scan replaces the heap
                if (!inTree[v] && (u == -1 || minDist[v] < minDist[u])) u = v;
            }
            inTree[u] = true;
```

```
            total += minDist[u];
            for (int v = 0; v < n; v++) {
                if (!inTree[v]) {
                    int d = Math.abs(points[u][0] - points[v][0])
                          + Math.abs(points[u][1] - points[v][1]);   // <= 4e6, fits in int
                    if (d < minDist[v]) minDist[v] = d;
                }
            }
        }
        return (int) total;
    }
}
```

Time $O(V^2)$ **Space** $O(V)$

Pitfall. Kruskal over all pairs is accepted but is $O(V^2 \log V)$ time and $O(V^2)$ memory. The complete graph is always connected, so no -1 case exists. A single edge is at most $4 \cdot 10^6$, but the running total is accumulated in `long` as a habit, because sums of $V - 1$ such edges can exceed int in general.

LC 1135 Connecting Cities With Minimum Cost

MEDIUM Premium

Task. Cities are labelled 1..n; `connections[i] = [x, y, cost]` is an undirected candidate road. Return the minimum cost to connect all cities, or -1 if impossible.

KEY INSIGHT

The input is a sparse edge list with a possible disconnected answer, which is exactly Kruskal's natural input. Connectivity is checked for free by counting accepted edges.

WHY THE ALGORITHM APPLIES

"Connect all cities at minimum total cost" is the MST objective. Kruskal's accepted edges form the minimum spanning forest. Fewer than $n - 1$ of them means the graph is disconnected.

APPROACH

1. Sort the connections by cost.
2. Union the endpoints (1-indexed, array size n + 1). Add the cost on a successful union.
3. Return the total if $n - 1$ edges were accepted, else -1.

JAVA SOLUTION LeetCode 1135 · Solution.java

```
import java.util.*;

class Solution {
    private int[] parent, size;

    public int minimumCost(int n, int[][] connections) {
        parent = new int[n + 1];                          // cities are 1-indexed
        size = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
        Arrays.sort(connections, (a, b) -> Integer.compare(a[2], b[2]));
        int total = 0, used = 0;
        for (int[] c : connections) {
            if (union(c[0], c[1])) {                      // undirected road
                total += c[2];
                if (++used == n - 1) return total;        // spanning tree complete
            }
```

```
        }
        return n == 1 ? 0 : -1;                    // fewer than n-1 edges: disconnected
    }

    private int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        size[ra] += size[rb];
        return true;
    }
}
```

Time O(E log E) **Space** O(n) DSU + O(log E) sort stack

Pitfall. With n = 1 no edge is needed, so return 0. Bounds are $n \le 10^4$ and cost $\le 10^5$, so the total stays below 10^9 and `int` is safe here. Sorting the input array in place is acceptable on LeetCode; copy it if the caller still needs the original order.

Kadane's Algorithm

A one-pass dynamic programme for the best contiguous subarray. The best subarray ending at index i either extends the best one ending at i − 1 or starts fresh at i.

A5.1 Purpose and practical applications

Purpose	Maximum (or minimum) sum over all non-empty contiguous subarrays in O(n) time and O(1) space.
Applications	Maximum subarray, circular maximum (via a minimum-subarray complement), best time to buy and sell once (Kadane on price differences), maximum-sum submatrix (Kadane over compressed column sums, $O(R^2 \cdot C)$), and "maximum gain from flipping a segment" style problems.
Key insight	Only the best sum *ending exactly at i* needs to be carried forward. A negative running sum can never help any future subarray, so drop it.

A5.2 Recognition signals

Signal	Structural reason
Contiguous subarray, objective is a *sum* (max or min), no length or target constraint	The objective decomposes as "best ending here" with a two-way choice: extend or restart.
Negative values present	With all non-negative values the answer is the whole array. Negatives make Kadane, not a sliding window, the right tool.
Wraparound allowed	A circular subarray is either a normal subarray or the complement of a normal one (total − minimum subarray).
"Profit", "gain", "net change" over a contiguous period	Convert to a difference array and run Kadane over it.

> **Ask yourself.** "Is the objective a sum of a contiguous block with no other constraint?" If there is a target sum, use prefix sums + hash map (Pattern 4). If there is a length bound, use prefix sums with a monotonic deque. If the objective is a product, track both max and min.

A5.3 Core concept and invariant

Let `end(i)` be the maximum sum of a non-empty subarray that ends exactly at index i, and `best(i)` the maximum over all subarrays within a[0..i].

```
Invariant after processing index i:
  cur  = end(i)  = max sum of a[j..i] over 0 ≤ j ≤ i
  best = max(end(0), …, end(i))
```

Every subarray ends somewhere, so the answer is `max over i of end(i)`. That is the whole correctness argument: local best (ending here) feeds the global best (anywhere so far).

A5.4 Important formulas and derivations

```
end(0) = a[0]
end(i) = a[i] + max(0, end(i-1))  =  max(a[i], end(i-1) + a[i])
answer = max over i of end(i)
```

Derivation: a subarray ending at i is either [i] alone, or a[j..i−1] followed by a[i]. The best of the second kind is end(i−1) + a[i]. So keep the previous run only if it is positive.

Prefix-sum view. sum(a[j..i]) = pre[i+1] − pre[j], so end(i) = pre[i+1] − min(pre[0..i]). Kadane is the same as tracking the running minimum prefix. This view generalises to length-constrained versions: use a minimum over a sliding range of prefixes (monotonic deque).

Circular case. A wrapping subarray takes a suffix and a prefix, which is the complement of a middle block a[j..i]. Its sum is `total − sum(a[j..i])`, maximised when the middle block is the minimum subarray:

```
circular = max( maxSubarray(a),  total − minSubarray(a) )
```

Exception: if every element is negative, minSubarray = total, so total − min = 0 corresponds to the empty subarray, which is not allowed. Return maxSubarray(a) in that case (detect it with maxSubarray < 0).

Empty-subarray conventions. If the problem allows an empty subarray (answer ≥ 0), start with `cur = 0, best = 0` and clamp `cur = max(0, cur + a[i])`. If subarrays must be non-empty (LeetCode 53, 918), initialise both with `a[0]`. Initialising `best = 0` returns 0 instead of −1 on `[-3,-1,-2]`.

Extensions worth knowing

- **Recovering indices.** Record `start` whenever you restart (`cur` becomes a[i]) and copy (start, i) when best improves.
- **Maximum product subarray (LeetCode 152).** Multiplying by a negative swaps max and min, so carry both `maxEnd` and `minEnd`. A plain Kadane is wrong here.
- **K-concatenation (LeetCode 1191).** The answer uses at most two copies plus (k − 2) · total when total > 0. Run Kadane over two copies.
- **2D maximum-sum rectangle.** Fix a pair of rows, compress columns into a 1D array, run Kadane: $O(R^2 \cdot C)$.
- **At most one deletion (LeetCode 1186).** Two states: best ending here with no deletion, and with one deletion used.

A5.5 Reusable Java implementation

JAVA TEMPLATE Kadane.java

```java
/** Kadane variants. Sums are accumulated in long so n * |a[i]| cannot overflow. */
class Kadane {
    /** Maximum sum of a NON-EMPTY contiguous subarray. Requires a.length >= 1. */
    static long maxSubarray(int[] a) {
        long cur = a[0], best = a[0];     // not 0: all-negative input must return max
        for (int i = 1; i < a.length; i++) {
            cur = Math.max(a[i], cur + a[i]);     // extend or restart
            best = Math.max(best, cur);
        }
        return best;
```

```
    }

    /** Maximum sum when the EMPTY subarray (sum 0) is allowed. */
    static long maxSubarrayAllowEmpty(int[] a) {
        long cur = 0, best = 0;
        for (int x : a) {
            cur = Math.max(0, cur + x);
            best = Math.max(best, cur);
        }
        return best;
    }

    /** Returns {bestSum, start, end} (inclusive) of a non-empty maximum subarray. */
    static long[] maxSubarrayWithIndices(int[] a) {
        long cur = a[0], best = a[0];
        int start = 0, bestL = 0, bestR = 0;
        for (int i = 1; i < a.length; i++) {
            if (cur < 0) {                // previous run can only hurt: restart at i
                cur = a[i];
                start = i;
            } else {
                cur += a[i];
            }
            if (cur > best) {
                best = cur;
                bestL = start;
                bestR = i;
            }
        }
        return new long[]{best, bestL, bestR};
    }

    /** Minimum sum of a non-empty subarray (mirror image). */
    static long minSubarray(int[] a) {
        long cur = a[0], best = a[0];
        for (int i = 1; i < a.length; i++) {
            cur = Math.min(a[i], cur + a[i]);
            best = Math.min(best, cur);
        }
        return best;
    }
}
```

A5.6 Time and space complexity

Time	O(n), a single pass. The circular version is also O(n) (one pass tracking max, min and total).
Auxiliary space	O(1). The DP table `end[]` collapses to one variable because end(i) depends only on end(i − 1).

A5.7 Common pitfalls

- **Zero initialisation** when subarrays must be non-empty: returns 0 for all-negative input.
- **Updating best before cur**, or resetting `cur = 0` *before* adding a[i]. That variant silently allows the empty subarray.
- **Circular all-negative case**: `total − min` = 0 is the empty subarray. Guard with `if (max < 0) return max`.
- **Using Kadane with a target or length constraint.** Kadane cannot enforce either; switch to prefix sums.
- **Overflow** on long arrays of large values. LeetCode 53 fits in int ($n \le 10^5$, $|a| \le 10^4$), but a template should use long.

A5.8 Comparison with related algorithms

Technique	Use when	Why not Kadane
Prefix sum + hash map (P4)	Subarray sum equals k, divisible by k, longest with sum k	Kadane optimises; it cannot match an exact value.
Sliding window (P2)	Non-negative values with a sum bound	With non-negative values and no bound, the whole array is optimal, so Kadane is unnecessary.
Prefix sum + monotonic deque	Maximum sum with length ≤ L, shortest subarray with sum ≥ K (LeetCode 862)	Needs the minimum prefix over a *bounded* range.
Divide and conquer	When a segment tree must answer max-subarray queries on ranges with updates	Kadane is offline and single-range; the (sum, prefix, suffix, best) merge supports range queries.
General DP (P12)	Extra state: deletions, products, k concatenations	Kadane is the one-state special case of this DP.

A5.9 Practice problems

LC 53 Maximum Subarray

MEDIUM

Task. Return the largest sum of a non-empty contiguous subarray.

KEY INSIGHT

A negative running sum is a liability to every later subarray, so restart whenever extending is worse than starting at the current element.

WHY THE ALGORITHM APPLIES

The objective is a contiguous sum with no constraint, and negatives are present. This is exactly the end(i) recurrence.

APPROACH

1. `cur = best = nums[0]`.
2. For each later x: `cur = max(x, cur + x)`, `best = max(best, cur)`.

JAVA SOLUTION LeetCode 53 · Solution.java

```java
class Solution {
    public int maxSubArray(int[] nums) {
        int cur = nums[0], best = nums[0];    // non-empty: start from the first element
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);  // |sum| <= 1e9: fits in int
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Time O(n) **Space** O(1)

Pitfall. `best = 0` fails on `[-1]`. The follow-up asks for divide and conquer: O(n log n), merging the best crossing sum (best suffix of the left half + best prefix of the right half). It is worth sketching, but Kadane is the expected answer.

LC 918 Maximum Sum Circular Subarray

MEDIUM

Task. Return the maximum sum of a non-empty subarray of a circular array, where each element may be used at most once.

KEY INSIGHT

A wrapping subarray is the complement of a non-wrapping middle block, so its best sum is `total - minSubarray`. The answer is the larger of the ordinary Kadane maximum and that complement, except when everything is negative.

WHY THE ALGORITHM APPLIES

Both cases reduce to linear Kadane runs (one for the maximum, one for the minimum), which can be done in the same pass.

APPROACH

1. In one pass, track `total`, the max-ending-here / global max, and the min-ending-here / global min.
2. If `maxSum < 0`, every element is negative: return `maxSum`.
3. Otherwise return `max(maxSum, total - minSum)`.

JAVA SOLUTION LeetCode 918 · Solution.java

```java
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0;
        int curMax = 0, maxSum = nums[0];
        int curMin = 0, minSum = nums[0];
        for (int x : nums) {
            curMax = Math.max(x, curMax + x);   // best non-wrapping subarray ending here
            maxSum = Math.max(maxSum, curMax);
            curMin = Math.min(x, curMin + x);   // worst subarray ending here
            minSum = Math.min(minSum, curMin);
            total += x;                         // |total| <= 3e4 * 3e4 = 9e8: fits in int
        }
        // All negative: total - minSum == 0 would be the (forbidden) empty subarray.
        if (maxSum < 0) return maxSum;
        return Math.max(maxSum, total - minSum);
    }
}
```

Time O(n) **Space** O(1)

Pitfall. Forgetting the all-negative guard returns 0 for `[-3,-2,-3]` (expected −2). Do not guard with `total == minSum`: that also holds for inputs like `[0,-1]`, where the minimum subarray is the whole array but a non-negative element exists. There `total - minSum = 0` is still correct, but the condition is fragile to reason about. The clean test is `maxSum < 0`, which says exactly "no non-negative element exists". An alternative is a prefix-sum + monotonic deque over the doubled array (window length ≤ n), which is O(n) time and O(n) space.

KMP String Matching

Knuth–Morris–Pratt finds every occurrence of a pattern in linear time by precomputing, for each prefix of the pattern, its longest proper border. A mismatch then jumps straight to the next viable alignment.

A6.1 Purpose and practical applications

Purpose	Exact matching of a pattern P (length m) in a text T (length n) in O(n + m) worst case. Naive matching is O(n·m).
Applications	First or all occurrences, counting overlapping matches, the longest prefix that is also a suffix (LeetCode 1392), the smallest period of a string (LeetCode 459), the longest palindromic prefix (LeetCode 214), and rotation checks (B is a substring of A + A).
Key insight	When P[0..j−1] has matched and P[j] fails, the text's last j characters *are* P[0..j−1]. The next alignment that could succeed keeps the longest proper prefix of P[0..j−1] that is also its suffix, and that length is precomputed.

A6.2 Recognition signals

Signal	Structural reason
Exact substring search with adversarial worst cases (`aaaa…ab`)	Naive search degrades to O(n·m); KMP never moves backward in the text.
"Longest prefix that is also a suffix", "border", "period", "repeated pattern"	These are exactly the values of the prefix function.
Prefix of S matched against a suffix of another string	Concatenate with a separator and read the final prefix-function value.
Streaming text, with one pass and no backtracking	The matcher's state is a single integer j (the matched prefix length).

> **Ask yourself.** "Is this about a prefix of a string reappearing later (as a suffix or inside another string)?" If yes, a prefix function on a well-chosen concatenation usually answers it. For *many* patterns at once, use Aho–Corasick or a trie (P19). For approximate or multi-length comparisons, rolling hashes are often simpler.

A6.3 Core concept and invariant

Prefix function. For a string P, `pi[i]` = the length of the longest *proper* prefix of P[0..i] that is also a suffix of P[0..i]. Proper means shorter than i + 1.

```
P   = a b a b c a b a b a
pi  = 0 0 1 2 0 1 2 3 4 3
```

pi[8] = 4: "abab" is both a prefix and a suffix of "ababcabab". pi[9] = 3: "aba".

Search invariant. After reading T[i], `j` = the length of the longest prefix of P that is a suffix of T[0..i]. A match is reported when j = m, after which j becomes pi[m − 1] so overlapping matches are found.

Why matched characters need not be rechecked. Suppose P[0..j−1] equals T[i−j..i−1] and P[j] ≠ T[i]. Any shift that keeps fewer characters aligned must line a prefix of P up against a suffix of T[i−j..i−1] = P[0..j−1], which means a border of P[0..j−1]. Shifts that do not correspond to a border are mismatched on characters already known, so they are impossible. Trying borders in decreasing length (pi[j−1], pi[pi[j−1]−1], …) visits every viable alignment and skips the impossible ones without reading the text again.

A6.4 Important formulas and derivations

Construction phase (pattern against itself). Borders of P[0..i] are obtained by extending borders of P[0..i−1]:

```
pi[0] = 0
for i = 1..m−1:
    k = pi[i−1]                          longest border of P[0..i−1]
    while k > 0 and P[i] ≠ P[k]:  k = pi[k−1]    next shorter border
    if P[i] == P[k]:  k++
    pi[i] = k
```

This is correct because the borders of a string are exactly pi[i−1], pi[pi[i−1]−1], …, so the loop tries every candidate border in decreasing order and takes the first that extends by P[i].

Search phase. This is the same loop with T[i] in place of P[i], continuing to compare against P[j].

Linear-time (amortised) proof. In each phase, k (or j) increases by at most 1 per character, so by at most n in total. Each iteration of the inner `while` loop decreases it by at least 1, and it never goes below 0. Therefore the total number of fallbacks is at most the total number of increases, and the work is O(m) for construction plus O(n) for search.

Useful identities.

```
Longest border of S                = pi[|S| − 1]
Smallest period of S               = p = |S| − pi[|S| − 1]
S is a repetition of a block       ⇔  pi[|S|−1] > 0  and  |S| mod p == 0
Longest palindromic prefix of S    = pi[last] of  S + '#' + reverse(S)
```

A6.5 Reusable Java implementation

JAVA TEMPLATE KMP.java

```java
import java.util.*;

/** Knuth–Morris–Pratt. pi[i] = longest proper border of p[0..i]. */
class KMP {
    static int[] prefixFunction(String p) {
        int m = p.length();
        int[] pi = new int[m];
        for (int i = 1; i < m; i++) {
            int k = pi[i − 1];
            while (k > 0 && p.charAt(i) != p.charAt(k)) k = pi[k − 1];  // shorter border
            if (p.charAt(i) == p.charAt(k)) k++;
            pi[i] = k;
        }
        return pi;
    }

    /** All start indices of pattern in text (overlapping matches included). */
```

```
    static List<Integer> findAll(String text, String pattern) {
        List<Integer> res = new ArrayList<>();
        int m = pattern.length();
        if (m == 0) return res;          // convention: caller decides empty-pattern rule
        int[] pi = prefixFunction(pattern);
        int j = 0;                                  // length of matched prefix of pattern
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            while (j > 0 && c != pattern.charAt(j)) j = pi[j - 1];  // never move i back
            if (c == pattern.charAt(j)) j++;
            if (j == m) {
                res.add(i - m + 1);
                j = pi[m - 1];                  // continue for overlapping matches
            }
        }
        return res;
    }

    /** Smallest period of s (s.length() if s is not a repetition of a shorter block). */
    static int smallestPeriod(String s) {
        int n = s.length();
        if (n == 0) return 0;
        int p = n - prefixFunction(s)[n - 1];
        return n % p == 0 ? p : n;
    }
}
```

A6.6 Time and space complexity

Time	O(m) preprocessing + O(n) search = O(n + m) worst case, independent of the alphabet.
Auxiliary space	O(m) for the prefix function; O(n + m) if you build a concatenated string.

A6.7 Common pitfalls

- `if` **instead of** `while` for the fallback: it stops after one shorter border, so it can compare against the wrong candidate and produce invalid pi values. For pattern `aacaaab`, the final value comes out as 1 instead of 0, because the last `b` is compared with `a` after only one fallback.
- **Off-by-one in the link**: the fallback is `pi[k - 1]`, not `pi[k]`; `pi` is indexed by the *last index* of the prefix.
- **Proper border**: pi[0] must be 0 and the loop starts at i = 1. Otherwise every string is its own border.
- **Separator** in concatenation tricks: it must not occur in either string, or it must be a character the alphabet excludes. This keeps the border from crossing the boundary.
- **Resetting j to 0 after a full match** misses overlapping occurrences (`"aa"` in `"aaa"`). Use `j = pi[m-1]`.
- **Library comparison**: `String.indexOf` is naive O(n·m) in the worst case but very fast in practice. Mention KMP's guarantee when asked about worst cases.

A6.8 Comparison with related algorithms

Algorithm	Complexity	When to prefer it
Naive / `indexOf`	O(n·m) worst, fast average	Short patterns, non-adversarial input, interview "warm-up" answers.
KMP	O(n + m) worst	Guaranteed linear time; borders, periods and prefix/suffix questions.
Z-algorithm	O(n + m)	Equivalent power. z[i] = the longest common prefix of S and S[i..]. Some find it easier for "prefix occurs at position i" questions.

Algorithm	Complexity	When to prefer it
Rabin–Karp (rolling hash)	O(n + m) expected	Many lengths or many substring comparisons (longest duplicate substring with binary search). Requires collision handling.
Aho–Corasick / Trie	O(n + Σm + matches)	Many patterns at once (dictionary matching).
Manacher	O(n)	All palindromic substrings. KMP only gives palindromic *prefixes* via the reverse trick.

A6.9 Practice problems

LC 28 Find the Index of the First Occurrence in a String

EASY

Task. Return the index of the first occurrence of `needle` in `haystack`, or −1. Both strings have length ≥ 1.

KEY INSIGHT

After a partial match, the prefix function tells you how much of the match survives a shift, so the text pointer never moves backward.

WHY THE ALGORITHM APPLIES

This is direct exact matching. Constraints ($\leq 10^4$) also allow naive search, but KMP is the linear-worst-case answer interviewers usually probe for.

APPROACH

1. Build `pi` for the needle.
2. Scan the haystack while maintaining `j`; return `i - m + 1` the first time `j == m`.

JAVA SOLUTION LeetCode 28 · Solution.java

```java
class Solution {
    public int strStr(String haystack, String needle) {
        int m = needle.length();
        if (m == 0) return 0;                       // defensive: constraints say m >= 1
        int[] pi = new int[m];
        for (int i = 1; i < m; i++) {               // construction phase
            int k = pi[i - 1];
            while (k > 0 && needle.charAt(i) != needle.charAt(k)) k = pi[k - 1];
            if (needle.charAt(i) == needle.charAt(k)) k++;
            pi[i] = k;
        }
        int j = 0;
        for (int i = 0; i < haystack.length(); i++) {   // search phase
            char c = haystack.charAt(i);
            while (j > 0 && c != needle.charAt(j)) j = pi[j - 1];
            if (c == needle.charAt(j)) j++;
            if (j == m) return i - m + 1;
        }
        return -1;
    }
}
```

Time O(n + m) **Space** O(m)

> **Pitfall.** Returning `i - m` instead of `i - m + 1`: at the moment j reaches m, index i is the *last* character of the match.

LC 214 Shortest Palindrome

HARD

Task. Add the fewest characters to the *front* of `s` to make it a palindrome; return the result. ($0 \le |s| \le 5 \cdot 10^4$, lowercase letters.)

KEY INSIGHT

Keep the longest palindromic prefix of s and prepend the reverse of the remaining suffix. A prefix of s is a palindrome exactly when it equals a suffix of reverse(s), so the longest one is the final prefix-function value of `s + '#' + reverse(s)`.

WHY THE ALGORITHM APPLIES

"Longest prefix of A that is a suffix of B" is a border question on A + sep + B. The separator stops the border from being longer than |s|. The naive check of every prefix is $O(n^2)$, which is borderline at $5 \cdot 10^4$ with adversarial input such as `aaaa…ab`.

APPROACH

1. `rev = reverse(s)`, `t = s + "#" + rev`.
2. `k = pi(t)[|t| − 1]` = the length of the longest palindromic prefix of s.
3. Return `rev.substring(0, n − k) + s`: the reversed unmatched suffix in front.

JAVA SOLUTION LeetCode 214 · Solution.java

```java
class Solution {
    public String shortestPalindrome(String s) {
        int n = s.length();
        String rev = new StringBuilder(s).reverse().toString();
        String t = s + "#" + rev;               // '#' never occurs in s (lowercase only)
        int[] pi = new int[t.length()];
        for (int i = 1; i < t.length(); i++) {
            int k = pi[i - 1];
            while (k > 0 && t.charAt(i) != t.charAt(k)) k = pi[k - 1];
            if (t.charAt(i) == t.charAt(k)) k++;
            pi[i] = k;
        }
        int pal = pi[t.length() - 1];               // longest palindromic prefix of s
        return rev.substring(0, n - pal) + s;   // prepend reverse of the remaining suffix
    }
}
```

Time O(n) **Space** O(n)

Pitfall. Without the separator, a border of `s + rev` can extend past |s| (for `s = "aa"`, `"aaaa"` has border 3), giving a wrong length. Handle the empty string: the code returns `""` because `t = "#"` and pal = 0.

Bellman–Ford Algorithm

Relax every edge, repeatedly. After round i, every shortest path that uses at most i edges has been found. This one fact gives negative-weight support, negative-cycle detection and edge-budgeted shortest paths.

A7.1 Purpose and practical applications

Purpose	Single-source shortest paths in a directed (or undirected, with care) graph whose edge weights may be negative. It also detects negative cycles reachable from the source.
Applications	Shortest paths with at most K edges (LeetCode 787), currency arbitrage (a negative cycle of −log rates), difference constraints ($x_v - x_u \le w$), and routing protocols (distance-vector routing is distributed Bellman–Ford).
Key insight	Without a reachable negative cycle, some shortest path to every vertex is simple, so it has at most V − 1 edges. Round i extends correct "≤ i − 1 edges" distances by one edge.

> **Honest scope note.** LeetCode has very few mainstream problems with genuinely negative edge weights. Its main Bellman–Ford application is the *edge-budget* variant (787). Negative weights and cycle detection are still standard interview discussion topics, so the template below implements them in full.

A7.2 Recognition signals

Signal	Structural reason
Negative edge weights	Dijkstra's "popped means final" argument fails; Bellman–Ford needs no ordering.
"At most K edges / stops / transfers"	Round-limited relaxation computes exactly the best path with ≤ i edges after round i (with snapshots).
Detect arbitrage or an infinitely improvable cycle	A V-th round that still relaxes an edge proves a negative cycle is reachable.
Small V·E (e.g. V ≤ 100, E ≤ 10^4)	O(V·E) fits the budget, and the code is shorter than Dijkstra's.

> **Ask yourself.** "Is there a negative weight or an edge-count limit?" If neither applies and weights are non-negative, Dijkstra (A2) is faster. If all pairs are needed and V ≤ ~400, use Floyd–Warshall.

A7.3 Core concept and invariant

Relaxation. `if dist[u] + w(u, v) < dist[v] then dist[v] = dist[u] + w(u, v)`. A relaxation never sets dist[v] below the weight of some real path to v, so distances are always upper bounds (or ∞).

```
Invariant after round i (snapshot version):
  dist_i[v] = min weight over walks src ⇝ v using at most i edges

dist_0[src] = 0, dist_0[v] = ∞ otherwise
dist_i[v]   = min( dist_{i-1}[v],  min over edges (u,v,w) of dist_{i-1}[u] + w )
```

This is a DP over "number of edges used". With in-place updates (the classical algorithm), dist after round i is *at most* dist_i: it may already include longer paths. That does not affect correctness for unconstrained shortest paths, but it breaks any edge budget.

A7.4 Important formulas and derivations

Why V − 1 rounds suffice. If no negative cycle is reachable from the source, every shortest walk can have its cycles removed (each has weight ≥ 0) without increasing its cost. The result is a simple path with at most V − 1 edges. By the invariant, round V − 1 has considered every path with ≤ V − 1 edges, so the distances are final.

Negative-cycle detection. If after V − 1 rounds some edge (u, v) can still be relaxed, with dist[u] finite, then a shortest path would need ≥ V edges, so it repeats a vertex. That repeated section must be a negative cycle, and it is reachable from src (dist[u] is finite). Conversely, a reachable negative cycle always leaves at least one relaxable edge: summing `dist[v] ≤ dist[u] + w` around the cycle would give 0 ≤ (cycle weight) < 0.

```
Detection:  ∃ (u,v,w): dist[u] ≠ ∞ and dist[u] + w < dist[v]  after V-1 rounds
         ⇔  a negative cycle is reachable from src

Vertices whose distance is −∞ : every vertex reachable FROM a vertex on (or fed by) that
cycle.
Propagate −∞ with V more rounds, or BFS from every vertex that still relaxes.
```

Early termination. If a full round changes nothing, no later round can change anything, because each round depends only on the previous one. Stop early. When distances stabilise, no reachable negative cycle can exist.

A7.5 Reusable Java implementation

Template A — classical Bellman–Ford with early exit, detection and −∞ propagation. Directed edges `{u, v, w}`; weights may be negative.

JAVA TEMPLATE BellmanFord.java

```java
import java.util.*;

/** Single-source shortest paths, weights may be negative. Directed edges {u, v, w}. */
class BellmanFord {
    static final long INF = Long.MAX_VALUE / 4;      // sentinel; INF + w cannot overflow
    static final long NEG_INF = Long.MIN_VALUE;      // marks "arbitrarily small" distances

    /**
     * Returns dist[] where dist[v] == INF if v is unreachable and dist[v] == NEG_INF if a
     * negative cycle reachable from src can reach v (no finite shortest path exists).
     */
    static long[] shortestPaths(int n, int[][] edges, int src) {
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 1; round <= n - 1; round++) {
            boolean changed = false;
```

```
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                if (dist[u] != INF && dist[u] + e[2] < dist[v]) {   // skip unreachable u
                    dist[v] = dist[u] + e[2];
                    changed = true;
                }
            }
            if (!changed) return dist;              // stable: no reachable negative cycle
        }
        // Any edge still relaxable lies on or after a reachable negative cycle.
        // n more rounds propagate NEG_INF along every path leaving such edges.
        for (int round = 0; round < n; round++) {
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                if (dist[u] == INF) continue;
                if (dist[u] == NEG_INF || dist[u] + e[2] < dist[v]) dist[v] = NEG_INF;
            }
        }
        return dist;
    }

    /** True iff some negative cycle is reachable from src. */
    static boolean hasReachableNegativeCycle(int n, int[][] edges, int src) {
        for (long d : shortestPaths(n, edges, src)) if (d == NEG_INF) return true;
        return false;
    }
}
```

Template B — shortest path with at most K edges (snapshot per round).

JAVA TEMPLATE BoundedEdgesBellmanFord.java

```
import java.util.*;

/** min-cost walk src -> every v using AT MOST k edges. Directed edges {u, v, w}. */
class BoundedEdgesBellmanFord {
    static final long INF = Long.MAX_VALUE / 4;

    static long[] atMostKEdges(int n, int[][] edges, int src, int k) {
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 1; round <= k; round++) {
            long[] next = dist.clone();             // reads use round-(i-1) values only
            boolean changed = false;
            for (int[] e : edges) {
                int u = e[0], v = e[1];
                if (dist[u] != INF && dist[u] + e[2] < next[v]) {
                    next[v] = dist[u] + e[2];
                    changed = true;
                }
            }
            dist = next;
            if (!changed) break;                    // further rounds cannot help
        }
        return dist;
    }
}
```

> **Why the snapshot matters.** With in-place updates, one round can relax a → b and then b → c using the *new* dist[b], so it uses two edges in a single round. If the edge list happens to be in path order, one round can traverse a whole chain, and the "≤ i edges after round i" guarantee disappears.

A7.6 Time and space complexity

Time	O(V·E) worst case (V − 1 rounds × E edges); O(K·E) for the edge-budget variant; often far less with early termination.
Auxiliary space	O(V) for distances (+ O(V) for the snapshot). The edge list is the input.

A7.7 Common pitfalls

- **Relaxing from unreachable vertices**: `INF + (negative w)` is less than INF and creates fake distances. Always check `dist[u] != INF`.
- **Overflow**: using `Integer.MAX_VALUE` as INF and adding w overflows. Use a guarded check or `Long.MAX_VALUE / 4`.
- **In-place updates under an edge budget** (see the warning above).
- **Undirected graphs with negative edges**: an undirected negative edge is a negative cycle of length 2 (u → v → u). Bellman–Ford is meaningless there unless the problem forbids reusing an edge.
- **Detecting any negative cycle, not only reachable ones**: add a virtual source with 0-weight edges to all vertices (or start with dist = 0 everywhere).
- **K stops vs K edges**: in 787, K *stops* means K + 1 *flights* (edges).

A7.8 Comparison with related algorithms

Algorithm	Weights	Complexity	Notes
BFS (P10)	All equal	O(V + E)	Layer order = distance order.
Dijkstra (A2)	≥ 0	O((V + E) log V)	Greedy finalisation. Incorrect with negative edges, even without negative cycles.
Bellman–Ford	Any; detects reachable negative cycles	O(V·E)	Round structure gives edge-budgeted paths for free.
SPFA (queue-based BF)	Any	O(V·E) worst, often fast	Relax only from vertices whose distance changed. Same worst case; adversarial inputs exist.
Floyd–Warshall	Any; negative cycle ⇔ dist[i][i] < 0	$O(V^3)$	All pairs; simplest for V ≤ ~400.
Topological DP	Any, DAG only	O(V + E)	Relax in topological order. No cycles means no negative-cycle issue.

A7.9 Practice problem

LC 787 Cheapest Flights Within K Stops — MEDIUM

Task. Directed flights `[from, to, price]` with price ≥ 1. Return the cheapest price from `src` to `dst` using at most K intermediate stops (so at most K + 1 flights), or −1.

KEY INSIGHT

"At most K stops" is "at most K + 1 edges". After round i of snapshot Bellman–Ford, `dist[v]` is exactly the cheapest walk with ≤ i edges, so run K + 1 rounds and read `dist[dst]`.

WHY THE ALGORITHM APPLIES

The weights are positive, so Dijkstra is tempting. But a vertex's cheapest path may use too many edges, while a dearer path with fewer edges is what later vertices need. Plain Dijkstra on vertices discards that path. Bellman–Ford's round structure tracks the edge count directly. Chapter 18 solves the same problem with Dijkstra on the expanded state (vertex, edges used).

APPROACH

1. `dist[src] = 0`, others ∞.
2. Repeat K + 1 times: copy `dist` to `next`; relax every flight reading from `dist` and writing to `next`; set `dist = next`. Stop early if nothing changed.
3. Return `dist[dst]`, or −1 if it is still ∞.

JAVA SOLUTION LeetCode 787 · Solution.java

```java
import java.util.*;

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        final int INF = Integer.MAX_VALUE;
        int[] dist = new int[n];
        Arrays.fill(dist, INF);
        dist[src] = 0;
        for (int round = 0; round <= k; round++) {   // k stops = k + 1 edges = k + 1 rounds
            int[] next = dist.clone();               // snapshot: one more edge per round
            boolean changed = false;
            for (int[] f : flights) {
                int u = f[0], v = f[1], price = f[2];
                // Max cost <= 101 flights * 1e4 = ~1e6, so dist[u] + price cannot overflow.
                if (dist[u] != INF && dist[u] + price < next[v]) {
                    next[v] = dist[u] + price;
                    changed = true;
                }
            }
            dist = next;
            if (!changed) break;
        }
        return dist[dst] == INF ? -1 : dist[dst];
    }
}
```

Time O((K + 1) · E) **Space** O(V)

> **Pitfall.** Relaxing in place (no snapshot) can chain several flights in one round and return a price that needs more than K stops. Running exactly K rounds instead of K + 1 is the other common off-by-one.

PART IV

Revision Toolkit

Compact material for the last pass before an interview: one quick-reference sheet for every technique, a checklist to run before writing code, a complexity reference for the Java collections you will actually use, and a tracking grid with a self-assessment framework that measures readiness by observable behaviour rather than by feel.

PART IV · A

Quick-reference sheet

Table A1 — The 20 patterns

Pattern	Recognition signal	Core idea	Typical complexity	Most important pitfall
1 · Two Pointers	Pair or triplet over sorted data; score set by the two ends	Each move discards a row or column of candidate pairs	O(n) after an O(n log n) sort	Sorting when original indices are needed; moving the taller side
2 · Sliding Window	Contiguous range; validity monotone in the window	Expand right; shrink left while invalid (or to fixed size)	O(n)	Negative values break the shrink condition
3 · Fast & Slow	Linked list, or iterated function x → f(x)	The speed gap closes the distance by 1 per step inside a cycle	O(n), O(1)	Null checks on `fast.next`; entry-point reset
4 · Prefix Sum	Range sums; exact sum or modular conditions	sum(i..j) = pre[j+1] − pre[i]; hash prior prefixes	O(n) time, O(n) map	Missing the empty prefix (pre = 0 at index −1)
5 · Binary Search	Sorted or monotone-predicate index space	Keep the answer inside [lo, hi] and halve it	O(log n)	Mixing inclusive and half-open bounds; infinite loops
6 · Binary Search on Answer	"Minimise the maximum"; smallest feasible speed or capacity	Monotone feasible(x); search the value range	O(n log V)	Wrong bounds; a predicate that is not monotone
7 · Merge Intervals	Overlap, coverage, rooms, free time	Sort by start; compare only with the last merged interval	O(n log n)	Touching endpoints: closed vs half-open
8 · Backtracking	Enumerate all subsets, permutations or placements	Choose → explore → undo, with pruning	$O(2^n \cdot n)$, O(n!·n)	Adding the mutable path without copying it
9 · DFS	Tree aggregates, components, path values	Return a subtree summary; keep the global answer separate	O(V + E)	Recursion depth; marking visited too late
10 · BFS	Fewest steps with unit-weight edges; spread from sources	FIFO layers equal distance layers; multi-source start	O(V + E)	Marking visited on pop instead of on push
11 · Topological Sort	Dependencies, prerequisites, ordering constraints	Emit indegree-0 nodes (Kahn) or reverse postorder (DFS)	O(V + E)	Treating a cycle as a valid order; edge direction
12 · DP 1D	Linear sequence of decisions; count or optimise	dp[i] from a bounded set of earlier states	O(n·choices)	A state that misses information the future needs

Pattern	Recognition signal	Core idea	Typical complexity	Most important pitfall
13 · DP 2D	Grid paths; two-sequence alignment	dp[i][j] from neighbours; fill in dependency order	O(m·n)	Base row and column; wrong order when rolling arrays
14 · Greedy	Local choice provably safe (interval scheduling)	Exchange argument; sort by the right key	O(n log n)	Sorting by start instead of end for maximum compatible sets
15 · Heap	Top-K, k-th element, repeated extremes, streams	Keep k items in an opposite-order heap; two heaps for medians	O(n log k)	Comparator subtraction overflow
16 · Bit Manipulation	Multiplicity mod k, masks, integer ranges	XOR cancellation; per-bit counting; `x & -x`	O(n·32) or O(n)	`>>` vs `>>>`; sign bit; `1 << 31`
17 · Union-Find	Incremental connectivity; grouping; cycle on insertion	Root per component; path compression + union by size	O(α(n)) amortised	Union of nodes rather than roots; needing deletions
18 · Dijkstra	Weighted shortest path, weights ≥ 0	Pop the minimum tentative distance; relax neighbours	O((V+E) log V)	Not skipping stale heap entries; extra constraints need a bigger state
19 · Trie	Prefix queries over many words; word-board search	Shared prefix paths with end-of-word markers	O(L) per operation	Confusing "is a prefix" with "is a word"
20 · Segment / Fenwick	Range queries interleaved with updates	Tree of merged aggregates / low-bit prefix sums	O(log n) per operation	0- vs 1-based indexing; Fenwick for non-invertible operations

Table A2 — The 7 essential algorithms

Algorithm	Recognition signal	Core idea	Typical complexity	Most important pitfall
A1 · Monotonic Stack	Next or previous greater/smaller; spans; histogram; contribution sums	Pop elements the new one resolves; the stack stays monotone	O(n) amortised	Strict vs non-strict comparisons with duplicates (double counting)
A2 · Dijkstra	Single-source shortest path, non-negative weights	Greedy finalisation via a min-heap; lazy deletion	O((V+E) log V)	Negative edges; overflow in path counts or sums
A3 · Union-Find	Equivalence classes; components after edge additions	find with compression; union by size	O(α(n)) amortised	Counting components without decrementing on successful unions
A4 · MST	Connect all nodes at minimum total edge cost (undirected)	Cut property: the lightest crossing edge is safe	O(E log E) / $O(V^2)$	Disconnected graph (spanning forest, not a tree)
A5 · Kadane	Max or min contiguous-sum subarray	Best ending here = max(a[i], previous + a[i])	O(n), O(1)	Zero initialisation on all-negative input
A6 · KMP	Exact matching; borders; periods; palindromic prefix	Fall back along the longest proper borders; never re-read the text	O(n + m)	`if` instead of `while`; `pi[k]` instead of `pi[k-1]`
A7 · Bellman–Ford	Negative weights; negative cycles; at most K edges	V−1 rounds of relaxing every edge	O(V·E)	In-place updates under an edge budget; relaxing from INF

Interview recognition checklist

Run this before writing any code. Each phase should take about a minute for a Medium problem, and saying it aloud doubles as communication with the interviewer.

1. Understand

- ☐ Restate the task in one sentence: input → output.
- ☐ Clarify contiguous vs any subset; distinct vs repeated values; 0- or 1-based output.
- ☐ Read the constraints; write down the complexity budget.
- ☐ Work a small example by hand, including one edge case.

2. Recognise

- ☐ Input structure: array, string, intervals, tree, graph, grid, operations.
- ☐ Question type: existence, count, optimise, order, connectivity, shortest path.
- ☐ Exploitable property: order, monotonicity, invariant, overlapping subproblems, a safe greedy choice.
- ☐ Name the hypothesis *and* the closest wrong alternative, and why it is wrong.

3. Design

- ☐ State the invariant or recurrence in one sentence.
- ☐ Choose the data structure and its exact contents (indices or values? which comparator?).
- ☐ Fix the interval convention ([l, r] or [l, r)) and keep it.
- ☐ Decide types: `long` for sums and products; a modulus if counts grow.

4. Verify

- ☐ Dry-run the hand example through the code, not through your intention.
- ☐ Empty or single input; all equal; all negative; zero.
- ☐ Duplicates; touching intervals; cycles; disconnected parts; unreachable targets.
- ☐ Overflow at the maximum constraints; recursion depth.
- ☐ Loop termination: does every iteration shrink the search space?

5. Communicate

- ☐ State time and auxiliary space, including sort, heap, map and recursion stack.
- ☐ Mention the brute force and why the chosen approach improves on it.
- ☐ Pre-empt likely follow-ups: streaming, updates, k variants, memory limits.
- ☐ If stuck for more than 3 minutes, revisit the "Recognise" phase rather than patching code.

PART IV · C

Complexity reference

Table C1 — Java collections used in this handbook

Structure	Operations	Notes
`ArrayList`	`get/set` O(1); `add` at end O(1) amortised; `add/remove(index)` O(n); `contains` O(n)	`remove(int)` vs `remove(Object)` overload trap on `List<Integer>`.

Structure	Operations	Notes
`ArrayDeque`	`offer/poll/peek` at both ends O(1) amortised	Use it for stacks and queues (not `Stack` or `LinkedList`). Null elements are not allowed.
`HashMap` / `HashSet`	`get/put/remove/contains` O(1) expected	Since Java 8, heavily colliding bins become trees (O(log n) worst for Comparable keys). Iteration order is unspecified.
`TreeMap` / `TreeSet`	`get/put/remove` O(log n); `floor/ceiling/lower/higher`, `first/last`, `pollFirst` O(log n)	Red-black tree. The comparator defines equality, so a comparator returning 0 merges distinct items.
`PriorityQueue`	`offer/poll` O(log n); `peek` O(1); `remove(Object)/contains` O(n); construction from a collection O(n)	Binary min-heap by default. Iteration order is *not* sorted. Use lazy deletion instead of `remove(Object)`.
`Arrays.sort(int[])`	O(n log n)	Dual-pivot quicksort; not stable (irrelevant for primitives). The JDK 14+ implementation falls back to heapsort on adversarial inputs.
`Arrays.sort(T[])`, `Collections.sort`, `List.sort`	O(n log n), O(n) extra	TimSort, **stable**, near O(n) on almost-sorted data. Sorting `int[][]` with a comparator uses this path.
`String` / `StringBuilder`	`charAt` O(1); `substring` O(k) copy; `sb.append` O(1) amortised	String `+` in a loop is $O(n^2)$. `equals` is O(n), and `hashCode` is cached after the first call.

Table C2 — Algorithm complexities (this handbook)

Algorithm / operation	Time	Auxiliary space	Condition
Two pointers, sliding window, prefix sums, Kadane	O(n)	O(1) / O(k) / O(n)	Single pass; window state or map size as noted
Binary search (index) / on answer	O(log n) / O(n log V)	O(1)	Monotone predicate
Sorting-based sweeps (intervals, greedy)	O(n log n)	O(log n) to O(n)	Dominated by the sort
Heap top-k / two-heap median	O(n log k) / O(log n) per add, O(1) per median	O(k) / O(n)	Quickselect: O(n) expected, $O(n^2)$ worst
DFS / BFS / topological sort	O(V + E)	O(V)	Adjacency list; grids have V = R·C, E ≈ 4V
Union-Find (compression + size)	O(α(n)) amortised per operation	O(n)	Unions only, no splits
Dijkstra (binary heap, lazy)	O((V + E) log V)	O(V + E)	Weights ≥ 0
Bellman–Ford / bounded K edges	O(V·E) / O(K·E)	O(V)	Any weights; detects reachable negative cycles
Kruskal / Prim (heap) / Prim (array)	O(E log E) / O(E log V) / $O(V^2)$	O(V + E)	Undirected; dense graphs favour the $O(V^2)$ Prim
Trie insert / search	O(L)	O(total chars · alphabet) worst	L = word length
Segment tree / Fenwick tree	O(n) build; O(log n) query and update	O(4n) / O(n)	Lazy propagation for range updates
KMP	O(n + m)	O(m)	Worst-case linear
Backtracking: subsets / permutations	$O(2^n \cdot n)$ / O(n!·n)	O(n) path + output	Output-sensitive
DP 1D / 2D	O(states × transitions)	O(states), often one row	Rolling arrays when dependencies allow

Recursion stack notes. The Java default thread stack (often 512 KB–1 MB) typically fits a recursion depth in the low tens of thousands for small frames. Expect trouble around 10^5: a degenerate tree or linked list, or a 1000×1000 grid DFS whose depth can reach 10^6. Use an explicit `ArrayDeque` stack, BFS, or DSU in those cases, and always count recursion depth in the space complexity (O(h) for trees, O(V) for graphs).

Java value traps. `Integer` objects compared with `==` are only reliable within the cached range −128..127; use `equals` or unbox. `Math.abs(Integer.MIN_VALUE)` is negative. `(a + b) / 2` overflows, so use `a + (b − a) / 2`. `a − b` as a comparator overflows, so use `Integer.compare`.

PART IV · D

Revision checklist

Tick a box only when you have *demonstrated* it without notes: on a blank editor for templates, aloud for invariants, on an unseen problem for the last column. A printed copy works as a progress tracker.

Technique	Recognise	Explain invariant	Write template unaided	Edge cases	Complexity	Solve unfamiliar
1 · Two Pointers	☐	☐	☐	☐	☐	☐
2 · Sliding Window	☐	☐	☐	☐	☐	☐
3 · Fast & Slow Pointers	☐	☐	☐	☐	☐	☐
4 · Prefix Sum	☐	☐	☐	☐	☐	☐
5 · Binary Search	☐	☐	☐	☐	☐	☐
6 · Binary Search on Answer	☐	☐	☐	☐	☐	☐
7 · Merge Intervals	☐	☐	☐	☐	☐	☐
8 · Backtracking	☐	☐	☐	☐	☐	☐
9 · Recursion / DFS	☐	☐	☐	☐	☐	☐
10 · BFS & Level Order	☐	☐	☐	☐	☐	☐
11 · Topological Sort	☐	☐	☐	☐	☐	☐
12 · DP 1D	☐	☐	☐	☐	☐	☐
13 · DP 2D	☐	☐	☐	☐	☐	☐
14 · Greedy	☐	☐	☐	☐	☐	☐
15 · Heap / Priority Queue	☐	☐	☐	☐	☐	☐
16 · Bit Manipulation	☐	☐	☐	☐	☐	☐
17 · Union-Find	☐	☐	☐	☐	☐	☐
18 · Dijkstra	☐	☐	☐	☐	☐	☐
19 · Trie	☐	☐	☐	☐	☐	☐
20 · Segment / Fenwick Tree	☐	☐	☐	☐	☐	☐
A1 · Monotonic Stack	☐	☐	☐	☐	☐	☐
A2 · Dijkstra	☐	☐	☐	☐	☐	☐

Technique	Recognise	Explain invariant	Write template unaided	Edge cases	Complexity	Solve unfamiliar
A3 · Union-Find / DSU	☐	☐	☐	☐	☐	☐
A4 · Minimum Spanning Tree	☐	☐	☐	☐	☐	☐
A5 · Kadane	☐	☐	☐	☐	☐	☐
A6 · KMP	☐	☐	☐	☐	☐	☐
A7 · Bellman–Ford	☐	☐	☐	☐	☐	☐

PART IV · E

Self-assessment framework

Rate each technique on six independent dimensions. Being strong in one does not imply the others: many candidates recognise a pattern instantly but cannot prove it correct, or write a template fluently but mis-size the state on a new problem.

Dimension	Not yet	Developing	Interview-ready
Recognition	Identifies the pattern only after seeing a hint or the tag	Names the right pattern on familiar problems but is misled by surface wording	Names the pattern from structure within ~2 minutes on unseen problems, and says why the nearest alternative fails
Conceptual understanding	Can describe the steps but not why they work	States the invariant or recurrence for the template's standard form	Adapts the invariant to a variant (e.g. at most K vs exactly K, circular, weighted) without guidance
Template fluency	Needs to look up the skeleton	Writes it from memory with one or two boundary bugs found in testing	Writes it bug-free in under 5 minutes, with correct bounds and types on the first attempt
Independent implementation	Solves only problems already seen	Solves unseen Easy/Medium problems, but slower than 30 minutes	Solves unseen Mediums in about 20–25 minutes and Hards in about 40 minutes, including edge cases
Correctness proof	Argues by examples	Gives an informal invariant or exchange argument when prompted	Volunteers a concise proof (invariant, exchange, cut property, monotonicity) and identifies the assumption that would break it
Complexity analysis	States time only, often ignoring sorting or the recursion stack	Gives correct time and space for the standard form	Includes amortised arguments, the stack, heap and map sizes, worst vs expected, and how the bound changes under follow-ups

How to use it

1. **Timed cold attempts.** Pick an unseen problem tagged with the technique (or, better, untagged). Set a timer and do not look anything up. Record the time, the bugs and the hints you would have needed.
2. **Explain aloud.** Before coding, state the invariant and complexity in under a minute, as if to an interviewer. If you hesitate, the "Conceptual" and "Proof" levels are not yet interview-ready.

3. **Template drill.** Rewrite the chapter template on a blank editor, then compile and test it. Any boundary bug keeps "Template fluency" at Developing.
4. **Mixed sets.** Readiness is judged on *interleaved* problems from several chapters. Recognition practised on one chapter at a time overstates skill.
5. **Spaced review.** Re-test Developing items after about 3 days and Interview-ready items after about 2 weeks. Downgrade any item that fails.

> **Readiness rule of thumb.** A technique is interview-ready when you can solve two unseen problems with it, under time, with a stated proof and correct complexity, after at least a week without reviewing it.

PART IV · F

References

Problem titles, numbers, difficulties and premium status were verified against LeetCode's public problem catalogue. Problem pages:

- **P1** — 167. Two Sum II - Input Array Is Sorted · 15. 3Sum · 11. Container With Most Water · 42. Trapping Rain Water
- **P2** — 643. Maximum Average Subarray I · 3. Longest Substring Without Repeating Characters · 1493. Longest Subarray of 1's After Deleting One Element · 76. Minimum Window Substring
- **P3** — 876. Middle of the Linked List · 142. Linked List Cycle II · 547. Number of Provinces · 202. Happy Number
- **P4** — 724. Find Pivot Index · 560. Subarray Sum Equals K · 523. Continuous Subarray Sum · 325. Maximum Size Subarray Sum Equals k
- **P5** — 704. Binary Search · 34. Find First and Last Position of Element in Sorted Array · 162. Find Peak Element · 4. Median of Two Sorted Arrays
- **P6** — 1283. Find the Smallest Divisor Given a Threshold · 875. Koko Eating Bananas · 1011. Capacity To Ship Packages Within D Days · 410. Split Array Largest Sum
- **P7** — 56. Merge Intervals · 57. Insert Interval · 253. Meeting Rooms II · 759. Employee Free Time
- **P8** — 78. Subsets · 46. Permutations · 39. Combination Sum · 51. N-Queens
- **P9** — 104. Maximum Depth of Binary Tree · 200. Number of Islands · 547. Number of Provinces · 124. Binary Tree Maximum Path Sum
- **P10** — 733. Flood Fill · 994. Rotting Oranges · 542. 01 Matrix · 127. Word Ladder
- **P11** — 210. Course Schedule II · 207. Course Schedule · 802. Find Eventual Safe States · 269. Alien Dictionary
- **P12** — 70. Climbing Stairs · 198. House Robber · 1416. Restore The Array · 494. Target Sum
- **P13** — 62. Unique Paths · 64. Minimum Path Sum · 1143. Longest Common Subsequence · 72. Edit Distance
- **P14** — 252. Meeting Rooms · 435. Non-overlapping Intervals · 56. Merge Intervals · 986. Interval List Intersections
- **P15** — 1046. Last Stone Weight · 215. Kth Largest Element in an Array · 347. Top K Frequent Elements · 295. Find Median from Data Stream
- **P16** — 136. Single Number · 137. Single Number II · 260. Single Number III · 201. Bitwise AND of Numbers Range
- **P17** — 1971. Find if Path Exists in Graph · 684. Redundant Connection · 547. Number of Provinces · 721. Accounts Merge
- **P18** — 1971. Find if Path Exists in Graph · 743. Network Delay Time · 1631. Path With Minimum Effort · 787. Cheapest Flights Within K Stops
- **P19** — 208. Implement Trie (Prefix Tree) · 211. Design Add and Search Words Data Structure · 648. Replace Words · 212. Word Search II
- **P20** — 307. Range Sum Query - Mutable · 315. Count of Smaller Numbers After Self · 2407. Longest Increasing Subsequence II · 699. Falling Squares
- **A1** — 739. Daily Temperatures · 907. Sum of Subarray Minimums

- **A2** — 1514. Path with Maximum Probability · 1976. Number of Ways to Arrive at Destination
- **A3** — 1319. Number of Operations to Make Network Connected · 990. Satisfiability of Equality Equations
- **A4** — 1584. Min Cost to Connect All Points · 1135. Connecting Cities With Minimum Cost
- **A5** — 53. Maximum Subarray · 918. Maximum Sum Circular Subarray
- **A6** — 28. Find the Index of the First Occurrence in a String · 214. Shortest Palindrome
- **A7** — 787. Cheapest Flights Within K Stops

Algorithm and library references:

- Java SE 17 API — PriorityQueue (https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/PriorityQueue.html) · Arrays (https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html) · TreeMap (https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/TreeMap.html) · ArrayDeque (https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayDeque.html)
- cp-algorithms — Prefix function / KMP (https://cp-algorithms.com/string/prefix-function.html) · Fenwick tree (https://cp-algorithms.com/data_structures/fenwick.html) · Segment tree (https://cp-algorithms.com/data_structures/segment_tree.html) · Disjoint set union (https://cp-algorithms.com/data_structures/disjoint_set_union.html) · Bellman–Ford (https://cp-algorithms.com/graph/bellman_ford.html) · Dijkstra (https://cp-algorithms.com/graph/dijkstra.html) · Kruskal (https://cp-algorithms.com/graph/mst_kruskal.html) · Prim (https://cp-algorithms.com/graph/mst_prim.html)
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press — chapters on single-source shortest paths, minimum spanning trees, disjoint sets and string matching.